

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



HCMUTE

**TIỂU LUẬN MÔN HỌC
KIẾN TRÚC VÀ GIAO THỨC IOT**

THIẾT KẾ MÔ HÌNH IOT AQUANOVA

Giáo viên hướng dẫn: Ths. Trịnh Quốc Thanh

Nhóm thực hiện: Nhóm 2

Mã lớp học: ITAP338264_01

TP. Hồ Chí Minh, tháng 10 năm 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



HCMUTE

TIỂU LUẬN MÔN HỌC
KIẾN TRÚC VÀ GIAO THỨC IOT

THIẾT KẾ MÔ HÌNH IOT AQUANOVA

GVHD: Ths. Trịnh Quốc Thanh

SVTH: Nhóm 2

1: Đoàn Minh Duy Bình

MSSV : 23139005

2: Hoàng Ngọc Dung

MSSV : 23139006

3: Trần Hữu Dương

MSSV : 23139009

TP. Hồ Chí Minh, tháng 10 năm 2025

DANH SÁCH NHÓM VIẾT BÁO CÁO CUỐI KỲ MÔN KIẾN TRÚC
GIAO THỨC IOT
HỌC KỲ 1 NĂM HỌC 2025–2026

1. **Mã lớp môn học:** ITAP338264_01
2. **Giảng viên hướng dẫn:** Ths. Trịnh Quốc Thanh
3. **Tên đề tài:** Thiết kế mô hình IoT AquaNova
4. **Danh sách nhóm và phân công nhiệm vụ**

STT	Họ và tên sinh viên	MSSV	Nội dung thực hiện	Kí tên
01	Đoàn Minh Duy Bình	23139005	- Thiết kế sơ đồ nguyên lý, bố trí chân cảm biến và servo feeder. - Lập trình STM32: đọc ADC (độ ẩm), 1-Wire DS18B20 (nhiệt độ), PWM servo cho ăn. - Thực hiện hiệu chuẩn cảm biến và kiểm thử phần cứng	
02	Hoàng Ngọc Dung	23139006	- Khảo sát yêu cầu, chọn cảm biến và đề xuất mô hình hệ thống. - Lập trình giao diện web Flask + Dashboard hiển thị dữ liệu. - Thiết kế API Flask và MQTT (Feed Now, Scheduler). - Tổng hợp, biên soạn và chỉnh sửa báo cáo.	
03	Trần Hữu Dương	23139009	- Lập trình ESP32/IoT: Wi-Fi, MQTT (TLS, QoS=1), bridge UART–MQTT. - Thiết kế cơ khí bộ cho ăn, lắp đặt chống ẩm và nguồn. - Hỗ trợ hiệu chuẩn cảm biến và test hệ thống thực tế.	

Nhận xét của giảng viên

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Ngày tháng năm 2025

Giáo viên chấm điểm

LỜI CAM ĐOAN

Chúng em xin cam đoan đề tài nghiên cứu là trung thực. Những kết quả số liệu, thông tin phục vụ cho quá trình xử lý và hoàn thành bài nghiên cứu đều được thu thập từ các nguồn khác nhau, có ghi rõ nguồn gốc. Tất cả các thông tin số liệu, bảng, hình ảnh và kết quả nghiên cứu được đều được trình bày một cách minh bạch và chân thực, không chứa bất kỳ sự biến đổi hay chỉnh sửa nào có thể làm ảnh hưởng đến tính chính xác của kết quả.

Các nguồn dữ liệu và thông tin được sử dụng trong nghiên cứu đã được thu thập từ các nguồn đáng tin cậy và đã được ghi rõ nguồn gốc. Bất kỳ tài liệu tham khảo, nghiên cứu trước đây hoặc ý kiến cá nhân được sử dụng đều đã được đặc tả một cách rõ ràng để đảm bảo tính minh bạch và khách quan. Chúng em xin chịu hoàn toàn trách nhiệm, kỷ luật của bộ môn và nhà trường nếu như có bất cứ vấn đề nào.

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành đến Ths. Trịnh Quốc Thanh giảng viên bộ môn Kiến trúc và giao thức IoT lớp ITAP338264_01. Trong suốt quá trình học tập, Thầy đã rất tâm huyết dạy và hướng dẫn cho chúng tôi nhiều điều bổ ích trong môn học và kỹ năng làm một bài nghiên cứu để chúng tôi có đủ kiến thức thực hiện bài báo cáo này.

Tuy nhiên vì kiến thức bản thân còn nhiều hạn chế và sự tìm hiểu chưa sâu sắc nên không tránh khỏi những thiếu sót. Mong Thầy sẽ châm chước và cho chúng em những lời góp ý để bài nghiên cứu của chúng tôi sẽ hoàn thiện hơn. Một lần nữa, chúng tôi xin gửi lời cảm ơn sâu sắc đến Thầy và chúc Thầy luôn mạnh khỏe, hạnh phúc và thành công trong sự nghiệp.

TP. Hồ Chí Minh, tháng 10 năm 2025

MỤC LỤC

DANH MỤC HÌNH	i
DANH MỤC BẢNG BIỂU	iii
DANH MỤC CÁC CHỮ VIẾT TẮT	iv
CHƯƠNG 1. GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI	1
1. Tình hình nghiên cứu	1
2. Đặt vấn đề.....	2
3. Mục tiêu của đề tài.....	2
4. Nội dung nghiên cứu.....	3
5. Đối tượng, phạm vi nghiên cứu	4
6. Phương pháp nghiên cứu	5
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT	6
2.1. Mô hình giám sát chất lượng nước và cung cấp thức ăn tự động.....	6
2.1.1. Mô hình giám sát chất lượng nước, độ đục và cho cá ăn	6
2.1.2. Nguyên lý đo độ đục của nước	7
2.1.2. Hiện tượng phản xạ sóng âm trong cảm biến siêu âm	8
2.1.3. Hình thức cung cấp thức ăn	10
2.1.4. Mô hình chữ V.....	11
2.2. Cơ sở dữ liệu	13
2.2.1. Firsebase	13
2.2.2. Cloud Firestore	14
2.3. UI (User interface) và UX (User experience)	16
2.3.1. HTML (HyperText Markup Language)	16
2.3.2. CSS (Cascading Style Sheets)	17
2.3.3. JavaScript (JS)	18
2.3.4. HTTP (Hypertext Transfer Protocol)	19
2.3.5. IP Addresss	19
2.3.6. Flask (micro web framework for Python)	20
2.4. Các chuẩn truyền thông	22
2.4.1. Các chuẩn truyền thông không dây	23
2.4.2. Các chuẩn truyền thông có dây	24

2.5. Giao thức IoT: MQTT	26
2.6. Thiết bị phần cứng.....	32
2.6.1. Vi điều khiển	32
2.6.2. Cảm biến và các linh kiện khác	33
CHƯƠNG 3 THIẾT KẾ VÀ THI CÔNG MÔ HÌNH.....	39
3.1. Yêu cầu về tính năng của mô hình	40
3.2. Yêu cầu về kỹ thuật	41
3.2.1. Chức năng kỹ thuật.....	41
3.2.2. Thông số kỹ thuật	41
3.3. Mô hình tổng quan hệ thống.....	42
3.4. Thiết kế kiến trúc của mô hình	44
3.4.1. Khối nguồn	45
3.4.2. Khối cảm biến.....	46
3.4.3. Khối xử lý trung tâm	47
3.4.4. Kết nối toàn bộ hệ thống	47
3.5. Thiết kế cơ khí.....	49
3.5.1. Cài đặt phần mềm	49
3.5.2. Chi tiết thiết kế	50
3.5.3. Lắp ráp sản phẩm.....	51
3.6. Thiết kế nhúng	52
3.6.1. Cài đặt phần mềm và cấu hình	53
3.6.2. STM32.....	64
3.6.3. ESP32	93
3.7. Thiết kế website	105
3.7.1. Cài đặt phần mềm và cấu hình	105
3.7.2. Cấu trúc tổ chức chương trình	107
3.7.3. Frontend.....	109
3.7.4. Backend	120
3.7.5. Database.....	130
3.7.6. Giao thức IoT – MQTT	132
3.8. Hiệu chuẩn và kiểm thử	136

3.8.1. Đánh giá độ chính xác dữ liệu của môi trường và thiết bị đo	136
3.8.2. Đánh giá đường truyền kết nối wifi.....	136
3.8.3. Độ trễ và tần suất cập nhật	139
CHƯƠNG 4 THỰC NGHIỆM, KIỂM CHỨNG VÀ ĐÁNH GIÁ	142
4.1. Kết quả thiết kế phần cứng.....	142
4.1. Kết quả thiết kế phần mềm.....	143
Phản hồi ngược về server đã hoàn tất trong vòng dưới 0.5 giây.....	147
4.2. Mô phỏng một số kịch bản.....	148
4.3. Nhận xét, đánh giá và kết quả	150
CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	151
5.1. Kết luận	151
5.2. Hướng phát triển	152
TÀI LIỆU THAM KHẢO.....	154
PHỤ LỤC	156

DANH MỤC HÌNH

Hình 1: Logo mô hình AquaNova	6
Hình 2: Sơ đồ nguyên lý hoạt động của cảm biến độ đục theo tán xạ ánh sáng.	8
Hình 3: Sơ đồ minh họa nguyên lý hoạt động của cảm biến siêu âm.	9
Hình 4: Mô hình chữ V.....	11
Hình 5: Sơ đồ quy trình hoạt động của Firebase Cloud Messaging.....	13
Hình 6: Cơ chế đồng bộ dữ liệu thời gian thực của Cloud Firestore.....	14
Hình 7: Giao diện lưu trữ dữ liệu cảm biến trên Cloud Firestore.	15
Hình 8: UI/UX (HTML, CSS, JavaScript)	16
Hình 9: Sơ đồ truyền gói tin qua giao thức IP trong mô hình AquaNova.....	20
Hình 10: Minh họa sơ lược chuẩn truyền thông có dây và không dây.....	23
Hình 11: Cách kết nối của chuyên truyền SPI.....	24
Hình 12: Cách kết nối của chuẩn truyền UART.....	25
Hình 13: Mô hình hoạt động của giao thức MQTT theo cơ chế Publish/Subscribe	26
Hình 14: Cảm biến nhiệt độ MKE-S15 DS18B20	35
Hình 15: Cảm biến đo độ đục MJKDZ Turbidity Sensor	37
Hình 16: Phần mềm sử dụng để xây dựng AquaNova	41
Hình 17: Tổng quan mô hình AquaNova	Error! Bookmark not defined.
Hình 18: Kết nối MQTT vào ESP32 C3 kết hợp STM32	43
Hình 19: Kiến trúc tổng quan của hệ thống IoT 3 lớp của mô hình AquaNova	43
Hình 20: Sơ đồ khối của mô hình AquaNova	44
Hình 21: Khối cấp nguồn của mô hình AquaNova	46
Hình 22: Sơ đồ các cảm biến trong mô hình	46
Hình 23: UART giữa vi điều khiển STM32 và ESP32 trong mô hình AquaNova	47
Hình 24: Sơ đồ nguyên lý mô hình IoT AquaNova	48
Hình 25: Phần mềm vẽ cơ khí	49
Hình 26: Giao diện phần mềm.....	49
Hình 27: Hình vẽ 3d giá đỡ giữ thức ăn và servo và phần cố định (mặt trên)	50
Hình 28: Hình vẽ 3d giá đỡ giữ thức ăn và servo và phần cố định (mặt dưới)	50
Hình 29: Giá đỡ phần giữ thức ăn và servo.....	51
Hình 30: Cơ khí cho ăn đã gắn servo và que chặn	52

Hình 31: Hình ảnh cơ khí trong mô hình thực tế.....	52
Hình 32: Use Case các tính năng của mô hình AuquaNova	53
Hình 33: Logo các phần mềm code vi điều khiển STM32	54
Hình 34: Giao diện tải STM32CubeMx trên web	54
Hình 35. Giao diện STM32CubeMx	55
Hình 36: Giao diện KeilC	56
Hình 37: Cấu hình Clock Configuration	57
Hình 38: Cấu hình GPIO	58
Hình 39: Cấu hình ngắt.....	59
Hình 40: Sơ đồ chân sau khi đã cấu hình trong STM32CubeMx	59
Hình 41: Giao diện trang web chính thức tải phần mềm Arduino IDE.....	60
Hình 42: Cài đặt gói hỗ trợ ESP32 trong Arduino IDE – Board Manager	61
Hình 43: Cài đặt thư viện phụ trợ ESP32 trong Arduino IDE – Library Manager	61
Hình 44: Một số thư viện chính dùng trong dự án	62
Hình 45: Cài đặt thư viện PubSubClient trong Arduino IDE.....	62
Hình 46: Sơ đồ tuần tự cho module cảm biến đo độ đục MJKDZ	66
Hình 47: Sơ đồ tuần tự cho module cảm biến nhiệt độ DS18B20	68
Hình 48: Sơ đồ tuần tự cho module đặt lịch cung cấp thức ăn.....	73
Hình 49: Kết nối Wifi và đồng bộ thời gian.....	95
Hình 50: Hình ảnh gửi dữ liệu thành công từ STM32 đến ESP32 (8 bit).....	100
Hình 51: Quá trình bắt tay ba bước giữa esp32 và hivemq broker	101
Hình 52: Chuỗi gói tin khởi tạo kết nối với các cờ TCP [SYN], [SYN, ACK] và [ACK] được bắt trong Wireshark	101
Hình 53: Gói tin TCP SYN khởi tạo kết nối MQTT bảo mật giữa ESP32 và Broker	102
Hình 54: Các gói tin bắt tay TLS dùng để thiết lập phiên mã hóa sau khi kênh TCP đã được mở (26, 29, 30, 31, 34, 35, 36, 38)	103
Hình 55: Truyền dữ liệu qua giao thức TLS giữa ESP32 và MQTT Broker	104
Hình 56: Tính toán độ trễ dựa trên phân tích gói tin trong Wireshark.....	105
Hình 57: Sơ đồ kết nối Backend, Frontend, Firebase và MQTT	106
Hình 58: Bố cục giao diện của trang web người dùng chính	110
Hình 59: Collection truyền đến firebase.....	132

Hình 60: Luồng kết nối MQTT	134
Hình 61: Mô hình AquaNova khi gắm nguồn.....	Error! Bookmark not defined.
Hình 62: Khởi nguồn hiện đèn báo khi cắm sạc.....	142
Hình 63: Mô hình AquaNova khi bật đèn sáng	Error! Bookmark not defined.
Hình 64: Hình ảnh thực tế mô hình AquaNova vận hành thành công	143
Hình 65: Giao diện web trang Feed Timer (menu bên trái và biểu đồ bên phải).....	144
Hình 66: Giao diện home hiển thị chính của web AquaNova.....	145
Hình 67: Giao diện web trang Turbidity đo độ đục	145

DANH MỤC BẢNG BIỂU

Bảng 1: So sánh đặc điểm giữa giao thức MQTT và CoAP	27
Bảng 2: Cấu trúc và chức năng của các gói tin connect MQTT.....	29
Bảng 3: Cấu trúc và chức năng của các gói tin publish MQTT	30
Bảng 4: Danh sách các topic MQTT và vai trò trong mô hình AquaNova.....	32
Bảng 5: Thông số kỹ thuật của Module relay SRD-5VDC-SL-C.....	34
Bảng 6: Thông số kỹ thuật của module SG90 Servo 180 Độ	34
Bảng 7: Bảng thông số và đặc điểm kỹ thuật cảm biến DS18B20.....	36
Bảng 8: Bảng phân tích và thông số cảm biến MJKDZ Turbidity Sensor.....	37
Bảng 9: Xác định yêu cầu hệ thống và linh kiện sử dụng	40
Bảng 10: Bảng liệt kê chi tiết các requirements	107
Bảng 11: Liên kết backend với các chức năng.....	115
Bảng 12: Bảng chi tiết vai trò các module backend	120
Bảng 13: Luồng hoạt động của Module Control	129
Bảng 14: Danh sách collection lưu trên cloud Firestore	131

DANH MỤC CÁC CHỮ VIẾT TẮT

IOT	Internet of things
MQTT	Message Queuing Telemetry Transport
QoS	Quality of Service
TLS	Transport Layer Security
JSON	JavaScript Object Notation
API	Application Programming Interface
MCU	Microcontroller Unit
DB	Database
GUI	Graphical User Interface
FE	Frontend
BE	Backend

CHƯƠNG 1

GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI

1. Tình hình nghiên cứu

Nuôi cá cảnh tại Việt Nam đã và đang trở thành một hoạt động phổ biến, không chỉ phục vụ nhu cầu giải trí và thư giãn mà còn mang lại giá trị kinh tế đáng kể cho các địa phương và cộng đồng. Với sự phát triển mạnh mẽ trong những năm qua, nuôi cá cảnh không chỉ dừng lại ở việc làm đẹp không gian sống mà còn góp phần vào việc cải thiện đời sống người dân, tạo ra nguồn thu nhập ổn định. TP.HCM, trung tâm lớn nhất về nuôi và xuất khẩu cá cảnh, đã sản xuất khoảng 110 triệu con cá cảnh trong năm 2023, tăng 17,3% so với năm 2022. Trong đó, 12 triệu con đã được xuất khẩu, mang về 12,5 triệu USD, cho thấy sự phát triển mạnh mẽ và tiềm năng xuất khẩu của ngành cá cảnh tại thành phố này [1]. Đây là một minh chứng rõ ràng cho tiềm năng phát triển của nghề nuôi cá cảnh ở nhiều khu vực khác nhau của Việt Nam.

Bên cạnh đó, một nghiên cứu tại TP.HCM cho thấy, 75% người nuôi cá cảnh tự chăm sóc cá của mình, trong khi 13% giao cho người thân chăm sóc và 12% cả gia đình cùng chăm sóc. Điều này cho thấy sự gắn bó, tình yêu và trách nhiệm của người nuôi cá đối với thú chơi này. Về thời gian chăm sóc và ngắm cá, kết quả nghiên cứu cũng chỉ ra rằng 59% người nuôi dành dưới 30 phút/ngày để chăm sóc và theo dõi cá, 28% dành từ 30 đến 60 phút/ngày, và 13% dành hơn 60 phút/ngày cho việc chăm sóc cá cảnh của mình [2]. Như trong nghiên cứu [2] đã chỉ ra kết quả khảo sát cho thấy tỉ lệ người có số năm nuôi từ 1-3 năm chiếm cao nhất khoảng 58,00 % , 4-6 năm chiếm 23,00 % , 7-10 năm chiếm 17,00 % , 11-14 năm và 15-18 năm có cùng tỉ lệ là 1,00 %. Kiểu hồ nuôi cá cảnh phổ biến nhất là bể kiếng đặt trên giá đỡ chiếm tỉ lệ cao nhất 78,64%. Kiểu hồ treo tường chiếm tỉ lệ ít nhất 2,91% trong khi các kiểu hồ khác như hồ non bộ, hồ xi măng cạn, chậu, hũ... chiếm 18,45%. Mặc dù nghề nuôi cá cảnh đang phát triển mạnh, đa số người nuôi vẫn chăm sóc hồ cá theo phương pháp thủ công – từ việc cho ăn, kiểm tra nhiệt độ nước, thay nước định kỳ đến theo dõi chất lượng môi trường. Việc này tốn nhiều thời gian và công sức, đặc biệt là đối với người nuôi có lịch trình bận rộn hoặc thường xuyên đi vắng.

Trong điều kiện đô thị hóa, môi trường khép kín và thay đổi khí hậu, việc duy trì ổn định các yếu tố môi trường nước (nhiệt độ, độ đục, oxy hòa tan, lượng thức ăn) là một

thách thức lớn, ảnh hưởng trực tiếp đến sức khỏe, màu sắc và tuổi thọ của cá cảnh. Chính vì vậy, nhu cầu ứng dụng công nghệ tự động hóa và Internet of Things (IoT) trong việc giám sát và điều khiển hồ cá đang trở thành một xu hướng tất yếu.

2. Đặt vấn đề

Nhu cầu về một hệ thống chăm sóc cá cảnh tự động trong gia đình là rất lớn. Để đáp ứng nhu cầu này, nhóm đã phát triển hệ thống giám sát môi trường sống và cung cấp thức ăn tự động trong hồ cá tại nhà như một giải pháp hiệu quả. Bên cạnh đó, nhiều người nuôi cá hiện nay bận rộn với công việc hoặc thường xuyên vắng nhà, dẫn đến bỏ quên việc cho ăn đúng giờ hoặc kiểm soát môi trường hồ cá không kịp thời. Điều này làm tăng nguy cơ cá bị thiếu dinh dưỡng, stress, hoặc chết hàng loạt do biến động môi trường nước. Vì vậy, nhu cầu về một hệ thống giám sát điều khiển hồ cá thông minh, hoạt động tự động và có khả năng cảnh báo từ xa ngày càng trở nên cần thiết. Sự phát triển của IoT và các giao thức truyền thông như MQTT (Message Queuing Telemetry Transport) đã mở ra hướng đi mới cho việc xây dựng hệ thống giám sát môi trường hồ cá theo thời gian thực. MQTT cho phép các thiết bị nhúng như ESP32 hoặc STM32 truyền dữ liệu cảm biến nhiệt độ, độ đục, mức thức ăn lên máy chủ hoặc đám mây, giúp người dùng có thể theo dõi mọi thông số trên giao diện web mọi lúc mọi nơi.

Tuy nhiên, việc triển khai hệ thống này cũng đặt ra những thách thức, chẳng hạn như độ ổn định của kết nối Wi-Fi, khả năng bảo mật dữ liệu hay việc mở rộng hệ thống cho nhiều hồ cá khác nhau. Những vấn đề này sẽ được phân tích và thảo luận trong báo cáo, đồng thời đề xuất các giải pháp nhằm tối ưu hóa hiệu quả.

3. Mục tiêu của đề tài

Đề tài hướng đến việc thiết kế và triển khai một mô hình giám sát và chăm sóc cá cảnh tự động AquaNova với những mục tiêu cụ thể:

- Mục tiêu 1: Sử dụng vi điều khiển STM32 làm bộ xử lý trung tâm có nhiệm vụ thu thập dữ liệu từ các cảm biến kết hợp với ESP32 truyền gửi nhận dữ liệu.
- Mục tiêu 2: Thiết kế chức năng hẹn giờ và điều khiển cho ăn tự động có đèn sáng/tắt.
- Mục tiêu 3: Thiết kế module cảm biến đo độ đục, nhiệt độ và đo mức thức ăn tích hợp cơ chế cảnh báo khi gần hết thức ăn và ghi nhận lịch sử tiêu thụ.
- Mục tiêu 4: Tích hợp ESP32 cho truyền dữ liệu và giám sát từ xa, đồng bộ hóa thông tin qua cơ sở dữ liệu và nền tảng ứng dụng, hỗ trợ cấu hình và điều khiển qua mạng.

- Mục tiêu 5: Xây dựng website để hiển thị dashboard từ dữ liệu các cảm biến và hỗ trợ người dùng tương tác từ xa,
- Mục tiêu 6: Hiệu chỉnh các thông số đồng thời phân tích số liệu nhiệt độ, độ đục, mức thức ăn nhằm nâng cao hiệu quả quản lý.

4. Nội dung nghiên cứu

Đề tài AquaNoVa – mô hình IoT giám sát và điều khiển hồ cá tập trung thiết kế, xây dựng và thử nghiệm hệ thống giúp theo dõi môi trường nước và cho ăn tự động cho hồ cá cảnh trong gia đình.

- Nghiên cứu lý thuyết:
 - o Tìm hiểu nguyên lý hoạt động của các cảm biến nhiệt độ, độ đục và siêu âm đo lượng thức ăn.
 - o So sánh các giao thức IoT (MQTT, CoAP, HTTP) và chọn MQTT do hỗ trợ truyền dữ liệu ổn định, độ trễ thấp.
 - o Nghiên cứu kiến trúc IoT ba lớp: Thiết bị – Gateway – Cloud, sử dụng Firestore làm cơ sở dữ liệu thời gian thực.
- Thiết kế phần cứng nhúng
 - o Xây dựng mạch cảm biến, mô-đun STM32 điều khiển servo cho ăn và kết nối UART với ESP32 gửi nhận dữ liệu
 - o Thử nghiệm trực tiếp với hồ cá 40 lít, dòng chảy trung bình 0.05–0.1 m/s để đảm bảo đo chính xác và duy trì môi trường ổn định
- Thiết kế phần mềm (Backend & Frontend)
 - o Phát triển máy chủ Flask xử lý API và giao tiếp với MQTT Broker.
 - o Xây dựng các module: telemetry (nhận dữ liệu), control (điều khiển), dashboard (hiển thị).
 - o Giao diện web (HTML, CSS, JS, Chart.js) hiển thị biểu đồ, bảng dữ liệu, Feed Timer và điều khiển Feed Now.
 - o Bộ lập lịch APScheduler kiểm tra thời gian định kỳ và tự động gửi lệnh cho ăn.
- Tích hợp và kiểm thử
 - o Kết nối hoàn chỉnh: STM32 → ESP32 → MQTT Broker → Flask Server → Firestore → Web Dashboard.

- Kiểm thử độ trễ truyền dữ liệu, độ ổn định kết nối và hoạt động của lịch cho cá ăn.
- Ứng dụng và mở rộng
 - Đề xuất tích hợp Machine Learning để dự đoán lượng thức ăn.
 - Mở rộng cho nhiều hồ cá hoặc hệ thống nuôi thủy sản nhỏ.

5. Đối tượng, phạm vi nghiên cứu

Đối tượng nghiên cứu: Các loài cá cảnh nước ngọt nuôi trong hồ tại nhà:

- Cá Molly (Poecilia spp.) là loài ăn tạp thiên về thực vật, thích hợp trong bể cộng đồng. Trong hồ $50 \times 27 \times 30$ cm, nên cho ăn lượng vừa đủ để cá tiêu thụ hết trong 1–2 phút, từ 1–3 lần/ngày. Trung bình ước lượng thức ăn 5g/ngày.
- Cá Neon tetra (Paracheirodon innesi) là loài cá nhỏ bơi đàn, cần khẩu phần rất ít. Mỗi lần chỉ cho ăn lượng vừa đủ trong 1–2 phút, tần suất 1–2 lần/ngày. Thức ăn phù hợp là thức ăn cho cá cảnh và ước lượng trung bình 1g/ngày.
- Cá Vàng (Carassius auratus) có tập tính ăn nhiều, dễ gây bẩn nước nên cần kiểm soát khẩu phần. Cho ăn vừa đủ trong vòng 2 phút, 1 lần/ngày.

Phạm vi nghiên cứu:

- Kiến trúc điều khiển: STM32, ESP32 C3 Mini
- Độ chính xác đo:
 - Nhiệt độ: sai số mục tiêu $\leq \pm 0,5$ °C sau hiệu chuẩn đa điểm.
 - Độ đục: độ lặp lại (SD) $\leq 5\%$ trong dải quan tâm; thiết lập đường cong ADC $\rightarrow$ NTU tối thiểu 5 điểm.
 - Định lượng cho ăn: sai số mục tiêu $\leq \pm 10\%$ so với giá trị gram tham chiếu ăn hết trong ≤ 2 phút.
 - Độ tin cậy vận hành: uptime $\geq 90\%$ /tuần; mất Wi-Fi hệ thống vẫn cho ăn theo lịch cục bộ; độ trễ MQTT trong LAN mục tiêu ≤ 1 s.
- Môi trường thực nghiệm: Hồ cá tại nhà kích thước $50 \times 27 \times 30$ cm và thức ăn khô dạng mảnh cho cá cảnh. Trong đó giới hạn mực nước tối thiểu 30 lít và tối đa 40 lít.
- Giả định về dòng chảy và áp lực nước:
 - Hệ thống hồ cá sử dụng bơm tuần hoàn mini DC 5–12V với lưu lượng danh định khoảng 300L/giờ tương đương 5 L/phút.

- Khi tính theo thể tích hồ tốc độ dòng chảy được giả định vào khoảng 0,05–0,08 m/s trong vùng trung tâm hồ đủ tạo dòng tuần hoàn nhẹ giúp phân bố đều oxy và tránh lắng cặn thức ăn. Áp lực nước tại đầu ra được giả định trong khoảng 3–5 kPa, tương ứng với chiều cao cột nước 0,3–0,5 m.

6. Phương pháp nghiên cứu

Phương pháp thu thập số liệu:

- Sử dụng phương pháp quan sát: Nhóm tiến hành quan sát trực tiếp các yếu tố tác động đến chất lượng nước, môi trường sống của cá và quá trình chăm sóc cá trong các hồ cá thực tế. Qua việc quan sát, nhóm thu thập thông tin về các yếu tố như độ đục nước, nhiệt độ nước, và các thói quen chăm sóc cá của người nuôi.
- Tiến hành phỏng vấn: Nhóm thực hiện các cuộc phỏng vấn với người nuôi cá và các chuyên gia trong ngành để hiểu rõ yêu cầu và kỳ vọng của họ đối với mô hình giám sát và chăm sóc cá tự động. Thông qua phỏng vấn, nhóm thu thập thông tin về các vấn đề mà người nuôi đang gặp phải.

Phương pháp thực nghiệm

- Thiết kế và triển khai hệ thống: Nhóm tiến hành thiết kế và triển khai một mô hình quan sát và cho cá ăn tự động
- Kiểm thử: lịch cho ăn, cảnh báo hết thức ăn, phát hiện kẹt motor/timeout, mất điện/mạng và khởi động lại.
- Đánh giá hiệu quả: Sau khi triển khai mô hình, mô hình tiến hành đánh giá hiệu quả của mô hình dựa trên các tiêu chí như độ chính xác của cảm biến, hiệu quả cảnh báo và khả năng tự động hóa các tác vụ chăm sóc cá.

Phương pháp phân tích và tổng kết

- Phân tích dữ liệu: Nhóm phân tích dữ liệu thu thập từ các cảm biến và mô hình giám sát để đánh giá độ tin cậy và tính chính xác của thông tin về môi trường nước, trạng thái của cá và các yếu tố chăm sóc.
- Tổng kết kinh nghiệm: Dựa trên kết quả phân tích, nhóm tổng kết kinh nghiệm từ quá trình triển khai mô hình. Đánh giá các ưu điểm và hạn chế của mô hình, đồng thời đề xuất các giải pháp và khuyến nghị để cải thiện hiệu quả hoạt động của mô hình giám sát và chăm sóc cá tự động.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1. Mô hình giám sát chất lượng nước và cung cấp thức ăn tự động

2.1.1. Mô hình giám sát chất lượng nước, độ đục và cho cá ăn

Mô hình giám sát chất lượng nước trong bể cá được thiết kế nhằm theo dõi các thông số môi trường và tự động hóa quá trình chăm sóc. Hệ thống sử dụng các cảm biến đo nhiệt độ, độ đục của nước, sau đó truyền dữ liệu về vi điều khiển trung tâm ESP32 C3 Mini. Thông qua kết nối Wi-Fi tích hợp, ESP32 C3 Mini gửi dữ liệu đến máy chủ để lưu trữ và xử lý. Tại đây, thông tin được trực quan hóa dưới dạng biểu đồ và bảng số liệu, đồng thời hệ thống có khả năng phát cảnh báo khi thông số vượt ngưỡng cho phép. Mô hình được đặt tên là AquaNova, trong đó “Aqua” xuất phát từ tiếng Latin nghĩa là nước, còn “Nova” có nghĩa là mới, đổi mới hoặc sáng tạo. Tên gọi AquaNova thể hiện mục tiêu hướng đến giải pháp công nghệ mới trong giám sát và quản lý môi trường nước, biểu trưng cho sự kết hợp giữa tự nhiên và công nghệ IoT hiện đại trong việc chăm sóc và bảo vệ hệ sinh thái hồ cá.



Hình 1: Logo mô hình AquaNova

Ngoài chức năng giám sát, mô hình còn được tích hợp cơ cấu chấp hành điều khiển bộ cho ăn tự động thông qua relay. Người sử dụng có thể cài đặt lịch cho ăn hoặc kích hoạt từ xa thông qua ứng dụng trên thiết bị di động. Điều này giúp duy trì chế độ chăm sóc ổn định, giảm thiểu sự phụ thuộc vào sự có mặt trực tiếp của người nuôi, đồng thời nâng cao hiệu quả quản lý.

Luồng hoạt động tổng thể của mô hình có thể được mô tả như sau: dữ liệu được cảm biến thu thập và truyền về ESP32 C3 Mini → dữ liệu được gửi đến máy chủ để xử lý và hiển thị → người sử dụng theo dõi thông tin, nhận cảnh báo và đưa ra lệnh điều

khởi từ xa → hệ thống kích hoạt bộ cho ăn thông qua relay khi nhận tín hiệu điều khiển. Với cấu trúc này, mô hình đảm bảo tính tự động, khả năng giám sát theo thời gian thực và sự tương tác linh hoạt giữa người sử dụng và hệ thống.

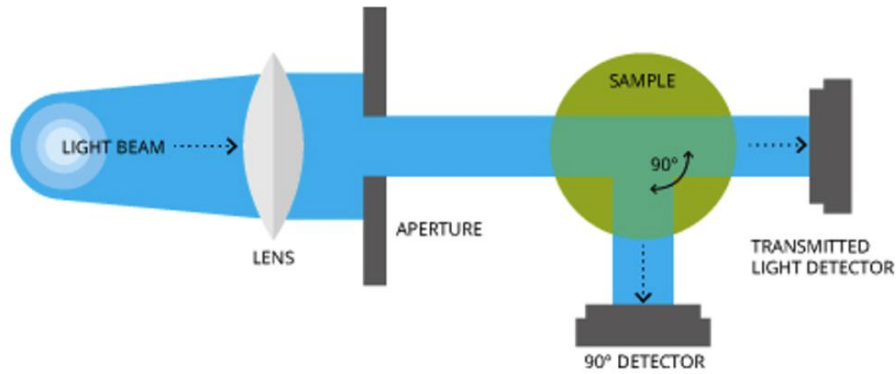
2.1.2. Nguyên lý đo độ đục của nước

Độ đục của nước là một chỉ tiêu quan trọng trong đánh giá chất lượng môi trường nước, thường được biểu thị qua các đơn vị NTU (Nephelometric Turbidity Units), FNU (Formazin Nephelometric Units), FTU (Formazin Turbidity Units) và FAU (Formazin Attenuation Units) [3]. Các đơn vị này được xem là tương đương nhau:

$$1 \text{ NTU} = 1 \text{ FNU} = 1 \text{ FTU} = 1 \text{ FAU}$$

Độ đục bắt nguồn từ sự hiện diện của các hạt lơ lửng trong nước như tảo, bụi bẩn, khoáng chất, dầu, vi khuẩn và nhiều hợp chất khác, khiến nước mất đi tính trong suốt và thay đổi màu sắc. Cần phân biệt độ đục với tổng chất rắn hòa tan (TDS – Total Dissolved Solids). Trong khi độ đục phản ánh khả năng truyền sáng của nước thông qua hiện tượng tán xạ và hấp thụ ánh sáng, thì TDS đo tổng khối lượng các chất hòa tan. Do vậy, độ đục không chỉ là biểu hiện trực quan về sự “trong” hay “đục”, mà còn phản ánh những yếu tố vật lý, hóa học và sinh học liên quan đến chất lượng nước [4]. Nước có độ đục cao thường hấp thụ nhiều nhiệt, dẫn đến tăng nhiệt độ và làm giảm nồng độ oxy hòa tan. Đồng thời, lớp nước đục hạn chế ánh sáng xuyên qua, gây cản trở quang hợp và làm suy giảm khả năng sản sinh oxy của hệ sinh thái thủy sinh [5]. Điều này ảnh hưởng trực tiếp đến sự sống và phát triển của nhiều loài thủy sinh. Các nguyên nhân gây độ đục rất đa dạng, bao gồm xói mòn đất, dòng chảy đô thị, hoạt động nông nghiệp, xả thải công nghiệp và các quá trình tự nhiên. Theo Hình 2 mô tả khi hiển thị LED phát sáng thì mẫu nước được detector góc 90° thu ánh sáng tán xạ [8].

Hiện nay, phương pháp đo độ đục phổ biến là Nephelometry dựa trên nguyên lý tán xạ ánh sáng. Theo tiêu chuẩn ISO 7027, độ đục được xác định bằng cách chiếu chùm sáng đơn sắc (thường $\sim 860 \text{ nm}$) vào mẫu nước và đo cường độ ánh sáng tán xạ ở góc 90° so với nguồn phát. Cách bố trí này giúp giảm ảnh hưởng của màu sắc nước và tăng độ chính xác trong phép đo [6], [7].



Hình 2: Sơ đồ minh họa nguyên lý hoạt động của cảm biến độ đục theo tán xạ ánh sáng.

2.1.2. Hiện tượng phản xạ sóng âm trong cảm biến siêu âm

Cảm biến siêu âm là một trong những loại cảm biến được sử dụng phổ biến trong các hệ thống đo khoảng cách, phát hiện vật cản và điều khiển tự động. Nguyên lý hoạt động của cảm biến này dựa trên hiện tượng phản xạ sóng âm, tức là khi sóng âm truyền trong không khí gặp bề mặt ngăn cách giữa hai môi trường có mật độ khác nhau, một phần năng lượng của sóng sẽ bị phản xạ trở lại môi trường ban đầu.

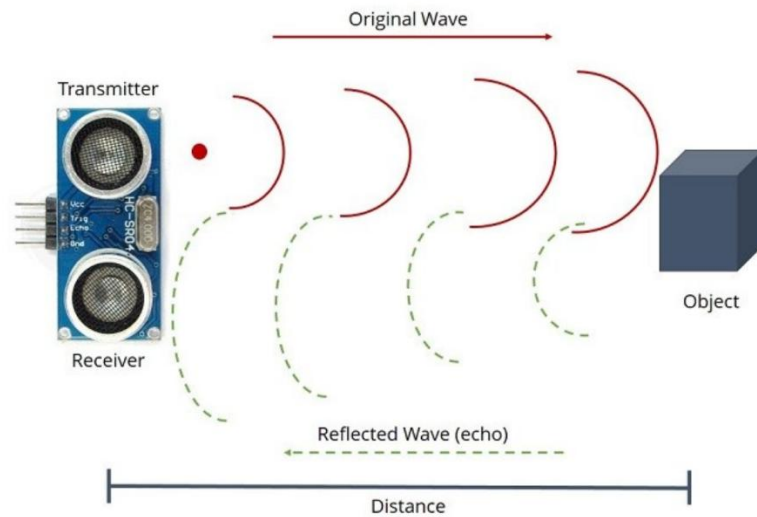
Bộ phát (Trigger) của cảm biến tạo ra chuỗi xung ngắn có độ rộng khoảng 10 μ s, phát ra chùm sóng siêu âm tần số 40 kHz lan truyền trong không khí với vận tốc trung bình 343 m/s ở 25°C. Khi sóng âm gặp vật cản, năng lượng của sóng phản xạ trở lại và được bộ thu (Echo) ghi nhận. Thời gian từ lúc phát sóng đến khi nhận lại sóng phản xạ được gọi là thời gian bay (Time of Flight – ToF), từ đó ta có thể xác định khoảng cách đến vật cản thông qua công thức:

$$D = \frac{V \times t}{2} \quad (1)$$

Trong đó: D là khoảng cách đến vật cản (m),

v là vận tốc truyền âm trong không khí (m/s),

và t là thời gian sóng đi và về (s).



Hình 3: Sơ đồ minh họa nguyên lý hoạt động của cảm biến siêu âm.

Trong thực tế, cảm biến HC-SR04, khi vi điều khiển kích hoạt chân Trig bằng xung 10 μ s, cảm biến sẽ phát ra 8 xung siêu âm ở tần số 40 kHz, sau đó đợi sóng phản xạ quay về. Khi tín hiệu phản xạ được ghi nhận, cảm biến xuất ra xung tại chân Echo, trong đó độ rộng xung tỷ lệ thuận với thời gian bay của sóng âm. Vi điều khiển sẽ đo độ rộng của xung này để tính toán khoảng cách dựa theo công thức được cung cấp trong datasheet[8]:

$$D = \frac{343 \times T_{\text{echo}}}{2}$$

Trong đó: T_{echo} là thời gian tín hiệu Echo ở mức cao (tính bằng giây), vận tốc âm thanh là 343 m/s.

Hiện tượng phản xạ sóng âm trong cảm biến chịu ảnh hưởng của nhiều yếu tố vật lý và môi trường như góc phản xạ, vật liệu bề mặt, nhiệt độ môi trường và nhiễu âm thanh. Góc phản xạ không vuông góc với hướng truyền của sóng sẽ khiến tín hiệu thu yếu hoặc mất ổn định. Vật liệu hấp thụ âm như cao su, vải làm giảm biên độ sóng phản xạ, trong khi vật liệu rắn như kim loại cho tín hiệu phản xạ mạnh hơn. Vận tốc truyền âm phụ thuộc vào nhiệt độ, được xác định theo công thức gần đúng:

Hiện tượng phản xạ sóng âm được ứng dụng rộng rãi trong các lĩnh vực như robot tự hành, nông nghiệp thông minh, đo mức nước, giám sát an ninh và tự động hóa công nghiệp. Một ví dụ điển hình là mô-đun đo lượng thức ăn tự động, trong đó cảm biến

HC-SR04 được gắn trên nắp hộp chứa; khi thức ăn giảm, khoảng cách phản xạ tăng, hệ thống sẽ điều khiển động cơ nạp thêm thức ăn dựa trên dữ liệu phản xạ sóng âm.

$$V = 331 + 0.6T$$

Hiện tượng phản xạ sóng âm được ứng dụng rộng rãi trong các lĩnh vực như robot tự hành, nông nghiệp thông minh, đo mức nước, giám sát an ninh và tự động hóa công nghiệp. Một ví dụ điển hình là mô-đun đo lượng thức ăn tự động, trong đó cảm biến HC-SR04 được gắn trên nắp hộp chứa; khi thức ăn giảm, khoảng cách phản xạ tăng, hệ thống sẽ điều khiển động cơ nạp thêm thức ăn dựa trên dữ liệu phản xạ sóng âm.

2.1.3. Hình thức cung cấp thức ăn

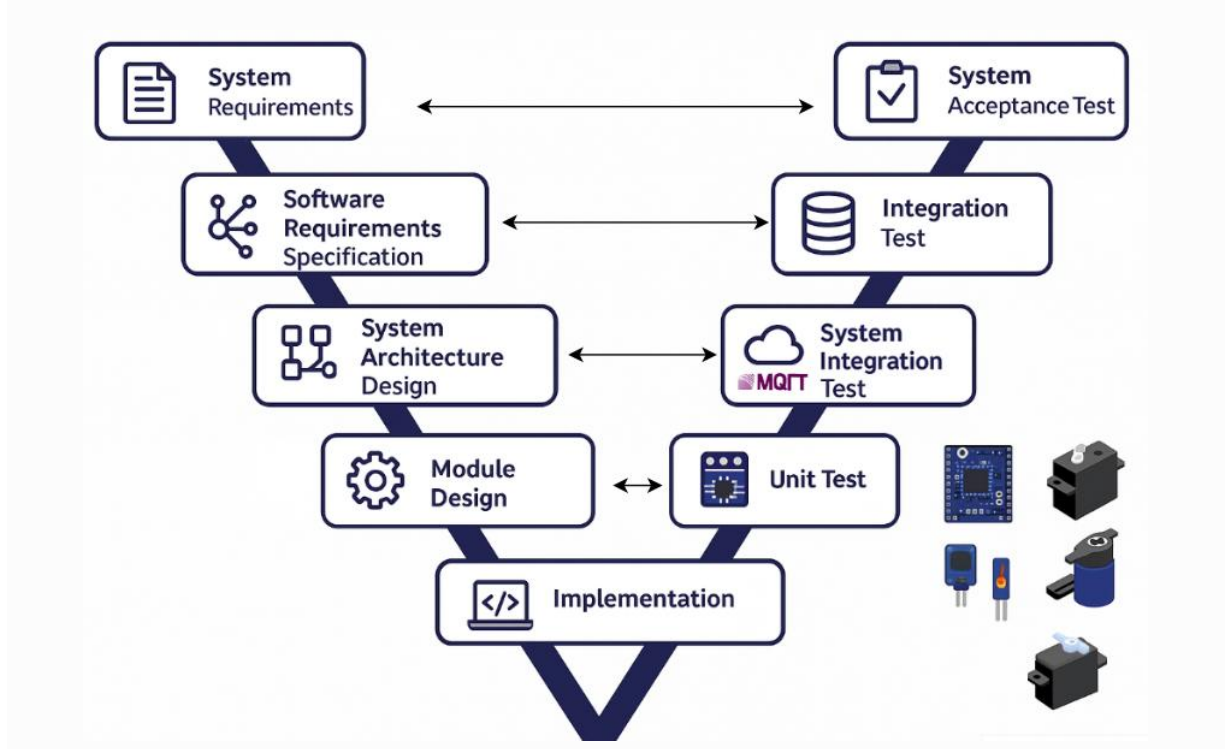
Mô hình cơ khí được xây dựng dựa trên nguyên tắc tối giản hóa, tận dụng triệt để trọng lực để cấp liệu và sử dụng một cơ cấu chấp hành trực tiếp để kiểm soát dòng chảy. Thiết kế này tập trung vào độ tin cậy, chi phí thấp và sự đơn giản trong chế tạo cũng như vận hành, loại bỏ các chi tiết truyền động phức tạp như trục xoắn hay băng tải. Cấu trúc của thiết bị bao gồm một chai nhựa thông thường, được đảo ngược và cố định chắc chắn trên một giá đỡ, đóng vai trò như một phễu chứa (hopper) tiện lợi. Thiết kế này cho phép thức ăn dạng hạt tự động dồn xuống miệng chai dưới tác dụng của trọng lực, luôn ở trạng thái sẵn sàng để được phân phối. Điểm cốt lõi của cơ cấu chấp hành nằm ở một động cơ servo được định vị chính xác ngay tại miệng ra của chai. Cánh tay của servo (servo horn), hoặc một tấm gạt nhỏ được gắn vào đó, hoạt động như một van chặn trực tiếp, đóng vai trò như một cánh cửa đóng-mở linh hoạt.

Nguyên lý hoạt động của thiết bị diễn ra một cách tuần tự và rõ ràng. Ở trạng thái chờ, servo giữ cánh tay gạt ở vị trí đóng kín miệng chai, ngăn không cho thức ăn rơi ra ngoài. Khi nhận được tín hiệu điều khiển từ vi điều khiển theo lịch trình đã được cài đặt, servo sẽ lập tức quay, di chuyển tấm chặn ra khỏi miệng chai. Hành động này tạo ra một khoảng hở, cho phép thức ăn rơi tự do xuống bể cá. Lượng thức ăn được định lượng gián tiếp thông qua khoảng thời gian mà van được giữ ở trạng thái mở; thời gian mở càng lâu, lượng thức ăn rơi xuống càng nhiều. Sau khi hết thời gian quy định, servo sẽ quay trở về vị trí ban đầu, đóng kín cửa ra và kết thúc một chu trình cấp liệu. Đây là một giải pháp cơ khí thông minh, tận dụng các linh kiện phổ biến để tạo ra một hệ thống tự động hóa hiệu quả. Mặc dù độ chính xác về định lượng không cao bằng các cơ cấu phức

tập hơn như trục xoắn, thiết kế này hoàn toàn đáp ứng được yêu cầu của các ứng dụng phổ thông, đặc biệt là khi ưu tiên sự đơn giản, độ bền và tối ưu hóa chi phí.

2.1.4. Mô hình chữ V

Mô hình chữ V (V model), hay mô hình xác minh (Verification) và xác nhận (Validation), là một mô hình vòng đời phát triển phần mềm (Software Development Life Cycle). Mô hình ưu tiên kiểm thử (testing), một cách nghiêm ngặt trong suốt quá trình phát triển phần mềm và được biểu diễn trực quan như hình chữ V[10]. Hình 7 minh họa mô hình V với nhánh trái mô tả các giai đoạn thiết kế và đặc tả hệ thống (Verification) và Nhánh phải: mô tả các giai đoạn kiểm thử và đánh giá (Validation).



Hình 4: Mô hình chữ V

1. Xác định yêu cầu hệ thống (System Requirements)

Mục tiêu: Xác định nhu cầu tổng thể của người dùng và chức năng chính.

Đầu ra:

- Hệ thống tự động cho cá ăn theo lịch trình, điều khiển đèn
- Giám sát nhiệt độ, độ đục nước và lượng thức ăn theo thời gian thực.
- Điều khiển qua web (Flask backend + Firestore + MQTT).
- Cảnh báo khi vượt ngưỡng (qua MQTT và dashboard).

2. Kiểm thử tương ứng (System Acceptance Test)

Kiểm thử thực tế toàn hệ thống: lịch hoạt động đúng, sensor hoạt động, MQTT phản hồi

3. Thiết kế yêu cầu phần mềm (Software Requirement Specification)

Mục tiêu: Phân rã chức năng thành module logic.

Đầu ra:

- Biểu đồ chức năng, use case, API endpoints.
- Module đo nhiệt độ: DS18B20.
- Module đo độ đục: cảm biến turbidity.
- Module điều khiển servo feeder.
- Flask backend: /feed, /schedule, /sensor.
- Firestore: collection “Schedule”, “Telemetry”.
- MQTT topics:
 - aquanova/telemetry
 - aquanova/control/feeder
 - aquanova/control /light

4. Kiểm thử tương ứng (Integration Test)

Kiểm tra giao tiếp giữa Flask ↔ MQTT ↔ STM32 hoạt động đúng.

5. Thiết kế kiến trúc hệ thống (System Architecture Design)

- Mục tiêu: Xác định cấu trúc tổng thể phần cứng & phần mềm.
- output: Sơ đồ khối hệ thống.
- Hardware: STM32 + DS18B20 + Turbidity Sensor + Servo + WiFi module (ESP32).
- Communication: MQTT (HiveMQ Cloud).
- Software: Flask server + Firestore cloud + Web Dashboard.

6. Kiểm thử tương ứng (System Integration Test)

Kiểm tra kết nối MQTT, Flask REST API, database cập nhật realtime.

7. Thiết kế chi tiết module (Module Design)

- Mục tiêu: Thiết kế từng module riêng biệt.
- Module Sensor: đọc giá trị ADC, tính toán độ đục.
- Module Feeder: điều khiển servo 0–180° với timer.
- Module Network: publish/sub MQTT.
- Flask Blueprint: telemetry.py, control.py, admin.py.

8. Kiểm thử tương ứng (Unit Test)

Test từng module độc lập: đọc cảm biến, servo quay đúng góc, MQTT gửi đúng payload.

9 Cài đặt (Implementation / Coding)

- Mục tiêu: Lập trình và tích hợp code theo thiết kế.
- MCU: STM32CubeIDE (C code).
- Server: Flask Python, MQTT client, Firestore (Firebase)
- Frontend: HTML/JS giao diện điều khiển.

9. Kiểm thử tương ứng (Code Verification / Unit Testing)

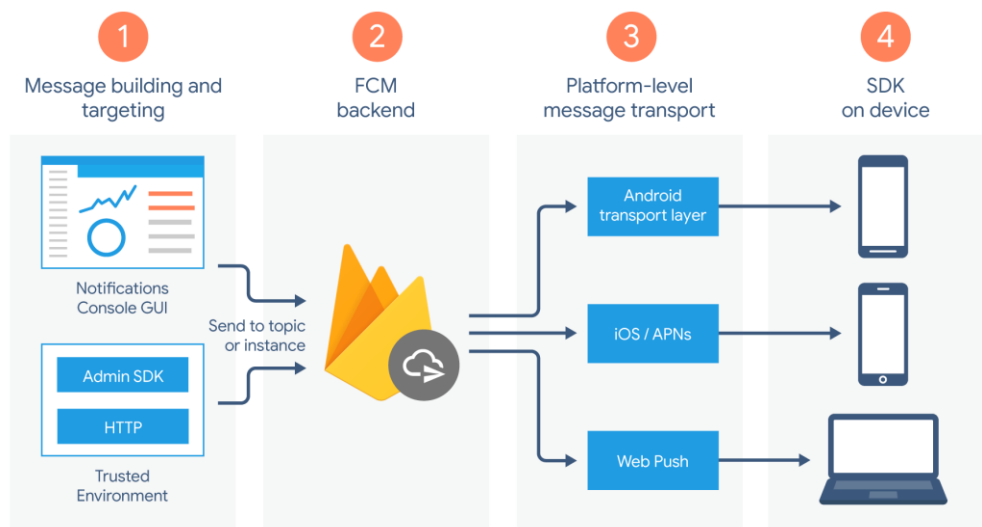
Chạy test, debug từng phần bằng console log hoặc serial monitor.

2.2. Cơ sở dữ liệu

2.2.1. Firsebase

Firebase[10] là một nền tảng phát triển ứng dụng được Google cung cấp, cho phép lập trình viên xây dựng, triển khai và quản lý các ứng dụng web hoặc di động một cách nhanh chóng. Firebase tích hợp nhiều dịch vụ như Realtime Database, Cloud Firestore, Authentication, Cloud Storage và Hosting, giúp đồng bộ dữ liệu thời gian thực giữa client và server, lưu trữ dữ liệu trên nền tảng đám mây, và mở rộng quy mô.

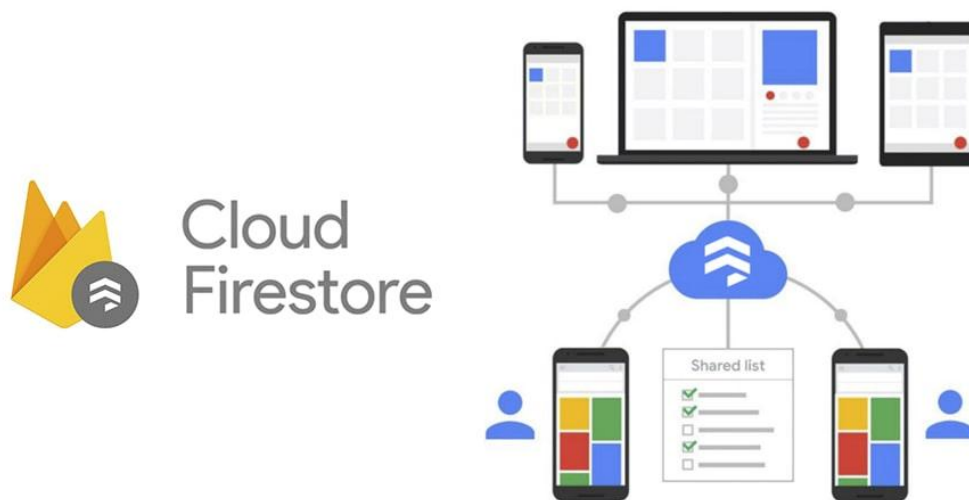
Trong hệ thống AquaNova, Firebase được sử dụng chủ yếu với Cloud Firestore để lưu trữ dữ liệu cảm biến, lịch cho ăn và nhật ký điều khiển, giúp đảm bảo khả năng đồng bộ tức thời, bảo mật cao và quản lý dữ liệu tập trung.



Hình 5: Sơ đồ quy trình hoạt động của Firebase Cloud Messaging

2.2.2. Cloud Firestore

Cloud Firestore là cơ sở dữ liệu NoSQL linh hoạt và có khả năng mở rộng, được thiết kế để lưu trữ và đồng bộ dữ liệu giữa các ứng dụng khách và máy chủ theo thời gian thực. Firestore tổ chức dữ liệu dưới dạng collection và document, cho phép truy vấn mạnh mẽ, cập nhật tức thời và hỗ trợ hoạt động ngoại tuyến. Ngoài ra, Firestore còn tích hợp cơ chế bảo mật thông qua Firebase Security Rules và khả năng mở rộng toàn cầu trên hạ tầng Google Cloud [12]. Một ưu điểm quan trọng của Firestore là khả năng đồng bộ thời gian thực, cho phép dữ liệu được cập nhật ngay lập tức trên tất cả các thiết bị khi có thay đổi. Hệ thống có khả năng mở rộng mạnh mẽ, hỗ trợ xử lý khối lượng lớn dữ liệu và người dùng đồng thời. Bên cạnh đó, Firestore cung cấp cơ chế truy vấn đa dạng như lọc, sắp xếp và phân trang dữ liệu, đáp ứng tốt nhu cầu khai thác dữ liệu trong các ứng dụng IoT. Về bảo mật, Firestore hỗ trợ Firebase Security Rules ở mức tài liệu và tập hợp, cho phép kiểm soát truy cập chi tiết và an toàn. Ngoài ra, Firestore còn có độ tin cậy cao nhờ multi-region replication, giúp đảm bảo tính sẵn sàng và hạn chế nguy cơ mất mát dữ liệu.

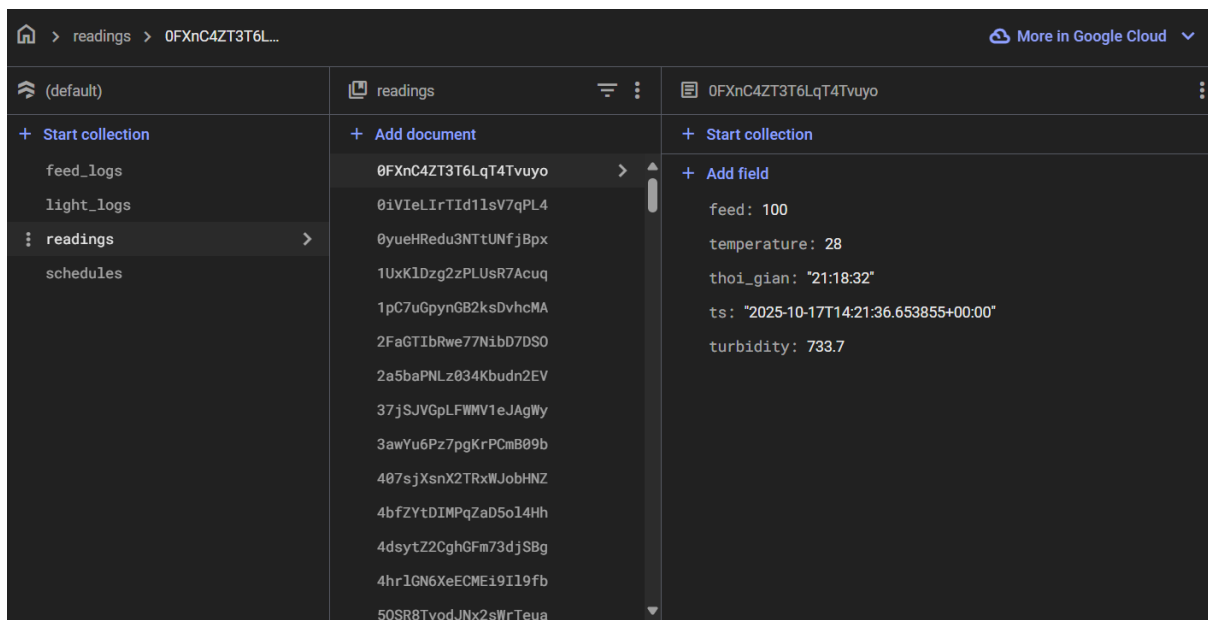


Hình 6: Cơ chế đồng bộ dữ liệu thời gian thực của Cloud Firestore

Để minh họa rõ hơn về cơ chế hoạt động và cách tổ chức dữ liệu trong hệ thống, Hình 4 và Hình 5 trình bày mô hình đồng bộ dữ liệu thời gian thực của Cloud Firestore và giao diện lưu trữ dữ liệu cảm biến trong ứng dụng AquaNova. Cơ chế này cho phép dữ liệu được cập nhật và phản ánh tức thời giữa các thiết bị và máy chủ thông qua nền tảng đám mây. Khi các cảm biến gửi dữ liệu mới, thông tin sẽ

tự động được đồng bộ và hiển thị trên dashboard mà không cần thao tác tải lại trang, đảm bảo tính liên tục, chính xác và ổn định trong quá trình giám sát môi trường hồ cá.

Hình 7 dưới đây minh họa cấu trúc cơ sở dữ liệu Cloud Firestore được sử dụng trong mô hình AquaNova. Dữ liệu được tổ chức theo dạng collection và document, trong đó mỗi collection đại diện cho một nhóm dữ liệu, còn mỗi document lưu trữ các trường thông tin (fields) với collection “readings” chứa các document tương ứng với từng lần cập nhật dữ liệu từ thiết bị.



Hình 7: Giao diện lưu trữ dữ liệu cảm biến trên Cloud Firestore.

Mỗi document bao gồm các trường:

- device_id: mã định danh thiết bị gửi dữ liệu (ví dụ “sensor-3”);
- temperature: giá trị nhiệt độ nước (°C) đo được;
- turbidity: giá trị độ đục (NTU);
- feed: lượng thức ăn được cung cấp (g);
- ts: dấu thời gian (timestamp) ghi nhận dữ liệu.

Mô hình lưu trữ này cho phép các bản ghi dữ liệu cảm biến được đồng bộ hóa và truy xuất theo thời gian thực, hỗ trợ tốt cho việc hiển thị và phân tích trên Flask Dashboard hoặc ứng dụng người dùng.

2.3. UI (User interface) và UX (User experience)

Giao diện người dùng (UI) và trải nghiệm người dùng (UX) đóng vai trò quan trọng trong việc tạo nên tính trực quan và dễ sử dụng của hệ thống. Ba công nghệ nền tảng HTML, CSS và JavaScript được sử dụng để xây dựng và hoàn thiện giao diện web.

- HTML chịu trách nhiệm tạo cấu trúc và bố cục nội dung.
- CSS định dạng và thiết kế giao diện trực quan, màu sắc, phông chữ.
- JavaScript giúp trang web trở nên tương tác, xử lý dữ liệu động và cập nhật thời gian thực từ Flask Server.

Nhờ sự kết hợp này, mô hình AquaNova mang lại trải nghiệm sử dụng thân thiện, dễ thao tác và đồng bộ dữ liệu cảm biến một cách trực quan trên dashboard.



Hình 8: UI/UX (HTML, CSS, JavaScript)

2.3.1. HTML (HyperText Markup Language)

HTML (HyperText Markup Language) là một ngôn ngữ đánh dấu được sử dụng như một công cụ giúp xây dựng cấu trúc và định dạng nội dung trên trang web. HTML là ngôn ngữ căn bản và chủ yếu sử dụng các thẻ (tag) để xác định các phần tử và sắp xếp chúng thành các thành phần như tiêu đề, đoạn văn bản, hình ảnh, liên kết và bảng.

Cấu trúc cơ bản của HTML: Một tài liệu HTML sẽ luôn bắt đầu bằng thẻ <html> và kết thúc cũng bằng thẻ </html>. Trong tài liệu HTML, chúng ta sẽ thấy các phần tử như <head>, <body>, <title>, <h1>, <p>, , <a>, <table>, <form> và nhiều phần tử khác. HTML cung cấp các thẻ để định dạng nội dung như chữ in đậm (), chữ in nghiêng (), danh sách (, ,), và định dạng văn bản (<h1>, <p>,) để làm nổi bật nội dung và cung cấp cấu trúc cho trang web. Ngoài ra, HTML

còn cho phép chúng ta tạo liên kết đến các nơi khác bằng cách sử dụng thẻ <a> và tạo hình ảnh bằng cách sử dụng thẻ . Chúng ta có thể định rõ đường dẫn, mô tả và thuộc tính khác cho các liên kết và hình ảnh này. Cuối cùng, nhằm mục đích trực quan hóa dữ liệu một cách tốt hơn, HTML hỗ trợ việc tạo bảng dữ liệu nhờ các thẻ <table>, <tr>, <td>, và các thẻ liên quan. Chúng ta cũng có thể tạo biểu đồ đơn giản bằng cách sử dụng các phần tử HTML và CSS.

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Tiêu đề trang</title>
</head>
<body>
  <h1>Đây là tiêu đề chính</h1>
  <p>Đây là một đoạn văn bản trong HTML.</p>
  
  <a href="https://www.example.com">Đây là một liên kết</a>
</body>
</html>
```

2.3.2. CSS (Cascading Style Sheets)

CSS (Cascading Style Sheets) là ngôn ngữ định kiểu được sử dụng để mô tả cách trình bày và hiển thị của các phần tử trong tài liệu HTML. Nếu HTML quy định cấu trúc nội dung của một trang web thì CSS đóng vai trò như lớp “giao diện” phủ lên, giúp điều chỉnh màu sắc, kích thước, khoảng cách, phông chữ, hiệu ứng và bố cục, từ đó mang lại trải nghiệm trực quan và thẩm mỹ hơn cho người dùng.

Lựa chọn phần tử: CSS sử dụng các bộ chọn (selectors) để xác định phần tử mà chúng ta muốn kiểm soát. Các bộ chọn có thể là tên thẻ HTML (h1, p), lớp (class), ID (id), thuộc tính ([attribute]), và nhiều lựa chọn khác.

Thuộc tính và giá trị: CSS cho phép chúng ta thiết lập các thuộc tính như color, font size, margin, padding, background-color, border, text-align, và nhiều thuộc tính khác. Chúng ta có thể thiết lập giá trị của thuộc tính để định dạng và kiểm soát hiển thị của các phần tử HTML.

Lớp và ID: CSS hỗ trợ việc định nghĩa lớp (class) và ID (id) cho các phần tử HTML, giúp chúng ta áp dụng kiểu dáng chỉ định cho từng phần tử cụ thể hoặc nhóm phần tử. Các phương pháp kiểu dáng: CSS hỗ trợ các phương pháp kiểu dáng bao gồm

Inline, Internal và External. Inline CSS áp dụng kiểu dáng trực tiếp vào các phần tử HTML, Internal CSS được định nghĩa trong thẻ <style> trong tài liệu HTML, và External CSS được định nghĩa trong một tệp riêng biệt và được liên kết tài liệu HTML.

```
<p style="color: red;">Văn bản màu đỏ</p>
<style>
  h1 {
    color: green
    font-size: 24px;
  }
  p { color: blue; }
</style>
<link rel="stylesheet" href="style.css">
```

2.3.3. JavaScript (JS)

JavaScript (JS) là ngôn ngữ lập trình được sử dụng rộng rãi trong phát triển web, cho phép các trang web trở nên tương tác và động. JS có thể thay đổi nội dung HTML, xử lý sự kiện người dùng, và giao tiếp với server để cập nhật dữ liệu mà không cần tải lại trang. Trong hệ thống hồ cá thông minh, JavaScript được dùng để hiển thị dữ liệu cảm biến thời gian thực và gửi lệnh điều khiển đến server thông qua các yêu cầu HTTP. Thông qua các phương thức HTTP như GET và POST, JavaScript có thể gửi và nhận dữ liệu với server Flask mà không cần tải lại trang, giúp việc điều khiển và giám sát hệ thống trở nên mượt mà hơn. Trong ứng dụng mô hình giám sát hồ cá, JS sử dụng Fetch API để trao đổi dữ liệu:

- Phương thức GET: dùng để truy vấn và nhận dữ liệu từ server (ví dụ: lấy danh sách lịch cho ăn hoặc dữ liệu cảm biến).
- Phương thức POST: dùng để gửi dữ liệu mới hoặc yêu cầu điều khiển (ví dụ: thêm lịch, cho ăn ngay).

Ví dụ mã JavaScript cơ bản gửi yêu cầu đến server bằng phương thức POST: Đoạn mã trên minh họa cách JS sử dụng Fetch API để gửi dữ liệu dạng JSON đến server và nhận phản hồi trả về, giúp tạo nên cầu nối giữa giao diện người dùng và hệ thống backend.

```
// Gửi dữ liệu đến server Flask qua HTTP POST
fetch("/control/feed-now", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    device_id: "esp32-feeder-1",
    amount: 30
  })
})
```

```
    })  
  })  
  .then(response => response.json())  
  .then(data => console.log("Server response:", data))  
  .catch(error => console.error("Error:", error));
```

2.3.4. HTTP (Hypertext Transfer Protocol)

HTTP (Hypertext Transfer Protocol) là giao thức truyền tải siêu văn bản được sử dụng phổ biến trong các ứng dụng web. Đây là nền tảng cho việc trao đổi dữ liệu giữa client (trình duyệt web, ứng dụng người dùng) và server (máy chủ web). Giao thức này hoạt động theo mô hình yêu cầu – phản hồi (request – response), trong đó client gửi yêu cầu (HTTP Request) đến server, và server trả về phản hồi (HTTP Response) chứa dữ liệu hoặc thông tin trạng thái.

Trong mô hình AquaNova, giao thức HTTP được sử dụng để kết nối giữa giao diện web (HTML/CSS/JS) và máy chủ Flask. Khi người dùng thao tác trên web, các yêu cầu như hiển thị dữ liệu, bật/tắt đèn, hoặc kích hoạt cho ăn sẽ được gửi tới Flask thông qua các API endpoint dưới dạng HTTP POST hoặc GET. Flask xử lý yêu cầu, đồng bộ dữ liệu với Firebase Cloud Firestore, và phản hồi lại cho trình duyệt để cập nhật trạng thái thời gian thực.

HTTP giúp hệ thống AquaNova hoạt động ổn định, dễ dàng tích hợp với các dịch vụ cloud (như Firebase), đồng thời mang lại khả năng mở rộng và tương thích cao với nhiều thiết bị người dùng (máy tính, điện thoại, tablet).

2.3.5. IP Addresss

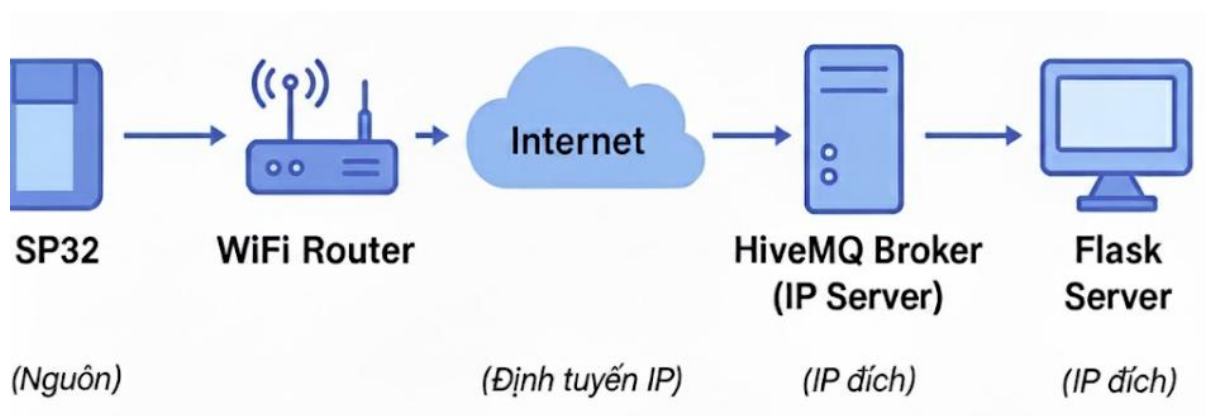
Địa chỉ IP (Internet Protocol Address) là định danh duy nhất được gán cho mỗi thiết bị trong mạng Internet hoặc mạng nội bộ, giúp các thiết bị có thể định tuyến và giao tiếp với nhau. Mỗi gói tin dữ liệu khi được gửi qua mạng sẽ mang địa chỉ IP nguồn và IP đích, giúp hệ thống xác định nơi gửi và nơi nhận.

Có hai phiên bản chính của địa chỉ IP:

- IPv4 (Internet Protocol version 4): Sử dụng cấu trúc 32 bit, thường được biểu diễn dưới dạng bốn số thập phân, ví dụ: 192.168.1.100.
- IPv6 (Internet Protocol version 6): Sử dụng cấu trúc 128 bit, được thiết kế để khắc phục giới hạn về số lượng địa chỉ của IPv4, ví dụ: 2001:0db8:85a3::8a2e:0370:7334.

Trong mô hình AquaNova, cả ESP32, Flask Server, và HiveMQ Cloud Broker đều giao tiếp với nhau thông qua địa chỉ IP và cổng TCP/8883 (MQTT bảo mật).

- Chức năng chính của IP:
 - Định danh thiết bị bằng địa chỉ IP (IPv4/IPv6).
 - Phân mảnh, đóng gói và tái hợp dữ liệu khi truyền qua các mạng khác.
 - Định tuyến (routing) gói tin thông qua nhiều nút mạng trung gian.
 - Không đảm bảo độ tin cậy — chỉ đảm nhiệm việc “best-effort delivery”, tức là cố gắng chuyển đến nơi nhận mà không xác nhận.
- Ứng dụng trong mô hình AquaNova:
 - ESP32 kết nối WiFi và nhận IP cục bộ (LAN).
 - Dữ liệu cảm biến gửi qua Internet bằng địa chỉ IP của HiveMQ Broker.
 - Flask Server lắng nghe và xử lý dữ liệu thông qua IP Public của broker.
 - Gói tin đi từ ESP32 đi qua Internet, định tuyến qua IP và tới broker HiveMQ trước khi đến Flask Server.



Hình 9: Sơ đồ truyền gói tin qua giao thức IP trong mô hình AquaNova

2.3.6. Giao thức tầng giao vận TCP (Transmission Control Protocol)

TCP là giao thức hoạt động ở tầng giao vận (Transport Layer), được xây dựng trên nền IP nhằm đảm bảo độ tin cậy, toàn vẹn và đúng thứ tự của dữ liệu trong quá trình truyền thông. Khác với UDP, TCP thiết lập kết nối ba bước (three-way handshake) trước khi truyền dữ liệu, nhờ đó hạn chế mất gói và sai lệch.

Đặc điểm chính của TCP:

- Kết nối hướng dòng (connection-oriented):
- Thiết lập kết nối logic giữa hai đầu (client – server) trước khi gửi dữ liệu.
- Cơ chế kiểm soát lỗi:

- Mỗi gói tin có số thứ tự (sequence number) và mã kiểm tra (checksum).
- Cơ chế truyền lại (retransmission):
- Nếu bên nhận không gửi ACK (acknowledge) trong thời gian quy định, gói tin sẽ được gửi lại.
- Kiểm soát luồng và tắc nghẽn (flow & congestion control):
- Giúp tối ưu tốc độ truyền và tránh quá tải mạng.

Vai trò trong hệ thống AquaNova:

- Trong mô hình giám sát và điều khiển qua MQTT:
- MQTT Client (ESP32) kết nối đến MQTT Broker (HiveMQ) qua TCP/TLS.
- Flask Server cũng sử dụng TCP để subscribe hoặc publish topic.
- TCP đảm bảo:
 - Dữ liệu cảm biến (nhiệt độ, độ ẩm, feed) không bị mất.
 - Các lệnh điều khiển (feed-now, light-on) được gửi đi an toàn và theo thứ tự.

Giao thức	Tầng OSI	Vai trò trong hệ thống AquaNova	Ghi chú
IP	Tầng Mạng	Định tuyến gói tin từ ESP32 đến Broker và Flask	Không đảm bảo độ tin cậy
TCP	Tầng Giao vận	Đảm bảo truyền dữ liệu an toàn, có xác nhận, không mất gói	Dùng cho MQTT qua TLS

2.3.7. Flask (micro web framework for Python)

Flask là một framework Python nhẹ và linh hoạt, cung cấp các cấu hình và quy ước mặc định giúp người phát triển có thể triển khai nhanh chóng các ứng dụng web. Tài liệu chính thức của Flask mô tả chi tiết các thành phần cấu thành framework và cách chúng có thể được sử dụng, tùy chỉnh để phù hợp với từng mục đích cụ thể.

Ngoài các module có sẵn, Flask còn hỗ trợ nhiều extension được cộng đồng bảo trì, cho phép tích hợp các tính năng như xác thực người dùng, kết nối cơ sở dữ liệu, RESTful API, hoặc xử lý giao thức MQTT:

Nhờ đó, Flask rất phù hợp để phát triển các hệ thống IoT thời gian thực, chẳng hạn như mô hình AquaNova, nơi Flask đảm nhiệm vai trò backend điều phối dữ liệu, nhận và phát lệnh MQTT, cũng như quản lý giao diện Dashboard.

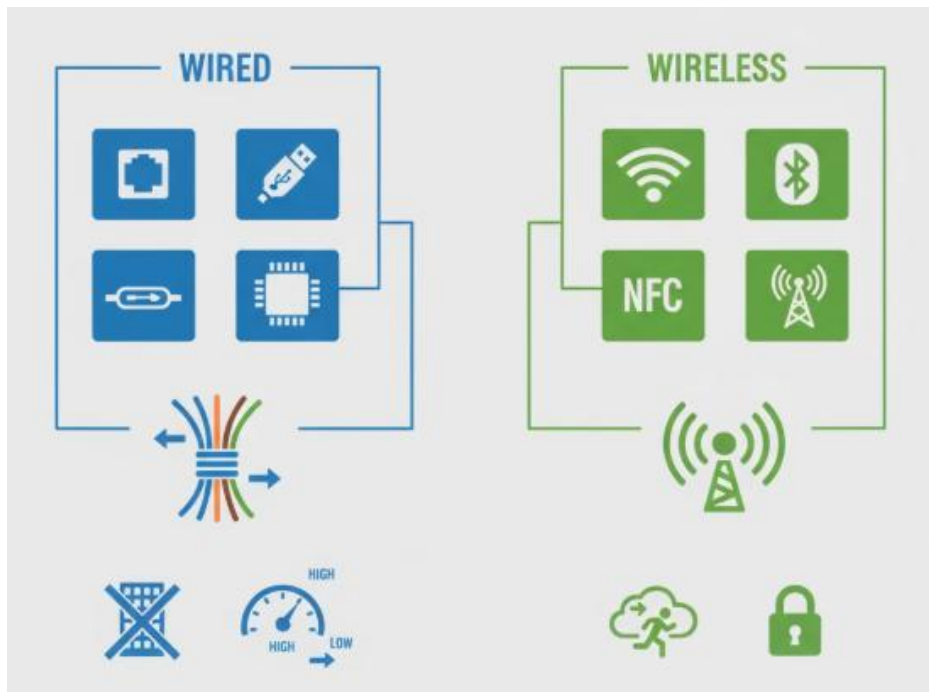
Một số đặc điểm chính:

- Nhẹ và đơn giản: Flask được thiết kế với kiến trúc tối giản, chỉ cung cấp những thành phần cốt lõi cần thiết (routing, template engine, HTTP handling).
- Mở rộng dễ dàng: có thể tích hợp thêm nhiều thư viện/phần mở rộng như quản lý cơ sở dữ liệu (SQLAlchemy), xác thực người dùng, gửi mail.
- Routing linh hoạt: dễ dàng định nghĩa các đường dẫn (URL) và ánh xạ tới các hàm xử lý trong ứng dụng.
- Hỗ trợ Jinja2: cho phép tạo giao diện HTML động kết hợp dữ liệu từ server.
- Thân thiện với REST API: thường được dùng để triển khai backend cho ứng dụng IoT, mobile, hoặc web vì có thể trả về dữ liệu JSON nhanh gọn.
- Ứng dụng trong hệ thống IoT: Flask đóng vai trò là máy chủ web (Web Server):
- Nhận dữ liệu từ MQTT và lưu trữ vào cơ sở dữ liệu (Firebase Firestore).
- Cung cấp API để dashboard hoặc ứng dụng web/mobile truy xuất dữ liệu.
- Xử lý lệnh điều khiển (bật/tắt, cho ăn, lên lịch) và gửi ngược lại cho thiết bị.

2.4. Các chuẩn truyền thông

Các chuẩn truyền thông là tập hợp những quy tắc và giao thức cho phép các thiết bị điện tử trao đổi dữ liệu với nhau một cách hiệu quả. Chúng được phân thành hai loại chính: có dây và không dây. Chuẩn truyền thông có dây, như Ethernet, USB, I2C, và SPI, sử dụng các kết nối vật lý như cáp đồng hoặc cáp quang để truyền tín hiệu. Phương pháp này thường đảm bảo tốc độ cao, độ trễ thấp và kết nối ổn định, nhưng bị hạn chế về sự linh hoạt và khoảng cách vật lý.

Ngược lại, chuẩn truyền thông không dây, bao gồm Wi-Fi, Bluetooth, NFC và mạng di động (4G/5G), truyền dữ liệu qua không gian bằng sóng điện từ như sóng vô tuyến. Lợi thế lớn nhất của phương thức này là sự tiện lợi, khả năng di động và dễ dàng triển khai, cho phép các thiết bị kết nối mà không cần đến hệ thống dây dẫn phức tạp. Việc lựa chọn giữa truyền thông có dây và không dây phụ thuộc vào yêu cầu cụ thể của từng ứng dụng, cân bằng giữa các yếu tố như tốc độ, độ tin cậy, bảo mật và tính di động.



Hình 10: Minh họa sơ lược chuẩn truyền thông có dây và không dây

2.4.1. Các chuẩn truyền thông không dây

Chuẩn truyền IEEE 802.11, hay còn được gọi là WiFi, là một chuẩn truyền không dây phổ biến được sử dụng để truyền dữ liệu qua mạng local area network (LAN). Được phát triển bởi Hiệp hội Kỹ sư Điện và Điện tử (IEEE), chuẩn truyền này cho phép truyền dữ liệu giữa các thiết bị mạng không dây như máy tính, điện thoại di động, máy tính bảng và các thiết bị kết nối Internet khác.

Các phiên bản của chuẩn truyền IEEE 802.11 WiFi:

- + IEEE 802.11a: Được phát triển vào năm 1999 và hoạt động trong dải tần 5GHz, cung cấp tốc độ truyền dữ liệu lên đến 54Mbps.
- + IEEE 802.11b: Được phát triển cùng thời với IEEE 802.11a, hoạt động trong dải tần 2.4GHz cung cấp tốc độ truyền dữ liệu lên đến 11Mbps.
- + IEEE 802.11g: Được phát triển vào năm 2003 và hoạt động trong cả dải tần 2.4GHz cung cấp tốc độ truyền dữ liệu lên đến 54Mbps và sử dụng kỹ thuật OFDM như IEEE 802.11a.
- + IEEE 802.11n: Được phát triển vào năm 2009 hoạt động trong cả dải tần 2.4GHz và 5GHz và cung cấp tốc độ truyền dữ liệu lên đến 600Mbps.
- + IEEE 802.11ac: Được phát triển vào năm 2013, hoạt động trong dải tần 5GHz và cung cấp tốc độ truyền dữ liệu lên đến hàng trăm Mbps hoặc thậm chí hàng ngàn Mbps.

Các đặc điểm của chuẩn truyền IEEE 802.11 WiFi:

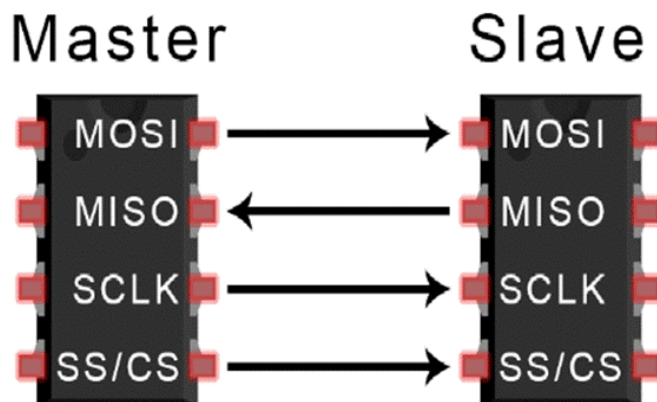
- Tốc độ truyền dữ liệu cao.
- Phổ sóng rộng và xuyên vật cản
- Độ bảo mật cao.
- Tương thích ngược với các phiên bản cũ.

2.4.2. Các chuẩn truyền thông có dây

a) Chuẩn truyền thông SPI

Chuẩn truyền SPI (Serial Peripheral Interface) là một giao thức truyền thông đồng bộ (synchronous) được sử dụng để kết nối và truyền dữ liệu giữa các thiết bị điện tử trong hệ thống nhúng (embedded systems). Nó được phát triển bởi Motorola vào những năm 1980 và sau đó trở thành một tiêu chuẩn ngành công nghiệp.

SPI bao gồm hai thành phần chính: Master (thiết bị chủ) và Slave (thiết bị con). Master là nguồn điều khiển gửi lệnh và nhận dữ liệu từ Slave. Các Slave là các thiết bị nhận và trả lời các lệnh từ Master. SPI sử dụng hai đường truyền chính. Master gửi dữ liệu tới Slave thông qua đường truyền dữ liệu (MOSI - Master Output Slave Input). Slave trả lời Master thông qua đường truyền dữ liệu (MISO - Master Input Slave Output)



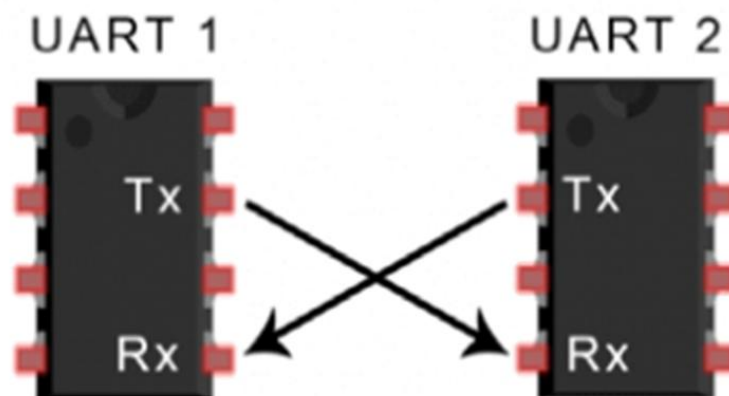
Hình 11: Cách kết nối của chuẩn truyền SPI.

Tốc độ truyền dữ liệu: SPI cho phép truyền dữ liệu với tốc độ cao. Tuy nhiên, tốc độ truyền dữ liệu phụ thuộc vào tốc độ đồng hồ và cấu hình của hệ thống. SPI không yêu cầu giao thức truyền cụ thể. Nó chỉ cung cấp cơ chế truyền dữ liệu đồng bộ giữa Master và Slave. SPI cho phép cấu hình các tham số như tốc độ truyền, kiểu truyền (full-duplex, half-duplex) và thứ tự truyền (LSB-first, MSB-first). SPI có tính tương thích ngược, cho phép kết nối và giao tiếp giữa các thiết bị SPI từ các nhà sản xuất khác nhau.

b) Chuẩn truyền UART

Chuẩn truyền UART (Universal Asynchronous Receiver-Transmitter) là một giao thức truyền thông bất đồng bộ (asynchronous) được sử dụng trong việc kết nối và truyền dữ liệu giữa các thiết bị điện tử. Giao thức UART cho phép truyền dữ liệu qua một đường truyền đơn (simplex) hoặc hai đường truyền song song (duplex).

Giao thức UART sử dụng cơ chế truyền dữ liệu không đồng bộ, không yêu cầu một tín hiệu đồng hồ chung giữa các thiết bị. Nó dựa trên việc sử dụng một chuỗi các bit để truyền dữ liệu. Các bit được truyền theo thứ tự từ bit thấp đến bit cao (LSB first) hoặc từ bit cao đến bit thấp (MSB first). Giao thức UART bao gồm hai tín hiệu chính: Tín hiệu truyền (TX - Transmitter) và tín hiệu nhận (RX - Receiver). Thiết bị gửi dữ liệu sẽ sử dụng tín hiệu này để truyền dữ liệu đến thiết bị nhận. Thiết bị nhận dữ liệu sẽ sử dụng tín hiệu này để nhận dữ liệu từ thiết bị gửi.

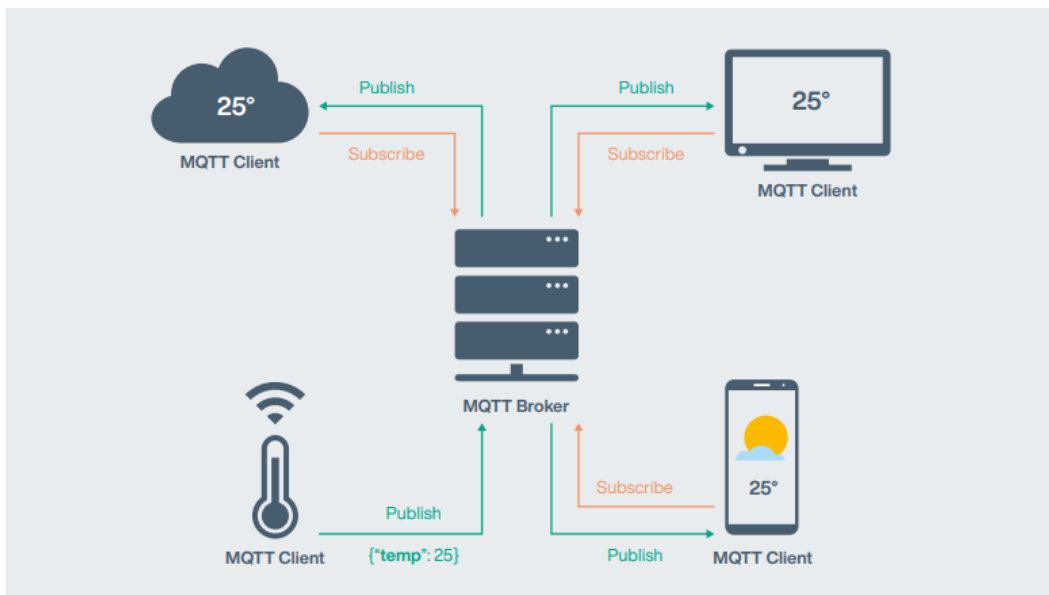


Hình 12: Cách kết nối của chuẩn truyền UART.

Tốc độ truyền dữ liệu của giao thức UART được xác định bằng số lượng bit truyền trong một giây (baud rate). Các tốc độ phổ biến bao gồm 9600, 19200, 38400, 115200, và nhiều tốc độ khác. Khả năng truyền dữ liệu không đồng bộ: Giao thức UART cho phép truyền dữ liệu không đồng bộ, nghĩa là không cần một tín hiệu đồng hồ chung giữa các thiết bị. Chuẩn truyền UART có tính tương thích ngược, cho phép kết nối cũng như giao tiếp giữa các thiết bị UART từ các nhà sản xuất khác nhau. Giao thức UART là giao thức truyền thông đơn giản và phổ biến trong ứng dụng nhúng và vi điều khiển.

2.5. Giao thức IoT: MQTT

MQTT được phát triển bởi Andy Stanford-Clark (IBM) và Arlen Nipper (Eurotech) vào năm 1999, hiện là tiêu chuẩn mở ISO/IEC 20922. MQTT (Message Queuing Telemetry Transport) là một giao thức nhắn tin theo mô hình xuất bản/đăng ký (Publish/Subscribe), được thiết kế để trở nên nhẹ (lightweight), phù hợp với các thiết bị bị hạn chế tài nguyên (bộ nhớ, CPU) và các mạng có băng thông thấp, độ trễ cao hoặc không ổn định. Đây là một giao thức lý tưởng cho Internet Vạn Vật (IoT) và giao tiếp Machine-to-Machine (M2M). Ngoài MQTT COAP là một giao thức IoT phổ biến. Cả hai đều được thiết kế để truyền dữ liệu nhẹ, phù hợp với các thiết bị nhưng có tài nguyên hạn chế. Tuy nhiên, chúng khác nhau về kiến trúc, cơ chế truyền tin và mô hình kết nối.



Hình 13: Mô hình hoạt động của giao thức MQTT theo cơ chế Publish/Subscribe

Hình 9 trên minh họa cơ chế truyền nhận dữ liệu của giao thức MQTT, trong đó các thiết bị (MQTT Client) gửi và nhận dữ liệu thông qua Broker trung gian. Các Client có thể đóng vai trò Publisher (gửi dữ liệu cảm biến) hoặc Subscriber (nhận dữ liệu hiển thị). Bảng dưới trình bày sự khác biệt giữa hai giao thức truyền thông phổ biến trong IoT là MQTT và CoAP, dựa trên các tiêu chí như mô hình giao tiếp, độ trễ, độ tin cậy và khả năng mở rộng. MQTT phù hợp với mô hình AquaNova, nơi yêu cầu đồng bộ dữ liệu ổn định và truyền nhận thông tin theo thời gian thực.

Bảng 1: So sánh đặc điểm giữa giao thức MQTT và CoAP

Tiêu chí	MQTT	CoAP
Mô hình giao tiếp	Publish/Subscribe thông qua Broker trung gian	Client/Server trực tiếp
Giao thức nền	TCP/IP (đảm bảo kết nối)	UDP (nhẹ hơn nhưng không đảm bảo)
Cơ chế tin cậy	Hỗ trợ QoS (0, 1, 2) xác nhận truyền tin	Có ACK nhưng phụ thuộc vào UDP
Cách tổ chức hệ thống	Các thiết bị gửi dữ liệu lên Broker, bên khác đăng ký (subscribe) để nhận	Thiết bị gửi yêu cầu GET/POST đến server
Bảo mật	Hỗ trợ SSL/TLS, xác thực người dùng	Hỗ trợ DTLS, cấu hình phức tạp hơn
Độ trễ	Trung bình (do TCP)	Rất thấp (UDP)
Khả năng mở rộng	Cao, chỉ cần thêm topic	Hạn chế, cần thiết lập từng kết nối
Phù hợp cho	Hệ thống lớn, nhiều node, truyền dữ liệu thường xuyên	Hệ thống cảm biến nhỏ, đơn giản

Mô hình Publish/Subscribe: MQTT hoạt động dựa trên ba thành phần chính:

- Publisher (Bên Xuất bản/Client): Thiết bị hoặc ứng dụng gửi dữ liệu.
- Subscriber (Bên Đăng ký/Client): Thiết bị hoặc ứng dụng nhận dữ liệu.
- Broker (Máy chủ/Server): Thành phần trung gian, nhận thông điệp từ Publisher và chuyển tiếp đến các Subscriber đã đăng ký (subscribe) một Topic cụ thể. Broker giúp tách biệt bên gửi và bên nhận về mặt thời gian và không gian.

Lý do chọn : Lý do chọn giao thức MQTT trong hệ thống IoT cho cá ăn tự động

- Nhẹ và tối ưu cho IoT: MQTT sử dụng băng thông thấp, overhead nhỏ, phù hợp cho các thiết bị tài nguyên hạn chế như STM32 và ESP32.
- Thích hợp cho môi trường có kết nối Wi-Fi không ổn định.
- Mô hình Publish/Subscribe đơn giản: Dữ liệu cảm biến (nhiệt độ, độ ẩm, lượng thức ăn) từ ESP32 chỉ cần publish một lần. Nhiều subscriber (web dashboard,

app di động, server Firebase) đồng thời nhận dữ liệu mà không cần MCU gửi riêng cho từng thiết bị.

- Độ tin cậy cao: Hỗ trợ các mức QoS (Quality of Service) 0, 1, 2 cho phép lựa chọn giữa tốc độ và độ đảm bảo truyền dữ liệu. Có cơ chế Last Will Message và KeepAlive để xử lý mất kết nối.
- Khả năng mở rộng tốt MQTT broker (như HiveMQ Cloud) có thể quản lý hàng ngàn thiết bị cùng lúc. Hệ thống dễ dàng mở rộng thêm cảm biến (nhiệt độ, pH, oxy hòa tan) hoặc actuator (máy bơm, quạt oxy) chỉ bằng cách thêm topic mới.
- Tích hợp thuận tiện với nền tảng đám mây: MQTT có sẵn plugin, bridge kết nối với Firebase, Node-RED, InfluxDB, Grafana, giúp dễ dàng lưu trữ, trực quan hóa và điều khiển từ xa.
- Bảo mật tốt: Hỗ trợ xác thực Username/Password, kết nối TLS/SSL để bảo vệ dữ liệu trong môi trường mạng công cộng (Wi-Fi).

a) Mức Chất lượng Dịch vụ (Quality of Service - QoS)

MQTT cung cấp ba mức QoS để đảm bảo độ tin cậy của việc truyền thông điệp:

QoS 0: At most once (Tối đa một lần)

- Thông điệp được gửi một lần, không có xác nhận (ACK).
- Truyền tải nhanh nhất nhưng có thể bị mất.
- Tương đương: Gửi bưu thiếp thông thường.
- QoS 0: phù hợp cho dữ liệu không quan trọng, như cập nhật liên tục nhiệt độ mỗi giây (chấp nhận mất gói).

QoS 1: At least once (Tối thiểu một lần)

- Thông điệp được đảm bảo đến đích ít nhất một lần. Bên nhận sẽ gửi gói PUBACK xác nhận.
- Thông điệp có thể bị trùng lặp (nếu bên gửi không nhận được PUBACK và gửi lại).
- Tương đương: Gửi thư bảo đảm có biên lai xác nhận.
- QoS 1: phù hợp cho dữ liệu quan trọng vừa phải, như trạng thái cho ăn (nếu trùng lặp vẫn không sao).

QoS 2: Exactly once (Duy nhất một lần)

- Thông điệp được đảm bảo đến đích đúng một lần. Sử dụng giao thức bắt tay bốn bước (handshake) để đảm bảo không bị mất hoặc trùng lặp.

- Đáng tin cậy nhất nhưng tốn nhiều băng thông và thời gian nhất.
- Tương đương: Gửi thư bảo đảm yêu cầu chữ ký và xác nhận giao hàng chi tiết.
- QoS 2: dùng cho dữ liệu cực kỳ quan trọng, như giao dịch tài chính, lệnh điều khiển chỉ được thực hiện một lần duy nhất.

b) MQTT Connection

Luồng gói tin MQTT

CONNECT: Client gửi đến Broker để yêu cầu thiết lập kết nối.

CONNACK: Broker phản hồi cho gói CONNECT, xác nhận kết nối thành công hay kết nối thất bại.

```
[sensor-1] log: Sending CONNECT (u1, p1, wr0, wq0, wf0, c1, k60)
client_id=b'sim-sensor-1'
[sensor-1] log: Received CONNACK (0, 0)
[sensor-1] Connected with result code Success
```

- u1, p1: username & password
- c1 : cleanSession = 1 | wq0: mức QoS
- k60: keepAlive = 60 giây
- client_id= 'sim-sensor-1': định danh client
- wf0: khai báo Last Will
- wr0: cờ retain Last Will message.

Sau đó broker trả về CONNACK (0,0) → thành công.

Bảng 2: Cấu trúc và chức năng của các gói tin connect MQTT

Gói tin	Mô tả	Cấu trúc (Header + Payload)
CONNECT	Client gửi đến Broker để yêu cầu thiết lập kết nối.	Fixed Header + Variable Header (Tên và phiên bản Protocol, cờ kết nối, Keep Alive) + Payload (Client ID, Username/Password, Last Will & Testament)
CONNACK	Broker phản hồi cho gói CONNECT, xác nhận kết nối thành công hay không	Fixed Header + Variable Header (cờ xác nhận, lý do)

DISCONNECT	Client gửi đến Broker để ngắt kết nối một cách an toàn.	Chỉ có Fixed Header
PINGREQ / PINGRESP	Client gửi PINGREQ để giữ kết nối hoạt động khi không có dữ liệu trao đổi; Broker phản hồi bằng PINGRESP.	Chỉ có Fixed Header

c) MQTT Publisher

Luồng gói tin MQTT

- PUBLISH: Thiết bị gửi dữ liệu cảm biến (telemetry) lên topic định sẵn.
- PUBACK: Broker xác nhận đã nhận dữ liệu (QoS=1).
- Broker trả PUBACK (mid=1) → nhận gói tin thành công
- DISCONNECT: Client chủ động ngắt kết nối.

```
[sensor-1] log: Sending PUBLISH (d0, q1, r0, m1),
  'b'aquanova/devices/sensor-1/telemetry'', ... (123 bytes)
[sensor-1] -> publishing {'device_id': 'sensor-1', 'ts': '2025-09-30T10:01:05.944232+00:00', 'temperature': 30.53, 'turbidity': 48.55, 'feed': 51.6}
```

- d0: dupFlag = 0 | r0: retain = 0
- q1: QoS 1 (phải có PUBACK xác nhận)
- m1: messageId = 1
- Topic: aquanova/devices/sensor-1/telemetry
- Payload: JSON chứa thông số sensor

```
2025-09-30T17:14:44.996069 [sensor-1] log: Received PUBACK (Mid: 1)
2025-09-30T17:14:44.996898 [sensor-1] Published mid=1,
reason=Success
2025-09-30T17:15:34.852617 [sensor-1] log: Sending DISCONNECT
2025-09-30T17:15:34.855632 [sensor-1] log: ... Disconnected,
reason=0
```

Bảng 3: Cấu trúc và chức năng của các gói tin publsih MQTT

Gói tin	Mô tả	Cấu trúc (Header + Payload)
PUBLISH	Client (hoặc Broker) gửi thông điệp ứng dụng đến Topic.	Fixed Header (QoS, DUP, RETAIN) + Variable Header

		(Tên Topic, Packet Identifier) + Payload (Dữ liệu thực tế)
PUBACK	Phản hồi cho PUBLISH QoS 1, xác nhận đã nhận thông điệp.	Fixed Header + Variable Header (Packet Identifier, Mã lý do)
PUBREC, PUBREL, PUBCOMP	Các gói tin bắt tay (handshake) để đảm bảo giao tiếp QoS 2 - PUBREC (Received): Broker xác nhận đã nhận PUBLISH. - PUBREL (Release): Client xác nhận đã sẵn sàng hoàn tất. - PUBCOMP (Complete): Broker xác nhận đã hoàn tất giao dịch.	

d) MQTT Subscriber

- Luồng kết nối MQTT
- CONNECT → client gửi yêu cầu kết nối.
- CONNACK → broker xác nhận kết nối thành công
- SUBSCRIBE → client gửi yêu cầu đăng ký topic.
- SUBACK → broker xác nhận quyền đăng ký với QoS

SUBSCRIBE → client gửi yêu cầu đăng ký topic.

```
2025-10-01T17:03:37.123756 [MQTT] LISTENING
d2b265...hivemq.cloud:8883 topic=aquanova/telemetry
2025-10-01T17:03:37.389492 [MQTT] CONNECTED with code=Success
2025-10-01T17:03:37.390013 [MQTT] SUBSCRIBE sent
topic=aquanova/telemetry mid=1 result=0
```

Subscriber gửi gói CONNECT và broker phản hồi CONNACK với code Success, xác nhận kết nối MQTT

- topic: aquanova/devices/+/telemetry
- mid=1: ID duy nhất của gói đăng ký
- result=0: gửi thành công về phía client library

SUBACK → broker xác nhận quyền đăng ký với QoS

```
2025-10-01T17:03:37.601655 [MQTT] SUBACK received mid=1,
granted_qos=[ReasonCode(Suback, 'Granted QoS 1')]
```

Bảng 4: Danh sách các topic MQTT và vai trò trong mô hình AquaNova

Thành phần	Subscribe topic	Ý nghĩa
Server/WebApp	aquanov/telemetry	Nhận dữ liệu từ tất cả sensor
MCU Feeder	Aquanova/feeder	Nhận lệnh cho ăn
MCU Light	aquanova/light	Nhận lệnh bật/tắt đèn
AI Engine	aquanova/devices/+/telemetry	Phân tích dữ liệu môi trường, đưa ra hành động

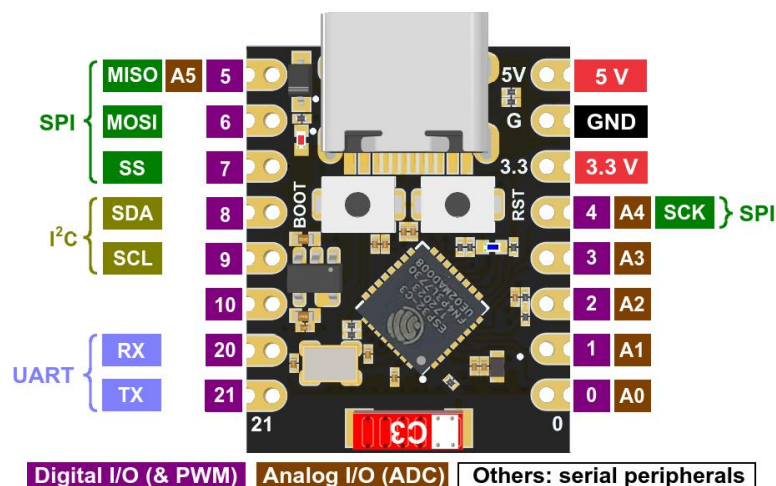
2.6. Thiết bị phần cứng

2.6.1. Vi điều khiển

a) ESP32 C3 Mini

ESP-01 có tích hợp sẵn một bộ phát WiFi, đủ để kết nối với mạng internet và truyền dữ liệu. Module này còn được tích hợp một cổng giao tiếp chuẩn UART, cho phép truyền dữ liệu giữa ESP32 và các thiết bị khác.

Thông số kỹ thuật của Module Wifi ESP32:



Hình 2: Sơ đồ chân Esp32 c3 mini

b) STM32

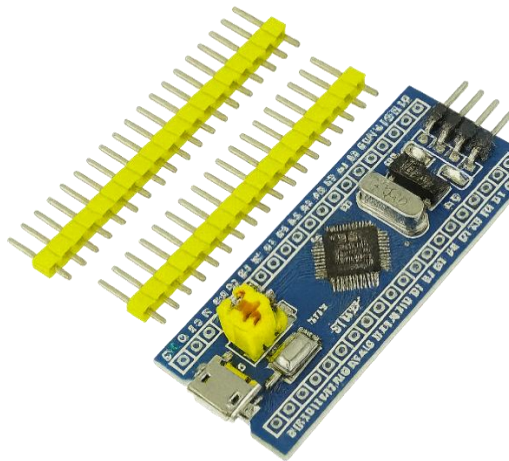
STM32: một dòng vi điều khiển mạnh mẽ của STMicroelectronics. STM32 hỗ trợ rất nhiều giao thức giao tiếp, giúp kết nối với các thiết bị ngoại vi, cảm biến, và các mô-đun khác:

- USART/UART: Giao tiếp nối tiếp (Serial communication).
- SPI (Serial Peripheral Interface): Giao tiếp đồng bộ tốc độ cao.

- I2C: Giao tiếp nối tiếp với nhiều thiết bị trên cùng một bus.
- CAN (Controller Area Network): Giao thức giao tiếp mạnh mẽ trong ngành ô tô và tự động hóa.
- USB: Giao tiếp qua cổng USB, cho phép kết nối với các thiết bị như máy tính hoặc các thiết bị ngoại vi.

Ethernet: Một số vi điều khiển STM32 hỗ trợ Ethernet để kết nối mạng.

Cổng vào/ra (GPIO - General Purpose Input/Output: Các vi điều khiển STM32 có các chân GPIO để kết nối với các thiết bị bên ngoài như công tắc, đèn LED, cảm biến, động cơ, v.v. Các chân này có thể cấu hình để làm input (nhận tín hiệu từ thiết bị ngoài) hoặc output (gửi tín hiệu điều khiển đến thiết bị ngoài).USB



Hình 3: Mạch STM32F103C8T6 Micro USB

2.6.2. Cảm biến và các linh kiện khác

a) Relay

Module relay SRD-5VDC-SL-C hoạt động với điện áp đầu vào là 5V DC (Direct Current). Nó có khả năng chuyển đổi tải điện từ mạch điều khiển sang tải ngoại vi (ví dụ: đèn, motor, thiết bị điện...). Module này được điều khiển bằng nguồn điện DC và tín hiệu điều khiển từ mạch điện tử thông qua một đường tín hiệu kích hoạt (triggers) để mở hoặc đóng relay.



Hình 5: Relay SRD-5VDC-SL-C.

Bảng 5: Thông số kỹ thuật của Module relay SRD-5VDC-SL-C

Điện áp đầu vào	5V
Tiếp điểm	SPDT
Dòng điện chuyển mạch	10A
Kiểu chân	Xuyên lỗ
Series	SRD

b) SG90 Động Cơ Servo 180 Độ



Hình 6: SG90 Servo 180 độ

SG90 có kích thước nhỏ, giá thành thấp là loại động cơ servo phổ biến. Động cơ servo này có các bánh răng được làm bằng nhựa, được tích hợp sẵn Driver điều khiển động cơ, dễ dàng điều khiển góc quay bằng phương pháp điều độ rộng xung PWM.

- Phạm vi xung PWM 1ms - 2ms: góc quay tối đa 90 độ
- Phạm vi xung PWM 0.5ms - 2.5ms: góc quay tối đa 180 độ

Bảng 6: Thông số kỹ thuật của module SG90 Servo 180 Độ

Kích thước	21.5 x 11.8 x 22.7mm
Góc quay	180 độ
Trọng lượng	9g

Tốc độ không tải	0.12 giây / 60° (4.8V)
Mô-men xoắn	1.2-2.4 kg.cm (4.8V)
Nhiệt độ hoạt động	-30 → +60 ° C
Điện áp hoạt động	4.8 → 6 V dc

c) Cảm biến nhiệt độ MKE-S15 DS18B20



Hình 14: Cảm biến nhiệt độ MKE-S15 DS18B20

Cảm biến DS18B20 là một đầu dò nhiệt độ kỹ thuật số chống nước, được chế tạo với vỏ bọc kín nhằm đảm bảo độ bền và khả năng hoạt động trong nhiều điều kiện môi trường khắc nghiệt. Thiết bị được phát triển bởi Dallas Semiconductor (nay thuộc Maxim Integrated [9]) và được cung cấp dưới dạng dây dài tiêu chuẩn 1 mét, thuận tiện cho việc triển khai trong thực tế. DS18B20 hoạt động dựa trên nguyên lý chuyển đổi trực tiếp nhiệt độ sang tín hiệu số, nhờ bộ chuyển đổi tín hiệu tương tự – số (ADC) tích hợp với độ phân giải đến 12 bit. Cảm biến sử dụng giao thức truyền thông nối tiếp 1-Wire, cho phép mỗi thiết bị được định danh duy nhất bằng một mã nối tiếp 64 bit, đảm bảo khả năng kết nối và nhận diện nhiều cảm biến trên cùng một đường dữ liệu. Ngoài ra, DS18B20 có thể vận hành trong chế độ nguồn điện ký sinh, giúp giảm số chân kết nối cần thiết. Với đặc tính là cảm biến 1-Wire, DS18B20 chỉ yêu cầu một chân dữ liệu cùng chân GND để giao tiếp với vi điều khiển. Thiết bị hỗ trợ dải đo từ -55 °C đến +125 °C, với độ chính xác khoảng ± 0.5 °C trong phạm vi thông thường và sai số lớn hơn ở biên nhiệt độ. Nguồn nuôi cho cảm biến dao động từ 3.0 V đến 5.5 V, với dòng tiêu thụ tối đa khoảng 1 mA. Một ưu điểm nổi bật của DS18B20 là chức năng cảnh báo nhiệt độ, cho phép người dùng thiết lập ngưỡng giới hạn cao hoặc thấp. Khi nhiệt độ vượt quá ngưỡng này, cảm biến sẽ phát tín hiệu cảnh báo qua đường dữ liệu.

Bảng 7: Bảng thông số và đặc điểm kỹ thuật cảm biến DS18B20

Mục	Chi tiết
Loại cảm biến	Cảm biến nhiệt độ kỹ thuật số có thể lập trình, giao tiếp qua giao thức 1-Wire
Điện áp hoạt động	3.0 V – 5.5 V
Độ chính xác	$\pm 0.5\text{ }^{\circ}\text{C}$ trong dải $-10\text{ }^{\circ}\text{C}$ đến $+85\text{ }^{\circ}\text{C}$ (<i>chính xác, không $\pm 5\text{ }^{\circ}\text{C}$</i>)
Dải nhiệt độ hoạt động	$-55\text{ }^{\circ}\text{C}$ đến $+125\text{ }^{\circ}\text{C}$ (tương đương $-67\text{ }^{\circ}\text{F}$ đến $+257\text{ }^{\circ}\text{F}$)
Độ phân giải	Lựa chọn 9 đến 12 bit
Giao tiếp	1-Wire, chỉ cần một chân dữ liệu, không cần linh kiện ngoài
Mã định danh	Mỗi chip có ID 64-bit duy nhất, cho phép nhiều cảm biến dùng chung một bus dữ liệu
Tính năng đặc biệt	Hệ thống cảnh báo giới hạn nhiệt độ
Thời gian truy vấn	$< 750\text{ ms}$
Kết nối dây tiêu chuẩn	- Dây đỏ: VCC (3.0–5.5 V)- Dây đen: GND- Dây vàng: DATA
Thông số vật lý (vỏ bọc chống nước)	- Ống thép không gỉ, đường kính 6 mm, dài 35 mm- Cáp: đường kính 4 mm, dài 95 cm ($\sim 37.4\text{ inch}$)- Đầu dò: đường kính 7 mm, dài 26 mm- Tổng chiều dài dây: $\sim 6\text{ feet}$
Bảo vệ	Vỏ thép không gỉ chống ẩm, chống nước, chống gỉ; bên trong có keo niêm phong và ống co nhiệt chống đoản mạch
Yêu cầu điện trở kéo lên	Bắt buộc có điện trở pull-up ($4.7\text{ k}\Omega$) nối giữa dây Data và VCC để đảm bảo tín hiệu ổn định
Kết nối với vi điều khiển	- VCC $\leftrightarrow$ nguồn 3.0–5.5 V- GND $\leftrightarrow$ chân GND- Data $\leftrightarrow$ chân digital I/O- Có điện trở kéo lên Data–VCC

d) Cảm biến đo độ đục MJKDZ Turbidity Sensor



Hình 15: Cảm biến đo độ đục MJKDZ Turbidity Sensor

Cảm biến đo độ đục MJKDZ hoạt động dựa trên nguyên lý tán xạ ánh sáng (Nephelometry). Trong cấu trúc của module, một nguồn sáng, thường là diode phát quang (LED), chiếu ánh sáng vào mẫu chất lỏng cần phân tích. Khi đi qua môi trường chứa các hạt lơ lửng, chùm sáng sẽ xảy ra hiện tượng tán xạ do sự tương tác với các hạt có kích thước, hình dạng và đặc tính quang học khác nhau. Mức độ tán xạ này phản ánh trực tiếp nồng độ các hạt trong dung dịch và do đó đặc trưng cho độ đục của mẫu. Một bộ dò quang (photodiode hoặc phototransistor) được bố trí ở góc thích hợp (thường là 90° so với hướng chiếu sáng) để thu nhận phần ánh sáng tán xạ. Cường độ ánh sáng mà bộ dò thu được tỷ lệ thuận với mật độ các hạt lơ lửng trong chất lỏng. Tín hiệu quang sau đó được chuyển đổi thành tín hiệu điện, trong đó biên độ tín hiệu điện tỷ lệ thuận với cường độ ánh sáng tán xạ, và do đó tỷ lệ thuận với độ đục của mẫu. Tín hiệu điện này được đưa vào mạch xử lý và vi điều khiển để thực hiện quá trình số hóa, hiệu chuẩn và tính toán giá trị độ đục theo các đơn vị chuẩn như NTU. Kết quả đo được hiển thị trên màn hình kỹ thuật số hoặc truyền tới hệ thống thu thập dữ liệu. Phương pháp này đảm bảo tính trực quan, nhanh chóng và chính xác, đồng thời phù hợp với các ứng dụng trong giám sát chất lượng nước sinh hoạt, nuôi trồng thủy sản.

Bảng 8: Bảng phân tích và thông số cảm biến MJKDZ Turbidity Sensor

Mục	Chi tiết
Thông tin cảm biến – phân tích lựa chọn linh kiện	- Xuất xứ: Trung Quốc - Giá thành: Rẻ (~100–150k VND) - Ưu điểm: Dễ mua, chi phí thấp - Hạn chế: Độ chính xác và ổn định thấp hơn so với SEN0182
Thông số kỹ thuật	- Điện áp hoạt động: 5 V

	- Ngõ ra: Analog (0 – 4.5 V), Digital (ngưỡng chỉnh bằng biến trở) - Dải đo: ~0 – 1000 NTU (độ chính xác kém hơn SEN0182) - Độ ổn định: Thấp, tín hiệu nhiễu nhiều, cần thêm mạch lọc
Yêu cầu kết nối phần cứng	- Kết nối tương tự SEN0182 - Nguồn nuôi: 5 V - Ngõ ra AOUT → ADC của vi điều khiển (cần chia áp khi dùng STM32 3.3 V) - Khuyến nghị: Thêm tụ lọc (0.1 – 1 μF) chống nhiễu trên đường tín hiệu
Giao thức phần mềm	- Đọc giá trị ADC và chuyển đổi thành điện áp - Hiệu chuẩn: Cần hiệu chuẩn thủ công bằng dung dịch chuẩn (không có công thức chuyển đổi sẵn) - Lọc tín hiệu: Nên lấy trung bình nhiều mẫu để giảm dao động và nhiễu

e) Cảm biến siêu âm HCSR04

Cảm biến siêu âm phát ra siêu âm, siêu âm sau khi chạm vào vật cản sẽ phản xạ lại và được thu bởi cảm biến (hiện tượng phản xạ sóng âm.)

- Bộ phát (Trigger – Trig) của cảm biến phát ra một chùm sóng âm tần số 40 kHz trong không khí.
- Khi sóng âm gặp vật cản, nó sẽ phản xạ ngược trở lại và được bộ thu (Echo) của cảm biến ghi nhận.
- Thời gian từ lúc phát sóng đến khi nhận được sóng phản xạ được gọi là thời gian bay của sóng âm (time of flight – ToF).

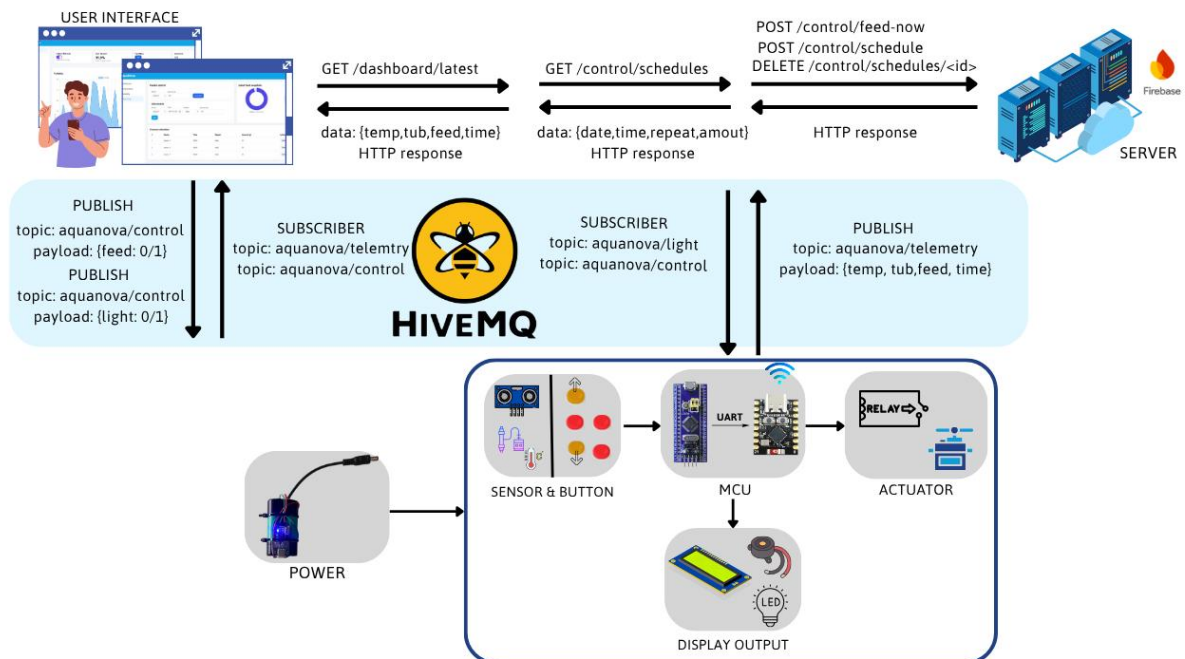
Do sóng âm truyền trong không khí với vận tốc $v \approx 343$ m/s (ở 25°C), ta có thể xác định khoảng cách đến vật cản dựa vào thời gian bay này. Vì vậy có thể đo khoảng cách bằng cách xác định thời gian từ lúc phát đến lúc nhận được xung phản xạ.

CHƯƠNG 3

THIẾT KẾ VÀ THI CÔNG MÔ HÌNH

Quá trình thiết kế và thi công mô hình AquaNova trong Chương 3 bao gồm các bước:

- Bước 1: xác định các yêu cầu về tính năng (như đo nhiệt độ, độ đục) và yêu cầu kỹ thuật (như dùng STM32, ESP32, MQTT).
- Bước 2: thiết kế mô hình tổng quan của hệ thống, bao gồm kiến trúc IoT 3 lớp (Perception, Communication, Application) và sơ đồ khối chi tiết (Khối nguồn, Khối cảm biến, Khối xử lý trung tâm).
- Bước 3: thiết kế cơ khí 3D cho bộ cấp thức ăn bằng phần mềm Autodesk Inventor và tiến hành in 3D, lắp ráp.
- Bước 4: thiết kế nhúng, tập trung vào lập trình vi điều khiển STM32 (dùng STM32CubeMX) để xử lý cảm biến, điều khiển servo, và lập trình ESP32 (dùng Arduino IDE) để kết nối Wi-Fi và làm cầu nối UART-MQTT.
- Bước 5: thiết kế website, bao gồm backend (dùng Flask) để xử lý API, kết nối cơ sở dữ liệu (Firestore) và MQTT, cùng với frontend (HTML/CSS/JS) để hiển thị dashboard. Bước 6: hiệu chuẩn các cảm biến và kiểm thử hệ thống, bao gồm đánh giá độ chính xác, độ ổn định của kết nối Wi-Fi và độ trễ giao tiếp.



Hình 16: Tổng quan mô hình AquaNova

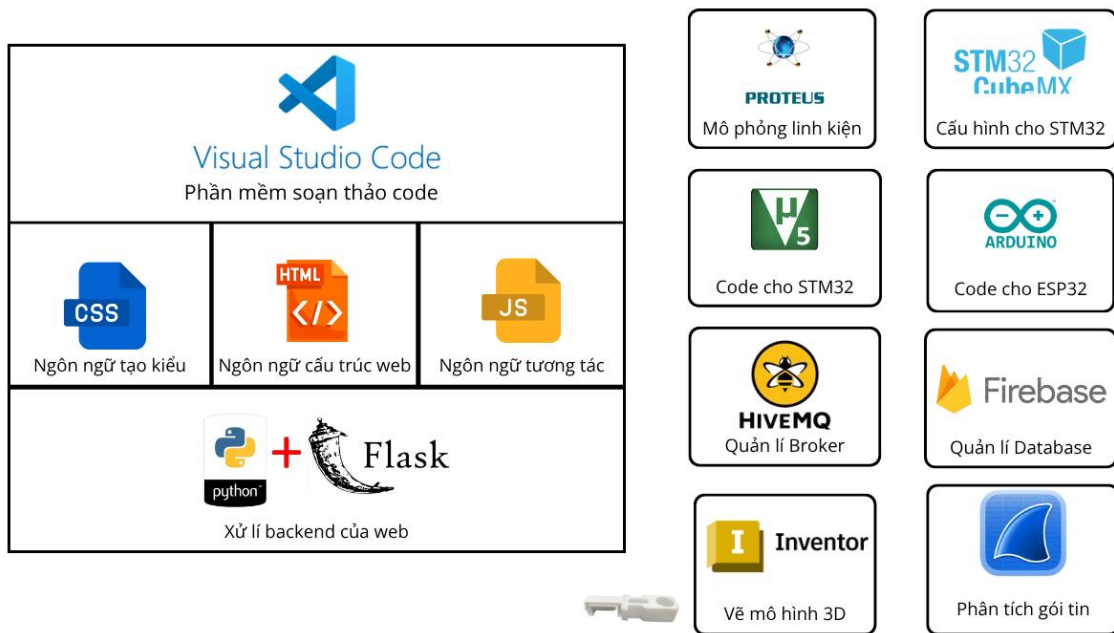
3.1. Yêu cầu về tính năng của mô hình

Để xây dựng mô hình IoT AquaNova hoạt động ổn định và đáp ứng đầy đủ các chức năng giám sát – điều khiển hồ cá, trước hết cần xác định rõ yêu cầu của hệ thống và lựa chọn các linh kiện phù hợp cho từng nhiệm vụ. Các yêu cầu này được tổng hợp trong Bảng 9, bao gồm các thành phần chính như cảm biến, vi điều khiển, module kết nối, và cơ cấu chấp hành, nhằm đảm bảo hệ thống có thể thu thập dữ liệu chính xác, xử lý hiệu quả và điều khiển tự động trong môi trường thực tế.

Chức năng hệ thống	Mô tả yêu cầu	Linh kiện sử dụng
1. Đo nhiệt độ nước	Cảm biến chính xác, chống nước, sai số nhỏ $\pm 0.5\text{ }^{\circ}\text{C}$	Cảm biến DS18B20
2. Đo độ đục của nước	Đo độ trong/đục để đánh giá chất lượng nước	Turbidity sensor (MJKDZ module)
3. Đo lượng thức ăn	Xác định lượng thức ăn để cảnh báo	HC-SR04 Ultrasonic sensor
4. Xử lý và hiển thị dữ liệu	Đọc giá trị cảm biến, hiển thị LCD, cảnh báo khi vượt ngưỡng	STM32F103C8, LCD I2C 16×2, Buzzer
5. Kết nối Internet và truyền dữ liệu	Gửi dữ liệu cảm biến lên server qua MQTT, nhận lệnh điều khiển	ESP32-C3 Mini, Wi-Fi module tích hợp
6. Đồng bộ thời gian	Lưu và hiển thị giờ thực, hỗ trợ hẹn lịch cho ăn	RTC DS3232 module
7. Điều khiển cơ cấu cho ăn	Quay servo định lượng thức ăn theo lịch hoặc lệnh tay	Servo motor SG90
8. Cấp nguồn cho hệ thống	Cung cấp điện ổn định 5 V–3.3 V, có thể dùng pin hoặc adapter	Nguồn pin Li-ion + Module sạc/ôn áp

Bảng 9: Xác định yêu cầu hệ thống và linh kiện sử dụng

3.2. Yêu cầu về kỹ thuật



Hình 17: Phần mềm sử dụng để xây dựng AquaNova

3.2.1. Chức năng kỹ thuật

- Đọc dữ liệu từ cảm biến (cảm biến nhiệt độ, cảm biến đo độ đục, cảm biến siêu âm), sau đó gửi dữ liệu và thông kê trên dashboard.
- Gửi yêu cầu cho ăn, bật tắt đèn tự động thông qua web server.
- Hẹn giờ, đặt lịch cho ăn; xóa lịch cho ăn.
- Cho biết phần trăm lượng thức ăn còn lại thông qua cảm biến siêu âm.
- Hiện thị thông tin nhiệt độ, độ đục, đặt lịch,.. qua màn hình LCD.
- Cảnh báo khi độ đục nước vượt ngưỡng cho phép.

3.2.2. Thông số kỹ thuật

- Vi điều khiển chính: ESP32 + STM32
- Giao thức kết nối có dây:
 - UART: truyền dữ liệu giữa STM32 và ESP32
 - I2C: giao tiếp giữa STM32 và LCD 16x2 và STM32 với bộ định thời DS3231
- Giao thức kết nối không dây: Wi-Fi chuẩn IEEE 802.11 a (2.4 GHz)
- Nguồn hoạt động: 5V DC (nguồn chính) cụ thể: Màn hình LCD 1602, cảm biến siêu âm HCSR04, Servo, relay, đèn

- Nguồn phụ cho cảm biến: 3.3V DC (được chuyển đổi từ 5V qua bộ ổn áp nội) cụ thể: cảm biến độ đục, cảm biến nhiệt độ DS18B20, module DS3231, buzzer
- Cảm biến tích hợp:
 - Cảm biến nhiệt độ DS18B20
 - Cảm biến độ đục (Turbidity Sensor)
 - Cảm biến siêu âm HC-SR04 (đo mức nước)
 - Màn hình hiển thị: LCD 16x2
- Tần suất cập nhật dữ liệu: 10 phút/lần
- Nguồn cấp cho servo/relay: 5V DC
- Kích thước mô hình tổng thể: (10 cm × 15 cm) + (15 cm × 3 cm × 15 cm)
- Thời gian khởi động và ổn định hệ thống: 8–10 giây kể từ khi cấp nguồn

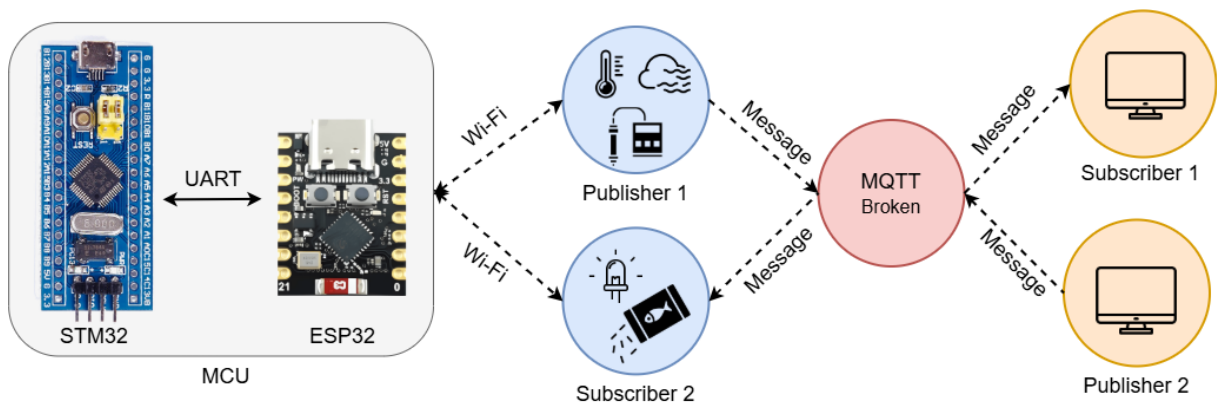
3.3. Mô hình tổng quan hệ thống

Mô hình giám sát và điều khiển AquaNova được xây dựng nhằm theo dõi chất lượng nước (nhiệt độ, độ đục) và tự động cho cá ăn thông qua kết nối Internet. Mô hình sử dụng ESP32-C3 Mini làm bộ điều khiển trung tâm kết hợp với STM32 để thu thập và xử lý tín hiệu cảm biến.

Dữ liệu đo được từ các cảm biến được truyền về ESP32-C3 Mini, sau đó gửi đến Flask Server thông qua giao thức MQTT. Tại đây, dữ liệu được lưu trữ trên Firebase Cloud Firestore và hiển thị trực quan dạng biểu đồ và bảng số liệu trên giao diện web.

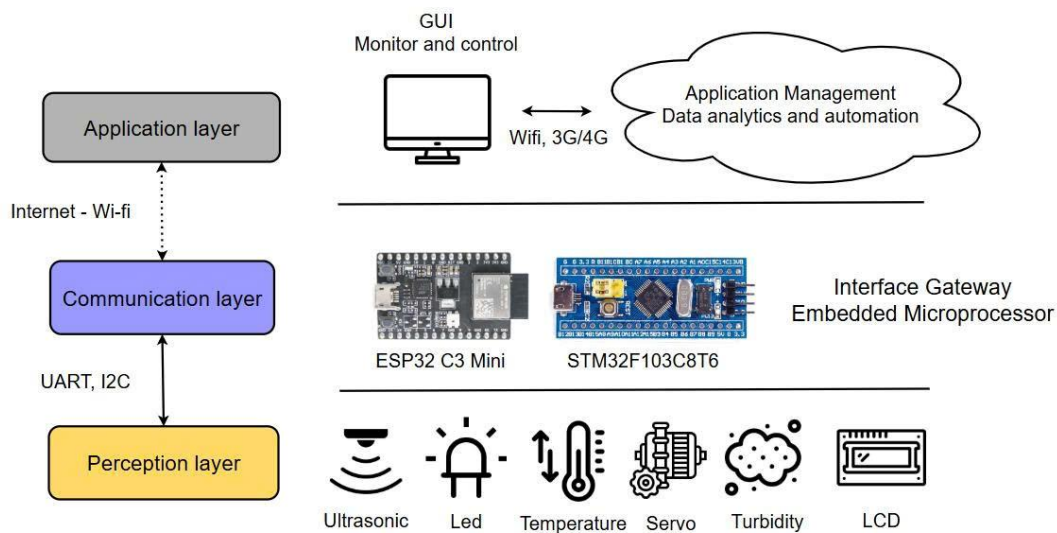
Người dùng có thể đặt lịch cho ăn hoặc kích hoạt cho ăn thủ công trực tiếp từ dashboard, lệnh điều khiển sẽ được gửi ngược lại qua MQTT Broker đến ESP32/STM32 để kích hoạt cơ cấu chấp hành (servo motor) thực hiện việc cho ăn. Quy trình vận hành đảm bảo dữ liệu được đồng bộ hai chiều giữa thiết bị và giao diện web, đồng thời cho phép người dùng theo dõi các chỉ số môi trường theo thời gian thực, hỗ trợ việc chăm sóc hồ cá hiệu quả và tự động hóa cao.

Cơ chế truyền thông giữa hai vi điều khiển được thể hiện chi tiết trong Hình 16, mô tả sơ đồ khối giao tiếp MQTT giữa ESP32-C3 và STM32 trong mô hình giám sát và điều khiển tự động AquaNova. Đây là nơi dữ liệu cảm biến được thu thập, truyền tải và xử lý theo thời gian thực.



Hình 18: Kết nối MQTT vào ESP32 C3 kết hợp STM32

Để xây dựng một hệ thống IoT hiệu quả, việc thiết kế kiến trúc hệ thống là rất quan trọng. Một trong những mô hình kiến trúc phổ biến là kiến trúc 3 lớp. Trong hình 17 minh họa kiến trúc tổng quan của mô hình IoT ba lớp (Three-Layer IoT Architecture) được áp dụng trong mô hình AquaNova. Cấu trúc này bao gồm ba tầng chính: lớp nhận thức (Perception Layer), lớp truyền thông (Communication Layer) và lớp ứng dụng (Application Layer).



Hình 19: Kiến trúc tổng quan của hệ thống IoT 3 lớp của mô hình AquaNova

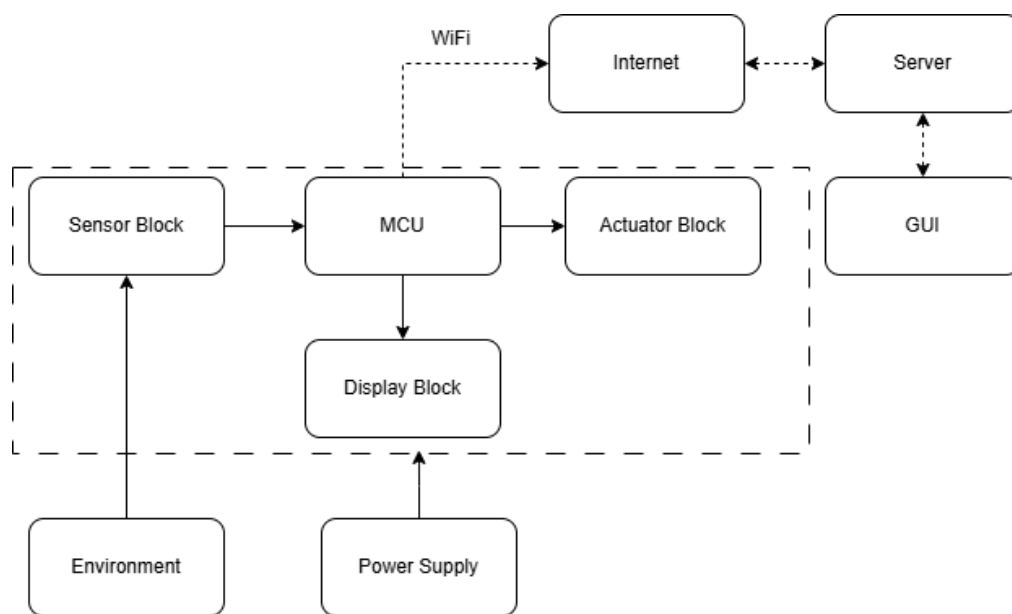
Lớp trên cùng là lớp ứng dụng (Application Layer), đóng vai trò giao tiếp trực tiếp với người dùng. Tại lớp này, dữ liệu được xử lý và hiển thị thông qua giao diện người dùng (Web Dashboard) được phát triển bằng HTML, CSS, JavaScript và quản lý bởi Flask Framework.

Người dùng có thể theo dõi các thông số như nhiệt độ, độ đục, mực nước, đồng thời gửi các lệnh điều khiển đến mô hình như cho ăn tự động, bật/tắt thiết bị hoặc hiệu chỉnh

ngưỡng cảnh báo. Các dữ liệu thu thập được từ lớp vật lý sẽ được ESP32-C3 Mini truyền qua lớp truyền thông (Communication Layer) bằng giao thức MQTT, sau đó Flask Server tiếp nhận và lưu trữ lên Cloud Firestore. Nhờ đó, toàn bộ quá trình thu thập – truyền – xử lý – hiển thị được thực hiện liên tục theo thời gian thực.

Kiến trúc ba lớp của mô hình AquaNova giúp phân tách rõ ràng vai trò giữa các tầng, đảm bảo tính mở rộng, khả năng bảo trì và độ ổn định cao. Mỗi lớp đều có thể được thay thế hoặc nâng cấp độc lập mà không ảnh hưởng đến toàn bộ mô hình, tạo điều kiện thuận lợi cho việc phát triển, mở rộng tính năng và tích hợp các dịch vụ IoT.

3.4. Thiết kế kiến trúc của mô hình



Hình 20: Sơ đồ khối của mô hình AquaNova

Mô hình được xây dựng bao gồm có 4 khối kết nối với nhau:

- Khối cảm biến: khối này dùng cảm biến đo siêu âm, servo, độ đục, nhiệt độ hiện việc theo dõi quan sát môi trường sống của cá và cảnh báo.
- (MCU) Microcontroller Unit: đây là khối trang bị các thuật toán, hàm để tương tác với dữ liệu được thu lại từ khối cảm biến và đồng thời chuyển dữ liệu đã xử lý đến khối điều khiển và khối hiển thị.
- Khối cơ cấu chấp hành: khối này bao gồm các relay, nhận các tín hiệu dùng cho mục đích điều khiển từ khối xử lý trung tâm để đóng cắt thiết bị điện bên ngoài.

- Khối hiển thị: là một màn hình LCD sử dụng để đưa ra các thông báo cho người dùng biết các thông tin đã và đang được xử lý trong khối xử lý trung tâm.
- Khối nguồn: là khối cấp nguồn 5VDC cho các khối cảm biến, khối xử lý trung tâm và khối điều khiển.
- Database: Hệ thống sử dụng các database để giao tiếp giữa MCU với Web.
- Web: một giao diện cơ bản được xây dựng từ HTML, CSS và JS, nhằm mục đích thu thập, xử lý và trực quan hóa các thông tin liên quan đến môi trường sống hồ cá như độ đục, nhiệt độ, lượng thức ăn.

Dựa trên yêu cầu đặt ra của sơ đồ tổng thể của hệ thống và các yêu cầu về mặt tính năng cũng như kỹ thuật, nhóm đã tìm hiểu, nghiên cứu và chọn ra các phần cứng phù hợp với từng khối, đáp ứng được các yêu cầu kể trên.

3.4.1. Khối nguồn

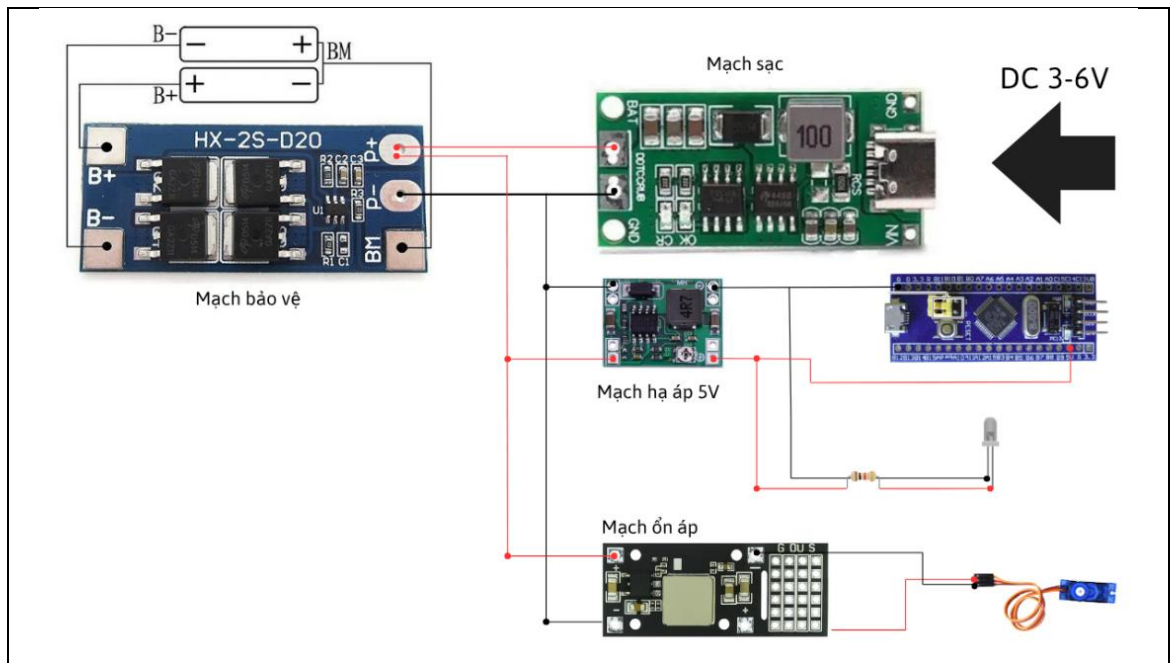
Trong mô hình hiện tại, toàn bộ hệ thống được gộp chung một nguồn pin Lithium, giúp giảm kích thước, khối lượng và sự phức tạp trong quản lý năng lượng. Tuy nhiên, để đảm bảo tính ổn định và độ sạch của điện áp cho từng loại tải, nguồn này được phân chia thành hai nhánh độc lập sau khi qua các mạch hạ áp chuyên dụng:

Nhánh 1 – Nguồn cho vi điều khiển STM32 và relay điều khiển đèn:

- Điện áp từ pin được hạ xuống 5V thông qua mạch MP1584, một bộ chuyển đổi xung (Buck Converter) hiệu suất cao.
- Nguồn 5V này cấp trực tiếp cho bo mạch STM32. Trên bo mạch, điện áp tiếp tục được ổn định xuống 3.3V bằng LDO tích hợp, đảm bảo dòng điện sạch, ít nhiễu cho nhân xử lý của vi điều khiển.
- Relay điều khiển đèn cũng được cấp từ nhánh 5V này, giúp đồng bộ hóa điều khiển và giảm số lượng nguồn phụ.

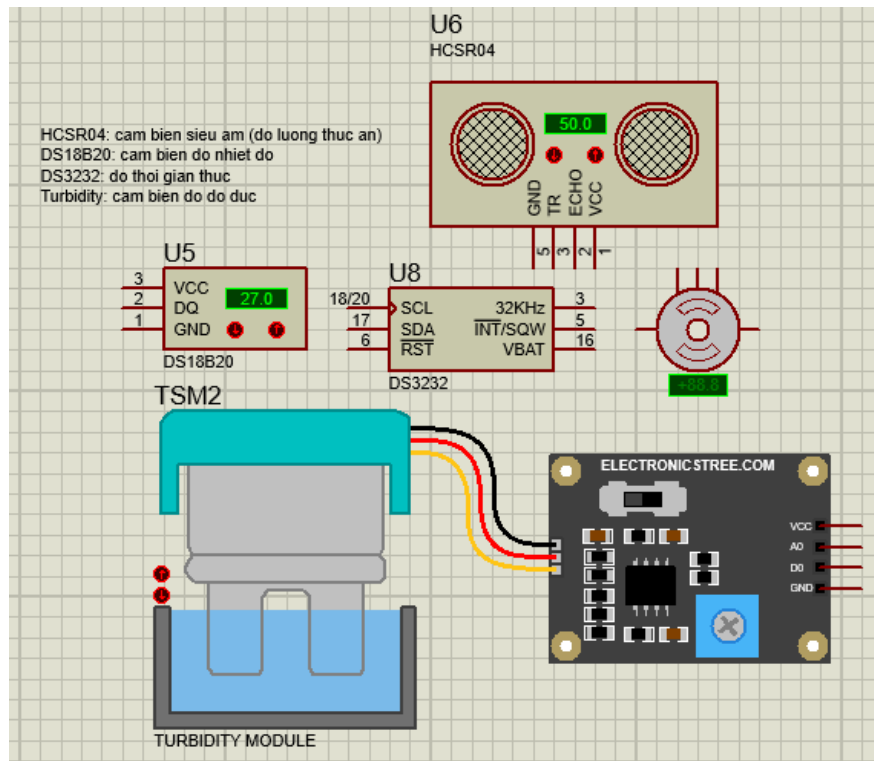
Nhánh 2 – Nguồn cho động cơ Servo:

- Servo yêu cầu dòng cao và độ ổn định tốt, nên được cấp qua mạch MP2482, hạ điện áp từ pin xuống mức phù hợp (thường 5–6V tùy loại servo).
- Việc tách riêng nhánh servo giúp tránh nhiễu dòng và sụt áp lan sang phần điều khiển khi động cơ hoạt động.



Hình 21: Khối cấp nguồn của mô hình AquaNova

3.4.2. Khối cảm biến



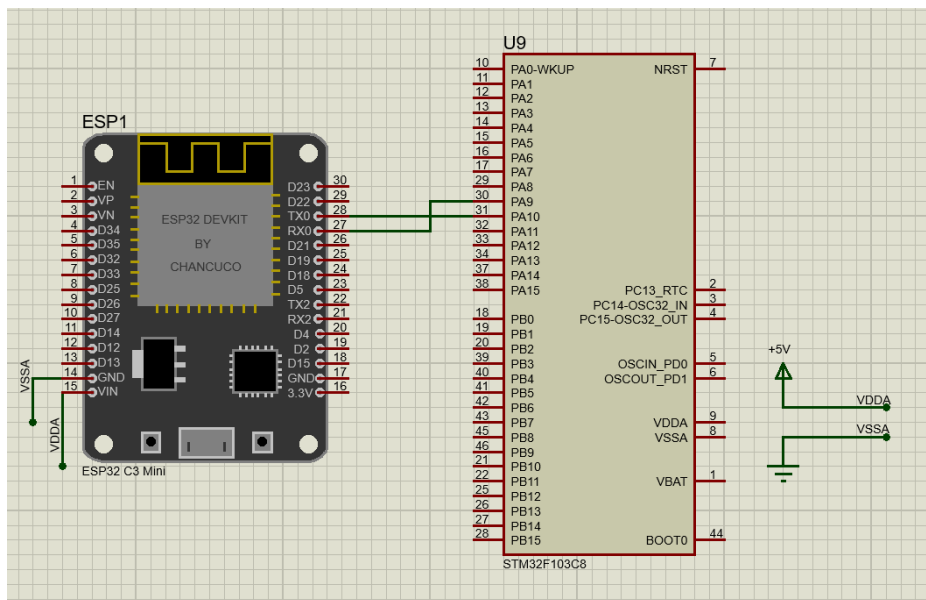
Hình 22: Sơ đồ các cảm biến trong mô hình

Khối cảm biến gồm các cảm biến chính: DS18B20 đo nhiệt độ nước, Turbidity sensor đo độ đục, và HC-SR04 đo lượng thức ăn. Các tín hiệu thu được được đưa về vi điều khiển STM32F103C8 để xử lý. Ngoài ra, mô hình còn tích hợp DS3231 RTC để

định thời và LCD I2C hiển thị thông số. Khi giá trị đo vượt ngưỡng, buzzer sẽ phát tín hiệu cảnh báo, giúp người dùng theo dõi tình trạng nước trong hồ cá một cách trực quan.

3.4.3. Khối xử lý trung tâm

Trong hệ thống, khối trung tâm bao gồm hai vi điều khiển chính là STM32 và ESP32. STM32 đảm nhiệm vai trò điều khiển và thu thập dữ liệu cảm biến, còn ESP32 đóng vai trò xử lý truyền thông mạng WiFi và gửi dữ liệu lên hệ thống IoT. Để hai vi điều khiển có thể trao đổi dữ liệu với nhau một cách hiệu quả, chuẩn giao tiếp UART (Universal Asynchronous Receiver – Transmitter) được lựa chọn làm phương thức kết nối chính. Mục đích của việc kết nối UART là để thiết lập một kênh truyền dữ liệu nối tiếp song công giữa STM32 và ESP32. UART là một trong những giao thức có kết nối đơn giản và hiệu quả nhất. Việc kết nối giữa STM32 và ESP32 chỉ cần hai dây cơ bản gồm: TX (Transmit) của STM32 nối với RX (Receive) của ESP32, RX của STM32 nối với TX của ESP32 chung để đảm bảo tín hiệu truyền ổn định. So với các giao thức khác như SPI (cần ít nhất 4 dây) hay I2C (đòi hỏi cấu hình phức tạp hơn), UART giúp tiết kiệm chân kết nối, đơn giản hóa mạch phần cứng. Nhờ vậy, việc phát hiện và khắc phục lỗi trong quá trình phát triển trở nên dễ dàng và nhanh chóng hơn.



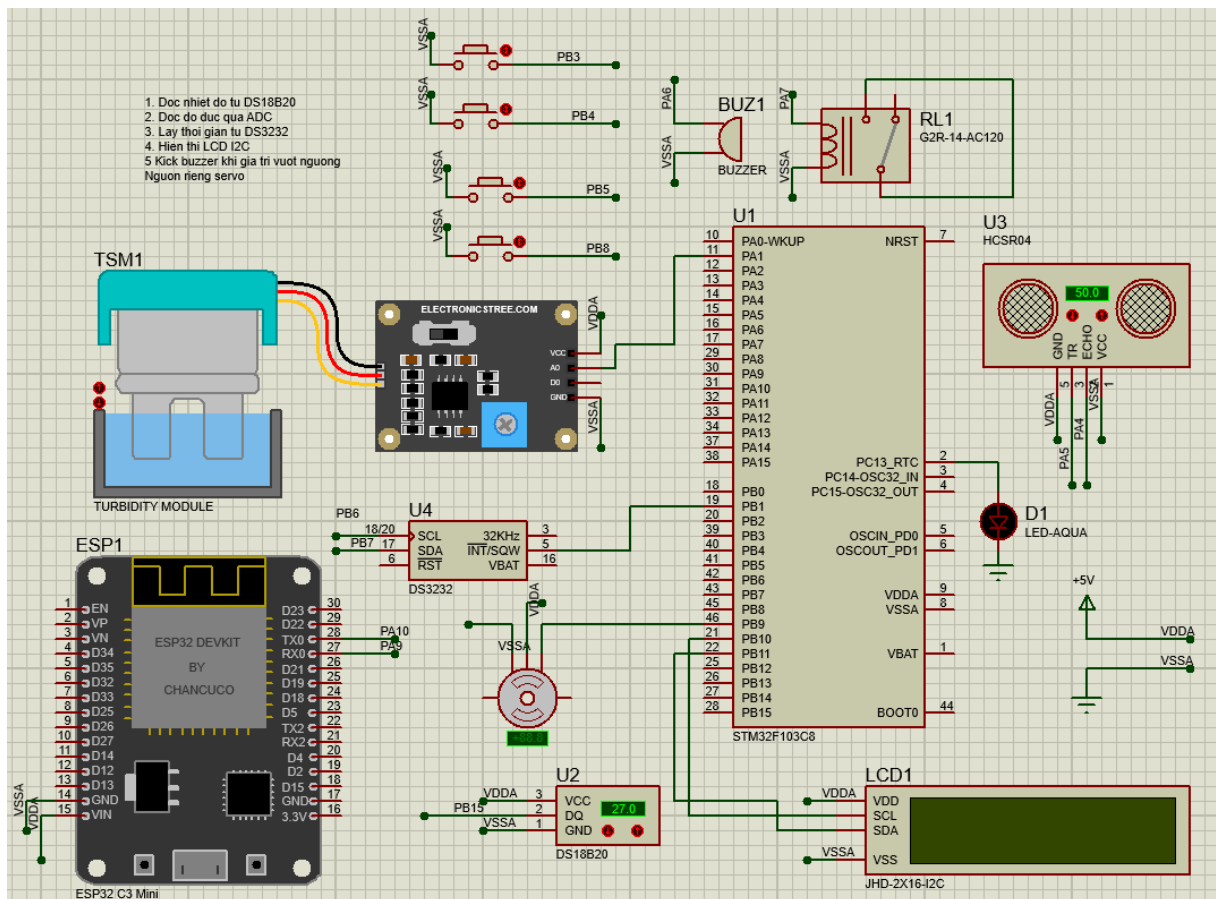
Hình 23: Sơ đồ kết nối UART giữa vi điều khiển STM32 và ESP32 trong mô hình AquaNova

3.4.4. Kết nối toàn bộ hệ thống

Sơ đồ nguyên lý thể hiện toàn bộ cấu trúc phần cứng của mô hình IoT AquaNova, bao gồm các khối cảm biến, vi điều khiển, truyền thông và cơ cấu chấp hành. Cảm biến DS18B20 đảm nhận đo nhiệt độ nước, cảm biến độ đục TSM1 đo độ trong của nước, và

cảm biến siêu âm HC-SR04 dùng để xác định mực nước trong hồ. Tín hiệu từ các cảm biến được đưa vào vi điều khiển STM32F103C8 để xử lý và hiển thị thông tin trên màn hình LCD I2C 16×2.

ESP32-C3 Mini được sử dụng làm bộ truyền thông IoT, đảm nhiệm việc gửi dữ liệu thu thập được lên máy chủ thông qua giao thức MQTT và nhận lệnh điều khiển từ web dashboard. Mô hình còn tích hợp module thời gian thực RTC DS3232 giúp đồng bộ lịch hẹn cho ăn và quản lý thời gian hoạt động. Các thiết bị ngoại vi như buzzer và relay được sử dụng để cảnh báo khi thông số vượt ngưỡng và điều khiển đèn hoặc servo cấp thức ăn. Sơ đồ này là cơ sở để tiến hành thiết kế mạch in (PCB) và triển khai hệ thống hoàn chỉnh trong giai đoạn tiếp theo.



Hình 24: Sơ đồ nguyên lý mô hình IoT AquaNova

3.5. Thiết kế cơ khí

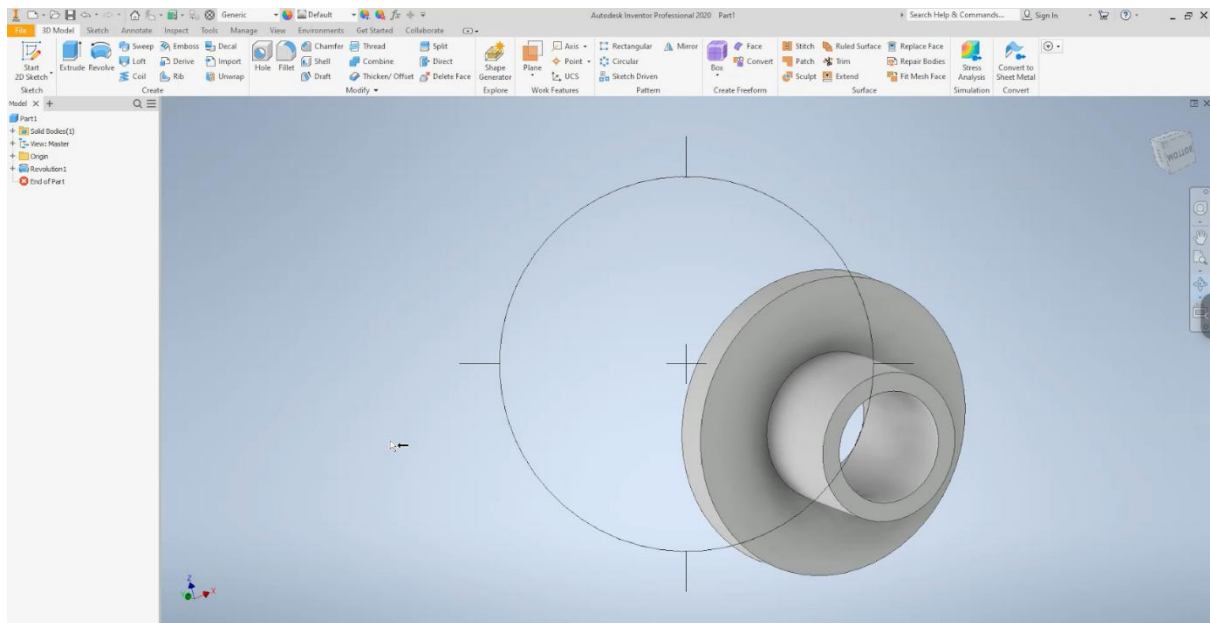
3.5.1. Cài đặt phần mềm



Hình 25: Phần mềm vẽ cơ khí

Để thực hiện việc thiết kế mô hình cơ khí cho hệ thống, nhóm sử dụng phần mềm Autodesk Inventor Professional, một công cụ chuyên dụng mạnh mẽ trong thiết kế 3D cơ khí. Phần mềm được cài đặt và cấu hình trên máy tính cá nhân với phiên bản mới nhất tương thích hệ điều hành Windows 10. Quá trình cài đặt gồm các bước cơ bản:

- Tải và cài đặt Autodesk Inventor từ trang chính thức của Autodesk Education để đảm bảo bản quyền và tính ổn định.
- Cấu hình ban đầu, lựa chọn đơn vị đo lường (millimeter), thư viện vật liệu, và môi trường làm việc dạng “Standard (mm)”.
- Tạo dự án thiết kế (Project) nhằm quản lý các file chi tiết, cụm lắp ráp và bản vẽ kỹ thuật trong cùng một thư mục.



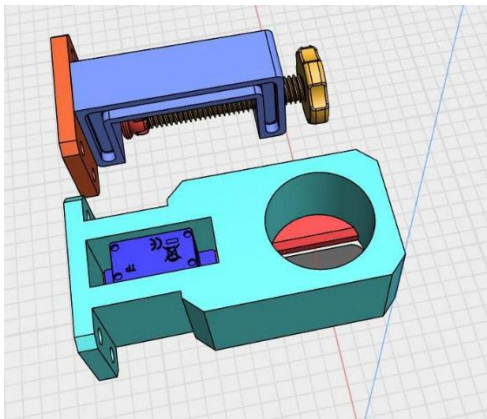
Hình 26: Giao diện phần mềm

Nhờ việc sử dụng Autodesk Inventor, nhóm có thể mô hình hóa chính xác các chi tiết cơ khí, kiểm tra va chạm, căn chỉnh vị trí lắp ráp, cũng như xuất trực tiếp bản vẽ kỹ thuật và file STL để phục vụ cho quá trình in 3D.

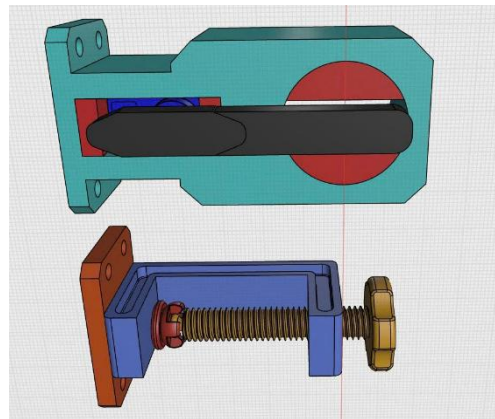
3.5.2. Chi tiết thiết kế

Phần cơ khí của hệ thống được thiết kế dựa trên nguyên tắc đơn giản – chắc chắn – dễ gia công, tận dụng trọng lực để cấp liệu và servo để điều khiển đóng mở dòng thức ăn. Tất cả các chi tiết đều được vẽ 3D trên Autodesk Inventor để đảm bảo độ chính xác và dễ dàng chỉnh sửa. Các chi tiết chính bao gồm:

- Phễu chứa (Hopper): Được tái sử dụng từ chai nhựa thông thường, có hình nón đảo ngược, giúp thức ăn dạng hạt tự động dồn xuống miệng chai nhờ trọng lực. Miệng chai được cắt gọn và mài nhẵn để đảm bảo bề mặt tiếp xúc phẳng van gạt.
- Bệ đỡ (Giá đỡ in 3D): Là chi tiết được thiết kế và in 3D bằng vật liệu PLA. Bệ gồm hai phần. Phần khung trên có vòng ôm giữ miệng chai, được thiết kế theo đường kính thực tế của chai và có ren hoặc chốt cố định. Phần thân dưới là vị trí gắn servo, có khoang vừa khít với kích thước servo SG90/MG90S, kèm hai lỗ bắt vít ở hai bên giúp cố định chắc chắn. Trục quay của servo được bố trí ngay tại miệng chai để dễ dàng điều khiển tấm gạt thức ăn.
- Tấm gạt (Van chặn): Được gắn trực tiếp vào servo horn thông qua ốc trung tâm. Tấm gạt được thiết kế mỏng, nhẹ, có thể làm bằng mica hoặc in 3D bằng PLA. Khi servo quay một góc 45° , tấm gạt sẽ tạo ra khe hở cho thức ăn rơi xuống; khi quay ngược lại, khe được đóng kín hoàn toàn.

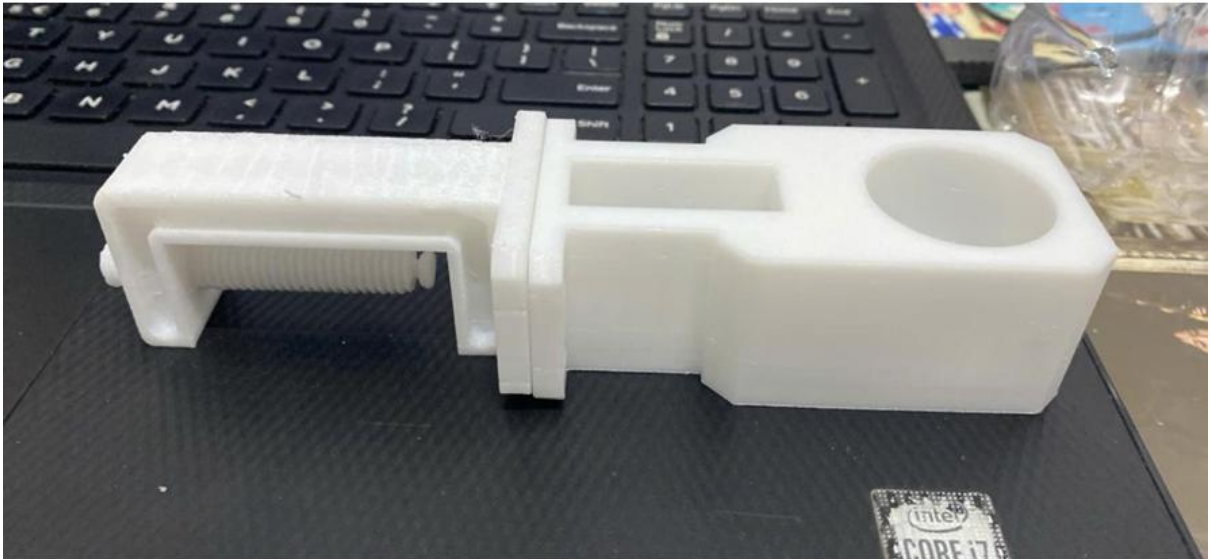


Hình 27: Hình vẽ 3d giá đỡ giữ thức ăn và servo và phần cố định (mặt trên)



Hình 28: Hình vẽ 3d giá đỡ giữ thức ăn và servo và phần cố định (mặt dưới)

Mô hình được thiết kế theo kiểu modular các chi tiết có thể tháo rời, dễ dàng thay thế, bảo trì hoặc nâng cấp. Sau khi hoàn tất, mô hình 3D được xuất ra định dạng .STL để tiến hành in 3D trên máy in Bambu.



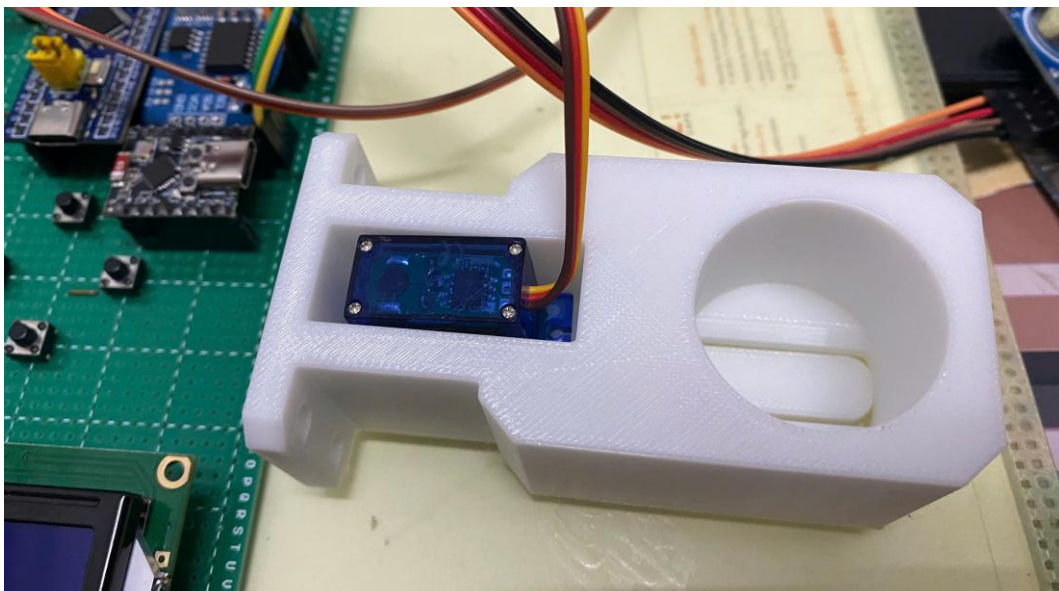
Hình 29: Giá đỡ phần giữ thức ăn và servo

3.5.3. Lắp ráp sản phẩm

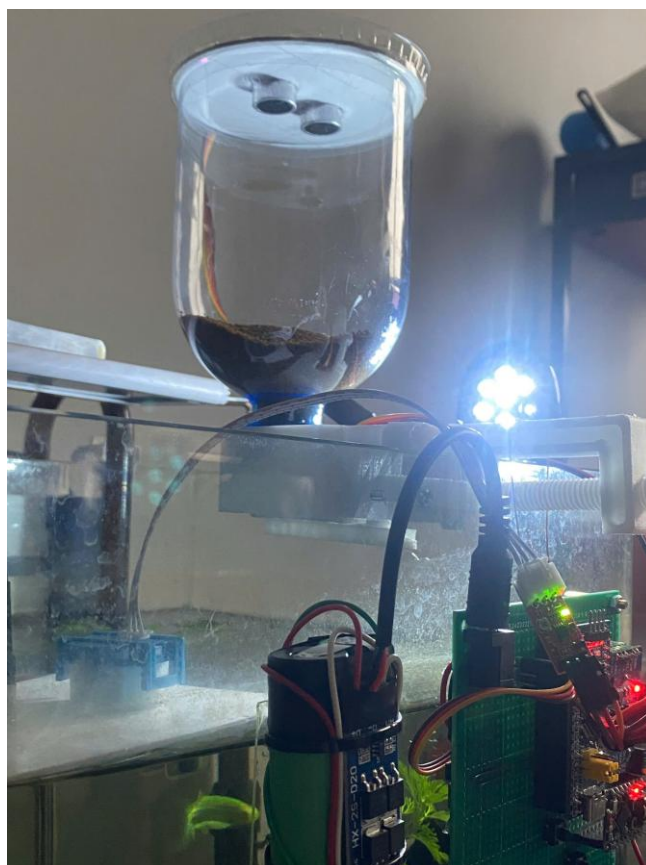
Sau khi các chi tiết in 3D hoàn thiện, nhóm tiến hành lắp ráp toàn bộ mô hình theo trình tự sau:

- Lắp servo vào bộ đỡ: Servo SG90 được đặt vào đúng khoang chứa, cố định bằng hai vít nhỏ qua tai gấn. Đảm bảo trục quay của servo hướng song song với mặt phẳng miệng chai.
- Gắn tấm gạt: Tấm gạt được vặn chặt vào servo horn bằng ốc trung tâm, căn chỉnh sao cho khi servo ở vị trí 0° , tấm gạt che kín hoàn toàn miệng chai; khi servo quay 90° , tấm gạt mở hoàn toàn khe thoát thức ăn.
- Cố định chai chứa: Phần miệng chai được luồn qua vòng giữ trên bộ đỡ và khóa chặt bằng ren hoặc chốt. Đảm bảo chai đứng vững và rung khi servo hoạt động.
- Kiểm tra cơ cấu: Nhóm vận hành thử nghiệm, quan sát góc mở, kiểm tra sự rơi của thức ăn và hiệu chỉnh lại vị trí servo nếu cần thiết.

Toàn bộ mô hình sau khi lắp ráp có kích thước nhỏ gọn, dễ đặt trên nắp bể cá. Cơ cấu hoạt động ổn định, servo quay nhẹ, không bị kẹt hay rung lắc. Thiết kế này đảm bảo tính thực tiễn, độ bền cao, đồng thời vẫn dễ bảo trì và tháo lắp khi cần.



Hình 30: Cơ khí cho ăn đã gắn servo và que chặn

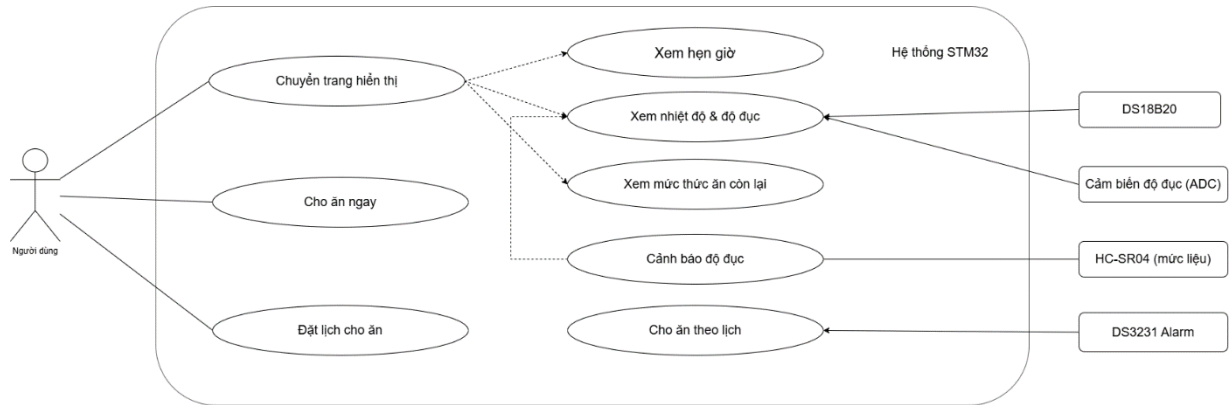


Hình 31: Hình ảnh cơ khí trong mô hình thực tế

3.6. Thiết kế nhúng

Thiết kế nhúng tập trung vào việc lập trình cho vi điều khiển STM32F103C8, đóng vai trò là bộ xử lý trung tâm của hệ thống. STM32 nhận dữ liệu từ các cảm biến (DS18B20 đo nhiệt độ, cảm biến độ đục đo chất lượng nước, HC-SR04 đo mực nước)

và xử lý để hiển thị thông số lên màn hình LCD I2C. Đồng thời, hệ thống được lập trình để cảnh báo khi giá trị vượt ngưỡng và điều khiển cơ cấu cho ăn tự động theo tín hiệu hẹn giờ từ DS3231 RTC hoặc lệnh thủ công của người dùng.



Hình 32: Use Case các tính năng của mô hình AuquaNova

Trong hình 25 mô tả người dùng có thể thực hiện các thao tác điều khiển và giám sát giao diện web hoặc trực tiếp trên thiết bị. Cụ thể:

- Xem thông tin cảm biến: Người dùng có thể theo dõi các giá trị nhiệt độ, độ đục và mực nước được STM32 thu thập từ các cảm biến DS18B20, TSM1 và HC-SR04. Các thông số này đồng thời được hiển thị lên màn hình LCD và cập nhật lên server qua module ESP32.
- Đặt lịch cho ăn: Cho phép người dùng tạo lịch cho ăn tự động thông qua giao diện web, hệ thống sẽ lưu lịch vào Firestore và thực thi lệnh dựa trên thời gian thực do DS3231 RTC cung cấp.
- Cho ăn ngay: Người dùng có thể gửi lệnh thủ công “Feed Now” để kích hoạt servo quay và cấp thức ăn cho cá.
- Cảnh báo vượt ngưỡng: Khi giá trị cảm biến vượt mức an toàn (ví dụ nhiệt độ cao, độ đục lớn hoặc mực nước thấp), hệ thống sẽ tự động kích hoạt cảnh báo bằng buzzer và hiển thị thông báo trên web dashboard.
- Chuyển trạng thái hệ thống: Người dùng có thể bật/tắt các thiết bị như đèn hoặc cơ cấu cho ăn thông qua MQTT để kiểm soát hệ thống linh hoạt.

3.6.1. Cài đặt phần mềm và cấu hình

Để cấu hình và lập trình cho vi điều khiển STM32, nhóm sử dụng phần mềm STM32CubeMX – một công cụ chính thức do STMicroelectronics cung cấp, hỗ trợ người dùng thiết kế, cấu hình và sinh mã tự động cho các dòng vi điều khiển STM32.

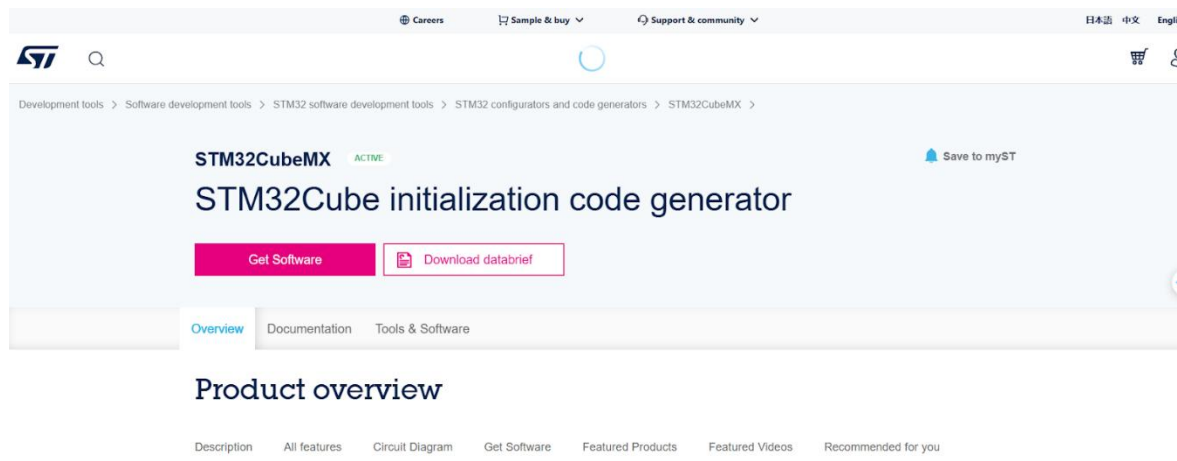
Phần mềm được cài đặt trên hệ điều hành Windows 10 và liên kết với KeilC để thực hiện biên dịch và nạp chương trình.



Hình 33: Logo các phần mềm code vi điều khiển STM32

Tải phần mềm:

- Truy cập trang chủ của STMicroelectronics tại địa chỉ <https://www.st.com>.
- Trong mục Tools & Software, tìm kiếm và tải về STM32CubeMX (phiên bản mới nhất tương thích với dòng chip STM32 đang sử dụng).
- Đăng ký hoặc đăng nhập tài khoản ST để kích hoạt quyền tải xuống.



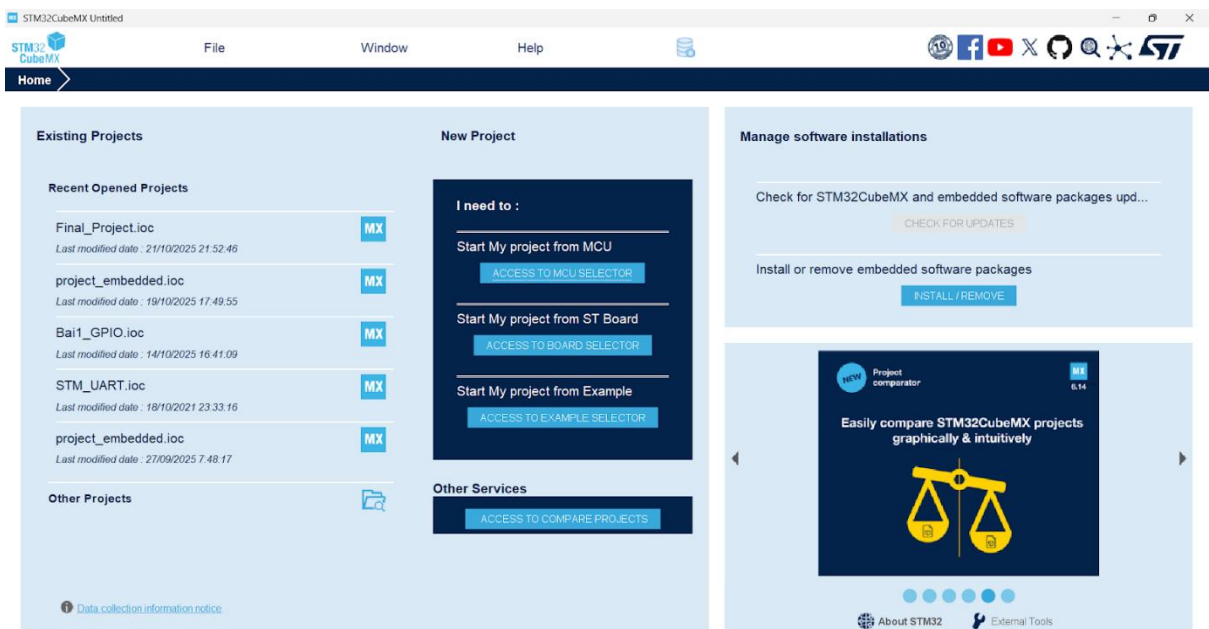
Hình 34: Giao diện tải STM32CubeMx trên web

Cài đặt:

- Sau khi tải xong, chạy file cài đặt .exe và tiến hành cài đặt theo hướng dẫn.
- Chọn thư mục lưu trữ và chấp nhận điều khoản sử dụng.
- Quá trình cài đặt hoàn tất sau vài phút, phần mềm có thể khởi động trực tiếp từ biểu tượng trên màn hình desktop.

Cấu hình ban đầu:

- Khi mở phần mềm lần đầu, người dùng được yêu cầu chọn thư mục lưu trữ mặc định cho các dự án.
- Kiểm tra và tải về gói thư viện MCU Packages tương ứng với dòng chip STM32 sử dụng (ví dụ: STM32F103RB, STM32F401RE, ...).
- Kiểm tra kết nối internet để phần mềm tự động cập nhật danh sách chip mới.
- Liên kết với môi trường phát triển:
- Trong mục Project Manager → Toolchain / IDE, chọn Keil C để xuất trực tiếp mã nguồn sang môi trường lập trình và biên dịch.
- Tùy chọn này giúp đồng bộ giữa CubeMX và Keil C.



Hình 35. Giao diện STM32CubeMx

Keil μ Vision tích hợp trình soạn thảo mã nguồn (code editor), trình biên dịch C/C++ (Arm Compiler), và bộ mô phỏng – nạp chương trình (debugger & programmer) trong cùng một môi trường. Phần mềm hỗ trợ trực tiếp các tệp mã được sinh ra từ STM32CubeMX, cho phép người dùng biên dịch và nạp chương trình lên vi điều khiển một cách thuận tiện.

Tải phần mềm:

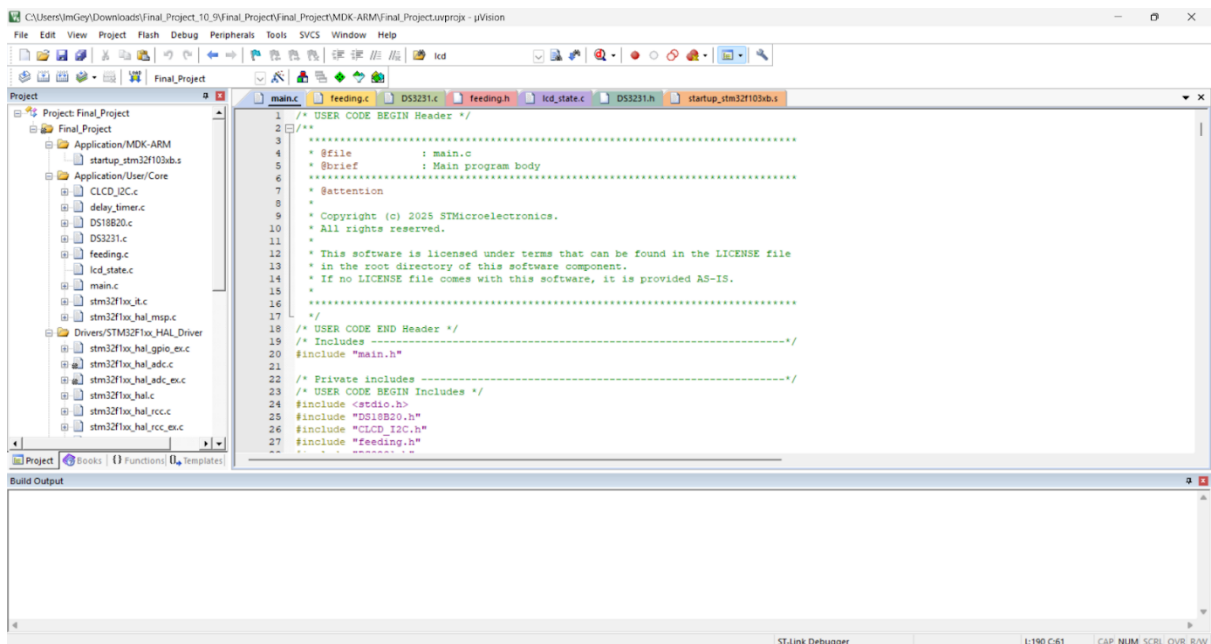
- Truy cập trang chính thức của Keil tại: <https://www.keil.com>.
- Chọn mục Downloads → MDK-ARM để tải phiên bản Keil μ Vision5 mới nhất.

Cài đặt:

- Mở tệp cài đặt .exe, chọn Next và chấp nhận các điều khoản sử dụng.
- Chọn đường dẫn cài đặt mặc định (thường là C:\Keil_v5).
- Sau khi hoàn tất, phần mềm có thể khởi chạy trực tiếp từ biểu tượng Keil μ Vision5 trên màn hình desktop.
- Cài đặt gói hỗ trợ thiết bị (Device Pack):
- Khi mở phần mềm lần đầu, vào menu Pack Installer để tải và cài đặt STM32 Series Device Family Pack tương ứng với dòng chip đang sử dụng (ví dụ: STM32F1, STM32F4, STM32L0...).
- Việc này giúp Keil nhận diện chính xác vi điều khiển, hỗ trợ biên dịch và nạp mã

Cấu hình trình nạp (Debugger/Programmer):

- Trong Project $\rightarrow$ Options for Target $\rightarrow$ Debug, chọn ST-Link Debugger nếu sử dụng ST-Link V2 để nạp chương trình.
- Kết nối bo mạch STM32 qua cáp USB, kiểm tra driver ST-Link đã được nhận dạng trong Device Manager.



Hình 36: Giao diện KeilC

Cấu hình cho các chân trong STM32CubeMx

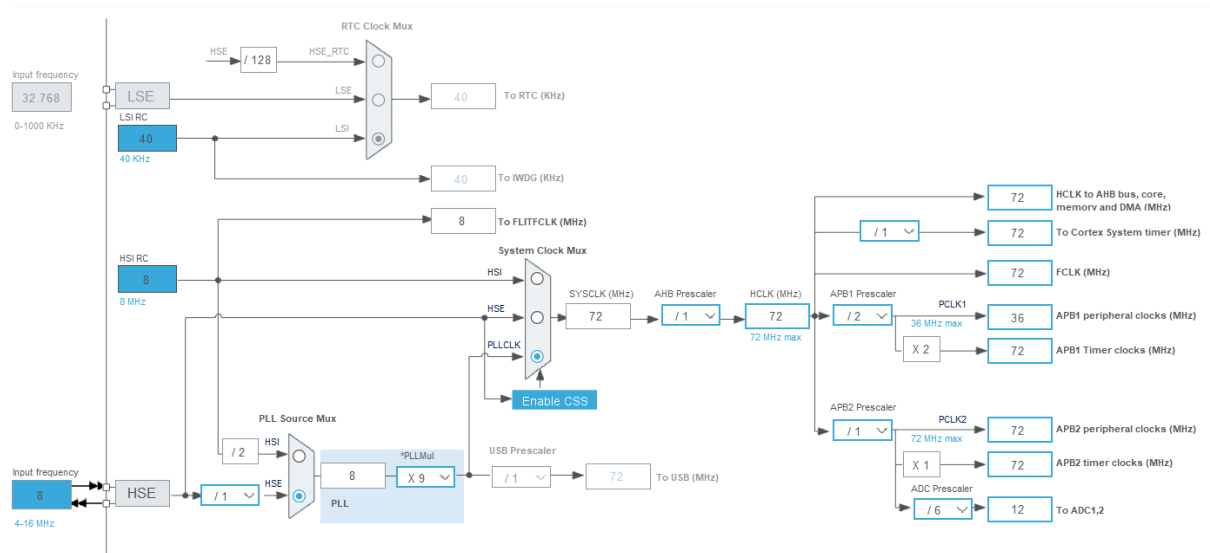
1. Thiết lập cơ bản hệ thống

- Vào mục System Core $\rightarrow$ SYS, tại phần Debug chọn Serial Wire để kích hoạt chức năng nạp và gỡ lỗi thông qua cổng SWD.

- Trong mục Timebase Source, chọn SysTick để sử dụng bộ đếm hệ thống làm nguồn định thời chính cho các tác vụ trong chương trình.

2. Cấu hình xung nhịp hệ thống (Clock Configuration)

- Trong phần RCC, mục High Speed Clock (HSE), chọn Crystal/Ceramic Resonator nhằm sử dụng thạch anh ngoài làm nguồn xung ổn định cho hệ thống.
- Vào Clock Configuration, trong ô HCLK (MHz) chỉnh thành 72.



Hình 37: Cấu hình Clock Configuration

3. Cấu hình các chân và ngoại vi

PA1: Chọn chức năng ADC1_IN1, đổi tên thành TURB (dùng để đọc tín hiệu cảm biến độ đục).

PC13: Chọn GPIO_Output, đổi tên thành LED (đèn báo trạng thái).

PB9: Chọn TIM4_CH4, đổi tên thành SERVO (điều khiển động cơ servo).

Trong phần Timers → TIM4, chỉnh:

- Prescaler = 144 - 1
- Counter Period = 10000 - 1
- Giúp tạo xung PWM phù hợp cho điều khiển servo.

PB8: Chọn GPIO_EXTI8, đổi tên thành BUTTON (nút nhấn chính).

PB5: Chọn GPIO_EXTI5, đổi tên thành BUTTON_UP (nút tăng giá trị).

PB4: Chọn GPIO_EXTI4, đổi tên thành BUTTON_DOWN (nút giảm giá trị).

PB3: Chọn GPIO_EXTI3, đổi tên thành BUTTON_CONTROL (nút điều khiển chính).

PA15: Chọn GPIO_EXTI15, đổi tên thành BUTTON_OFF (nút tắt hệ thống).

PB15: Chọn GPIO_Output, đổi tên thành DS18B20 (cảm biến nhiệt độ).

PB1: Chọn GPIO_EXTI1, đổi tên thành ALARM (ngắt cảnh báo).

PA5: Chọn GPIO_Output, đổi tên thành TRIG_HCSR04 (tín hiệu trigger cho cảm biến siêu âm).

PA6: Chọn GPIO_Output, đổi tên thành BUZZER (chuông cảnh báo).

PA7: Chọn GPIO_Output, đổi tên thành LIGHT (điều khiển đèn chiếu sáng).

PA4: Chọn GPIO_Input, đổi tên thành ECHO (tín hiệu phản hồi từ cảm biến siêu âm HCSR04).

Các nút nhấn (PB8, PB5, PB4, PB3, PB1) đều được cấu hình ở chế độ Pull-up và kích hoạt ngắt theo sườn lên (External Interrupt Mode with Rising Edge Trigger Detection) để đảm bảo tín hiệu ổn định, tránh nhiễu khi thao tác.

Pin Name	Signal on Pin	GPIO output L...	GPIO mode	GPIO Pull-up/...	Maximum out...	User Label	Modified
PB3	n/a	n/a	External Interr...	Pull-up	n/a	BUTTON_C...	✓
PB4	n/a	n/a	External Interr...	Pull-up	n/a	BUTTON_D...	✓
PB5	n/a	n/a	External Interr...	Pull-up	n/a	BUTTON_UP	✓
PB8	n/a	n/a	External Interr...	Pull-up	n/a	BUTTON	✓
PB15	n/a	Low	Output Push ...	No pull-up an...	Low	DS18B20	✓
PC13-TAMP...	n/a	Low	Output Push ...	No pull-up an...	Low	LED	✓

Pin Name	Signal on Pin	GPIO output L...	GPIO mode	GPIO Pull-up/...	Maximum out...	User Label	Modified
PA4	n/a	n/a	Input mode	No pull-up an...	n/a	ECHO	✓
PA5	n/a	Low	Output Push ...	No pull-up an...	Low	TRIG_HCSR...	✓
PA6	n/a	Low	Output Push ...	No pull-up an...	Low	BUZZER	✓
PA7	n/a	Low	Output Push ...	No pull-up an...	Low	LIGHT	✓
PA15	n/a	n/a	External Interr...	Pull-up	n/a	BUTTON_OFF	✓
PB1	n/a	n/a	External Interr...	Pull-up	n/a	ALARM	✓

Hình 38: Cấu hình GPIO

Trong phần NVIC, bật các ngắt tương ứng tại EXTI line 1, line 3, line 4, line [9:5], và line [15:10] (mức ưu tiên = 0). Đồng thời bật USART1 Global Interrupt với mức ưu tiên = 1.

4. Cấu hình giao tiếp và cảm biến

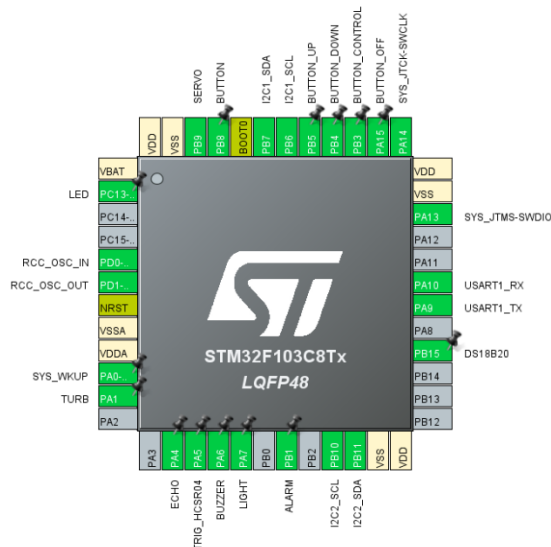
Trong mục Connectivity:

NVIC			
Code generation			
RCC global interrupt	☐	0	0
EXTI line1 interrupt	☒	0	0
EXTI line3 interrupt	☒	0	0
EXTI line4 interrupt	☒	0	0
ADC1 and ADC2 global interrupts	☐	0	0
EXTI line[9:5] interrupts	☒	0	0
TIM1 break interrupt	☐	0	0
TIM1 update interrupt	☐	0	0
TIM1 trigger and commutation interrupts	☐	0	0
TIM1 capture compare interrupt	☐	0	0
TIM2 global interrupt	☐	0	0
TIM4 global interrupt	☐	0	0
I2C1 event interrupt	☐	0	0
I2C1 error interrupt	☐	0	0
I2C2 event interrupt	☐	0	0
I2C2 error interrupt	☐	0	0
USART1 global interrupt	☒	1	0
EXTI line[15:10] interrupts	☒	0	0

Hình 39: Cấu hình ngắt

- I2C1: Bật chế độ I2C để giao tiếp với module DS3231 (đồng hồ thời gian thực).
- I2C2: Bật chế độ I2C cho LCD 1602/2004 (hiển thị thông tin).
- USART1: Chọn chế độ Asynchronous, dùng cho giao tiếp UART với ESP32 hoặc máy tính.
 - Baud Rate = 9600
 - Word Length = 8 bits
 - Stop Bits = 1

Sau khi hoàn tất việc gán chân và thiết lập các thông số, chọn Project → Generate Code, sau đó mở dự án bằng Keil μVision để tiến hành viết mã điều khiển và thử nghiệm.

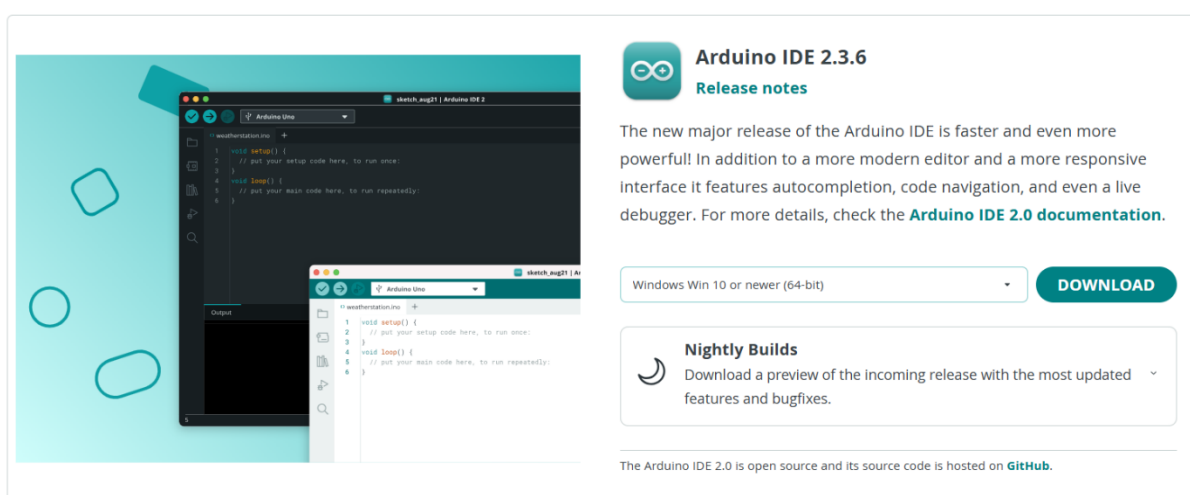


Hình 40: Sơ đồ chân sau khi đã cấu hình trong STM32CubeMx

Trong mô hình này, Arduino IDE (phiên bản 2.3.6) được lựa chọn làm môi trường phát triển chính cho vi điều khiển ESP32. Lý do lựa chọn nền tảng này bao gồm: tính phổ biến cao, cộng đồng người dùng rộng rãi, khả năng hỗ trợ tốt cho các dòng vi điều khiển IoT, cùng quy trình thiết lập đơn giản và ổn định.

Quy trình cài đặt Arduino IDE:

- Truy cập trang web chính thức của Arduino tại: <https://www.arduino.cc/en/software>.
- Tải xuống phiên bản Arduino IDE 2.3.6 (hoặc phiên bản mới nhất) phù hợp với hệ điều hành đang sử dụng (Windows, macOS hoặc Linux).
- Thực hiện cài đặt theo hướng dẫn của trình cài đặt.



Hình 41: Giao diện trang web chính thức tải phần mềm Arduino IDE

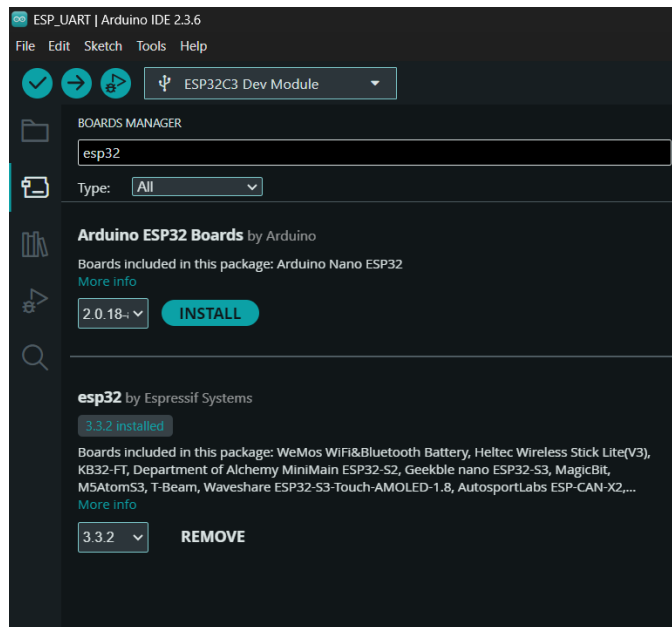
Cài đặt gói hỗ trợ (Board Manager) cho ESP32: để có thể biên dịch và nạp mã nguồn cho họ vi điều khiển ESP32, người dùng cần cài đặt gói hỗ trợ bo mạch của Espressif Systems thông qua Boards Manager của Arduino IDE.

Các bước thực hiện:

1. Mở Arduino IDE, điều hướng đến File > Preferences.
2. Trong mục “Additional Boards Manager URLs”, thêm đường dẫn sau:

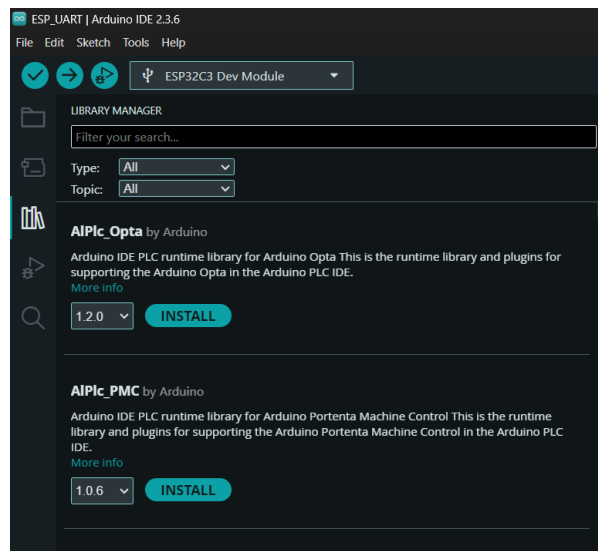
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

3. Tiếp tục vào Tools > Board > Boards Manager, tìm kiếm “esp32”, sau đó chọn và cài đặt gói “esp32 by Espressif Systems”.



Hình 42: Cài đặt gói hỗ trợ ESP32 trong Arduino IDE – Board Manager

Cài đặt thư viện phụ trợ: để mở rộng chức năng cho ESP32 và tích hợp các giao thức truyền thông, một số thư viện bên thứ ba được sử dụng và cài đặt Library Manager.



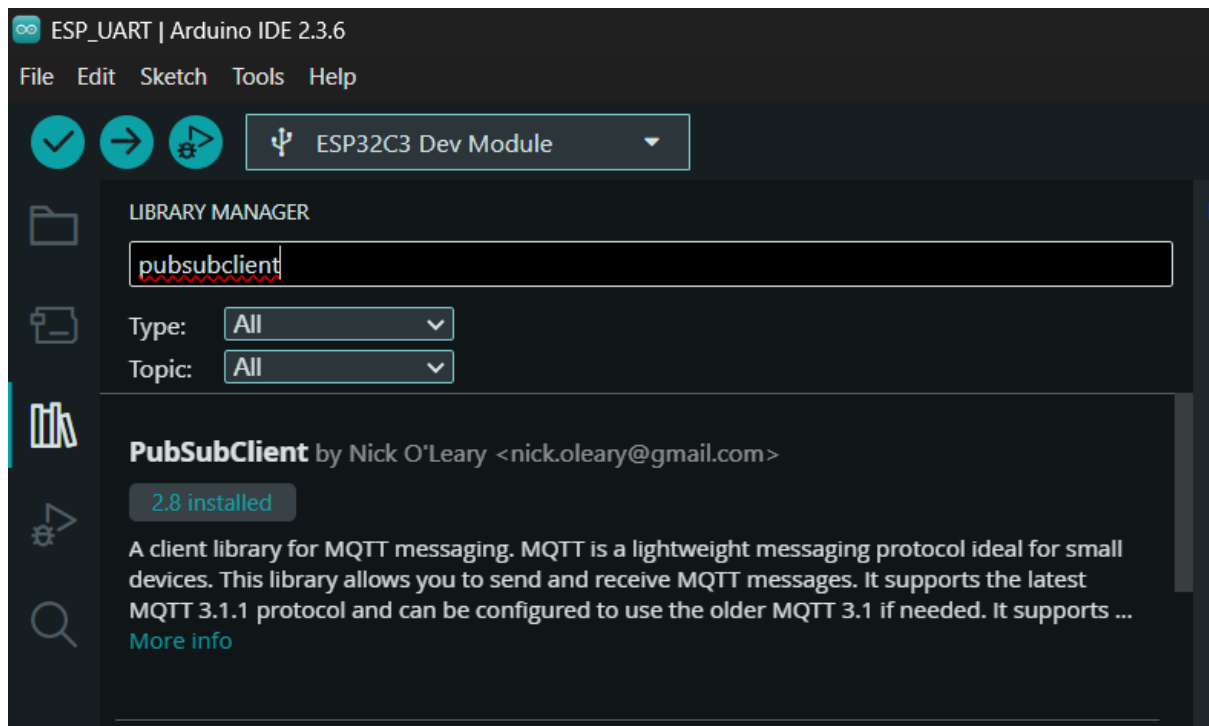
Hình 43: Cài đặt thư viện phụ trợ ESP32 trong Arduino IDE – Library Manager

Một số thư viện chính được sử dụng trong dự án được liệt kê qua bảng sau:

Tên thư viện	Tác giả	Chức năng chính
ArduinoJson	Benoît Blanchon	Tuần tự hóa dữ liệu cảm biến sang định dạng JSON và giải tuần tự gói tin JSON nhận từ máy chủ MQTT.

PubSubClient	Nick O'Leary	Cung cấp API gọn nhẹ cho giao thức MQTT, hỗ trợ kết nối, đăng ký (subscribe) và xuất bản (publish) dữ liệu lên broker.
NTPClient	Fabrice Weinberg	Truy vấn máy chủ NTP để đồng bộ hóa thời gian hệ thống theo giao thức UDP, đảm bảo độ chính xác thời gian.

Hình 44: Một số thư viện chính dùng trong dự án



Hình 45: Cài đặt thư viện PubSubClient qua Trình quản lý Thư viện trong Arduino IDE

Cấu hình tham số trong mã nguồn: trước khi tiến hành biên dịch và nạp chương trình, cần điều chỉnh các tham số trong mã nguồn để đảm bảo hệ thống hoạt động chính xác theo môi trường triển khai thực tế. Các tham số này liên quan đến cấu hình phần cứng (chân kết nối), thiết lập mạng không dây, dịch vụ điện toán đám mây (MQTT Broker), đồng bộ thời gian (NTP) và chu kỳ truyền dữ liệu cảm biến.

- Cấu hình giao tiếp UART giữa ESP32 và STM32.

```
const int STM32_RX_PIN = 20; // Chân RX của ESP32 (kết nối với TX của STM32)
const int STM32_TX_PIN = 21; // Chân TX của ESP32 (kết nối với RX của STM32)
```


Đây là phần cấu hình chân giao tiếp UART cho vi điều khiển ESP32-C3, nhằm thiết lập kênh truyền thông nối tiếp bất đồng bộ với vi điều khiển STM32.

- STM32_RX_PIN (GPIO20) là chân đầu vào nhận dữ liệu (RX) trên ESP32, được nối với chân TX (truyền) của STM32.
- STM32_TX_PIN (GPIO21) là chân đầu ra phát dữ liệu (TX) trên ESP32, được nối với chân RX (nhận) của STM32.

Giao tiếp UART cho phép hai vi điều khiển truyền dữ liệu song công (full-duplex) theo dạng chuỗi ký tự ASCII, giúp giảm độ trễ và tiết kiệm tài nguyên phần cứng so với giao thức phức tạp hơn như SPI hoặc I²C.

Việc xác định rõ chân GPIO giúp đảm bảo dữ liệu cảm biến từ STM32 được truyền chính xác sang ESP32 để xử lý và gửi lên đám mây. Tín hiệu điều khiển từ MQTT có thể phản hồi ngược lại STM32 mà không bị xung đột phần cứng cũng như dễ dàng mở rộng sang nhiều UART nếu cần mở rộng chức năng trong tương lai.

- Cấu hình kết nối WiFi.

```
const char* ssid = "Wifi";  
const char* password = "11111111";
```

Khai báo thông tin truy cập mạng WiFi mà ESP32 sẽ sử dụng để kết nối Internet.

- Ssid (Service Set Identifier) là tên định danh của mạng không dây nội bộ.
- Password là khóa bảo mật được sử dụng trong quá trình xác thực WPA2.

Khi chương trình khởi động, ESP32 sẽ gọi hàm WiFi.begin(ssid, password) để khởi tạo quá trình kết nối. Việc sử dụng mạng WiFi nội bộ giúp thiết bị gửi dữ liệu cảm biến đến máy chủ đám mây qua giao thức MQTT và nhận lệnh điều khiển từ xa qua Internet.

- Cấu hình MQTT Broker.

```
const char* mqtt_server =  
"fdca86b156bf4f86973c13d0a7fe4e2c.s1.eu.hivemq.cloud";  
const int mqtt_port = 8883;  
const char* mqtt_user = "aquanova_user";  
const char* mqtt_pass = "Aquanova123";
```

Định nghĩa các tham số cần thiết để ESP32 thiết lập kết nối với máy chủ MQTT Broker trên nền tảng HiveMQ Cloud.

- mqtt_server: Tên miền định danh máy chủ MQTT.
- mqtt_port: Cổng kết nối 8883 được chuẩn hóa cho giao thức MQTT bảo mật (SSL/TLS).

- mqtt_user / mqtt_pass: Thông tin xác thực người dùng, được cấu hình trên giao diện quản lý của HiveMQ.

Cấu hình đồng bộ thời gian (NTP)

```
WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", 25200); // 25200 = 7 * 60 * 60 (UTC+7)
```

Khởi tạo đối tượng NTPClient – một thư viện cho phép truy vấn máy chủ Network Time Protocol (NTP) để đồng bộ hóa thời gian hệ thống.

- "pool.ntp.org" là máy chủ NTP toàn cầu cung cấp thời gian chính xác theo chuẩn UTC.
- 25200 giây (7 × 3600) tương ứng với múi giờ UTC+7, tức Giờ Đông Dương (ICT) được sử dụng tại Việt Nam.
- Đối tượng ntpUDP đảm nhiệm việc gửi và nhận gói tin NTP qua giao thức UDP (User Datagram).

Cấu hình tần suất gửi dữ liệu.

```
const unsigned long SEND_INTERVAL = 60000;
```

Biến SEND_INTERVAL xác định chu kỳ gửi dữ liệu cảm biến lên máy chủ MQTT, đơn vị tính bằng mili-giây (ms).

- 60.000 ms = 60 giây ⇒ thiết bị sẽ gửi dữ liệu mỗi 30 giây một lần.
- Khoảng thời gian này được lựa chọn dựa trên sự cân bằng giữa độ cập nhật dữ liệu và tiêu thụ năng lượng mạng.

3.6.2. STM32

a) Module độ đục

Cảm biến MJKDZ sử dụng nguyên lý tán xạ ánh sáng để xác định

- Khi nước trong, lượng ánh sáng tán xạ ít thì điện áp đầu ra cao.
- Khi nước đục, ánh sáng bị tán xạ nhiều thì điện áp đầu ra giảm.

Điện áp đầu ra này được STM32 đọc qua bộ chuyển đổi ADC, sau đó xử lý và quy đổi ra giá trị NTU (Nephelometric Turbidity Unit) theo công thức tuyến tính được hiệu chuẩn thực nghiệm. Giá trị trung bình sau đó được chuyển đổi sang điện áp thực tế:

$$V = \frac{ADC \times 3.3}{4095} \quad (2)$$

Tiếp đến, chương trình áp dụng phương trình hồi quy bậc hai để quy đổi điện áp đo được sang giá trị độ đục (NTU):

$$\text{Turbidity} = -900.4 \times V^2 - 336.302 \times V + 2973.295 \quad (3)$$

Quy trình được mô tả qua Hình 4higf6 như sau:

- Bắt đầu đọc ADC bằng hàm HAL_ADC_Start() để khởi tạo bộ chuyển đổi.
- Chuyển đổi tín hiệu analog sang giá trị số (0–4095).
- Lấy trung bình nhiều mẫu để loại bỏ nhiễu (thường là 200 mẫu).
- Chuyển đổi giá trị ADC sang điện áp thực tế.
- Tính toán độ đục NTU theo công thức thực nghiệm.
- Giới hạn giá trị hợp lý (0–1000 NTU) để đảm bảo độ ổn định của dữ liệu.

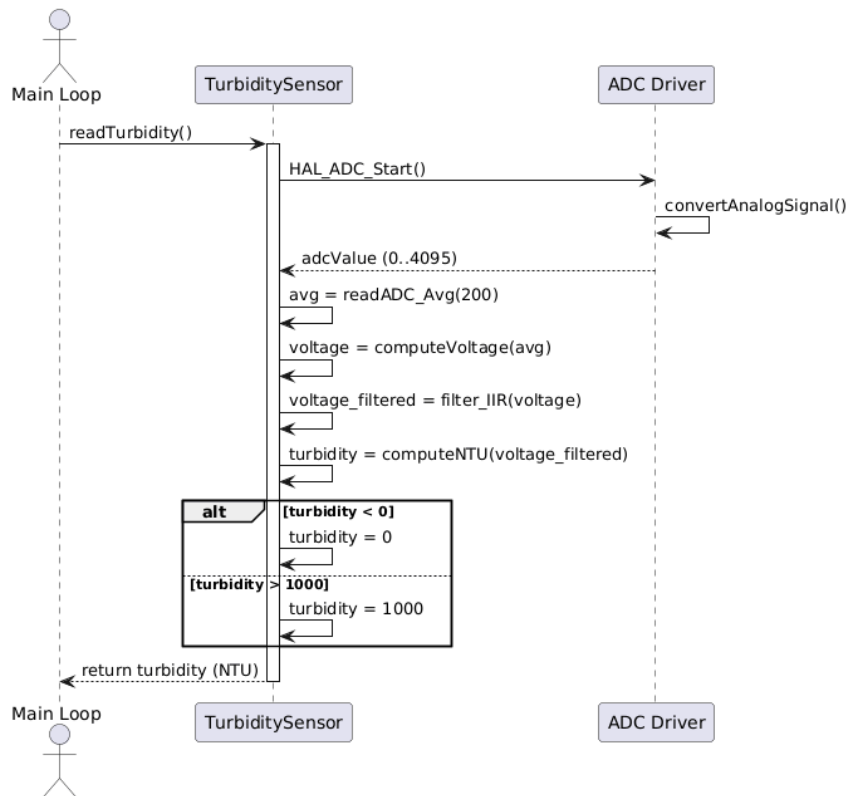
Cấu trúc biến toàn cục: Các biến trên dùng để lưu trữ giá trị ADC, điện áp, độ đục và trạng thái cảnh báo vượt ngưỡng.

```
uint32_t adcValue;  
float voltage, turbidity;  
// Biến cảnh báo  
uint8_t is_turbidity_warning_active = 0;  
uint8_t is_warning_silenced_by_user = 0;  
uint32_t last_beep_toggle_time = 0;
```

Đọc giá trị ADC: Hàm Read_ADC() khởi tạo, đọc và trả về giá trị ADC

```
uint16_t Read_ADC(void) {  
    uint16_t val = 0;  
    HAL_ADC_Start(&hadc1);  
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK) {  
        val = HAL_ADC_GetValue(&hadc1);  
    }  
    HAL_ADC_Stop(&hadc1);  
    return val;  
}
```

Lấy trung bình nhiều mẫu ADC: để tăng độ chính xác, hàm Read_ADC_Avg() được dùng để đọc nhiều mẫu và tính trung bình. Gọi Read_ADC_Avg(200) để lấy 200 mẫu, giúp triệt nhiễu đáng kể.



Hình 46: Sơ đồ tuần tự cho module cảm biến đo độ đục MJKDZ

```

uint32_t Read_ADC_Avg(uint8_t samples) {
    uint32_t sum = 0;
    for(uint8_t i = 0; i < samples; i++) {
        HAL_ADC_Start(&hadc1);
        HAL_ADC_PollForConversion(&hadc1, 10);
        sum += HAL_ADC_GetValue(&hadc1);
        HAL_ADC_Stop(&hadc1);
    }
    return sum / samples;
}
  
```

Xử lý và tính toán độ đục: Hàm process_turbidity() thực hiện toàn bộ quá trình xử lý giá trị ADC, chuyển đổi sang điện áp và tính toán độ đục (NTU):

```

void process_turbidity(void) {
    uint32_t avg_adc = Read_ADC_Avg(200);
    float voltage_raw = (avg_adc * 3.3f) / 4095.0f;
    float voltage = voltage_raw + 0.01f; // hiệu chỉnh nhiễu
    turbidity = -900.4f * (voltage * voltage) - 336.302f * voltage + 2973.295f;
    if (turbidity < 0) {
        turbidity = 0;
    } else if (turbidity > 1000) {
  
```

```

        turbidity = 1000;
    }
}

```

Trong đó: voltage_raw là giá trị điện áp đọc được.

- Công thức được xây dựng từ dữ liệu hiệu chuẩn thực nghiệm của cảm biến.
- Nếu tín hiệu bất thường (điện áp âm hoặc vượt quá dải đo), hệ thống sẽ tự động hiệu chỉnh về mức an toàn.

Cảnh báo và hiển thị dữ liệu: giá trị turbidity sau khi tính toán sẽ được gửi đến LCD I2C để hiển thị và truyền qua ESP32-MQTT lên Dashboard web. Nếu độ đục vượt quá ngưỡng an toàn (ví dụ > 200 NTU), chương trình sẽ bật còi cảnh báo:

```

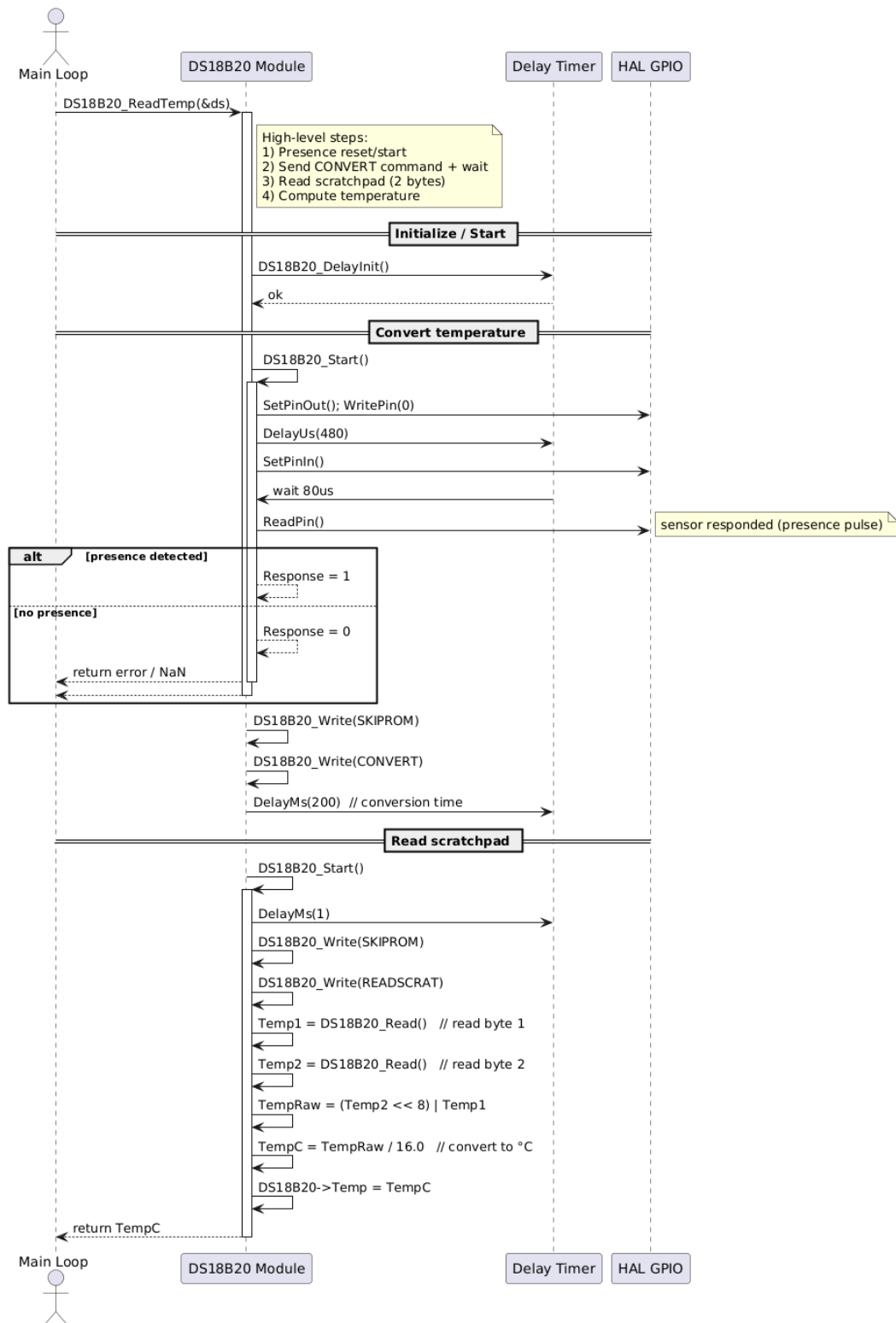
if (turbidity > 200 && !is_warning_silenced_by_user) {
    HAL_GPIO_WritePin(BUZZER_GPIO_Port, BUZZER_Pin, GPIO_PIN_SET);
    is_turbidity_warning_active = 1;
} else {
    HAL_GPIO_WritePin(BUZZER_GPIO_Port, BUZZER_Pin,
GPIO_PIN_RESET);
    is_turbidity_warning_active = 0;
}

```

b) Module nhiệt độ

Sơ đồ tuần tự mô tả quá trình đo và đọc nhiệt độ từ cảm biến DS18B20 trong chương trình. Khi hàm DS18B20_ReadTemp(&ds) được gọi từ Main Loop, modun DS18B20 tiến hành các bước chính gồm: khởi tạo, chuyển đổi nhiệt độ, đọc dữ liệu và tính toán.

Khi hàm DS18B20_ReadTemp() được kích hoạt từ Main Loop, mô-đun DS18B20 trước tiên thực hiện giai đoạn khởi tạo. Trong giai đoạn này, hàm DS18B20_DelayInit() được gọi để chuẩn bị bộ định thời, đảm bảo các khoảng trễ cần thiết được thiết lập chính xác. Sau đó, vi điều khiển gửi tín hiệu reset thông qua SetPinOut() và WritePin(0) nhằm yêu cầu cảm biến phản hồi tín hiệu. Cảm biến DS18B20 sẽ phản hồi bằng cách kéo mức điện áp xuống trong một khoảng thời gian ngắn. Việc có hoặc không có phản hồi này giúp hệ thống xác định xem cảm biến đang hoạt động bình thường hay không. Nếu phát hiện được tín hiệu hiện diện, biến phản hồi Response được gán bằng 1 và ngược lại, nếu không có tín hiệu, hàm sẽ trả về giá trị lỗi (NaN) và kết thúc quá trình đọc.



Hình 47: Sơ đồ tuần tự cho module cảm biến nhiệt độ DS18B20

Tiếp theo, hệ thống gửi lệnh SKIP ROM và CONVERT T để yêu cầu cảm biến thực hiện đo nhiệt độ. Sau thời gian chờ khoảng 200 ms cho quá trình chuyển đổi, mô-đun gửi tiếp các lệnh SKIP ROM và READ SCRATCHPAD để đọc hai byte dữ liệu nhiệt

độ lưu trong bộ nhớ tạm của cảm biến. Hai byte này được ghép lại thành giá trị thô TempRaw, sau đó chuyển đổi sang độ C bằng công thức:

$$TempC = \frac{TempRaw}{16.0} \quad (4)$$

Giá trị kết quả được lưu vào biến DS18B20->Temp và trả về cho Main Loop để hiển thị hoặc xử lý tiếp theo. Thư viện đo nhiệt độ

Tầng	Hàm tiêu biểu	Chức năng chính
Low Level (Hardware)	DS18B20_SetPinOut/In(), DS18B20_ReadPin(), DS18B20_DelayUs()	Điều khiển trực tiếp chân GPIO, tạo xung delay chính xác ở cấp vi giây
Middle Level (Protocol)	DS18B20_Start(), DS18B20_Write(), DS18B20_Read()	Thực thi giao thức 1-Wire (khởi tạo, đọc/ghi bit, nhận phản hồi presence)
High Level (Application)	DS18B20_Init(), DS18B20_ReadTemp()	Hàm ứng dụng cao nhất – khởi tạo cảm biến và đọc giá trị nhiệt độ thực tế

Luồng đo nhiệt độ

1. Reset / Presence:

- Master (MCU) kéo chân DQ xuống ~480 μs để reset bus 1-Wire, sau đó thả (đổi chân về Input).
- Slave (DS18B20) trả lời bằng một nhịp LOW ~60–240 μs (presence pulse) để báo cáo “tôi có mặt”

```
uint8_t Response = 0;
DS18B20_SetPinOut(DS18B20);    //GPIO sang Output (keo)
DS18B20_WritePin(DS18B20, 0);
DS18B20_DelayUs(DS18B20, 480);
DS18B20_SetPinIn(DS18B20);     // Thả về Input (pull-up kéo lên)
DS18B20_DelayUs(DS18B20, 80);
if (!(DS18B20_ReadPin(DS18B20))) Response = 1;
else Response = 0;
DS18B20_DelayUs(DS18B20, 400); // 480 us delay totally., kết
thúc khung reset
if (!(DS18B20_ReadPin(DS18B20))) Response = 1;
else Response = 0;
```

Nếu chân đang LOW thì Response = 1 (phát hiện presence); nếu chân HIGH thì Response = 0. Đúng với 1-Wire vì presence pulse của DS18B20 là active-LOW.

2. Convert: Gửi Skip ROM (0xCC) (vì bus có 1 cảm biến) rồi Convert T (0x44) để yêu cầu DS18B20 đo nhiệt độ bên trong nó.

```
DS18B20_Start(DS18B20); //Bắt đầu một phiên
DS18B20_DelayMs(DS18B20, 1);
DS18B20_Write(DS18B20, DS18B20_SKIPROM); //Skip ROM , Define
SKIPROM 0xCC
DS18B20_Write(DS18B20, DS18B20_CONVERT); //Convert T, Bắt đầu
đo, Define CONVERT 0x44
DS18B20_DelayMs(DS18B20, 200); //Chờ cố định 200ms
```

Khi nhận 0x44, DS18B20 tự đo và lưu kết quả vào scratchpad (bộ nhớ trong).

3. Read Scratchpad: sau khi đo xong, ta reset lại, gửi Skip ROM rồi Read Scratchpad (0xBE) và đọc 2 byte đầu (LSB, MSB).

```
DS18B20_Start(DS18B20);
DS18B20_DelayMs(DS18B20, 1);
DS18B20_Write(DS18B20, DS18B20_SKIPROM); //Skip ROM
DS18B20_Write(DS18B20, DS18B20_READSCRAT); //Read Scratchpad
Temp1 = DS18B20_Read(DS18B20); //lsb , byte 0
Temp2 = DS18B20_Read(DS18B20); //msb , byte 1
```

- Ghi bit '1': kéo LOW ~1 μ s, rồi thả (để HIGH) phần còn lại của slot
- Ghi bit '0': giữ LOW ~50 μ s, rồi thả.

```
static void DS18B20_Write(DS18B20_Name* DS18B20, uint8_t Data)
{
    DS18B20_SetPinOut(DS18B20);
    for(int i = 0; i<8; i++)
    {
        if((Data & (1<<i)) != 0) {
            DS18B20_SetPinOut(DS18B20);
            DS18B20_WritePin(DS18B20, 0);
            DS18B20_DelayUs(DS18B20, 1);
            DS18B20_SetPinIn(DS18B20);
            DS18B20_DelayUs(DS18B20, 50);
        }
        else{
            DS18B20_SetPinOut(DS18B20);
            DS18B20_WritePin(DS18B20, 0);
            DS18B20_DelayUs(DS18B20, 50);
        }
    }
}
```



```

        DS18B20_SetPinIn(DS18B20);
    }
}
}

```

Đọc bit: Master kéo LOW $\sim 1 \mu s$ để mở slot, sau đó thả và lấy mẫu khoảng $\sim 15 \mu s$.

```

static uint8_t DS18B20_Read(DS18B20_Name* DS18B20)
{
    uint8_t Value = 0;
    DS18B20_SetPinIn(DS18B20);
    for (int i = 0; i < 8; i++)
    {
        DS18B20_SetPinOut(DS18B20); // chuyển GPIO sang Output
        để có thể kéo
        DS18B20_WritePin(DS18B20, 0); // kéo DQ xuống LOW
        DS18B20_DelayUs(DS18B20, 1); // giữ LOW  $\sim 1 \mu s$  (đủ điều
        kiện mở slot)
        DS18B20_SetPinIn(DS18B20)
        if (DS18B20_ReadPin(DS18B20)) // đọc mức ngay sau khi
        thả
        {
            Value |= 1 << i;
        }
        // 4) ĐỢI HẾT SLOT
        DS18B20_DelayUs(DS18B20, 50); // đếm để tổng thời gian
        mỗi slot  $\sim 60 \mu s$ 
    }
    return Value; // trả về 1 byte đọc được
}

```

4. Raw -> C

Hai byte nhiệt độ (LSB, MSB) là số bù 2, 16-bit. Ghép lại rồi chia 16.0 để ra °C (vì DS18B20 12-bit có 0.0625°C/LSB). Nếu âm (dưới 0°C), bit dấu trong MSB sẽ set; phép bù 2 vẫn đúng khi ép về int16_t trước khi chia.

```
Temp = (Temp2<<8)|Temp1;  
DS18B20->Temp = (float)(Temp/16);
```

5. Temp -> LCD/UART

```
TEMP = DS18B20_ReadTemp(&DS1);  
sprintf(line, "Temp: %.1f C", TEMP);  
CLCD_I2C_WriteString(&LCD1, line);
```

Pseudo Flow

```
if (!ResetPresence()) return FAIL;           // B1  
Write(0xCC); Write(0x44);                     // B2: Convert T  
if (!ResetPresence()) return FAIL;           // B3  
Write(0xCC); Write(0xBE);  
lsb = ReadByte(); msb = ReadByte();  
raw = (int16_t)((msb<<8)|lsb);                // B4  
TempC = raw / 16.0f;  
publish_to_LCD_UART_Log(TempC);              // B5
```

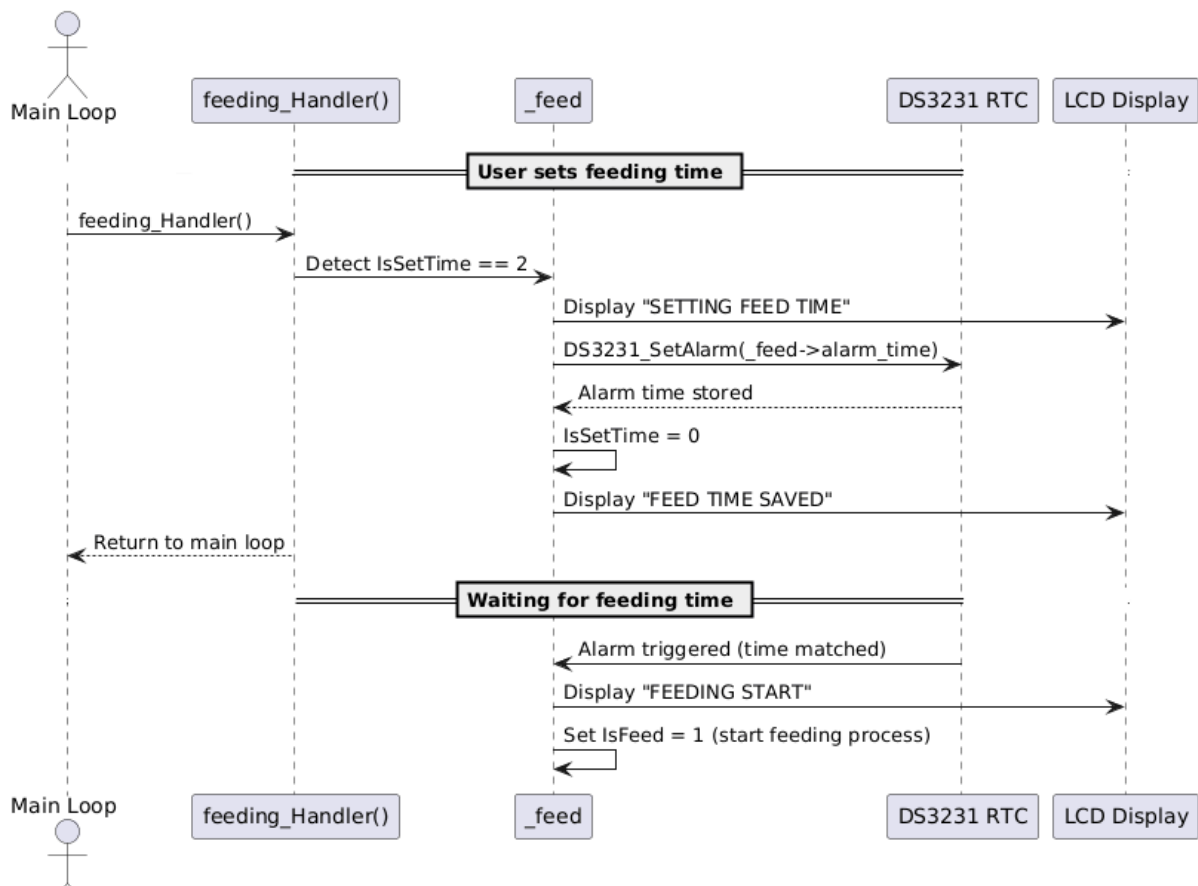
c) Module đặt lịch cung cấp thức ăn

Mô-đun đặt lịch cung cấp thức ăn có nhiệm vụ thiết lập và quản lý thời gian cho ăn tự động thông qua tương tác giữa vi điều khiển, người dùng và modun đồng hồ thời gian thực DS3231. Quá trình hoạt động được chia thành hai giai đoạn: thiết lập thời gian cho ăn và kích hoạt quá trình cho ăn.

Theo lưu đồ hình 31 chương trình chính chạy trong một vòng lặp vô hạn while(1), đảm bảo hệ thống hoạt động liên tục, chỉ dừng khi reset hoặc mất nguồn. Trong vòng lặp này, các chức năng được gọi theo chu kỳ bao gồm: đọc thời gian, hiển thị thông tin, xử lý nút nhấn và điều khiển servo cho ăn. Trong giai đoạn thiết lập, khi người dùng thực hiện thao tác đặt thời gian, hàm feeding_Handler() trong vòng lặp chính phát hiện biến cờ IsSetTime == 2. Khi đó, hệ thống hiển thị thông báo “SETTING FEED TIME” trên màn hình LCD, đồng thời gọi hàm DS3231_SetAlarm(_feed->alarm_time) để lưu thời điểm báo thức tương ứng với giờ cho ăn vào bộ nhớ của mô-đun DS3231. Sau khi

quá trình cài đặt hoàn tất, biến IsSetTime được đặt lại bằng 0 và LCD hiển thị “FEED TIME SAVED” để xác nhận người dùng đã đặt lịch thành công.

Module đặt lịch cung cấp thức ăn có nhiệm vụ thiết lập và quản lý thời gian cho ăn tự động thông qua tương tác giữa vi điều khiển, người dùng và mô-đun đồng hồ thời gian thực DS3231. Quá trình hoạt động được chia thành hai giai đoạn: thiết lập thời gian cho ăn và kích hoạt quá trình cho ăn, cụ thể:



Hình 48: Sơ đồ tuần tự cho module đặt lịch cung cấp thức ăn

Khởi tạo hệ thống: DS3231, PWM với

```

/* USER CODE BEGIN 2 */
feeding_Init(&feed);
/* USER CODE END 2 */

```

Nội dung hàm feeding_init

```

void feeding_Init(struct feeding_t *feed) {
    _feed = feed; // lay dia chi bien thong so
    DS3231_Init(_feed->hi2c);
    HAL_TIM_PWM_Start(_feed->htim, _feed->channel);
    feeding_Servo_Angle(0);
    // DS3231_SetTime(settime);
}

```

Hàm này được gọi một lần duy nhất trong giai đoạn khởi tạo hệ thống nhằm đảm bảo các thiết bị ngoại vi sẵn sàng trước khi thực hiện các chức năng điều khiển và hẹn giờ cho ăn. Cụ thể, hàm thực hiện:

- Gán con trỏ cấu trúc `_feed` để lưu tham chiếu đến các thông số phần cứng (I2C, Timer, kênh PWM, góc servo, thời gian hẹn).
- Khởi tạo module thời gian thực DS3231 thông qua giao tiếp I2C.
- Khởi động bộ định thời ở chế độ PWM để điều khiển động cơ servo.
- Đưa servo về góc ban đầu (0°) để đảm bảo cơ cấu cho ăn ở trạng thái đóng khi khởi động.

```
while (1)
{
    feeding_Handler();
    DS3231_GetTime(&timenow);
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
}
```

Trong vòng lặp vô hạn `while(1)`, hệ thống liên tục thực hiện hàm `feeding_Handler()` để xử lý chức năng cho ăn tự động và cập nhật thời gian thực từ module DS3231 thông qua hàm `DS3231_GetTime(&timenow)`, đảm bảo việc giám sát và điều khiển diễn ra liên tục.

Hàm `feeding_Handler`: Hàm này giúp tách biệt phần xử lý sự kiện điều khiển khỏi phần vòng lặp chính, giúp mã nguồn gọn và dễ mở rộng.

```
void feeding_Handler(void) {
    if (_feed->IsFeed != 0){
        feeding_Servo_Spin();
    }
    if (_feed->IsSetTime == 2){
        // Send feed schedule alarm
        DS3231_SetAlarm(_feed->alarm_time);
        _feed->IsSetTime = 0;
    }
}
```

Hàm `feeding_Handler()` được gọi liên tục trong vòng lặp `while(1)` để:

- Giám sát trạng thái chờ điều khiển (`IsFeed`, `IsSetTime`).
- Kiểm tra cờ `IsFeed` trong cấu trúc `_feed`.

- TH1: Nếu cờ khác 0 → tức là có yêu cầu cho cá ăn (do người dùng nhấn nút 1 chọn chế độ cho ăn thủ công hoặc đến giờ hẹn). Khi đó gọi hàm `feeding_Servo_Spin()` để điều khiển servo quay

Nội dung hàm `feeding_Servo_Spin`

```
void feeding_Servo_Spin() {
    switch(_feed->IsFeed) {
        case 1:
            feeding_Servo_Angle(180);
            _feed->IsFeed = 2;
            break;
        case 3:
            feeding_Servo_Angle(0);
            _feed->IsFeed = 0;
            break;
    }
}
```

Hàm này điều khiển chuyển động của servo theo trạng thái cờ `IsFeed`.

- Khi `IsFeed = 1`: servo quay đến góc 180° , tượng trưng cho mở nắp/đổ thức ăn.
- Khi `IsFeed = 3`: servo quay về góc 0° , tức đóng nắp lại sau khi cho ăn.
- Sau mỗi hành động, trạng thái `IsFeed` được cập nhật để tránh lặp thao tác.

Hàm `feeding_Servo_Angle` có chức năng tạo xung PWM điều khiển servo quay đến góc mong muốn.

```
static void feeding_Servo_Angle(int16_t angle) {
    if(angle > 180) angle = 180;
    if(angle < 0) angle = 0;

    int duty = -(angle * 50/9) + 1250;
    __HAL_TIM_SET_COMPARE(_feed->htim, _feed->channel, duty);
    _feed->angle = angle;
}
```

- Hàm nhận tham số `angle` ($0-180^\circ$) để xác định vị trí quay của servo và giới hạn góc hợp lệ từ 0 đến 180° nhằm tránh servo bị quá tải.
- Công thức `duty = -(angle * 50/9) + 1250` tính giá trị PWM tương ứng, điều chỉnh độ rộng xung để định vị servo.
- `__HAL_TIM_SET_COMPARE()` cập nhật giá trị `duty cycle` vào thanh ghi PWM của Timer (TIMx).
- Lưu lại góc hiện tại vào biến `_feed->angle` để tiện theo dõi trạng thái servo.

TH2: Khi IsSetTime == 2, nghĩa là quá trình cài đặt giờ đã hoàn tất. Nếu nhấn nút 2, 3 → chuyển sang Module đặt giờ cho ăn. Khi nhấn nút 2 → giờ tăng lên.

- Khi nhấn nút 3 → giờ giảm xuống. Khi chọn được mốc giờ mong muốn → nhấn nút 1 để lưu lại.
- Mốc giờ này sẽ được đồng hồ DS3231 ghi nhớ (hoặc lưu vào biến RAM).
- DS3231_SetAlarm(_feed->alarm_time)
- Gửi dữ liệu hẹn giờ mới (giờ, phút, giây) sang chip DS3231 qua giao tiếp I2C.
- RTC sẽ lưu thời điểm này và tự động phát tín hiệu ngắt báo thức khi đến giờ.

Hàm cấu hình thời điểm báo thức trên DS3231 để kích hoạt sự kiện

```
void DS3231_SetAlarm(DS3231_Time_t time) {
    uint8_t set_alarm[4];

    set_alarm[0] = decToBcd(time.seconds) & 0x7F;
    set_alarm[1] = decToBcd(time.minutes) & 0x7F;
    set_alarm[2] = decToBcd(time.hour) & 0x7F;
    set_alarm[3] = 0x80;

    while(HAL_I2C_Mem_Write(_i2c, DS3231_ADDRESS, 0x07, 1,
set_alarm, 4, 1000)!=HAL_OK);

    uint8_t ctrl;
    HAL_I2C_Mem_Read(_i2c, DS3231_ADDRESS, 0x0E, 1, &ctrl, 1,
1000);
    ctrl |= (1<<2) | (1<<0);
    while(HAL_I2C_Mem_Write(_i2c, DS3231_ADDRESS, 0x0E, 1, &ctrl,
1, 1000)!=HAL_OK);

    uint8_t status;
    HAL_I2C_Mem_Read(_i2c, DS3231_ADDRESS, 0x0F, 1, &status, 1,
1000);
    status &= ~(1<<0);
    while(HAL_I2C_Mem_Write(_i2c, DS3231_ADDRESS, 0x0F, 1, &status,
1, 1000)!=HAL_OK);
}
```

Hàm có nhiệm vụ thiết lập thời gian báo thức (Alarm) cho module DS3231 thông qua giao tiếp I2C. Các bước chính:

- Chuyển đổi thời gian từ định dạng thập phân sang BCD và ghi vào các thanh ghi báo thức (0x07 → 0x0A).

- Bật ngắt báo thức (Alarm 1 Interrupt) bằng cách set bit A1IE và INTCN trong thanh ghi Control (0x0E).
- Xóa cờ báo thức cũ (Alarm Flag) trong thanh ghi Status (0x0F) để tránh kích hoạt sai.
- Các vòng lặp while(... != HAL_OK) đảm bảo việc ghi dữ liệu I2C thành công tuyệt đối trước khi sang bước tiếp theo.
- Tiếp theo sẽ đặt lại cờ _feed->IsSetTime = 0;
- _feed->IsSetTime = 0;
- Reset cờ để tránh thiết lập báo thức lặp lại nhiều lần.
- Hệ thống trở lại trạng thái chờ và quay về vòng lặp chính So sánh thời gian hiện tại với mốc giờ đã đặt: Nếu trùng khớp → thực hiện Cho ăn tự động.

```
void DS3231_GetTime(DS3231_Time_t *time) {
    uint8_t get_time[7];

    HAL_I2C_Mem_Read(_i2c, DS3231_ADDRESS, 0x00, 1, get_time, 7,
    HAL_MAX_DELAY);

    time->seconds    = bcdToDec(get_time[0]);
    time->minutes    = bcdToDec(get_time[1]);
    time->hour       = bcdToDec(get_time[2]);
    time->dayofweek   = bcdToDec(get_time[3]);
    time->dayofmonth  = bcdToDec(get_time[4]);
    time->month       = bcdToDec(get_time[5]);
    time->year        = bcdToDec(get_time[6]);
}
```

Hàm có nhiệm vụ đọc thời gian thực từ RTC DS3231 thông qua giao tiếp I2C. DS3231 lưu trữ 7 byte dữ liệu thời gian bắt đầu từ địa chỉ thanh ghi 0x00 với:

- Giờ; Phút; Giờ; Thứ trong tuần; Ngày trong tháng; Tháng; Năm
- Hàm HAL_I2C_Mem_Read(...) thực hiện đọc liên tục 7 byte dữ liệu này.
- Mỗi byte thời gian được mã hóa theo chuẩn BCD (Binary Coded Decimal), nên cần chuyển đổi sang giá trị thập phân thông qua hàm bcdToDec().
- Sau khi chuyển đổi, dữ liệu được gán vào cấu trúc DS3231_Time_t để sử dụng trong chương trình

Bên cạnh lưu đồ chính thể hiện vòng lặp vô hạn và các chức năng tuần tự còn có 2 lưu đồ ngắt systick và ngắt ngoài và ngắt chỉ xảy ra bất chợt khi sự kiện đến, xử lý xong rồi quay về vòng lặp chính.

Thay vì chặn chương trình bằng delay(2000), bạn dùng SysTick cùng với counter → hệ thống vừa cho ăn, vừa vẫn chạy vòng lặp chính (đọc cảm biến, cập nhật LCD, kiểm tra nút nhấn). Đây là cách lập trình non-blocking (không chặn) rất phổ biến trong hệ thống nhúng.

```
void HAL_SYSTICK_Callback(void)
{
    static uint32_t counter = 0;
    if(counter >= 2000) { // 2000 * 1ms = 2s
        feed.IsFeed = 3;
        counter = 0;
    }
    if (feed.IsFeed == 2) {
        counter++;
    }
}
```

Đây là hàm ngắt SysTick (được kích hoạt mỗi 1 ms) dùng để đếm thời gian và điều khiển hành vi động cơ servo trong quá trình cho ăn.

- Biến tĩnh counter dùng làm bộ đếm mili-giây:
- Khi cò feed.IsFeed == 2, bộ đếm bắt đầu tăng.
- Khi bộ đếm đạt 2000 ms (2 giây), chương trình gán feed.IsFeed = 3 để báo cho hệ thống rằng động cơ đã quay đủ thời gian cho ăn và cần quay về vị trí ban đầu.
- Sau đó counter được reset về 0 để chuẩn bị cho chu kỳ tiếp theo.
- Cơ chế này giúp điều khiển thời gian quay servo chính xác mà không cần delay blocking, đảm bảo các tác vụ khác vẫn hoạt động bình thường.

Trong hệ thống cho cá ăn, các nút nhấn được cấu hình sử dụng ngắt ngoài (External Interrupt – EXTI) nhằm đảm bảo việc phản hồi nhanh chóng khi người dùng tác động. Cơ chế hoạt động như sau:

- Khi có sự kiện nhấn nút, ngắt EXTI được kích hoạt, hệ thống tạm dừng vòng lặp chính và nhảy vào hàm xử lý ngắt (ISR).
- Tại ISR, chương trình sẽ xác định nút nào được nhấn:
- Nút 1: Kích hoạt chế độ cho ăn thủ công, điều khiển servo mở nắp trong 2 giây.
- Nút 2: Tăng giá trị giờ cài đặt (set hour) cho chế độ cho ăn tự động.
- Nút 3: Giảm giá trị giờ cài đặt
- Sau khi xử lý, ISR cập nhật biến trạng thái (flag hoặc biến gio_set) để vòng lặp chính while(1) nhận biết và đồng bộ trạng thái hệ thống.


```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    feeding_ISR(GPIO_Pin);
}
```

Mỗi khi có một sự kiện ngắt ngoài (do nút nhấn hoặc tín hiệu báo thức từ RTC), thì feeding_ISR() sẽ được gọi với GPIO_Pin tương ứng.

Hàm feeding_ISR

```
void feeding_ISR(uint16_t GPIO_Pin) {
    if (GPIO_Pin == Button_Pin) {
        if(_feed->IsSetTime == 0)
            _feed->IsFeed = 1;
        else
            _feed->IsSetTime = 2;
    }

    if (GPIO_Pin == Alarm_Pin) {
        uint8_t status;
        HAL_I2C_Mem_Read(_feed->hi2c, 0xD0, 0x0F, 1, &status, 1,
1000);
        status &= ~(1<<0);
        HAL_I2C_Mem_Write(_feed->hi2c, 0xD0, 0x0F, 1, &status, 1,
1000);
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        _feed->IsFeed = 1;
    }

    if (GPIO_Pin == Button_Up_Pin) {
        _feed->IsSetTime = 1;
        _feed->alarm_time.hour++;
        if (_feed->alarm_time.hour >= 24) _feed->alarm_time.hour =
0;
    }

    if (GPIO_Pin == Button_Dw_Pin) {
        _feed->IsSetTime = 1;
        _feed->alarm_time.hour--;
        if (_feed->alarm_time.hour < 0) _feed->alarm_time.hour =
23;
    }
}
```

Khi chân GPIO_Pin có tín hiệu kích cạnh thì sẽ chạy vào 2 trường hợp:

- Khi không trong chế độ cài giờ (IsSetTime == 0) → nhấn nút này để thực hiện cho ăn thủ công (IsFeed = 1).

- Khi đang trong chế độ cài giờ (IsSetTime != 0) → nhấn nút này để xác nhận lưu hoặc thoát cài đặt (IsSetTime = 2).

```
if (GPIO_Pin == Alarm_Pin) {
    uint8_t status;
    HAL_I2C_Mem_Read(_feed->hi2c, 0xD0, 0x0F, 1, &status, 1, 1000);
    status &= ~(1<<0);
    HAL_I2C_Mem_Write(_feed->hi2c, 0xD0, 0x0F, 1, &status, 1,
1000);
    HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
    _feed->IsFeed = 1;
}
```

Đây là ngắt đến từ chân Alarm của module DS3231 RTC (địa chỉ I2C 0xD0).

- Bước 1: Đọc thanh ghi trạng thái (0x0F).
- Bước 2: Xóa cờ A1F (bit 0) — nếu không xóa, ngắt sẽ lặp lại mãi.
- Bước 3: Ghi lại giá trị mới để reset cờ báo thức.
- Bước 4: Toggle LED (bật/tắt để báo hiệu cho ăn).
- Bước 5: Đặt cờ _feed->IsFeed = 1; → hệ thống biết đến giờ cho ăn tự động
- Trường hợp nút tăng giờ

```
if (GPIO_Pin == Button_Up_Pin) {
    _feed->IsSetTime = 1;
    _feed->alarm_time.hour++;
    if (_feed->alarm_time.hour >= 24) _feed->alarm_time.hour = 0;
}
```

- Khi nhấn nút Up, hệ thống chuyển sang chế độ cài đặt giờ (IsSetTime = 1).
- Tăng giá trị giờ báo thức (alarm_time.hour).
- Nếu vượt quá 23 thì quay vòng về 0.
- Trường hợp nút giảm giờ

```
if (GPIO_Pin == Button_Dw_Pin) {
    _feed->IsSetTime = 1;
    _feed->alarm_time.hour--;
    if (_feed->alarm_time.hour < 0)
        _feed->alarm_time.hour = 23;
}
```

- Khi nhấn nút Down, cũng vào chế độ cài giờ (IsSetTime = 1).
- Giảm giá trị giờ báo thức (alarm_time.hour).
- Nếu nhỏ hơn 0, thì quay vòng lại 23.
- Dùng enum cho IsSetTime

```
typedef enum {
    MODE_NORMAL = 0,
```

```

MODE_SET_TIME = 1,
MODE_CONFIRM = 2
} FeedMode;

```

d) Module siêu âm (đo lượng thức ăn)

Khi hệ thống khởi động, chân Trigger của cảm biến siêu âm được kích ở mức cao trong một khoảng thời gian ngắn (khoảng 10 micro giây) để phát xung siêu âm ra môi trường. Sóng siêu âm này lan truyền trong không khí, gặp vật cản sẽ phản xạ lại và được chân Echo thu về.

Bộ vi điều khiển STM32 ghi nhận thời gian truyền và phản hồi của sóng siêu âm. Dựa vào thời gian này, vi điều khiển tính khoảng cách đến vật cản theo công thức:

$$\text{Distance} = \frac{\text{Time} \times 0.034}{2} \quad (4)$$

Đo khoảng cách bằng HC-SR04: Vi điều khiển gửi xung 10 μs đến chân TRIG để kích hoạt cảm biến. Sau đó đọc tín hiệu phản hồi từ chân ECHO, tính toán thời gian phản xạ bằng TIM2 (Timer). Cuối cùng chuyển đổi sang đơn vị centimet (cm)

```

float D_empty = 12.0f;    // cm, mức trống hoàn toàn
float D_full = 7.0f;      // cm, mức đầy
float percent;            // phần trăm mức đầy

void HCSR04_Read(void)
{
    HAL_GPIO_WritePin(TRIG_HCSR04_GPIO_Port, TRIG_HCSR04_Pin,
GPIO_PIN_SET);
    __HAL_TIM_SET_COUNTER(&htim2, 0);
    while (__HAL_TIM_GET_COUNTER(&htim2) < 10); // tạo xung 10us
    HAL_GPIO_WritePin(TRIG_HCSR04_GPIO_Port, TRIG_HCSR04_Pin,
GPIO_PIN_RESET);

    pMillis = HAL_GetTick();
    while (!(HAL_GPIO_ReadPin(ECHO_GPIO_Port, ECHO_Pin)) &&
(HAL_GetTick() - pMillis) < 20);
    Value1 = __HAL_TIM_GET_COUNTER(&htim2);

    pMillis = HAL_GetTick();
    while ((HAL_GPIO_ReadPin(ECHO_GPIO_Port, ECHO_Pin)) &&
(HAL_GetTick() - pMillis) < 50);
    Value2 = __HAL_TIM_GET_COUNTER(&htim2);

    Distance = (float)(Value2 - Value1) * 0.034 / 2;
}

```

Tính toán phần trăm mực đầy: Dựa trên khoảng cách đo được từ cảm biến, chương trình quy đổi sang phần trăm thức ăn hoặc nước còn lại trong bể. D_empty và D_full là hai giá trị hiệu chuẩn được đo thủ công (khi bể trống và đầy).

```
void measure_food_level(void)
{
    HCSR04_Read(); // Đọc khoảng cách hiện tại
    percent = ((D_empty - Distance) / (D_empty - D_full)) * 100.0f;

    if (percent < 0) percent = 0;
    if (percent > 100) percent = 100;
}
```

Dữ liệu đo từ HC-SR04 được gửi đến LCD hiển thị trực tiếp dưới dạng “Feed: xx %”. Đồng thời được truyền đến ESP32-MQTT → Flask Server → Firebase Dashboard, cho phép người dùng xem trạng thái bể cá từ xa. Kết quả đo này còn được dùng để điều khiển servo quay nắp thức ăn khi đến thời gian hẹn hoặc khi lượng thức ăn thấp.

e) Module hiển thị LCD

Mỗi chu kỳ của vòng lặp chính (while(1)), hàm LCD_Update() sẽ được gọi để kiểm tra trạng thái hiện tại của hệ thống và hiển thị nội dung phù hợp.

Mô hình gồm các trạng thái hiển thị chính sau:

Trạng thái hiển thị	Nội dung hiển thị LCD	Mục đích
LCD_STATE_NORMAL	Nhiệt độ và độ đục nước	Hiển thị thông số môi trường cơ bản
LCD_STATE_FEEDING	“FEEDING...”	Báo đang tiến hành cho ăn
LCD_STATE_TIMESET	Thời gian hẹn giờ	Hiển thị khi người dùng cài đặt lịch
LCD_STATE_INFO	Thời gian hiện tại và báo thức	Hiển thị thông tin thời gian thực
LCD_STATE_DISTANCE	Khoảng cách và phần trăm thức ăn	Đo và báo mức thức ăn còn lại
LCD_STATE_WARNING	Cảnh báo “TURBIDITY HIGH”	Hiển thị cảnh báo độ đục vượt ngưỡng

LCD_STATE_SET	Thông báo “Time's been set”	Hiển thị thông báo hẹn giờ đã được cài
LCD_STATE_SET_OFF	Thông báo “Alarm's deleted ”	Hiển thị thông báo hẹn giờ đã được xóa
LCD_STATE_LIGHT_ON	Thông báo “Light was on”	Hiển thị thông báo đèn đã được bật
LCD_STATE_LIGHT_OFF	Thông báo “Light was off”	Hiển thị thông báo đèn đã được tắt

Hàm LCD_Update() thực hiện toàn bộ quá trình hiển thị dữ liệu đo được và trạng thái hoạt động của mô hình. LCD có nhiệm vụ hiển thị các thông tin: nhiệt độ, độ đục, mực nước, trạng thái cho ăn, cảnh báo và thời gian.

Khởi tạo module LCD và giao tiếp I2C

```
I2C_HandleTypeDef hi2c2;
CLCD_I2C_Name LCD1;

MX_I2C2_Init(); // Khởi tạo giao tiếp I2C2 để truyền dữ liệu giữa
STM32 và LCD.
CLCD_I2C_Init(&LCD1, &hi2c2, 0x4E, 16, 2);
```

Cảnh báo độ đục vượt ngưỡng: Khi giá trị độ đục vượt ngưỡng cảnh báo, biến is_turbidity_warning_active được kích hoạt. LCD sẽ hiển thị thông báo “!!WARNING!! TURBIDITY HIGH”, cảnh báo người dùng về tình trạng nước bẩn. Đồng thời, hàm thoát ra (return) để ngăn không cho ghi đè nội dung khác lên màn hình cảnh báo.

```
if (is_turbidity_warning_active == 1)
{
    CLCD_I2C_SetCursor(&LCD1, 0, 0);
    CLCD_I2C_WriteString(&LCD1, " !!WARNING!! ");
    CLCD_I2C_SetCursor(&LCD1, 0, 1);
    CLCD_I2C_WriteString(&LCD1, " TURBIDITY HIGH ");
    return;
}
```

Hiển thị trạng thái cho ăn: Khi hệ thống nhận lệnh cho ăn (thủ công hoặc theo lịch), trạng thái LCD chuyển sang LCD_STATE_FEEDING. Dòng chữ “FEEDING ...” hiển thị trong 2 giây, sau đó tự động trở lại chế độ bình thường. Biến lcd_timer dùng để đo thời gian hiển thị tạm thời bằng hàm HAL_GetTick() (đếm thời gian chạy).

```
case LCD_STATE_FEEDING:
    CLCD_I2C_SetCursor(&LCD1, 0, 0);
```

```

CLCD_I2C_WriteString(&LCD1, "FEEDING ... ");
CLCD_I2C_SetCursor(&LCD1, 0, 1);
CLCD_I2C_WriteString(&LCD1, " ");
if (HAL_GetTick() - lcd_timer >= 2000) {
    lcd_state = LCD_STATE_NORMAL;
    CLCD_I2C_Clear(&LCD1);
}
break;

```

Hiện thị thông số môi trường (Trạng thái bình thường): Đây là chế độ hiển thị chính, luôn được cập nhật định kỳ trong vòng lặp while(1). Dòng đầu hiển thị nhiệt độ nước từ cảm biến DS18B20. Dòng thứ hai hiển thị độ đục nước (NTU) từ module TSM1. Việc cập nhật luân phiên giúp người dùng theo dõi trực tiếp các thông số môi trường.

```

case LCD_STATE_NORMAL:
default:
    sprintf(buffer, "Temp: %.1f C", TEMP);
    CLCD_I2C_SegrptCursor(&LCD1, 0, 0);
    CLCD_I2C_WriteString(&LCD1, buffer);

    sprintf(buffer, "Turb: %.1f NTU", turbidity);
    CLCD_I2C_SetCursor(&LCD1, 0, 1);
    CLCD_I2C_WriteString(&LCD1, buffer);
    break;

```

Hiện thị thời gian thực và báo thức:

```

case LCD_STATE_INFO:
    sprintf(buffer, "Time: %02d:%02d:%02d", timenow.hour,
timenow.minutes, timenow.seconds);
    CLCD_I2C_SetCursor(&LCD1, 0, 0);
    CLCD_I2C_WriteString(&LCD1, buffer);

    if (feed.IsAlarmEnabled == 1) {
        sprintf(buffer, "Alarm: %02d:00:00", feed.alarm_time.hour);
    } else {
        sprintf(buffer, "No set alarm");
    }
    CLCD_I2C_SetCursor(&LCD1, 0, 1);
    CLCD_I2C_WriteString(&LCD1, buffer);
    break;

```

Sau khi người dùng bấm nút BUTTON_UP hay BUTTON_DOWN mà hình sẽ đưa vào trạng thái Timeset để cho người dùng biết trạng thái đang được cập nhật

```

case LCD_STATE_TIMESSET:

    //CLCD_I2C_Clear(&LCD1);

```

```

sprintf(buffer, "Set: %02d:%02d:%02d",
        feed.alarm_time.hour,
        feed.alarm_time.minutes,
        feed.alarm_time.seconds);
CLCD_I2C_SetCursor(&LCD1, 0, 0);
CLCD_I2C_WriteString(&LCD1, buffer);
        CLCD_I2C_SetCursor(&LCD1, 0, 1);
CLCD_I2C_WriteString(&LCD1, " ");

        if (HAL_GetTick() - lcd_timer >= 2000) {
            lcd_state = LCD_STATE_NORMAL;
            CLCD_I2C_Clear(&LCD1);
        }
break;

```

Sau khi đã vào trạng thái Timeset, người dùng bấm BUTTON thì nó sẽ lưu cài đặt hẹn giờ vào MCU và hiện sang trạng thái Set. Nếu người dùng bấm BUTTON_OFF trong lúc TimeSet thì sẽ hiện trạng thái báo tắt hẹn giờ.

```

case LCD_STATE_SET:
    CLCD_I2C_SetCursor(&LCD1, 0, 0);
    CLCD_I2C_WriteString(&LCD1, "Time's been set ");
    CLCD_I2C_SetCursor(&LCD1, 0, 1);
    CLCD_I2C_WriteString(&LCD1, " ");
    if (HAL_GetTick() - lcd_timer >= 2000) {
        lcd_state = LCD_STATE_NORMAL;
        CLCD_I2C_Clear(&LCD1);
    }
    break;

case LCD_STATE_SET_OFF:
    CLCD_I2C_SetCursor(&LCD1, 0, 0);
    CLCD_I2C_WriteString(&LCD1, "Alarm's deleted ");
    CLCD_I2C_SetCursor(&LCD1, 0, 1);
    CLCD_I2C_WriteString(&LCD1, " ");

    if (HAL_GetTick() - lcd_timer >= 2000) {
        lcd_state = LCD_STATE_NORMAL;
        CLCD_I2C_Clear(&LCD1);
    }
    break;

```

MCU nhận được tín hiệu bật đèn từ Broker của Publisher thì sẽ gửi tín hiệu cho LCD để vào trạng thái thông báo đã bật đèn và ngược lại nếu tín hiệu gửi về là tín hiệu tắt đèn thì sẽ vào trạng thái thông báo đã tắt đèn.

```

case LCD_STATE_LIGHT_ON:

```

```

        CLCD_I2C_SetCursor(&LCD1, 0, 0);
        CLCD_I2C_WriteString(&LCD1, "  Light was on  ");
        CLCD_I2C_SetCursor(&LCD1, 0, 1);
        CLCD_I2C_WriteString(&LCD1, "                    ");
        if (HAL_GetTick() - lcd_timer >= 1000) {
            lcd_state = LCD_STATE_NORMAL;
            CLCD_I2C_Clear(&LCD1);
        }
        break;
case LCD_STATE_LIGHT_OFF:
    CLCD_I2C_SetCursor(&LCD1, 0, 0);
    CLCD_I2C_WriteString(&LCD1, "  Light was off ");
    CLCD_I2C_SetCursor(&LCD1, 0, 1);
    CLCD_I2C_WriteString(&LCD1, "                    ");

    if (HAL_GetTick() - lcd_timer >= 1000) {
        lcd_state = LCD_STATE_NORMAL;
        CLCD_I2C_Clear(&LCD1);
    }
    break;

```

Module LCD I2C đóng vai trò là giao diện trực quan giữa người dùng và hệ thống AquaNova. Việc hiển thị dữ liệu theo trạng thái (State Machine) giúp tối ưu quá trình cập nhật, tránh lỗi xung đột khi nhiều thông tin cần hiển thị đồng thời. Ngoài ra, hệ thống cảnh báo trực tiếp trên LCD giúp người vận hành dễ dàng phát hiện tình trạng bất thường trong hồ cá.

f) Module Button nút nhấn điều khiển

Khối lệnh xử lý nút nhấn được đặt trong hàm ISR (Interrupt Service Routine - Trình phục vụ ngắt). Đoạn code đầu tiên không dành cho một nút cụ thể nào mà là một kỹ thuật chung gọi là chống dội phím (debouncing). Khi bạn nhấn một nút vật lý, các tiếp điểm kim loại bên trong nó sẽ rung và tạo ra nhiều tín hiệu điện nhiễu trong một khoảng thời gian rất ngắn. Vì điều khiển có thể hiểu nhầm rằng bạn đã nhấn nút nhiều lần. Đoạn code này đảm bảo rằng sau một lần nhấn được ghi nhận, mọi tín hiệu khác trong vòng 250 mili giây tiếp theo sẽ bị bỏ qua.

- static uint32_t last_interrupt_time = 0: Khai báo một biến tĩnh để lưu lại thời điểm cuối cùng mà một ngắt hợp lệ xảy ra. Biến static sẽ giữ nguyên giá trị của nó giữa các lần gọi hàm.
- if (HAL_GetTick() - last_interrupt_time < 250): Kiểm tra xem thời gian từ lần nhấn hợp lệ cuối cùng đến hiện tại có nhỏ hơn 250ms không.

- return;; Nếu có (tức là đang bị dội phím), hàm sẽ kết thúc ngay lập tức.
- last_interrupt_time = HAL_GetTick(): Nếu không (tức là một lần nhấn hợp lệ), nó sẽ cập nhật lại last_interrupt_time thành thời điểm hiện tại.

```
static uint32_t last_interrupt_time = 0;
if (HAL_GetTick() - last_interrupt_time < 250) {
    return;
}
last_interrupt_time = HAL_GetTick();
```

BUTTON là nút chính của bạn, có hai chức năng tùy thuộc vào trạng thái hiện tại của hệ thống, được điều khiển bởi biến _feed->IsSetTime.

Khi _feed->IsSetTime == 0 (Chế độ bình thường) thì lúc này sẽ thực hiện cho ăn ngay lập tức. Người dùng nhấn nút này và không ở trong chế độ cài đặt giờ, nó sẽ đặt biến cờ _feed->IsFeed lên 1. Vòng lặp chính (while(1)) trong code của bạn sẽ thấy cờ này và kích hoạt động cơ cho ăn. Đồng thời, màn hình LCD sẽ hiển thị thông báo "Đang cho ăn" trong một khoảng thời gian. Khi _feed->IsSetTime khác 0 (đang ở chế độ cài đặt giờ) thì lúc này sẽ xác nhận giờ hẹn đã cài. Người dùng đã dùng nút UP/DOWN để chỉnh giờ, việc nhấn nút này sẽ lưu lại và kích hoạt giờ hẹn. Biến _feed->IsSetTime được đặt thành 2 để báo hiệu đã xác nhận, và _feed->IsAlarmEnabled được bật lên. LCD sẽ hiển thị thông báo "Đã cài đặt".

```
if (GPIO_Pin == BUTTON_Pin) {
    if(_feed->IsSetTime == 0)
    {
        _feed->IsFeed = 1;
        lcd_state = LCD_STATE_FEEDING;
        lcd_timer = HAL_GetTick();
    }
    else
    {
        _feed->IsSetTime = 2;
        _feed->IsAlarmEnabled = 1;    // thông báo đã set giờ
        lcd_state = LCD_STATE_SET;
        lcd_timer = HAL_GetTick();
    }
}
```

- Nút BUTTON_OFF này dùng để hủy quá trình cài đặt giờ/tắt chế độ hẹn giờ.
- Khi _feed->IsSetTime == 1 (đang trong quá trình cài đặt) thì hành động được thực hiện hủy cài đặt và tắt hẹn giờ.

- Nếu người dùng đang chỉnh giờ (IsSetTime == 1) và nhấn nút này, hệ thống sẽ thoát khỏi chế độ cài đặt (IsSetTime = 0), tắt luôn chế độ hẹn giờ (IsAlarmEnabled = 0), và hiển thị thông báo tương ứng trên LCD.

```
if (GPIO_Pin == BUTTON_OFF_Pin) {
    if(_feed->IsSetTime == 1){
        _feed->IsSetTime = 0;
        _feed->IsAlarmEnabled = 0;
        lcd_state = LCD_STATE_SET_OFF;
        lcd_timer = HAL_GetTick();
    }
}
```

- ALARM không phải là một nút nhấn vật lý cho người dùng, mà là chân nhận tín hiệu ngắt từ module thời gian thực (RTC - Real-Time Clock) như DS3231 khi đến giờ đã hẹn.
- ALARM này sẽ thực hiện cho ăn tự động khi đến giờ hẹn.
- Xóa cờ ngắt RTC: Khi RTC báo thức, nó sẽ kéo chân ALARM_Pin xuống mức thấp và đặt một bit cờ trong thanh ghi trạng thái của nó.
- Bắt buộc phải xóa bit cờ này bằng giao tiếp I2C. Nếu không, chân ALARM sẽ bị giữ ở mức thấp và ngắt sẽ xảy ra liên tục. Đoạn code đọc thanh ghi trạng thái (0x0F), xóa bit 0 (status &= ~(1<<0);), rồi ghi ngược lại.
- Kích hoạt cho ăn: Tương tự như nhấn nút thủ công, nó đặt cờ _feed->IsFeed = 1 để vòng lặp chính thực hiện việc cho ăn.
- Tắt hẹn giờ: Sau khi hoàn thành, code này tắt chế độ hẹn giờ (IsAlarmEnabled = 0). Điều này có nghĩa là hẹn giờ chỉ diễn ra một lần duy nhất. Nếu muốn nó lặp lại hàng ngày, bạn cần phải sửa lại logic này.

```
if (GPIO_Pin == ALARM_Pin) {
    uint8_t status;
    HAL_I2C_Mem_Read(_feed->hi2c, 0xD0, 0x0F, 1, &status, 1, 1000);
    status &= ~(1<<0);
    HAL_I2C_Mem_Write(_feed->hi2c, 0xD0, 0x0F, 1, &status, 1, 1000);
    HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
    _feed->IsFeed = 1;
    lcd_state = LCD_STATE_FEEDING;
    lcd_timer = HAL_GetTick();
    _feed->IsSetTime = 0;
    _feed->IsAlarmEnabled = 0;
}
```

- `_feed->IsSetTime = 1`: Đặt biến cờ này thành 1 để báo cho hệ thống biết rằng người dùng đang ở trong chế độ cài đặt giờ. Điều này rất quan trọng, vì nó thay đổi chức năng của nút `BUTTON_Pin` (nút xác nhận)
- `_feed->alarm_time.hour++`: Tăng giá trị của biến lưu trữ giờ hẹn lên 1.
- `_feed->alarm_time.hour--`: Giảm giá trị của biến lưu trữ giờ hẹn xuống 1.
- `if (_feed->alarm_time.hour >= 24) _feed->alarm_time.hour = 0`; Đây là xử lý quay vòng (wrap-around). Nếu người dùng tăng giờ khi đang là 23, giá trị sẽ trở thành 24, và dòng lệnh này ngay lập tức đặt nó trở về 0. Ngược lại, `if (_feed->alarm_time.hour < 0) _feed->alarm_time.hour = 23`; nếu người dùng giảm giờ khi đang là 0, giá trị sẽ trở thành -1 (trong kiểu int), và dòng lệnh này ngay lập tức đặt nó trở về 23.
- `lcd_state = LCD_STATE_TIMESSET`: Chuyển trạng thái màn hình LCD để hiển thị giao diện cài đặt giờ (ví dụ: hiển thị giá trị giờ đang nhập nháy).
- `lcd_timer = HAL_GetTick()`; Đặt lại bộ đếm thời gian của LCD. Điều này thường dùng để đảm bảo màn hình cài đặt sẽ hiển thị trong một khoảng thời gian nhất định trước khi tự động thoát (time-out).

```

if (GPIO_Pin == BUTTON_UP_Pin) {
    _feed->IsSetTime = 1;
    _feed->alarm_time.hour++;
    if (_feed->alarm_time.hour >= 24) _feed->alarm_time.hour = 0;
    lcd_state = LCD_STATE_TIMESSET;
    lcd_timer = HAL_GetTick();
}
if (GPIO_Pin == BUTTON_DOWN_Pin) {
    _feed->IsSetTime = 1;
    _feed->alarm_time.hour--;
    if (_feed->alarm_time.hour < 0) _feed->alarm_time.hour = 23;
    lcd_state = LCD_STATE_TIMESSET;
    lcd_timer = HAL_GetTick();
}

```

Khi nhấn nút này, code sẽ kiểm tra xem màn hình hiện tại đang hiển thị cái gì:

- Nếu màn hình đang ở trạng thái `LCD_STATE_NORMAL` (ví dụ như đang hiển thị giờ giấc bình thường), thì nó sẽ đổi trạng thái sang `LCD_STATE_INFO` (để hiển thị thông tin cảm biến, như nhiệt độ hay độ đục chẳng hạn).

- Còn nếu màn hình đang ở trạng thái LCD_STATE_INFO rồi, mà nhấn thêm một lần nữa, thì nó sẽ chuyển tiếp sang LCD_STATE_DISTANCE. Nói chung, mỗi lần nhấn nút này là để "lật trang" thông tin trên màn hình, đi từ NORMAL, DISTANCE.

```

if (GPIO_Pin == BUTTON_CONTROL_Pin) {

    if (lcd_state == LCD_STATE_NORMAL)
    {
        lcd_state = LCD_STATE_INFO;
    } else if (lcd_state == LCD_STATE_INFO)
    {
        lcd_state = LCD_STATE_DISTANCE;
    }
    else
    {
        lcd_state = LCD_STATE_NORMAL;
    }
    lcd_timer = HAL_GetTick();
}

```

g) Module giao tiếp UART với ESP32

Trong dự án này, UART (USART1) có hai nhiệm vụ chính:

1. Gửi dữ liệu (Truyền - TX): Định kỳ 1 phút một lần, nó sẽ tự động gửi một chuỗi dữ liệu chứa thông tin cảm biến (nhiệt độ, độ ẩm, % thức ăn) và thời gian hiện tại qua cổng UART.
 2. Nhận dữ liệu (Nhận - RX): Nó luôn luôn lắng nghe dữ liệu đến. Khi nhận được một ký tự cụ thể (như 'L', 'I', hoặc 'F'), nó sẽ thực hiện một hành động ngay lập tức (như bật/tắt đèn, cho cá ăn).
- `huart1`: Đây là một "handle" (biến cấu trúc) do thư viện HAL cung cấp. Nó giống như "chứng minh thư" cho ngoại vi USART1. Mọi hàm của HAL muốn điều khiển USART1 (như Init, Transmit, Receive) đều cần truyền biến này vào để biết chúng đang làm việc với UART nào.
 - `rec`: Một biến char (1 byte) dùng để lưu trữ ký tự mà bạn nhận được. Khi bạn gõ 'L' từ máy tính, ký tự 'L' sẽ được lưu vào biến này.
 - `uart_tx_buffer`: Một mảng ký tự (chuỗi) dùng làm bộ đệm. Bạn không gửi trực tiếp các con số (như TEMP = 28.5), mà bạn "format" (định dạng) chúng thành

một chuỗi (ví dụ: "28.5,450.1,80,15:30:00\r\n") rồi lưu vào đây. Sau đó, bạn gửi cả chuỗi này đi.

- `last_uart_send_time` và `UART_SEND_INTERVAL`: Đây là các biến để thực hiện một tác vụ định kỳ mà không "treo" hệ thống (non-blocking delay). Logic là: "Nếu thời gian hiện tại trừ đi lần cuối gửi đã lớn hơn 1 phút, thì thực hiện gửi và cập nhật lại mốc thời gian."

```
UART_HandleTypeDef huart1;
char rec; // Biến lưu ký tự nhận được
char uart_tx_buffer[100]; // Bộ đệm để chuẩn bị dữ liệu gửi đi
uint32_t last_uart_send_time = 0; // Mốc thời gian lần cuối gửi dữ liệu
#define UART_SEND_INTERVAL 60000 // Khoảng thời gian gửi: 60000ms = 1 phút
```

Đây là một hàm callback xử lý ngắt UART. Nói cách khác, đây là hàm được vi điều khiển tự động gọi đến khi có sự kiện ngắt xảy ra — cụ thể là khi UART nhận được dữ liệu. Cách hoạt động có thể hiểu đơn giản như sau:

- Khi gọi lệnh `HAL_UART_Receive_IT(...)`, bạn đang “ra lệnh” cho UART bắt đầu nghe một byte dữ liệu đến. Sau khi gửi yêu cầu đó, vi điều khiển không cần chờ dữ liệu, nó vẫn tiếp tục chạy các công việc khác trong vòng lặp `while(1)` như bình thường.
- Khi có một byte dữ liệu thật sự được gửi đến cổng UART, phần cứng UART sẽ tự động nhận byte này, lưu nó vào biến `rec`, và ngay lập tức tạm dừng chương trình chính để “nhảy” sang thực thi hàm `HAL_UART_RxCpltCallback`.

Bên trong hàm callback:

- Dòng `if (huart->Instance == USART1)` dùng để kiểm tra xem ngắt đến từ UART nào (trong trường hợp bạn dùng nhiều UART).
- Lệnh `switch (rec)` sẽ xác định ký tự vừa nhận được là gì.
- Các case 'L', case 'l', case 'F' tương ứng với từng lệnh điều khiển — ví dụ bật/tắt đèn, cho ăn, v.v. Đây chính là phần logic điều khiển từ xa của hệ thống.
- Cuối cùng, dòng `HAL_UART_Receive_IT(...)` là bắt buộc phải có: nó báo cho UART rằng “tôi đã xử lý xong dữ liệu, bây giờ hãy tiếp tục nghe byte tiếp theo.”

Nếu không có dòng này, UART sẽ chỉ nhận một byte đầu tiên rồi ngừng lắng nghe, khiến chương trình không thể nhận thêm dữ liệu nữa.

```

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        switch (rec)
        {
            case 'L': // Bật đèn
                HAL_GPIO_WritePin(LIGHT_GPIO_Port, LIGHT_Pin,
GPIO_PIN_SET);
                lcd_state = LCD_STATE_LIGHT_ON;
                lcd_timer = HAL_GetTick();
                break;
        }
        HAL_UART_Receive_IT(&huart1, (uint8_t *)&rec, 1);
    }
}

```

Đầu tiên, chương trình kiểm tra xem đã đến thời điểm cần gửi dữ liệu hay chưa bằng câu lệnh:

```

if (HAL_GetTick() - last_uart_send_time >= UART_SEND_INTERVAL

```

Câu lệnh này có nghĩa là: nếu thời gian hiện tại đã cách thời điểm gửi trước đó đủ một khoảng (ví dụ 1 phút), thì thực hiện việc gửi dữ liệu mới. Sau đó, hàm `snprintf(...)` sẽ định dạng dữ liệu cảm biến — chẳng hạn như độ đục (turbidity), nhiệt độ (TEMP)... — thành một chuỗi văn bản có dạng:

```

<turbidity>,<TEMP>,<percent>,<timenow> \r\

```

Các giá trị được phân tách bằng dấu phẩy và kết thúc bằng ký tự xuống dòng để dễ đọc ở phía nhận. Tiếp theo, `HAL_UART_Transmit(...)` sẽ gửi toàn bộ chuỗi này qua UART. Đây là hàm blocking, nghĩa là chương trình sẽ tạm dừng và đợi cho đến khi toàn bộ chuỗi được gửi xong mới tiếp tục chạy. Sau khi gửi xong, dòng `last_uart_send_time = HAL_GetTick();` được dùng để cập nhật lại thời điểm gửi gần nhất, bắt đầu một chu kỳ đếm thời gian mới cho lần gửi tiếp theo.

Câu lệnh `HAL_UART_Receive_IT(...)` được gọi ngay trong vòng lặp `while(1)` (và cả bên trong hàm callback). Đây là một kỹ thuật lập trình phòng thủ, nhằm đảm bảo UART luôn trong trạng thái sẵn sàng nhận dữ liệu.

Lần đầu tiên chương trình chạy vòng lặp, lệnh này sẽ kích hoạt chế độ nhận ngắt cho UART. Sau đó, mỗi khi có dữ liệu đến, hàm callback `HAL_UART_RxCpltCallback` sẽ được gọi để xử lý và kích hoạt lại việc nhận ngay sau đó. Việc gọi lại

HAL_UART_Receive_IT(...) trong vòng lặp chính giúp phòng ngừa trường hợp lỗi ngắt, ví dụ như ngắt bị vô hiệu hóa ngoài ý muốn. Nhờ cơ chế này, UART sẽ luôn sẵn sàng nhận dữ liệu mới bất cứ lúc nào, đảm bảo hệ thống giao tiếp ổn định và liên tục.

```
void handle_uart_transmission(void)
{
    if (HAL_GetTick() - last_uart_send_time >= UART_SEND_INTERVAL)
    {
        snprintf(uart_tx_buffer, sizeof(uart_tx_buffer),
                 "%.1f,%.1f,%.0f,%02d:%02d:%02d\r\n",
                 turbidity,
                 TEMP,
                 percent,
                 timenow.hour,
                 timenow.minutes,
                 timenow.seconds);
        HAL_UART_Transmit(&huart1, (uint8_t*)uart_tx_buffer,
                           strlen(uart_tx_buffer), 1000);

        last_uart_send_time = HAL_GetTick();
    }
    HAL_UART_Receive_IT(&huart1, (uint8_t *)&rec, 1);
}
```

3.6.3. ESP32

Trong kiến trúc tổng thể của hệ thống AquaNova, module ESP32-C3 Mini đảm nhiệm vai trò cổng kết nối IoT (IoT Gateway), có nhiệm vụ trung gian giữa tầng cảm biến và nền tảng đám mây. Thiết bị thực hiện hai chức năng chính:

- Giao tiếp với vi điều khiển STM32 thông qua chuẩn UART để nhận dữ liệu cảm biến và truyền lệnh điều khiển.
- Thiết lập kết nối Wi-Fi và kênh truyền thông bảo mật MQTT/TLS với HiveMQ Broker, qua đó đảm bảo việc trao đổi dữ liệu giữa phần cứng và ứng dụng.

Cấu trúc code bao gồm:

- + Giai đoạn khởi tạo hệ thống `setup()`
- + Vòng lặp chính `loop()`
- + Xử lý mất kết nối `reconnect()`
- + Xử lý lệnh điều khiển `callback()`

a) Gửi và nhận dữ liệu trên ESP32 và HiveMQ MQTT

Giai đoạn khởi tạo hệ thống `setup()`

Chương trình nhúng trên ESP32 được thiết kế nhằm tự động hóa toàn bộ quá trình kết nối mạng, đồng bộ thời gian hệ thống, khởi tạo và duy trì phiên MQTT, đồng thời đảm nhiệm việc gửi dữ liệu đo lường (*telemetry*) và nhận lệnh điều khiển (*control*).

- Cấu hình kết nối WiFi.

```
const char* ssid = "Wifi";  
const char* password = "11111111";
```

Khai báo thông tin truy cập mạng WiFi mà ESP32 sẽ sử dụng để kết nối Internet.

- + Ssid (Service Set Identifier) là tên định danh của mạng không dây nội bộ.
- + Password là khóa bảo mật được sử dụng trong quá trình xác thực WPA2.

Khi chương trình khởi động, ESP32 sẽ gọi hàm `WiFi.begin(ssid, password)` để khởi tạo quá trình kết nối. Việc sử dụng mạng WiFi nội bộ giúp thiết bị gửi dữ liệu cảm biến đến máy chủ đám mây qua giao thức MQTT và nhận lệnh điều khiển từ xa qua Internet.

- Cấu hình MQTT Broker.

```
const char* mqtt_server =  
"fdca86b156bf4f86973c13d0a7fe4e2c.s1.eu.hivemq.cloud";  
const int mqtt_port = 8883;  
const char* mqtt_user = "aquanova_user";  
const char* mqtt_pass = "Aquanova123";
```

Định nghĩa các tham số cần thiết để ESP32 thiết lập kết nối với máy chủ MQTT Broker trên nền tảng HiveMQ Cloud.

- + `mqtt_server`: Tên miền định danh máy chủ MQTT.
- + `mqtt_port`: Cổng kết nối 8883 được chuẩn hóa cho giao thức MQTT bảo mật (SSL/TLS).
- + `mqtt_user` / `mqtt_pass`: Thông tin xác thực người dùng, được cấu hình trên giao diện quản lý của HiveMQ.
- Cấu hình đồng bộ thời gian (NTP)

```
WiFiUDP ntpUDP;  
NTPClient timeClient(ntpUDP, "pool.ntp.org", 25200); // 25200 = 7 *  
60 * 60 (UTC+7)
```

Khởi tạo đối tượng `NTPClient` – một thư viện cho phép truy vấn máy chủ Network Time Protocol (NTP) để đồng bộ hóa thời gian hệ thống.

- + "pool.ntp.org" là máy chủ NTP toàn cầu cung cấp thời gian chính xác theo chuẩn UTC.

- + 25200 giây (7×3600) tương ứng với múi giờ UTC+7, tức Giờ Đông Dương (ICT) được sử dụng tại Việt Nam.
- + Đối tượng ntpUDP đảm nhiệm việc gửi và nhận gói tin NTP qua giao thức UDP (User Datagram).

```
+ 17:06:04.528 -> [MQTT] Subscribed to: aquanova/control
+ 17:06:04.528 -> From STM32: 702.7,29.0,0,14:39:48
+ 17:40:40.173 -> === AquaNova ESP32-C3 MQTT ===
+ 17:40:40.173 -> [UART] Serial1 started for STM32 on RX:20,
TX:21
+ 17:40:40.173 -> [WiFi] Connecting to Zun
+ 17:40:40.346 -> ....
+ 17:40:41.840 -> [WiFi] Connected!
+ 17:40:41.840 -> [WiFi] IP: 10.77.235.196
+ 17:40:41.840 -> [NTP] Syncing time...
+ 17:40:41.992 -> [NTP] Time synced!
+ 17:40:41.992 -> [SYSTEM] Setup complete!
+ 17:40:41.992 -> [MQTT] Attempting connection... connected!
+ 17:40:44.264 -> [MQTT] Subscribed to: aquanova/control
```

Hình 4949: Kết nối Wifi và đồng bộ thời gian

Hình ảnh minh chứng cho thấy ESP32 đã khởi tạo và kết nối thành công với mạng Wi-Fi 2.4 GHz, với địa chỉ IP nội bộ được cấp là 10.183.170.196. Sau khi kết nối, module tiến hành đồng bộ thời gian thực (NTP sync) đảm bảo tính chính xác của các gói tin, đặc biệt trong ứng dụng ghi nhận dữ liệu theo thời gian thực.

- Cấu hình tần suất gửi dữ liệu.

```
const unsigned long SEND_INTERVAL = 60000;
```

Biến SEND_INTERVAL xác định chu kỳ gửi dữ liệu cảm biến lên máy chủ MQTT, đơn vị tính bằng mili-giây (ms).

- + 60.00000 ms = 600 giây $\Rightarrow$ thiết bị sẽ gửi dữ liệu mỗi 1 phút một lần.
- + Khoảng thời gian này được lựa chọn dựa trên sự cân bằng giữa độ cập nhật dữ liệu và tiêu thụ năng lượng mạng.

Vòng lặp chính **loop()**

- Duy trì kết nối MQTT:

Hệ thống kiểm tra liên tục trạng thái client.connected(). Khi kết nối bị gián đoạn, hàm reconnect() sẽ được gọi để khôi phục.

```
if (!client.connected()) {
    reconnect();
}
```

- Xử lý nền MQTT: Hàm client.loop() được thực thi định kỳ để xử lý các tác vụ nền của MQTT như nhận tin, gửi gói keep-alive (PINGREQ/PINGRESP), và xác nhận gói PUBACK (khi QoS > 0).

```
client.loop();
```

- Gửi dữ liệu định kỳ: Cơ chế phi chặn (*non-blocking*) sử dụng millis() để xác định thời điểm gửi dữ liệu. Nếu đã đủ khoảng thời gian SEND_INTERVAL = 30s, hàm publishSensorData() được gọi để thu thập và truyền dữ liệu mới từ STM32. Cùng lúc đó, timeClient.update() đảm bảo thời gian NTP luôn chính xác.

```
unsigned long now = millis();
if (now - lastMsg > SEND_INTERVAL) {
    lastMsg = now;
    if(client.connected()) {
        timeClient.update();
        publishSensorData();
    }
}
```

- Xử lý mất kết nối reconnect(): Hàm reconnect() chịu trách nhiệm tái thiết lập phiên MQTT khi bị ngắt. ESP32 tạo một client ID duy nhất bằng cách kết hợp chuỗi cố định và địa chỉ MAC. Sau khi kết nối thành công, thiết bị tự động đăng ký lại (subscribe) vào chủ đề điều khiển (*aquanova/control*). Nếu thất bại, chương trình in mã lỗi (client.state()) và tạm dừng 5 giây trước khi thử lại.

```
void reconnect() {
    while (!client.connected()) {
        Serial.print("[MQTT] Attempting connection...");

        String clientId = "AquaNova_ESP32_C3_";
        clientId += WiFi.macAddress();
        clientId.replace(":", "");

        if (client.connect(clientId.c_str(), mqtt_user, mqtt_pass))
        {
            Serial.println(" connected!");
            if(client.subscribe(subscribe_topic)) {
                Serial.print("[MQTT] Subscribed to: ");
                Serial.println(subscribe_topic);
            } else {
                Serial.println("[ERROR] Subscribe failed!");
            }
        }
    }
}
```

```

    } else {
        Serial.print(" failed, rc=");
        Serial.print(client.state());
        Serial.println(" - trying again in 5 seconds");
        delay(5000);
    }
}
}
}

```

– Xử lý lệnh điều khiển `callback()`

Hàm `callback()` được kích hoạt mỗi khi có bản tin đến từ topic điều khiển. Payload (chuỗi JSON) được phân tích bằng thư viện **ArduinoJson**, sau đó xác định lệnh tương ứng (“light”, “feeding”). ESP32 gửi ký tự điều khiển (‘L’, ‘F’) tới STM32 thông qua UART (`Serial1.write()`), để thực hiện hành động tương ứng.

```

void callback(char* topic, byte* payload, unsigned int length) {
    Serial.print("\n[MQTT] Message arrived on topic: ");
    Serial.println(topic);

    char message[length + 1];
    memcpy(message, payload, length);
    message[length] = '\0';

    Serial.print("[MQTT] Payload: ");
    Serial.println(message);

    DynamicJsonDocument doc(128);
    DeserializationError error = deserializeJson(doc, message);

    if (error) {
        Serial.print("[ERROR] JSON parse failed: ");
        Serial.println(error.c_str());
        return;
    }

    if (doc.containsKey("light")) {
        int light_status = doc["light"];
        if (light_status == 1) {
            Serial1.write('L');
            Serial.println("[CONTROL] Sent 'L' (Light ON) to STM32");
        } else {
            Serial1.write('l');
            Serial.println("[CONTROL] Sent 'l' (Light OFF) to STM32");
        }
    } else if (doc.containsKey("feeding")) {
        int feeding_status = doc["feeding"];
        if (feeding_status != 0) {

```

```

        Serial1.write('F');
        Serial.println("[CONTROL] Sent 'F' (Start Feeding) to
STM32");
    } else {
        Serial1.write('f');
        Serial.println("[CONTROL] Sent 'f' (Stop Feeding) to
STM32");
    }
} else Serial1.write('B');
}

```

b) Truyền nhận dữ liệu trên ESP32 và STM32

Giao tiếp giữa hai vi điều khiển được thiết lập qua Serial1 của ESP32, sử dụng chân RX=20, TX=21 ở tốc độ 9600 baud, 8N1.

```

const int STM32_RX_PIN = 20; // Chân RX của ESP32 (kết nối với TX của STM32)
const int STM32_TX_PIN = 21; // Chân TX của ESP32 (kết nối với RX của STM32)=

```

- STM32_RX_PIN (GPIO20) là chân đầu vào nhận dữ liệu (RX) trên ESP32, được nối với chân TX (truyền) của STM32.
- STM32_TX_PIN (GPIO21) là chân đầu ra phát dữ liệu (TX) trên ESP32, được nối với chân RX (nhận) của STM32.

```

Serial1.begin(9600, SERIAL_8N1, STM32_RX_PIN, STM32_TX_PIN);
Serial.printf("[UART] Serial1 started for STM32 on RX:%d, TX:%d\n",
STM32_RX_PIN, STM32_TX_PIN);

```

Giao tiếp UART cho phép hai vi điều khiển truyền dữ liệu song công (full-duplex) theo dạng chuỗi ký tự ASCII, giúp giảm độ trễ và tiết kiệm tài nguyên phần cứng so với giao thức phức tạp hơn như SPI hoặc I²C.

- Nhận dữ liệu cảm biến:

Vi điều khiển STM32 được sử dụng làm bộ xử lý trung tâm có nhiệm vụ thu thập dữ liệu từ các cảm biến. Sau khi xử lý và chuyển đổi tín hiệu cảm biến sang dạng số, STM32 sẽ truyền dữ liệu sang ESP32 thông qua giao tiếp UART (Serial) để tiếp tục xử lý và gửi lên nền tảng IoT. ESP32 trong hệ thống đóng vai trò là bộ truyền thông WiFi, chịu trách nhiệm đóng gói dữ liệu và gửi lên máy chủ MQTT hoặc các dịch vụ đám mây khác. Dữ liệu được STM32 truyền sang ESP32 ở dạng payload, tức là chuỗi dữ liệu thô bao gồm các giá trị đo được từ cảm biến. STM32 có thể gửi chuỗi "340,25.6\n" tương ứng với giá trị độ đọc là 340 và nhiệt độ là 25.6°C. Dữ liệu này không có cấu trúc cụ thể mà chỉ

đơn giản là các giá trị được phân tách bằng dấu phẩy hoặc ký tự xuống dòng. Việc truyền payload giúp giảm kích thước gói tin, đảm bảo tốc độ nhanh và hạn chế độ trễ trong giao tiếp nối tiếp giữa hai vi điều khiển.

Sau khi ESP32 nhận được chuỗi payload, vi điều khiển này sẽ thực hiện phân tách và xử lý dữ liệu, sau đó đóng gói lại thành chuỗi JSON để gửi lên hệ thống MQTT. Việc sử dụng JSON ở giai đoạn này giúp dữ liệu có cấu trúc rõ ràng, dễ dàng xử lý ở phía server và thuận tiện cho việc hiển thị hoặc lưu trữ. Ví dụ, chuỗi payload "340,25.6\n" sau khi xử lý sẽ được ESP32 chuyển thành chuỗi JSON như sau:

```
{"turbidity":340, "temperature":25.6}
```

Cấu trúc chuỗi JSON trên gồm hai thành phần chính:

- + "turbidity": 340 – biểu thị giá trị độ đục của nước được cảm biến đo được (đơn vị NTU).
- + "temperature": 25.6 – thể hiện nhiệt độ môi trường hoặc của nước tại thời điểm đo (đơn vị °C).

Việc sử dụng định dạng JSON trong giai đoạn này mang lại nhiều lợi ích kỹ thuật và logic dữ liệu. Trước hết, JSON có cấu trúc rõ ràng dưới dạng cặp *key-value*, giúp định danh từng giá trị cảm biến như "turbidity" (độ đục) hay "temperature" (nhiệt độ), từ đó hệ thống server dễ dàng trích xuất và xử lý dữ liệu mà không cần phân tích chuỗi ký tự thô. Bên cạnh đó, JSON có tính mở rộng linh hoạt, cho phép thêm cảm biến mới (như pH hoặc độ dẫn điện) chỉ bằng cách bổ sung cặp dữ liệu, ví dụ:

```
{"turbidity": 340, "temperature": 25.6, "pH": 7.2}
```

Chuỗi JSON này sau đó sẽ được xuất bản (publish) lên broker MQTT để gửi đến các thiết bị hoặc dịch vụ đăng ký nhận dữ liệu. Giao tiếp UART giữa STM32 và ESP32 được thiết lập theo chuẩn truyền 8 bit dữ liệu, còn gọi là cấu hình 8N1 (gồm 1 bit start, 8 bit dữ liệu, không parity và 1 bit stop). Điều đó có nghĩa là mỗi ký tự trong chuỗi payload đều được truyền đi dưới dạng 1 byte (8 bit). Quá trình truyền nhận diễn ra tuần tự, ESP32 đọc từng byte và ghép chuỗi lại để xử lý hoàn chỉnh. Tốc độ truyền (baud rate) thường được cài đặt ở các mức 9600 bps.

```
17:06:04.528 -> [PUBLISH]
{"turbidity":702.7,"temperature":29,"feed":0,"thoi_gian":"14:39:48"}
Topic: aquanova/telemetry
QoS: 0
17:06:04.49 -> [PUBLISH]
{"turbidity":702.7,"temperature":29,"feed":30,"thoi_gian":"14:39:49"}
```

```
Topic: aquanova/telemetry
QoS: 0
17:06:04.50 -> [PUBLISH]
{"turbidity":702.6,"temperature":29,"feed":35,"thoi_gian":"14:39:50"}
Topic: aquanova/telemetry
QoS: 0
```

Hình 50: Hình ảnh gửi dữ liệu thành công từ STM32 đến ESP32 (8 bit)

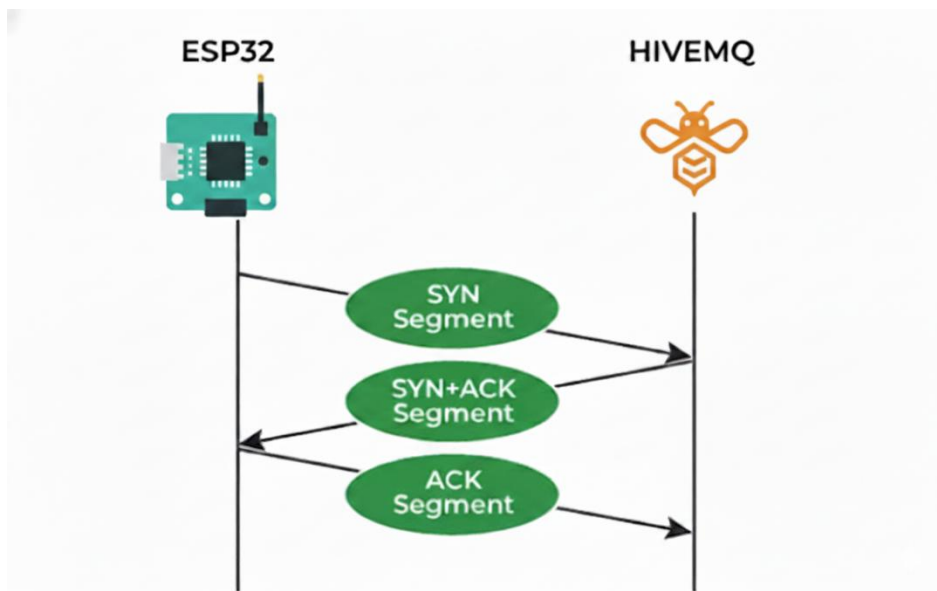
Trong hàm `publishSensorData()`, ESP32 kiểm tra `Serial1.available()`. Khi có dữ liệu, hàm `Serial1.readStringUntil('\n')` đọc toàn bộ chuỗi dữ liệu cho đến ký tự xuống dòng, đảm bảo nhận trọn vẹn một bản tin.

```
if (Serial1.available() > 0) {
    String input = Serial1.readStringUntil('\n');
    input.trim();
}
```

Trong hàm `callback()`, khi nhận lệnh điều khiển từ MQTT, ESP32 sẽ gửi ký tự điều khiển ('L', 'I', 'F', 'f', ...) đến STM32 qua UART, từ đó STM32 thực hiện hành động tương ứng như bật đèn, cho ăn hoặc dừng hệ thống.

c) Phân tích gói tin TCP trong quá trình truyền thông giữa ESP32 và HiveMQ Broker

Trong hệ thống IoT sử dụng vi điều khiển ESP32 làm thiết bị đầu cuối, giao thức MQTT được triển khai tại tầng ứng dụng (Application Layer) và hoạt động dựa trên giao thức TCP ở tầng vận chuyển (Transport Layer), nhằm đảm bảo độ tin cậy, tính toàn vẹn dữ liệu và thứ tự truyền gói tin. Khi ESP32 khởi tạo kết nối với HiveMQ Broker, thiết bị sẽ tiến hành quá trình bắt tay ba bước (three-way handshake) để thiết lập kênh truyền TCP ổn định.



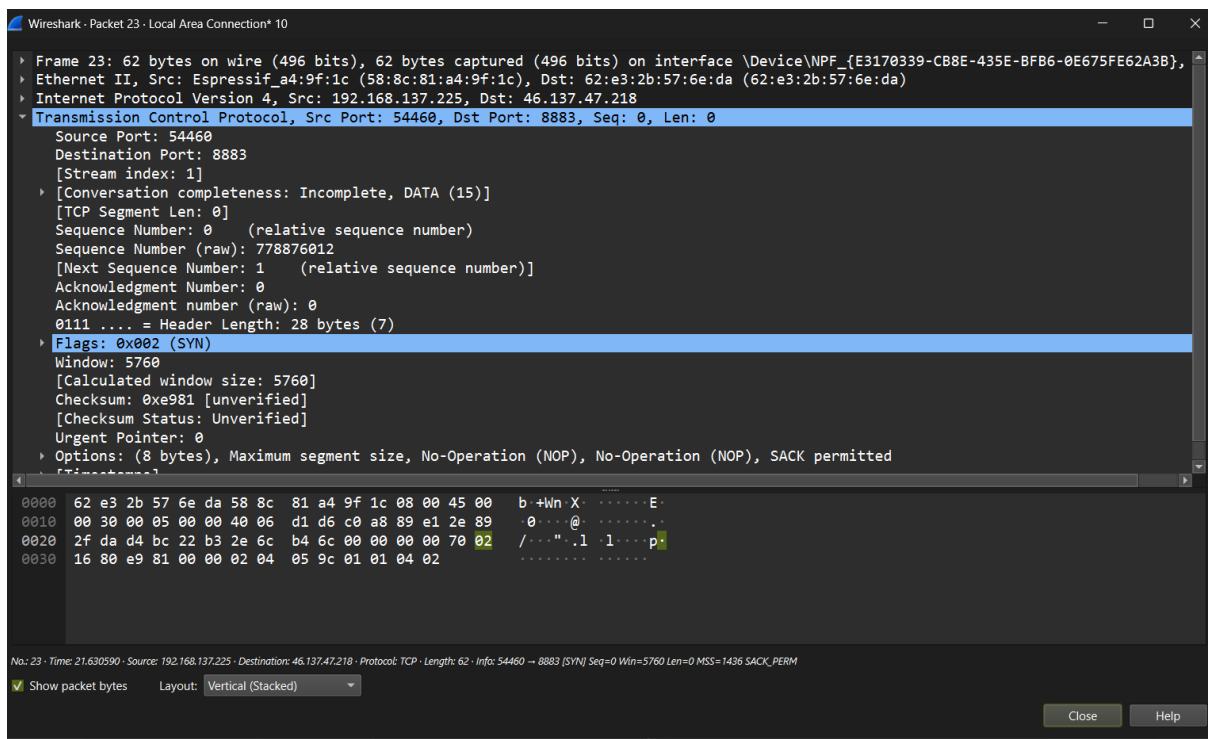
Hình 51: Quá trình bắt tay ba bước giữa esp32 và hivemq broker

Quan sát thực tế bằng Wireshark cho thấy rõ chuỗi gói tin khởi tạo kết nối với các cờ TCP [SYN], [SYN, ACK] và [ACK]. Sau khi thiết lập thành công, tất cả bản tin MQTT (CONNECT, PUBLISH, SUBSCRIBE, PINGREQ, DISCONNECT) đều được truyền qua kênh TCP này.

No.	Time	Source	Destination	Protocol	Length	Info
23	21.630590	192.168.137.225	46.137.47.218	TCP	62	54460 → 8883 [SYN] Seq=0 Win=5760 Len=0 MSS=1436 SACK_PERM
24	21.847734	46.137.47.218	192.168.137.225	TCP	62	8883 → 54460 [SYN, ACK] Seq=0 Ack=1 Win=62727 Len=0 MSS=1460 SACK_PERM
25	21.966267	192.168.137.225	46.137.47.218	TCP	54	54460 → 8883 [ACK] Seq=1 Ack=1 Win=5760 Len=0

Hình 52: Chuỗi gói tin khởi tạo kết nối với các cờ TCP [SYN], [SYN, ACK] và [ACK] được bắt trong Wireshark

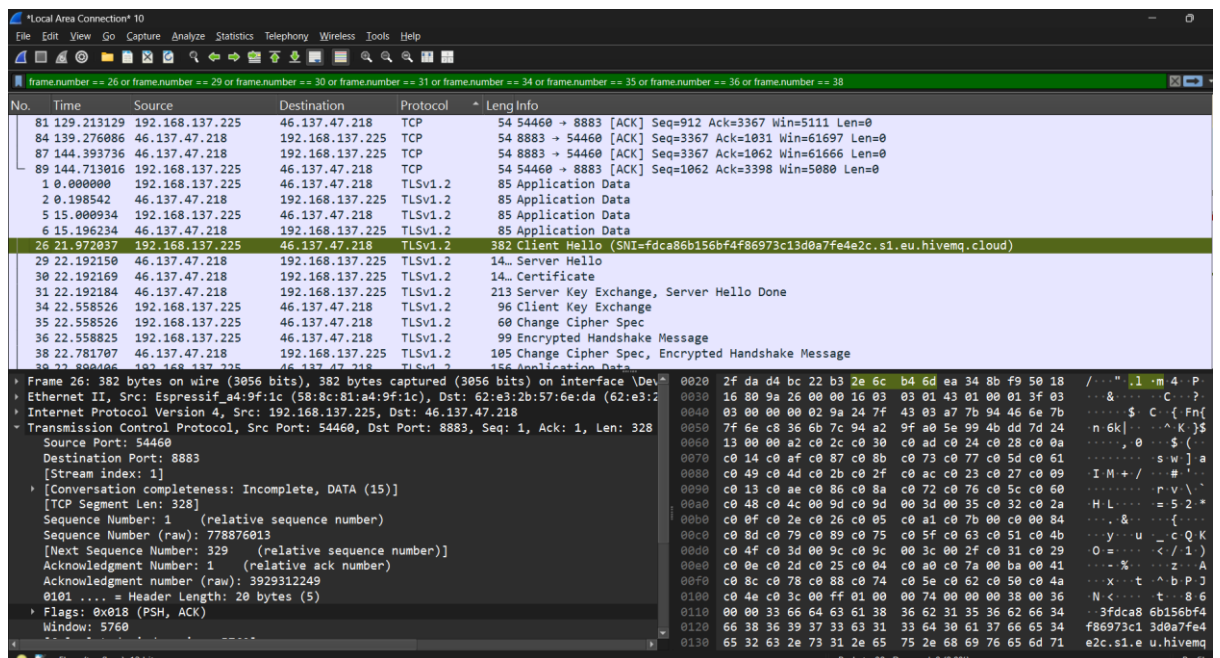
Mỗi gói tin TCP được cấu thành từ phần tiêu đề (Header) và phần dữ liệu (Payload). Phần tiêu đề bao gồm các trường quan trọng như Source Port, Destination Port, Sequence Number, Acknowledgment Number, Flags (SYN, ACK, PSH, FIN), Window Size và Checksum, đóng vai trò then chốt trong việc kiểm soát luồng và phát hiện lỗi.



Hình 53: Gói tin TCP SYN khởi tạo kết nối MQTT bảo mật giữa ESP32 và Broker
Cụ thể, gói tin SYN được gửi đi từ ESP32 để khởi tạo phiên kết nối an toàn với server.

- + Địa chỉ IP nguồn (Source IP): 192.168.1.105 — thuộc về ESP32, đại diện cho thiết bị IoT trong hệ thống.
- + Địa chỉ IP đích (Destination IP): 46.137.47.218 — thuộc về MQTT Broker (Server) trên nền tảng đám mây.
- + Source Port: 54460
- + Destination Port: 8883 (cổng mặc định cho giao thức MQTT Secure/TLS)
- + Window size: 5760 bytes
- + MSS (Maximum Segment Size): 1436 bytes
- + Protocol: TCP (Transmission Control Protocol)

Sau khi ESP32 gửi gói SYN, MQTT Broker phản hồi bằng gói SYN-ACK, và cuối cùng ESP32 gửi lại ACK để hoàn tất quá trình three-way handshake. Quy trình này đảm bảo rằng kết nối TCP được thiết lập thành công, tạo nền tảng cho việc truyền dữ liệu cảm biến, trạng thái thiết bị và lệnh điều khiển.



Hình 54: Các gói tin bắt tay TLS dùng để thiết lập phiên mã hóa sau khi kênh TCP đã được mở (26, 29, 30, 31, 34, 35, 36, 38)

Sau khi gói 38 được trao đổi thành công và xác minh, quá trình bắt tay TLS hoàn tất. Kênh giao tiếp TCP hiện tại đã được nâng cấp lên thành một kênh an toàn, mã hóa bằng TLS, sẵn sàng cho việc truyền tải dữ liệu ứng dụng MQTT. Trong khi đó, phần payload chứa bản tin MQTT, bao gồm Fixed Header, Variable Header và Payload. Ví dụ, khi ESP32 gửi một bản tin PUBLISH, phần dữ liệu TCP chứa nội dung “MQTT Publish Message, Topic Name:aquanova/control, Payload:

“turbidity”:801.4,”temperature”:29,”feed”:0”, và Wireshark hiển thị như:

```
Transmission Control Protocol
Source Port: 52231
Destination Port: 8883
Flags: PSH, ACK
```

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.137.225	46.137.47.218	TLSv...	85	Application Data
2	0.198542	46.137.47.218	192.168.137.225	TLSv...	85	Application Data
5	15.000934	192.168.137.225	46.137.47.218	TLSv...	85	Application Data
6	15.196234	46.137.47.218	192.168.137.225	TLSv...	85	Application Data
39	22.890406	192.168.137.225	46.137.47.218	TLSv...	156	Application Data
43	23.128091	46.137.47.218	192.168.137.225	TLSv...	87	Application Data
46	23.198548	192.168.137.225	46.137.47.218	TLSv...	106	Application Data
47	23.416335	46.137.47.218	192.168.137.225	TLSv...	88	Application Data
50	38.204031	192.168.137.225	46.137.47.218	TLSv...	85	Application Data
51	38.420230	46.137.47.218	192.168.137.225	TLSv...	85	Application Data
54	53.199227	192.168.137.225	46.137.47.218	TLSv...	85	Application Data
55	53.417674	46.137.47.218	192.168.137.225	TLSv...	85	Application Data
57	68.207163	192.168.137.225	46.137.47.218	TLSv...	85	Application Data
58	68.423404	46.137.47.218	192.168.137.225	TLSv...	85	Application Data
60	79.001022	192.168.137.225	46.137.47.218	TLSv...	173	Application Data
63	83.660596	192.168.137.225	46.137.47.218	TLSv...	85	Application Data
65	83.877783	46.137.47.218	192.168.137.225	TLSv...	85	Application Data

Transmission Control Protocol, Src Port: 49917, Dst Port: 8883, Seq: 1, Ack: 1, Len: 31

- Source Port: 49917
- Destination Port: 8883
- [Stream index: 0]
- [Conversation completeness: Incomplete (60)]
- [TCP Segment Len: 31]
- Sequence Number: 1 (relative sequence number)
- Sequence Number (raw): 407059153
- [Next Sequence Number: 32 (relative sequence number)]
- Acknowledgment Number: 1 (relative ack number)
- Acknowledgment number (raw): 1527416820
- 0101 = Header Length: 20 bytes (5)
- **Flags: 0x018 (PSH, ACK)**
- Window: 5049
- [Calculated window size: 5049]
- [Window size scaling factor: -1 (unknown)]
- Checksum: 0x71be [unverified]

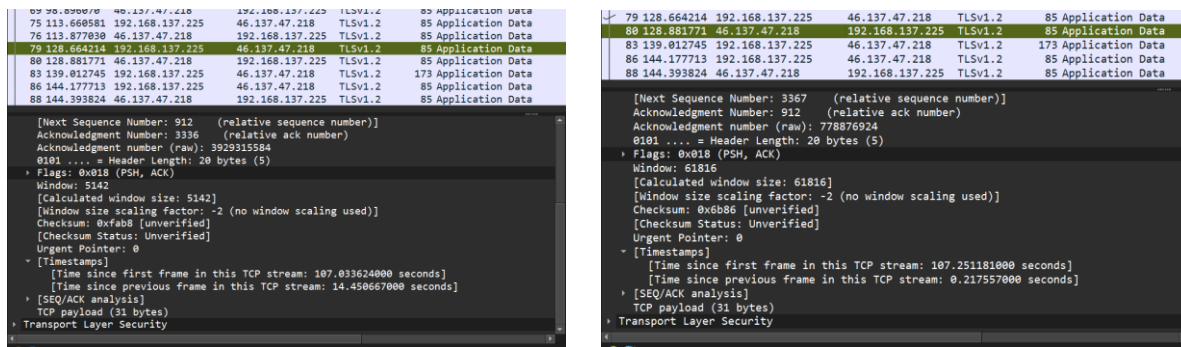
Hình 55: Minh họa quá trình truyền dữ liệu qua giao thức TLS giữa ESP32 và MQTT Broker

Các gói tin hiển thị trên Wireshark cho thấy dữ liệu ứng dụng (Application Data) được truyền an toàn qua cổng 8883, cổng mặc định của MQTT over TLS. Ta thấy toàn bộ nội dung MQTT được đóng gói bên trong TCP Payload, và cờ PSH cho phép gửi dữ liệu tức thời mà không cần chờ đệm, trong khi cờ ACK xác nhận dữ liệu trước đó đã được nhận chính xác.

Cơ chế kiểm soát độ tin cậy của TCP thể hiện qua việc sử dụng Sequence Number và Acknowledgment Number để theo dõi từng byte dữ liệu, cùng với Window Size giúp điều chỉnh tốc độ truyền, tránh tắc nghẽn (congestion control). Đặc biệt, trong mạng

WiFi mà ESP32 thường sử dụng, nhiễu sóng và mất gói có thể xảy ra thường xuyên, do đó TCP là lớp nền quan trọng để MQTT hoạt động ổn định và tin cậy.

Về mặt định lượng, các thử nghiệm thực tế cho thấy độ trễ trung bình khi truyền bản tin MQTT từ ESP32 đến Broker HiveMQ là tương đối nhỏ trong mạng WiFi ổn định, và tỷ lệ mất gói gần như bằng 0% nhờ cơ chế tái truyền của TCP. Khi có tình gây nhiễu (mô phỏng mất gói 10%), các gói bị mất được TCP tự động truyền lại, giúp đảm bảo dữ liệu ứng dụng không bị sai lệch, khẳng định ưu thế của TCP trong các môi trường IoT có độ tin cậy thấp.



Hình 56: Tính toán độ trễ dựa trên phân tích gói tin trong Wireshark

3.7. Thiết kế website

Quy trình thiết kế website điều khiển từ xa MCU gồm nhiều bước liên kết chặt chẽ giữa phần cứng và phần mềm. Trước hết, cần xác định yêu cầu hệ thống, luồng dữ liệu và các chức năng điều khiển cần thiết. Tiếp theo là xây dựng kiến trúc tổng thể gồm giao diện web (HTML, CSS, JavaScript), backend (Flask hoặc Node.js), cơ sở dữ liệu và giao thức truyền thông như MQTT hoặc WebSocket để kết nối với vi điều khiển. Hệ thống phải có các API gửi lệnh và nhận phản hồi từ MCU, giao diện hiển thị trạng thái thiết bị theo thời gian thực, đồng thời tích hợp các cơ chế bảo mật như xác thực người dùng và mã hóa dữ liệu. Cuối cùng, cần triển khai hệ thống lên server, kiểm thử khả năng hoạt động ổn định, và xây dựng tài liệu hướng dẫn vận hành để đảm bảo website có thể điều khiển thiết bị từ xa an toàn và hiệu quả.

3.7.1. Cài đặt phần mềm và cấu hình

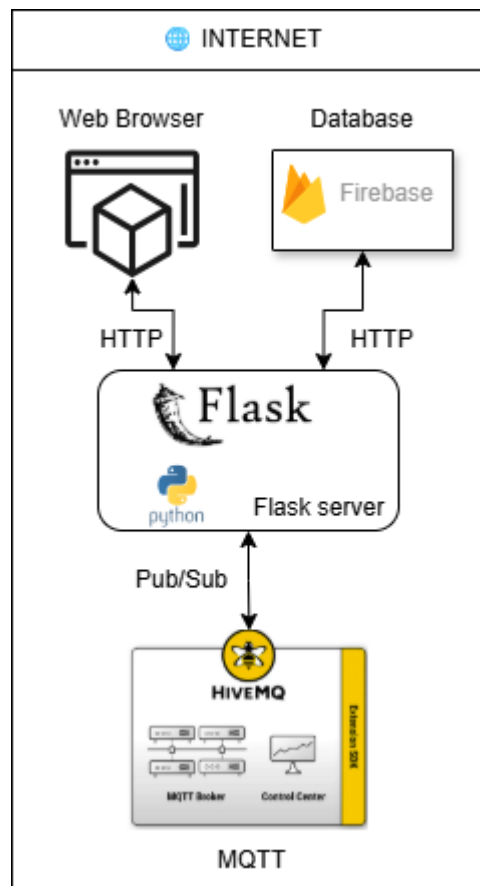
Giao diện web của mô hình AquaNova xây dựng dựa trên kiến trúc ba lớp chính:

- Lớp ứng dụng (Backend): sử dụng Flask (Python) làm web server chính, tạo HTTP server, kết nối MQTT và giao tiếp với cơ sở dữ liệu Firebase.

- Lớp giao diện (Frontend): viết bằng HTML, CSS, JavaScript, hiển thị dữ liệu đo được từ cảm biến theo thời gian thực.
- Lớp dữ liệu (Cloud Database): sử dụng Firebase Cloud Firestore để lưu trữ log đo lường và cấu hình hệ thống.

Giao tiếp giữa các lớp được thực hiện qua:

- Giao thức HTTP cho các request web.
- Giao thức MQTT cho trao đổi dữ liệu IoT (giữa STM32/ESP32 ↔ Flask).



Hình 57: Sơ đồ kết nối Backend, Frontend, Firebase và MQTT

Môi trường cần thiết để xây dựng web cho mô hình AquaNovo

```
Flask==3.0.0
firebase-admin==6.5.0
python-dotenv==1.0.1
Flask-Cors==4.0.0
google-cloud-firestore>=2.16
paho-mqtt>=1.6.1
APScheduler>=3.10.4
```

Tên package	Phiên bản	Chức năng chính	Vai trò trong dự án AquaNova
Flask	3.0.0	Framework web nhẹ, linh hoạt cho Python	Là backend chính để tạo API, routing, render HTML (dashboard, feed timer, telemetry...)
firebase-admin	6.5.0	SDK quản trị Firebase dành cho Python	Dùng để kết nối Firestore, đọc/ghi dữ liệu cảm biến, lịch cho ăn, thiết bị, người dùng
python-dotenv	1.0.1	Đọc file .env chứa biến môi trường	Lưu cấu hình bí mật (MQTT credentials, Firestore key path, API token...)
Flask-Cors	4.0.0	Cho phép CORS (Cross-Origin Resource Sharing)	Giúp frontend JS (Chart.js, fetch API) gọi API Flask mà không bị chặn bởi CORS policy
google-cloud-firestore	2.16	Thư viện truy cập Firestore (Google Cloud)	Giao tiếp với Cloud Firestore: lưu dữ liệu đo, logs, trạng thái, và các document theo collection
paho-mqtt	1.6.1	Thư viện MQTT cho Python	Dùng để publish / subscribe MQTT messages giữa Flask và ESP32/STM32 qua HiveMQ Cloud
APScheduler	3.10.4	Thư viện lập lịch	Dùng để tạo, quản lý và chạy lịch tự động — ví dụ: gửi lệnh cho ăn lúc 8:00, 12:00, 18:00 mỗi ngày

Bảng 10: Bảng liệt kê chi tiết các requirements

3.7.2. Cấu trúc tổ chức chương trình

Hệ thống AquaNova được tổ chức theo kiến trúc 3 lớp kết hợp mô hình MVC (Model – View – Controller), bao gồm:

Lớp	Chức năng	Thành phần trong dự án
Presentation Layer (Giao diện người dùng)	Hiển thị dữ liệu cảm biến, cho phép người dùng điều khiển, đặt lịch cho hệ thống cho ăn. Toàn bộ giao diện được xây dựng bằng HTML, CSS và JavaScript.	Thư mục templates/ (các file .html), static/css/, static/js/
Application Layer (Xử lý trung gian)	Xử lý logic ứng dụng, kết nối cơ sở dữ liệu và MQTT. Sử dụng Flask Framework để định	File app.py và các module trong blueprints/ gồm

	tuyến request và phản hồi dữ liệu JSON cho frontend.	dashboard, control, telemetry, admin
Data Layer (Lưu trữ và giao tiếp IoT)	Lưu trữ dữ liệu thời gian thực, bao gồm giá trị cảm biến, lịch cho ăn, log hoạt động. Dữ liệu được lưu trên Firebase Firestore và giao tiếp cảm biến qua MQTT Broker.	firebase_admin_init.py, collection readings, schedules, feed_logs

Cấu trúc thư mục chính:

```

aquanova/
├── app.py          # Khởi tạo Flask app, load config, đăng ký blueprint
├── blueprints/
│   ├── dashboard/routes.py    # Cung cấp API dữ liệu (temperature,
│   │                           turbidity, feed)
│   ├── control/routes.py      # Điều khiển cho ăn, tạo/xóa lịch
│   ├── control/scheduler.py   # Lập lịch tự động = và gửi lệnh feed
│   ├── telemetry/routes.py     # Ghi nhận dữ liệu từ MQTT (telemetry)
│   └── admin/routes.py         # Giao diện quản trị
├── firebase_admin_init.py     # Kết nối Firestore
├── MCU                        # STM32, ESP32
├── mqtt/
│   ├── listener.py           # sub
│   └── publisher.py
├── static /
│   ├── css/styles.css        # Giao diện hiển thị
│   └── js/dashboard.js        # Xử lý cập nhật dữ liệu & biểu đồ
└── templates/
    ├── index.html             # Dashboard tổng quan
    ├── feedtimer.html          # Giao diện đặt lịch cho ăn
    ├── temperature.html        # Biểu đồ nhiệt độ
    ├── turbidity.html          # Biểu đồ độ đục
    └── admin.html              # Trang quản trị

```

Chuẩn bị môi trường

```

# Database
FIREBASE_CREDENTIALS=C:\Users\ASUS\Documents\GitHub\IOT_AquaNova\Io
T_AquaNova\serviceAccount.json
FIREBASE_DB=firestore
SECRET_KEY=AquaNovaSecret123

# HiveMQ Cloud MQTT
MQTT_HOST=d2b2655505394c9b8e88c06c8d30d912.s1.eu.hivemq.cloud

```

```
MQTT_PORT=8883
MQTT_USER=aquanova_user
MQTT_PASS=12345678Aqua
MQTT_TOPIC=aquanova/telemetry
```

3.7.3. Frontend

a) Tổng quan giao diện

Giao diện người dùng của mô hình IoT AquaNova được xây dựng trên kết hợp với HTML, CSS và JavaScript.

- Frontend giao tiếp trực tiếp với Flask thông qua các API REST (ví dụ: /dashboard/latest, /control/feed-now, /control/light) và sử dụng Fetch API để lấy dữ liệu dưới dạng JSON.
- Dữ liệu được hiển thị trực quan bằng biểu đồ thời gian thực (Chart.js) và bảng thống kê nhằm cung cấp cái nhìn tổng thể về tình trạng ao nuôi (độ đục, nhiệt độ, lượng thức ăn còn lại,...).

Tất cả các trang web đều tuân theo bố cục hai phần chính:

- Sidebar (thanh điều hướng): chứa các liên kết đến các module chính như Dashboard, Temperature, Turbidity, Feed Timer, Admin.
- Main Content (nội dung chính): hiển thị dữ liệu, biểu đồ và bảng tương ứng theo từng module

Ví dụ đoạn HTML cấu trúc tổng quát:

```
<div class="layout">
  <aside>
    <nav class="menu">
      <a href="/dashboard" class="tab-link active">Dashboard</a>
      <a href="/temperature" class="tab-link">Temperature</a>
      <a href="/turbidity" class="tab-link">Turbidity</a>
      <a href="/feedtimer" class="tab-link">Feed Timer</a>
    </nav>
  </aside>
  <main>
    <!-- Nội dung từng module -->
  </main>
</div>
```

Mỗi trang đều dùng tệp styles.css để thống nhất màu chủ đạo:

```
:root {
  --brand: #0ea5e9; /* Xanh */
  --muted: #6b7280; /* Ghi nhạt */
```

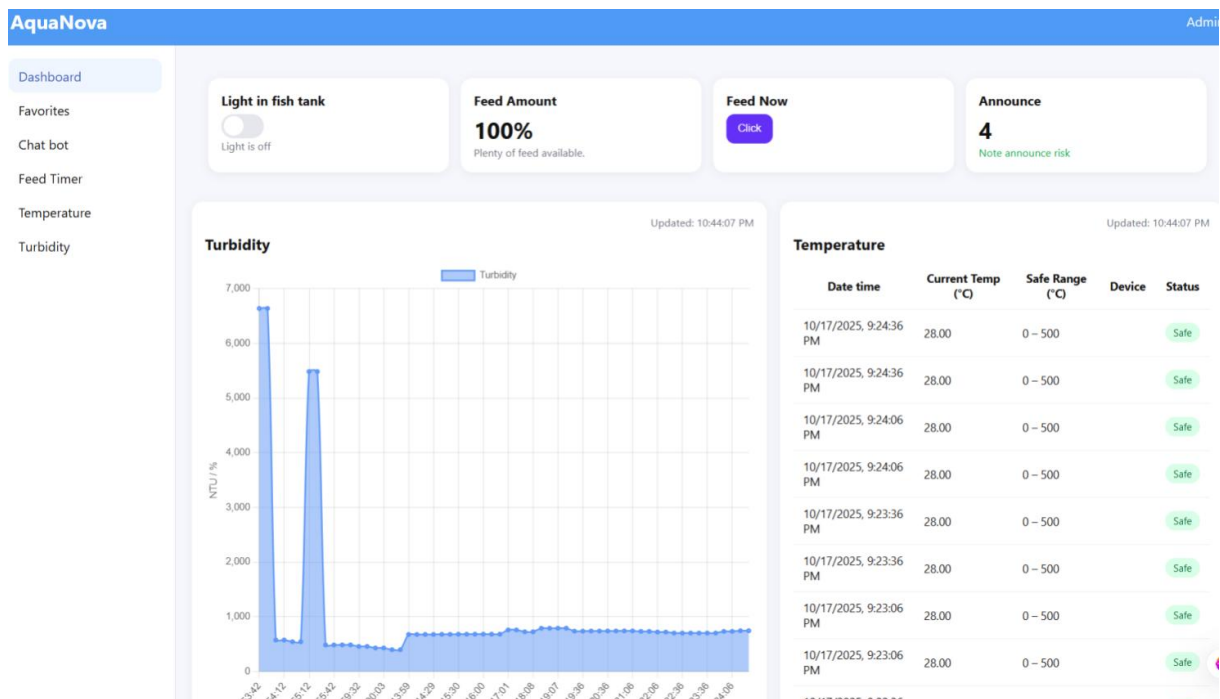


```

--bg: #f6f7fb;      /* Nền sáng */
}
.card {
  background: #fff;
  border-radius: 14px;
  box-shadow: 0 2px 12px rgba(0,0,0,.05);
}

```

b) Phân tích mã nguồn giao diện Dashboard (index.html)



Hình 58: Bộ cụ giao diện của trang web người dùng chính

Trang Dashboard là trung tâm giám sát mô hình như hình 53. Chức năng chính:

- Hiển thị các thông số tổng quan:
- Lượng thức ăn còn lại (%)
- Độ đục nước (NTU)
- Nhiệt độ trung bình (°C)
- Số lần cảnh báo (Announce)
- Biểu đồ đường (line chart) thể hiện độ đục nước theo thời gian.
- Bảng dữ liệu nhiệt độ: 10 giá trị gần nhất kèm đánh giá “Safe” hoặc “Alert”.

Khai báo HTML và thư viện

- CSS: định dạng bố cục (màu nền, lưới, thẻ card, sidebar).
- Chart.js: thư viện biểu đồ giúp hiển thị dữ liệu cảm biến (độ đục, nhiệt độ) dạng đồ thị tuyến tính.


```
<link rel="stylesheet" href="{{ url_for('static',
filename='css/styles.css') }}">
<script
src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min
.js"></script>
```

Phân cấu trúc giao diện:

- aside: tạo thanh điều hướng bên trái (menu liên kết đến các module).
- main: vùng nội dung chính chứa thẻ thông tin (cards) và biểu đồ + bảng dữ liệu.
- Card: hiển thị dữ liệu

```
<div class="layout">
  <aside>...</aside>
  <main>...</main>
</div>

<div class="cards">
  <div class="card">Light in fish tank</div>
  <div class="card">Feed Amount</div>
  <div class="card">Feed Now</div>
  <div class="card">Announce</div>
</div>
```

Phân điều khiển bằng JavaScript

- Điều khiển đèn hồ cá (Light Control): Khi người dùng nhấn nút chuyển, hàm toggleLight() gửi yêu cầu POST đến API /control/light. Flask backend sẽ publish MQTT message đến ESP32 để bật/tắt đèn hồ. Trạng thái phản hồi được cập nhật lại trên giao diện (“Light is turned ON/OFF”).

```
lightSwitch.addEventListener('click', () => {
  const isOn = lightSwitch.classList.toggle('on');
  toggleLight(isOn);
});
```

- Gửi lệnh cho ăn ngay: Khi người dùng nhấn nút “Feed Now”, hệ thống gửi yêu cầu POST đến Flask. Flask publish lệnh MQTT đến vi điều khiển (STM32/ESP32) để kích hoạt servo cho ăn. Sau khi gửi thành công, giao diện hiển thị thông báo “Feed command sent successfully!”.

```
const res = await fetch('/control/feed-now', {
  method: 'POST',
  body: JSON.stringify({ amount: 20 })
});
```

- Biểu đồ độ đục (Turbidity Chart): Hàm ensureChart() khởi tạo biểu đồ đường.

Dữ liệu được lấy từ API /dashboard/latest?n=60, mỗi mẫu gồm:

- ts: thời gian đo

- turbidity: giá trị độ đục
- Nhãn trục X hiển thị theo thời gian (UTC+7).
- Biểu đồ được cập nhật tự động mỗi 10 phút (setInterval(loadAll, 600000)).

```
turbChart = new Chart(ctx, {
  type: 'line',
  data: { labels, datasets: [{ label: 'Turbidity', data,
    borderWidth: 2, tension: .25, fill: true }] }
});
```

Bảng nhiệt độ (Temperature Table): Hiển thị 10 mẫu dữ liệu gần nhất gồm thời gian, nhiệt độ, thiết bị đo, và trạng thái an toàn. Nếu giá trị nằm ngoài khoảng [SAFE_MIN, SAFE_MAX] → cảnh báo “Alert” (màu đỏ).

```
items.slice(0,10).forEach(r=>{
  const t = Number(r.temperature);
  const badge = `<span class="badge
    ${ok?'safe':'alert'}">${ok?'Safe':'Alert'}</span>`;
});
```

c) Feed Timer – Giao diện điều khiển cho ăn (feedtimer.html)

Trang feed_timer.html là thành phần điều khiển mô hình AquaNova:

- Điều khiển cho cá ăn ngay lập tức với lượng tùy chọn.
- Tạo và quản lý lịch cho ăn tự động theo ngày, theo tuần hoặc hàng ngày.
- Xem danh sách các lịch hiện tại và xóa khi cần.
- Theo dõi tình trạng thức ăn còn lại thông qua biểu đồ dạng Pie chart.

Trang này hoạt động song song với Flask backend, giao tiếp qua các API:

- /control/feed-now, /control/schedule, /control/schedules

Header và bố cục chính: Hiển thị tiêu đề “AquaNova – Feed Timer” và liên kết đến trang quản trị. Trang được chia làm hai phần chính trong layout (như tổng quan)

```
<header>
  <h2>AquaNova</h2>
  <div>
    <span>Feed Timer</span>
    <a href="{ url_for('admin_page') }">Admin</a>
  </div>
</header>
```

Bên trái (Feeder Control)

Phần Feeder Control gồm 2 nhóm:

- Feed Now: cho ăn ngay.
- Add Schedule: tạo lịch định sẵn.

```
<!-- Feed Now -->
<input type="number" id="fnAmount" value="20">
<button id="feedNowBtn">Feed Now</button>
```

Người dùng nhập số gam muốn cho ăn và nhấn “Feed Now”: Người dùng chọn ngày, giờ, tần suất và lượng thức ăn để thêm lịch tự động.

```
<!-- Add Schedule -->
<input type="date" id="scDate">
<input type="time" id="scTime">
<select id="scRepeat">Once / Daily / Weekly</select>
<input type="number" id="scAmount" value="20">
<button id="addScheduleBtn">Add</button>
```

Bên phải (Feed Status – biểu đồ): Biểu đồ tròn (Pie Chart) thể hiện phần trăm thức ăn còn lại trong thùng. Được cập nhật từ dữ liệu Flask hoặc Firestore (qua file dashboard.js).

```
<canvas id="feedPie1"></canvas>
<b id="feedPie1Num">--%</b>
<div id="feedPie1Warn" class="tag" style="display:none">Low feed
(&lt;10%)</div>
```

Phân tích chi tiết JavaScript

- Khai báo API

```
const FEED_NOW_API = '/control/feed-now';
const SCHEDULE_API = '/control/schedule';
const SCHEDULES_API = '/control/schedules';
```

- Feed Now – Gửi lệnh cho ăn ngay

Luồng hoạt động:

1. Người dùng nhập lượng thức ăn.
2. JS gửi POST /control/feed-now với JSON { "amount": 20 }.
3. Flask backend nhận → publish MQTT đến ESP32 để kích hoạt servo feeder.
4. Nếu thành công → thông báo “Feed command sent.”
5. Nếu lỗi → hiển thị “Failed: ...”.

```
document.getElementById('feedNowBtn').onclick = async ()=>{
  const amount = Number(document.getElementById('fnAmount').value);
  const msg = document.getElementById('feedNowMsg');
  msg.textContent = 'Sending...';
  const r = await fetch(FEED_NOW_API, {
    method:'POST',
    headers:{'Content-Type':'application/json'},
    body: JSON.stringify({ amount })
  });
};
```

```
}
```

- Thêm lịch cho ăn mới

```
async function addSchedule(){
  const date = document.getElementById('scDate').value;
  const time = document.getElementById('scTime').value;
  const repeat = document.getElementById('scRepeat').value;
  const amount = Number(document.getElementById('scAmount').value
  || 0);
  const msg = document.getElementById('addScheduleMsg');

  const r = await fetch(SCHEDULE_API, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ date, time, repeat, amount })
  });
}
```

Hiển thị danh sách lịch hiện tại

```
async function refreshSchedules(){
  const r = await fetch(SCHEDULES_API);
  const j = await r.json();
  const list = j.items || [];
  list.forEach((s,idx)=>{
    const tr = document.createElement('tr');
    tr.innerHTML = `
      <td>${idx+1}</td>
      <td>${s.date || '-'}</td>
      <td>${s.time || ''}</td>
      <td>${s.repeat}</td>
      <td>${s.amount}</td>
      <td><button
onclick="deleteSchedule('${s.id}')">Delete</button></td>`;
  });
}
```

```
{
  "items": [
    { "id": "abcd123", "date": "2025-10-23", "time": "08:30",
    "repeat": "daily", "amount": 20 }
  ]
}
```

- Xóa lịch đã đặt: Gửi request DELETE /control/schedules/<id>

Flask xóa document tương ứng trong Firestore và gỡ job khỏi APScheduler.

```
async function deleteSchedule(id){
  const r = await fetch(`${SCHEDULES_API}/${id}`, { method: 'DELETE'
});
}
```

d) Temperature – giám sát nhiệt độ

Mục đích của trang là giám sát và phân tích dữ liệu nhiệt độ nước trong bể cá được gửi từ thiết bị ESP32/STM32 thông qua MQTT và Firestore. Người dùng có thể:

- Theo dõi xu hướng nhiệt độ bằng biểu đồ thời gian thực.
- Lọc dữ liệu theo thiết bị, khoảng thời gian, hoặc xuất dữ liệu ra file CSV.
- Quan sát các giá trị trung bình, min-max, và trạng thái an toàn.

Liên kết với backend

Chức năng	API Flask tương ứng	Mô tả hoạt động
Lấy dữ liệu nhiệt độ	/dashboard/latest	Flask truy vấn Firestore, trả JSON dữ liệu mới nhất
Lọc, xem chi tiết	client-side (JS)	Thực hiện hoàn toàn trên frontend sau khi tải dữ liệu
Xuất CSV	client-side (JS)	Tự động tạo file CSV từ dữ liệu đã lọc
Cảnh báo an toàn	Không cần API riêng	Kiểm tra ngưỡng [SAFE_MIN, SAFE_MAX] trên trình duyệt

Bảng 11: Liên kết backend với các chức năng

Chức năng và thành phần chính trong phần main (HTML)

Bộ lọc (Filter toolbar): evic Select: cho phép chọn cảm biến hoặc bo mạch cụ thể (ví dụ: ESP32-C3-01).

- From / To: lọc dữ liệu trong khoảng thời gian mong muốn.
- Apply: áp dụng bộ lọc để hiển thị lại bảng và biểu đồ.
- Export CSV: xuất toàn bộ dữ liệu nhiệt độ đang hiển thị ra file CSV để phân tích ngoại tuyến.

```
<div class="toolbar">
  <select id="deviceSelect">All devices</select>
  <input type="datetime-local" id="fromDt">
  <input type="datetime-local" id="toDt">
  <button id="applyBtn">Apply</button>
  <button id="exportBtn">Export CSV</button>
</div>
```

Biểu đồ nhiệt độ (Temperature Trend) (js)

```
tempChart = new Chart(ctx, {
  type: 'line',
  data: { labels: [], datasets: [{ label: 'Temperature (°C)', data: [],
    tension: .25 }] }
});
```

- Dùng Chart.js để vẽ biểu đồ dạng đường, thể hiện biến thiên nhiệt độ theo thời gian.
- Hàm renderChart() xử lý:
 - o Lấy dữ liệu từ biến filtered (đã lọc theo device hoặc thời gian).
 - o Giới hạn tối đa 200 điểm để giữ hiệu năng.
 - o Cập nhật thời gian (chartUpdated) khi biểu đồ được refresh.
- Dải nhiệt độ an toàn được hiển thị bên phải (Safe: 24–30 °C).

Bảng dữ liệu: hiển thị danh sách chi tiết từng mẫu đo (thời gian, giá trị, trạng thái).

Mỗi hàng được tô nhãn trạng thái và có phân trang (pagination):

- o PAGE_SIZE = 20 → 20 dòng mỗi trang.
- o Nút Prev / Next chuyển trang.

```
<table>
  <thead>
    <tr><th>#</th><th>Date time</th><th>Device</th><th>Temperature
(°C)</th><th>Status</th></tr>
  </thead>
  <tbody id="rows"></tbody>
</table>
```

```
const ok = t >= SAFE_MIN && t <= SAFE_MAX;
return `<span class="status
${ok?'ok':'bad'}">${ok?'Safe':'Alert'}</span>`;
```

Luồng hoạt động chính (JavaScript)

- Lấy dữ liệu từ API:
 - Gọi API /dashboard/latest (Flask backend) để nhận tối đa FETCH_N = 500 bản ghi gần nhất.
 - Mỗi bản ghi có dạng:

```
const res = await fetch(`/dashboard/latest?n=${FETCH_N}`);
const json = await res.json();
allItems = json.items || [];

{"ts":"2025-10-23T10:00:00Z","temperature":27.5}
```

- Lọc dữ liệu và hiển thị: Gọi renderChart() để vẽ lại biểu đồ, và renderTable() để cập nhật bảng.

```
function applyFilter(){
  const dev = document.getElementById('deviceSelect').value;
  const from = new Date(document.getElementById('fromDt').value);
  const to = new Date(document.getElementById('toDt').value);
  ...
  filtered = allItems.filter(...);
```

```
renderChart();
renderTable();
}
```

- Xuất dữ liệu CSV

```
document.getElementById('exportBtn').onclick = () => {
  const csv = lines.map(row => row.join(',')).join('\n');
  const blob = new Blob([csv], {type: 'text/csv'});
}
```

e) Turbidity – Giám sát độ đục nước

Trang Turbidity là một phần của mô hình giám sát chất lượng nước AquaNova:

- Hiển thị dữ liệu độ đục (NTU) thu thập từ cảm biến trong bể cá.
- Xem biểu đồ xu hướng (trend chart) theo thời gian.
- Lọc dữ liệu trong khoảng thời gian mong muốn.
- Xem bảng dữ liệu chi tiết và thống kê nhanh (min, max, avg).
- Xuất dữ liệu ra CSV để lưu trữ hoặc phân tích thêm.
- Trang này được dựng bằng HTML + CSS + JavaScript (Chart.js) và hoạt động như một client web giao tiếp với Flask backend thông qua API /dashboard/latest.

Header và Layout chính

```
<header>
  <h2>AquaNova</h2>
  <div>
    <span>Turbidity</span>
    <a href="{ url_for('admin_page') }">Admin</a>
  </div>
</header>
```

Css: sử dụng màu sắc xanh chủ đạo và nền sáng

Chức năng JavaScript chính:

- Khai báo hằng số và trạng thái: Cho phép cấu hình ngưỡng cảnh báo, số dữ liệu tải và phân trang.

```
const TURBIDITY_WARN = 70; // ngưỡng cảnh báo
const PAGE_SIZE = 20;      // số dòng mỗi trang
const FETCH_N = 500;       // tải tối đa 500 bản ghi
let allItems = [];
let filtered = [];
let page = 1;
```

Lấy dữ liệu từ Flask API:

- Gửi request GET đến endpoint /dashboard/latest của Flask.
- Flask trả về dữ liệu JSON dạng:

```

async function fetchData(){
  const res = await fetch(`/dashboard/latest?n=${FETCH_N}`);
  const json = await res.json();
  return json.items || [];
}

```

```

{
  "items": [
    {"ts": "2025-10-23T10:30:00Z", "turbidity": 45.3},
    {"ts": "2025-10-23T10:31:00Z", "turbidity": 72.1}
  ]
}

```

Lọc dữ liệu theo thời gian: Người dùng chọn thời gian “From – To”, JS lọc các bản ghi tương ứng và cập nhật giao diện.

```

function applyFilter(){
  const fromV = document.getElementById('fromDt').value;
  const toV    = document.getElementById('toDt').value;
  let arr = [...allItems];
  if (fromV){ const from = new Date(fromV); arr = arr.filter(x =>
    new Date(x.ts) >= from); }
  if (toV){   const to   = new Date(toV);   arr = arr.filter(x =>
    new Date(x.ts) <= to); }

  filtered = arr.sort(byTimeDesc);
  renderTable();
  renderChart();
}

```

Bảng dữ liệu (Table):

Tự động hiển thị 20 dòng mỗi trang, có phân trang “Prev/Next”.

statusBadge(v) hiển thị: OK (xanh lá) nếu <200 NTU, High (đỏ) nếu >200 NTU.

```

function renderTable(){
  const tbody = document.getElementById('rows');
  tbody.innerHTML = '';
  filtered.slice(start, start + PAGE_SIZE).forEach((r,i)=>{
    const v = Number(r.turbidity ?? NaN);
    tbody.innerHTML += `
      <tr>
        <td>${i+1}</td>
        <td>${fmtDate(r.ts)}</td>
        <td>${v.toFixed(2)}</td>
        <td>${statusBadge(v)}</td>
      </tr>`;
  });
}

```


Biểu đồ độ đục (Chart.js)

```
function ensureChart(ctx){
  turbChart = new Chart(ctx, {
    type:'line',
    data:{ labels:[], datasets:[{ label:'Turbidity (NTU / %)',
data:[], borderWidth:2 }] },
    options:{ scales:{ x:{title:{text:'Time'}}, y:{title:{text:'NTU
/ %'}}} } }
  });
}
```

Thông kê tổng quan: Tính toán nhanh min, max, trung bình cho giai đoạn hiển thị.

```
const min = Math.min(...data), max = Math.max(...data);
const avg = data.reduce((a,b)=>a+b,0)/data.length;
document.getElementById('summaryBox').innerHTML =
`Records: <b>${filtered.length}</b><br>
Range: <b>${min}</b> - <b>${max}</b><br>
Average: <b>${avg}</b>`;
```

Xuất dữ liệu ra CSV

```
document.getElementById('exportBtn').onclick = () => {
  const lines = [['timestamp','turbidity']];
  filtered.forEach(r=> lines.push([r.ts, r.turbidity]));
  ...
  a.download = `turbidity_export_${Date.now()}.csv`;
};
```

h) Tương tác giữa Front end với Flask API

Hoạt động	API Flask	Phương thức	Kết quả
Lấy dữ liệu cảm biến	/dashboard/latest	GET	Trả về danh sách các bản ghi mới nhất
Cho ăn ngay lập tức	/control/feed-now	POST	Gửi lệnh MQTT → thiết bị thực thi
Tạo lịch cho ăn	/control/schedule	POST	Lưu thông tin lịch vào Firestore
Lấy danh sách lịch	/control/schedules	GET	Hiển thị bảng “Current Schedules”
Xóa lịch	/control/schedules/<id>	DELETE	Xóa lịch tương ứng khỏi Firestore

3.7.4. Backend

Backend Flask trong mô hình AquaNova đóng vai trò:

- Cầu nối trung tâm giữa giao diện web, cơ sở dữ liệu Firebase, và thiết bị IoT (ESP32 qua MQTT).
- Tiếp nhận các yêu cầu từ người dùng (qua HTTP request).
- Gửi lệnh điều khiển (qua MQTT publish) hoặc truy vấn dữ liệu cảm biến (qua Firestore).
- Quản lý lịch tự động cho ăn (Feed Scheduler) sử dụng APScheduler.

a) Cấu trúc Flask và Blueprints

Thành phần	Chức năng
Flask Core	Tổ chức web server, routing, template rendering
Blueprints	Tách chức năng logic (telemetry, control, dashboard, admin)
Firestore (Firebase)	Lưu trữ dữ liệu thời gian thực và lịch sử
MQTT Listener	Nhận dữ liệu cảm biến từ ESP32
MQTT Publisher	Gửi lệnh điều khiển Feed Now, bật/tắt đèn, v.v.
APScheduler	Lập lịch gửi lệnh định kỳ
CORS	Cho phép frontend gọi API từ domain khác
Templates & Static	Giao diện HTML/CSS/JS hiển thị cho người dùng

Bảng 12: Bảng chi tiết vai trò các module backend

Để chạy server trên terminal gõ “python app.py”. Flask server chịu trách nhiệm xử lý API, định tuyến web, lưu trữ dữ liệu, và thực hiện lệnh điều khiển.

1. Cấu trúc tổng thể

```
from flask import Flask, render_template
from flask_cors import CORS
from config import Config
from firebase_admin_init import init_firebase
from blueprints.telemetry.routes import telemetry_bp
from blueprints.control.routes import control_bp
from blueprints.control.scheduler import start_scheduler
from blueprints.admin.routes import admin_bp
from blueprints.dashboard.routes import dashboard_bp
from mqtt.listener import start_mqtt_background
```

2. Hàm create_app() – Khởi tạo ứng dụng Tạo Flask app và định nghĩa thư mục:

```
def create_app():
```

```

app = Flask(__name__, static_folder="static",
template_folder="templates")
app.config.from_object(Config)
CORS(app) // ngăn truy cập trái phép

```

- static/ → chứa file CSS, JS, ảnh.
- templates/ → chứa các file HTML giao diện.
- Nạp cấu hình từ lớp Config.
- Kích hoạt CORS để cho phép truy cập từ web client.

```

import os
from dotenv import load_dotenv
load_dotenv()

class Config:
    SECRET_KEY = os.getenv("SECRET_KEY", "dev")

    # Firebase
    FIREBASE_CREDENTIALS = os.getenv("FIREBASE_CREDENTIALS")
    FIREBASE_DB = os.getenv("FIREBASE_DB", "firestore")

    # MQTT
    MQTT_HOST = os.getenv("MQTT_HOST", "localhost")
    MQTT_PORT = int(os.getenv("MQTT_PORT", "1883"))
    MQTT_USER = os.getenv("MQTT_USER", "") # optional
    MQTT_PASS = os.getenv("MQTT_PASS", "") # optional
    MQTT_TOPIC = os.getenv("MQTT_TOPIC", "aquanova/telemetry")

```

3. Khởi tạo firebase và Đăng ký các Blueprint (Module API)

```

init_firebase()

app.register_blueprint(telemetry_bp, url_prefix="/telemetry")
app.register_blueprint(control_bp, url_prefix="/control")
app.register_blueprint(admin_bp, url_prefix="/admin")
app.register_blueprint/dashboard_bp, url_prefix="/dashboard")

```

Bảng chi tiết

Blueprint	Đường dẫn	Chức năng chính
telemetry_bp	/telemetry	Nhận dữ liệu cảm biến gửi lên (nhiệt độ, độ đục, mức thức ăn)
control_bp	/control	Xử lý lệnh điều khiển: Feed Now, Add Schedule, Delete Schedule, Light

admin_bp	/admin	Quản trị hệ thống, cấu hình, xem log
dashboard_bp	/dashboard	Cung cấp API cho giao diện hiển thị dữ liệu (latest, summary, chart)

4. Định nghĩa các route giao diện

```
@app.get("/")
def home():
    return render_template("index.html")

@app.get("/admin")
def admin_page():
    return render_template("admin.html")

@app.get("/temperature")
def temperature_page():
    return render_template("temperature.html")

@app.get("/turbidity")
def turbidity_page():
    return render_template("turbidity.html")

@app.get("/feedtimer")
def feed_timer_page():
    return render_template("feedtimer.html")
```

5. Khởi động MQTT Listener & APScheduler

a. start_mqtt_background(app)

Tạo thread chạy nền kết nối MQTT broker (ví dụ HiveMQ Cloud).

Lắng nghe các topic:

- "aquanova/telemetry" → dữ liệu cảm biến gửi về.
- "aquanova/status" → phản hồi trạng thái thiết bị.

Khi có dữ liệu, Flask tự động cập nhật lên Firestore.

b. start_scheduler()

- Kích hoạt bộ lập lịch APScheduler – cho phép tự động thực hiện lệnh cho ăn đúng giờ.
- Mỗi khi người dùng thêm lịch qua /control/schedule, hệ thống tạo job mới trong scheduler.

Kết hợp MQTT + APScheduler giúp hệ thống hoạt động thời gian thực và tự động.

```
start_mqtt_background(app)
with app.app_context():
    start_scheduler()
```

6. Khởi chạy ứng dụng

Flask server chạy trên cổng mặc định (5000).

- debug=True cho phép theo dõi log khi phát triển.
- use_reloader=False tránh khởi chạy 2 lần (đặc biệt khi có thread MQTT).

```
if __name__ == "__main__":
    app = create_app()
    app.run(debug=True, use_reloader=False)
```

a) Module Dashboard (giao tiếp với giao diện)

Nhiệm vụ chính của module này là cung cấp dữ liệu quan trắc môi trường nước cho giao diện người dùng thông qua các API, đồng thời thực hiện thống kê và phát hiện các thông số bất thường dựa trên dữ liệu thu được từ các thiết bị IoT (ESP32/STM32) gửi lên Cloud Firestore.

Các chức năng chính bao gồm:

- Truy xuất dữ liệu cảm biến mới nhất để hiển thị trên Dashboard.
- Cung cấp thông tin tổng quan của hệ thống (số hồ nuôi, số thiết bị).
- Phát hiện và đếm số bản ghi bất thường để cảnh báo.
- Cung cấp bản ghi gần nhất phục vụ việc cập nhật tức thời trên web
- API /dashboard/latest – lấy danh sách bản ghi mới nhất

```
@dashboard_bp.get("/latest")
def latest():
    n = int(request.args.get("n", 100))
    db = firestore.client()
    docs = (
        db.collection("readings")
        .order_by("ts", direction=firestore.Query.DESCENDING)
        .limit(n)
        .stream()
    )
    items = []
    for doc in docs:
        d = doc.to_dict()
        items.append({
            "ts": d.get("ts"),
            "temperature": d.get("temperature"),
            "turbidity": d.get("turbidity"),
            "feed": d.get("feed"),
```

```

        "thoi_gian": d.get("thoi_gian")
    })
    return jsonify({"items": items})

```

Chức năng: API này thực hiện truy vấn n bản ghi mới nhất trong collection "readings" của Firestore. Mỗi bản ghi chứa thông tin:

- Thời gian đo (ts), nhiệt độ (temperature), độ đục (turbidity), mức thức ăn (feed), thời gian hiển thị (thoi_gian).

Dữ liệu trả về dạng JSON để giao diện web hiển thị biểu đồ và bảng thống kê. Kết quả trả về mẫu:

```

{
  "items": [
    {
      "ts": "2025-10-23T10:35:00Z",
      "temperature": 28.3,
      "turbidity": 56.2,
      "feed": 40
    }
  ]
}

```

API /dashboard/announce-count – Phát hiện và đếm cảnh báo

```

@dashboard_bp.get("/announce-count")
def announce_count():
    db = firestore.client()
    q = (
        db.collection("readings")
        .order_by("ts", direction=firestore.Query.DESCENDING)
        .limit(60)
    )
    cnt = 0
    for d in q.stream():
        r = d.to_dict()
        t = r.get("temperature")
        turb = r.get("turbidity")
        if t is not None and (t < 24 or t > 30):
            cnt += 1
        if turb is not None and turb > 200:
            cnt += 1
    return jsonify({"count": cnt})

```

Chức năng: Duyệt 60 bản ghi gần nhất và đếm số lần xuất hiện giá trị vượt ngưỡng an toàn: nhiệt độ ngoài khoảng 24–30 °C, độ đục vượt 200 NTU. Kết quả trả về là tổng số cảnh báo (count).

API /dashboard/summary – Thống kê tổng quan hệ thống

```

@dashboard_bp.get("/summary")
def summary():

```

```

db = firestore.client()
ponds = db.collection("readings").stream()
devices = db.collection("feed_logs").stream()

pond_count = len(list(ponds))
device_count = len(list(devices))
return jsonify({
    "ponds": pond_count,
    "devices": device_count
})

```

b) Module Control (điều khiển từ Web)

File control/routes.py chịu trách nhiệm nhận yêu cầu điều khiển từ web và gửi lệnh qua MQTT, cụ thể

- Gửi lệnh điều khiển đến thiết bị IoT thông qua giao thức MQTT (ví dụ: bật/tắt đèn, cho cá ăn)
- Quản lý và lưu trữ lịch cho ăn tự động trong Cloud Firestore.
- Cung cấp các API RESTful để giao diện web (Feed Timer Page) có thể gửi lệnh điều khiển, thêm, xóa hoặc hiển thị danh sách lịch.

Thông qua module này, người dùng có thể thao tác trực tiếp trên web để:

- Kích hoạt cho ăn ngay lập tức (“Feed Now”).
- Thiết lập thời gian, chu kỳ cho ăn (“Add Schedule”).
- Xem và quản lý danh sách lịch (“List/Delete Schedule”).

1. Cấu trúc tổng thể

```

from flask import Blueprint, request, jsonify, current_app
import paho.mqtt.client as mqtt
import json, uuid, time
from firebase_admin import firestore

control_bp = Blueprint("control_bp", __name__)
_mqtt_pub_client = None

```

- Blueprint("control_bp"): tạo module Flask riêng cho các API điều khiển.
- paho.mqtt.client: thư viện dùng để publish MQTT message đến broker (ví dụ: HiveMQ Cloud).
- firebase_admin.firestore: dùng để ghi lại log và lịch điều khiển vào Firestore.
- _mqtt_pub_client: lưu giữ MQTT client toàn cục, đảm bảo chỉ khởi tạo một lần trong suốt vòng đời ứng dụng.

2. Kết nối MQTT

```
def _get_pub():
    global _mqtt_pub_client
    if _mqtt_pub_client:
        return _mqtt_pub_client

    cfg = current_app.config
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,
client_id="aquanova-control-pub")
    user = cfg.get("MQTT_USER")
    if user:
        client.username_pw_set(user, cfg.get("MQTT_PASS"))

    client.tls_set()
    client.connect(cfg.get("MQTT_HOST"), int(cfg.get("MQTT_PORT",
8883)), keepalive=60)
    client.qos = 1
    _mqtt_pub_client = client
    return _mqtt_pub_client
```

Chức năng:

- Tạo MQTT publisher dùng để gửi lệnh điều khiển đến các thiết bị.
- Các tham số (host, port, user, password) được lấy từ tệp cấu hình config.py.
- Sử dụng TLS encryption (port 8883) đảm bảo an toàn khi truyền dữ liệu.
- Sau khi khởi tạo, client được lưu vào biến _mqtt_pub_client để tái sử dụng.

3. API /control/light – Điều khiển đèn chiếu sáng

```
@control_bp.post("/light")
def toggle_light():
    data = request.get_json(force=True) or {}
    light_val = data.get("light")

    if light_val not in (0, 1):
        return jsonify({"error": "light must be 0 or 1"}), 400

    topic = "aquanova/control"
    payload = {"light": light_val}
    _get_pub().publish(topic, json.dumps(payload), qos=1)
```

Chức năng: Gửi lệnh bật/tắt đèn đến thiết bị IoT qua topic aquanova/control. Dữ liệu gửi đi dạng JSON:

```
127.0.0.1 - - [16/Oct/2025 12:00:11] "POST /control/feed-now
HTTP/1.1" 200 - {'light': 1}
[Light] Publishing {'light': 1} → aquanova/control
{"light": 1} // Bật đèn
{"light": 0} // Tắt đèn
```


4. API /control/feed-now – Kích hoạt cho ăn ngay lập tức

```
@control_bp.post("/feed-now")
def feed_now():
    data = request.get_json(force=True)
    amount = data.get("amount", 20)
    topic = "aquanova/control"
    payload = {"feeding": 1}
    _get_pub().publish(topic, json.dumps(payload), qos=1)
    db.collection("feed_logs").add({
        "amount": amount,
        "timestamp": firestore.SERVER_TIMESTAMP
    })
    return jsonify({"ok": True})
```

Mã thành công

```
127.0.0.1 - - [16/Oct/2025 12:00:11] "POST /control/feed-now
HTTP/1.1" 200 -
{'cmd': 'feed', 'amount': 10}
[FEED-NOW] Publishing {'feeding': 1} → aquanova/control
```

Khi người dùng bấm “Feed Now” trên web, lệnh được gửi tới MQTT Broker qua topic aquanova/control với mức QoS = 1, đảm bảo lệnh không bị mất.

5. API /control/schedule – Tạo lịch cho ăn tự động

```
@control_bp.post("/schedule")
def add_schedule():
    data = request.get_json(force=True) or {}
    required = ("date", "time", "repeat")
    for key in required:
        if not data.get(key):
            return jsonify({"error": f"{key} required"}), 400

    sid = uuid.uuid4().hex[:12]
    item = {
        "id": sid,
        "date": data.get("date"),
        "time": data["time"],
        "repeat": data["repeat"],
        "created_at": int(time.time())
    }

    db = firestore.client()
    db.collection("schedules").document(sid).set(item)
```

Mã nhận thành công

```
127.0.0.1 - - [23/Oct/2025 08:30:15] "POST /control/schedule
HTTP/1.1" 200 -
{'cmd': 'add_schedule', 'date': '2025-10-23', 'time': '08:30',
'repeat': 'daily', 'amount': 20}
```

```
[SCHEDULE] Added: {'id': 'a4c19e3bdf21', 'date': '2025-10-23',  
'time': '08:30', 'repeat': 'daily', 'created_at': 1734947400}
```

Chức năng:

- Nhận dữ liệu từ giao diện web (ngày, giờ, chu kỳ, khối lượng thức ăn).
- Tạo mã định danh (UUID) cho mỗi lịch.
- Lưu lịch vào collection schedules trong Firestore.
- APScheduler sẽ tự động đọc và thực hiện các lịch này vào đúng thời điểm.

Bảng tóm tắt chuỗi sự kiện cho ăn

Giai đoạn	Giao thức	Dữ liệu gửi đi	Mục tiêu
1	HTTP POST	{ "cmd": "feed", "amount": 10 }	Gửi từ trình duyệt đến Flask
2	MQTT Publish	{ "feeding": 1 }	Flask gửi đến ESP32
3	Firestore Log	{ "timestamp": ..., "source": "manual" }	Lưu nhật ký cho ăn

6. API /control/schedules – Hiển thị danh sách lịch

Chức năng:

- Truy xuất tất cả các lịch đã tạo từ collection schedules.
- Dữ liệu trả về để hiển thị trong bảng “Current Schedules” trên trang web.

```
@control_bp.get("/schedules")  
def list_schedules():  
    db = firestore.client()  
    docs = db.collection("schedules").stream()  
    items = [doc.to_dict() for doc in docs]  
    return jsonify({"items": items})
```

7. API /control/schedules/<sid> – Xóa lịch cho ăn

```
@control_bp.delete("/schedules/<sid>")  
def delete_schedule(sid):  
    db = firestore.client()  
    ref = db.collection("schedules").document(sid)  
    if not ref.get().exists:  
        return jsonify({"error": "Schedule not found"}), 404
```

```
ref.delete()
return jsonify({"ok": True})
```

Bảng 13: Luồng hoạt động của Module Control

Bước	Thành phần	API / MQTT Topic	Mô tả hoạt động
1	Người dùng thao tác trên web (Feed Timer Page)	/control/feed-now	Flask gửi lệnh MQTT {feeding:1} đến ESP32
2	Người dùng thêm lịch	/control/schedule	Flask lưu lịch vào Firestore và APScheduler đăng ký job
3	Đến thời gian quy định	MQTT Topic aquanova/control	Flask gửi lệnh cho ăn tự động
4	ESP32 nhận lệnh và thực hiện	MQTT Subscribe	Servo kích hoạt rả thức ăn
5	Flask ghi log	feed_logs, schedules	Lưu lịch sử điều khiển và cập nhật trạng thái

c) Bộ lập lịch tự động (Scheduler)

Module Feeding Scheduler là thành phần tự động hóa quá trình cho ăn định kỳ trong mô hình AquaNova. Thay vì người dùng phải kích hoạt thủ công, bộ lập lịch sẽ:

- Đọc các lịch cho ăn đã được người dùng lưu trên Firestore (collection schedules).
- So sánh thời gian hiện tại với thời gian đã hẹn.
- Gửi lệnh MQTT đến thiết bị ESP32 để thực hiện cho ăn tự động.
- Ghi lại log hoạt động vào collection feed_logs.

Nhờ đó, việc cho ăn được diễn ra chính xác theo thời gian mà người dùng thiết lập, ngay cả khi không có thao tác từ giao diện web.

1. Cấu trúc tổng quan

```
from apscheduler.schedulers.background import BackgroundScheduler
from datetime import datetime, date, timedelta
from firebase_admin import firestore
import json
from .routes import _get_pub
scheduler = BackgroundScheduler()
```

Giải thích:

- APScheduler (Advanced Python Scheduler): thư viện giúp chạy các tác vụ định kỳ (cron / interval).
- BackgroundScheduler: hoạt động song song (chạy nền) cùng Flask mà không làm gián đoạn các request web.
- firestore: dùng để truy cập các lịch cho ăn lưu trong cơ sở dữ liệu Firebase.
- _get_pub(): hàm tái sử dụng từ module control.routes để gửi lệnh MQTT publish đến thiết bị.

2. Hàm check_schedules() – Kiểm tra và thực thi lịch cho ăn

Đây là hàm trung tâm, được gọi mỗi phút để kiểm tra có lịch thực hiện không.

```
//1 Truy xuất dữ liệu lịch từ Firestore
db = firestore.client()
docs = db.collection("schedules").stream()
//2 Kiểm tra thời gian thực tế so với lịch
now = datetime.now()
hh, mm = map(int, sched_time.split(":"))
is_time_match = (
    now.hour == hh and now.minute == mm and abs(now.second) < 30
)
//3 Xác định loại lặp lại (repeat)
if repeat == "none":          # chỉ thực hiện 1 lần, xóa sau khi chạy
elif repeat == "daily":       # thực hiện mỗi ngày cùng giờ
elif repeat == "weekly":      # thực hiện mỗi tuần cùng ngày & giờ
//4 Gửi lệnh cho ăn qua MQTT
topic = "aquanova/control"
payload = {"feeding": 1}
_get_pub().publish(topic, json.dumps(payload), qos=1)
//5 Ghi log sau khi thực thi
db.collection("feed_logs").add({
    "amount": amount,
    "timestamp": firestore.SERVER_TIMESTAMP,
})
//6 Xóa lịch nếu chỉ chạy 1 lần
if repeat == "none":
    db.collection("schedules").document(sched_id).delete()
    print(f"[SCHEDULE] Deleted one-time schedule {sched_id}")
```

3.7.5. Database

Kết nối cơ sở dữ liệu Firestore. Firestore được chọn vì khả năng đồng bộ thời gian thực, dễ mở rộng và hỗ trợ tốt với Python thông qua firebase_admin. Khởi tạo trong file firebase_admin_init.py:

```
import os
```

```

import firebase_admin
from firebase_admin import credentials, firestore
from config import Config

firebase_app = None
db = None

def init_firebase():
    global firebase_app, db
    cred_path = Config.FIREBASE_CREDENTIALS
    print(f"[Firebase Init] Using credentials: {cred_path}")

    if not cred_path or not os.path.isfile(cred_path):
        raise RuntimeError(f"Service account not found: {cred_path}")

    if not firebase_admin._apps:
        cred = credentials.Certificate(cred_path)
        firebase_app = firebase_admin.initialize_app(cred)
    db = firestore.client()
    return db

def get_db():
    global db
    if db is None:
        init_firebase()
    return db

```

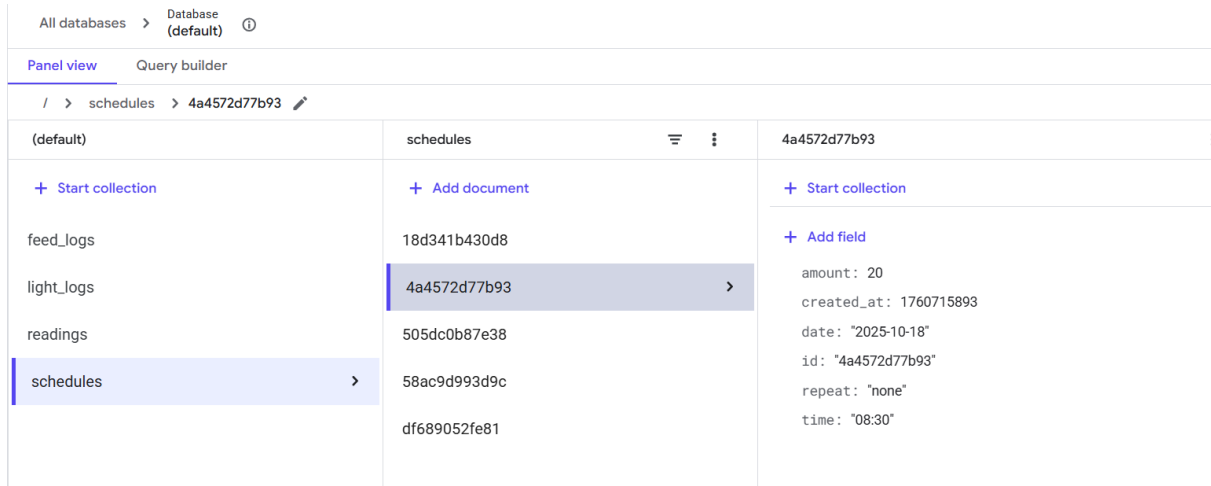
Bảng 14: Danh sách collection lưu trên cloud Firestore

Collection	Chức năng	Các trường chính	Nguồn dữ liệu
readings	Lưu dữ liệu cảm biến mới nhất	temperature, turbidity, ts, device_id	ESP32 gửi qua MQTT → Flask ghi vào Firestore
feed_logs	Ghi nhật ký các lần cho ăn	timestamp, source, amount	Flask khi gửi lệnh feed hoặc scheduler chạy
light_logs	Lưu trạng thái bật/tắt đèn	timestamp, state, source	Flask khi người dùng điều khiển
schedules	Lưu danh sách lịch cho ăn tự động	id, date, time, repeat, amount	Người dùng tạo qua giao diện web

Dữ liệu được tổ chức theo các collection:

- readings/ – lưu dữ liệu cảm biến (temperature, turbidity, feed, ts).
- schedules/ – lưu các lịch cho ăn tự động (date, time, repeat, amount).

- feed_logs/ – lưu lịch sử cho ăn thực tế.



Hình 59: Collection truyền đến firebase

3.7.6. Giao thức IoT – MQTT

a. MQTT Listener (Subscriber)

Module MQTT Listener có nhiệm vụ duy trì kết nối giữa hệ thống Flask và các thiết bị IoT (ESP32/STM32) thông qua giao thức MQTT. Giúp nhận dữ liệu cảm biến (temperature, turbidity, feed status) và tự động ghi vào Firestore Database, đồng thời ghi log hoạt động để phục vụ giám sát và gỡ lỗi.

Lưu ý: Sử dụng thư viện paho-mqtt làm client để giao tiếp với HiveMQ Cloud

Để thực hiện điều này, module MQTT Listener được xây dựng nhằm duy trì kết nối liên tục với HiveMQ Cloud thông qua giao thức MQTT: cụ thể lắng nghe các bản tin telemetry từ thiết bị, ghi dữ liệu vào Firestore, và ghi log sự kiện để phục vụ giám sát và khắc phục lỗi qua

1. Cơ chế kết nối ban đầu : Hàm on_connect đảm bảo hệ thống tự động bắt đầu lắng nghe dữ liệu cảm biến ngay khi có kết nối mạng ổn định.

```
def on_connect(client, userdata, flags, reason_code,
               properties=None):
    write_log(f"[MQTT] CONNECTED with code={reason_code}")
    try:
        result, mid = client.subscribe(cfg["MQTT_TOPIC"], qos=1)
        write_log(f"[MQTT] SUBSCRIBE sent
topic={cfg['MQTT_TOPIC']} mid={mid} result={result}")
    except Exception as e:
        write_log(f"[MQTT] SUBSCRIBE ERROR: {e}")
```

Chức năng:

- Được gọi tự động khi client kết nối thành công tới broker.
- Ghi log trạng thái CONNECTED cùng mã phản hồi (reason_code = 0 nghĩa là thành công).

2. Cơ chế xử lý ngắt kết nối (on_disconnect) : Tự động được gọi khi mất kết nối đến broker (do mất mạng hoặc lỗi TLS).

```
def on_disconnect(client, userdata, disconnect_flags, reason_code,
properties=None):
    write_log(f"[MQTT] DISCONNECTED (reason_code={reason_code}) –
attempting reconnect...")

    while True:
        try:
            client.reconnect()
            write_log("[MQTT] RECONNECTED successfully!")
            break
        except Exception as e:
            write_log(f"[MQTT] Reconnect failed: {e}")
            time.sleep(5)
```

Ví dụ

```
2025-10-23T17:00:11 [MQTT] DISCONNECTED (reason_code=4) –
attempting reconnect...
2025-10-23T17:00:17 [MQTT] Reconnect failed: [Errno 110]
Connection timed out
2025-10-23T17:00:22 [MQTT] RECONNECTED successfully!
```

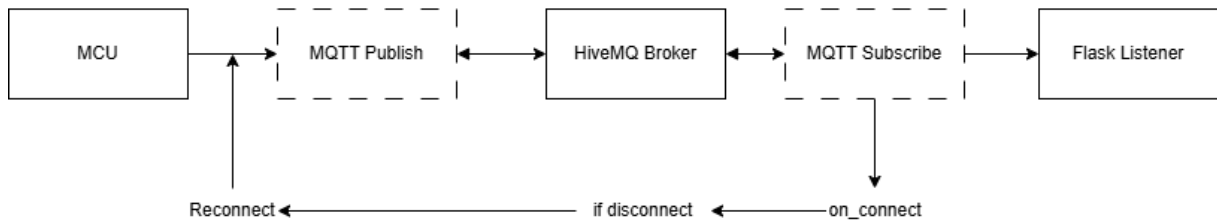
3. Khởi động kết nối nền

- Chạy vô hạn trong thread nền, đảm bảo Flask có thể xử lý web song song.
- loop_forever() giữ kết nối liên tục và kích hoạt callback mỗi khi có sự kiện (connect/disconnect/message).
- Khi kết nối thất bại (do mất mạng hoặc broker bảo trì), tiến trình sẽ:
 - o Ghi log lỗi.
 - o Ngủ 5 giây → thử kết nối lại.

```
def mqtt_loop():
    while True:
        try:
            write_log(f"[MQTT] 🔄 Connecting to
{cfg['MQTT_HOST']}:{cfg['MQTT_PORT']}")
            client.connect(cfg["MQTT_HOST"], int(cfg["MQTT_PORT"]),
keepalive=60)
            client.loop_forever(retry_first_connection=True)
        except Exception as e:
```

```
write_log(f"[MQTT] Connection error: {e}")
time.sleep(5)
```

Hình 60: Luồng kết nối MQTT



Khởi tạo và kết nối. Sau đó đăng kí chủ đề.

```
def start_mqtt_background(app):
    client = mqtt.Client(
        mqtt.CallbackAPIVersion.VERSION2,
        client_id=f"aquanova-server-{uuid.uuid4().hex[:6]}"
    )
    client.username_pw_set(cfg["MQTT_USER"], cfg["MQTT_PASS"])
    # tạo client với mã định danh duy nhất và đăng kí chủ đề
    aquanova/telemetry
    client.tls_set()
    client.connect(cfg["MQTT_HOST"], int(cfg["MQTT_PORT"]),
        keepalive=60)
    client.on_message = on_message
    client.subscribe(cfg["MQTT_TOPIC"], qos=1)
    client.loop_forever()
```

Khi nhận bản tin từ thiết bị: Mỗi khi có bản tin MQTT gửi đến, callback `on_message` được kích hoạt. Dữ liệu được giải mã, thêm timestamp (ts) và lưu vào Firestore collection readings

```
def on_message(client, userdata, msg):
    payload = msg.payload.decode("utf-8")
    data = json.loads(payload)
    data.setdefault("ts", datetime.now(timezone.utc).isoformat())
    db.collection("readings").add(data)
```

Ví dụ gói tin

```
2025-10-23T17:47:40.229846 [MQTT] MESSAGE topic=aquanova/telemetry
payload={'turbidity': 176.8, 'temperature': 29, 'feed': 0,
'thoi_gian': '17:46:39', 'ts': '2025-10-23T10:47:39.936766+00:00'}
```


Luồng dữ liệu hoạt động

Bước	Giao thức	Nguồn	Đích	Hành động
1	MQTT Publish	ESP32	Broker	Gửi dữ liệu cảm biến
2	MQTT Subscribe	Flask	Broker	Nhận bản tin qua topic aquanova/data
3	Python JSON	Flask	Firestore	Ghi dữ liệu vào collection readings
4	Flask API	Web dashboard	HTTP	Truy xuất dữ liệu để hiển thị biểu đồ

b. MQTT Publisher (Flask gửi lệnh điều khiển)

File `mqtt/publisher.py` định nghĩa lớp `MQTTPublisher`, chịu trách nhiệm publish lệnh đến thiết bị IoT.

Khởi tạo và kết nối:

```
self.client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2,
    client_id=f"aquanova-publisher-{datetime.now().timestamp()}"
)
self.client.username_pw_set(user, pwd)
self.client.tls_set()
self.client.connect(cfg["MQTT_HOST"], int(cfg["MQTT_PORT"]),
    keepalive=60)
self.client.loop_start()
```

Gửi lệnh điều khiển:

```
def publish(self, topic: str, payload: dict, qos: int = 1):
    msg = json.dumps(payload)
    result = self.client.publish(topic, msg, qos=qos)
    result.wait_for_publish()
```

Việc gọi `wait_for_publish()` đảm bảo gói tin được gửi hoàn tất trước khi tiếp tục xử lý, giúp hệ thống ổn định hơn.

Trong AquaNova, QoS = 1 được chọn cho cả subscribe và publish:

```
client.subscribe(cfg["MQTT_TOPIC"], qos=1)
client.publish("aquanova/control", msg, qos=1)
```

Điều này đảm bảo rằng:

- Dữ liệu hoặc lệnh sẽ đến ít nhất một lần, không bị mất.
- Flask và ESP32 đều nhận được xác nhận (PUBACK) từ Broker.
- Nếu có lỗi mạng, client sẽ tự gửi lại cho đến khi thành công.

3.8. Hiệu chuẩn và kiểm thử

3.8.1. Đánh giá độ chính xác dữ liệu của môi trường và thiết bị đo

Mạch điện tử ảnh hưởng trực tiếp đến khả năng đọc và xử lý tín hiệu cảm biến.

- Độ phân giải ADC: Vi điều khiển có ADC 12-bit (như STM32 hoặc ESP32) cho phép phép đo chính xác hơn so với ADC 10-bit trên Arduino.
- Nhiễu điện: Các thiết bị như bơm, đèn chiếu sáng có thể gây nhiễu tín hiệu, đặc biệt là đối với cảm biến analog. Một mạch tốt cần có bộ lọc nhiễu và cách ly quang để ổn định tín hiệu.
- Hiệu chuẩn (Calibration) là bước quan trọng nhất trong quá trình đảm bảo độ chính xác. Cảm biến dù tốt đến đâu cũng sẽ cho kết quả sai nếu không được hiệu chuẩn đúng cách. Cảm biến hóa học như pH, TDS hay ORP thường bị trôi giá trị (drift) theo thời gian do bụi bẩn hoặc tảo bám, vì vậy cần hiệu chuẩn định kỳ (khoảng 1–3 tháng/lần).

Cảm biến nhiệt độ:

- Sử dụng nhiệt kế chuẩn (thủy ngân hoặc điện tử phòng thí nghiệm) để so sánh.
- Đặt cảm biến và nhiệt kế vào cùng một vị trí trong nước, chờ ổn định 1–2 phút.
- Sai số từ $\pm 0.2^{\circ}\text{C}$ đến $\pm 0.5^{\circ}\text{C}$ được xem là tốt; $\pm 1.0^{\circ}\text{C}$ vẫn chấp nhận được.

Về cảm biến độ đục thì so sánh với mẫu nước chuẩn: Chuẩn bị các mẫu nước có độ đục đã biết (ví dụ 0, 50, 100, 400 NTU), thường do nhà sản xuất cung cấp hoặc được xác định bằng thiết bị chuẩn phòng thí nghiệm.

- Thực hiện đo nhiều lần: Ghi lại kết quả từ cảm biến và so sánh với giá trị chuẩn để tính sai lệch trung bình.
- Tốt: sai số dưới ± 5 NTU trong khoảng đo dưới 100 NTU.
- Chấp nhận được: sai số dưới ± 10 NTU hoặc dưới 10% giá trị đo.

3.8.2. Đánh giá đường truyền kết nối wifi

Việc đánh giá đường truyền kết nối WiFi đóng vai trò cực kỳ quan trọng nhằm đảm bảo khả năng kết nối, độ ổn định và hiệu suất truyền dữ liệu của module ESP32

trong hệ thống giám sát. Mục tiêu chính là xác nhận rằng dữ liệu từ các cảm biến có thể được truyền tải liên tục, nhanh chóng và không gặp sự cố đến MQTT Broker, từ đó đưa lên hệ thống giám sát.

Quá trình kiểm thử bắt đầu bằng việc đánh giá cường độ tín hiệu nhận được (RSSI) của mạng WiFi tại các vị trí triển khai dự kiến của mô hình. RSSI là một chỉ số logarit đo lường công suất tín hiệu nhận được, biểu thị bằng dBm. Giá trị này càng gần 0 dBm thì tín hiệu càng mạnh. Trong môi trường truyền dẫn không dây, cường độ tín hiệu giảm dần theo khoảng cách và bị suy hao bởi vật cản. Mối quan hệ giữa công suất phát P_T , công suất nhận P_R và suy hao đường truyền L_P có thể được mô tả đơn giản như sau:

$$P_R = P_T - L_P \quad (6) \text{ (dBm)}$$

Trong đó, L_P là tổng độ suy hao trên đường truyền, bao gồm suy hao không gian tự do (Free Space Path Loss - FSPL) và suy hao do vật cản.

FSPL được tính theo công thức:

$$FSPL = 20\log_{10}(d) + 20\log_{10}(f) + 20\log_{10}\left(\frac{4\pi}{c}\right) \text{ (dB)} \quad (7)$$

Với d là khoảng cách giữa thiết bị phát và nhận (mét), f là tần số sóng (Hz), và c là vận tốc ánh sáng (3×10^8 m/s).

Bằng cách sử dụng một đoạn mã đơn giản trên ESP32, giá trị RSSI của mạng đích sẽ được quét và ghi nhận. Sau đó, với firmware hoàn chỉnh, module được khởi động để kiểm tra khả năng tự động kết nối và nhận địa chỉ IP từ router. Thời gian từ khi khởi động đến khi kết nối thành công cũng được ghi nhận, đảm bảo tốc độ phản hồi của hệ thống. Cuối cùng, để kiểm tra khả năng truyền tải dữ liệu MQTT, sau khi ESP32 đã kết nối WiFi và Broker, một MQTT Client độc lập (ví dụ: MQTT Explorer) được sử dụng để đăng ký vào các chủ đề mà ESP32 gửi dữ liệu. Dữ liệu nhận được trên MQTT Client sẽ được đối chiếu với dữ liệu mà ESP32 báo cáo qua Serial Monitor để đảm bảo tính toàn vẹn và chính xác.

```
#include <WiFi.h>

const char* ssid = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";

void setup() {
```

```

Serial.begin(115200);
delay(10);
Serial.println();
Serial.print("Connecting to ");
Serial.println(ssid);

WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

Serial.println("");
Serial.println("WiFi connected!");
Serial.print("IP address: ");
Serial.println(WiFi.localIP());
Serial.print("RSSI: ");
Serial.print(WiFi.RSSI());
Serial.println(" dBm");
}

void loop() {
}

```

Một vấn đề quan trọng cần lưu ý là khả năng truy cập mạng. Một số mạng WiFi, đặc biệt là các mạng công cộng hoặc mạng của các tổ chức (ví dụ: mạng Wimesh-HCMUTE), thường áp dụng các chính sách bảo mật nghiêm ngặt. Các chính sách này có thể bao gồm việc chặn các cổng truyền thông nhất định (ví dụ: cổng 1883 cho MQTT non-TLS hoặc 8883 cho MQTTS), hoặc chỉ cho phép truy cập Internet sau khi người dùng đã xác thực qua một trang đăng nhập (Captive Portal). Đối với các trường hợp này, module ESP32 sẽ không thể thiết lập kết nối MQTT đến Broker do bị chặn ở tầng mạng hoặc không thể hoàn tất quá trình xác thực. Việc này yêu cầu người dùng phải chủ động cung cấp thông tin đăng nhập hoặc cấu hình mạng WiFi cho phép các kết nối outbound đến cổng MQTT. Do đó, việc lựa chọn mạng WiFi phù hợp và có quyền truy cập đầy đủ là điều kiện tiên quyết để hệ thống hoạt động.

Các tiêu chí đánh giá bao gồm: khả năng kết nối WiFi thành công 10/10 lần khởi động (trừ các trường hợp bị chặn bởi chính sách mạng), chỉ số RSSI duy trì ở mức ổn định và chấp nhận được (thường từ -30 dBm đến -75 dBm, với các giá trị thấp hơn -75 dBm có nguy cơ gây mất ổn định kết nối), dữ liệu MQTT được truyền tải đầy đủ và chính xác

100% so với dữ liệu gốc, và hệ thống duy trì kết nối liên tục không gián đoạn trong một khoảng thời gian dài kiểm tra (ví dụ: tối thiểu 2 giờ).

Trong quá trình thử nghiệm thực tế, khi mô hình được đặt cách router 3 mét không vật cản, chỉ số RSSI trung bình đạt -45 dBm. Khi di chuyển mô hình đến vị trí cách router 10 mét, xuyên qua một lớp tường, RSSI trung bình vẫn duy trì ở mức -68 dBm. Các giá trị này nằm trong ngưỡng hoạt động ổn định của ESP32. Ở cả hai vị trí, ESP32 đều thiết lập kết nối WiFi thành công trong 10/10 lần khởi động, với thời gian kết nối trung bình khoảng 1.5 giây. Dữ liệu từ các cảm biến như nhiệt độ và độ ẩm, được gửi lên MQTT Broker với tần suất 30 giây/lần, đã được MQTT Client nhận đầy đủ và chính xác, không ghi nhận bất kỳ sự sai lệch hay mất mát nào. Hệ thống được chạy liên tục trong 2 giờ và không xảy ra tình trạng mất kết nối WiFi hay gián đoạn truyền dữ liệu.

Tuy nhiên, khi thử nghiệm kết nối với mạng Wimesh-HCMUTE, module ESP32 đã không thể thiết lập kết nối MQTT thành công mặc dù đã kết nối được WiFi, điều này được xác định là do chính sách bảo mật mạng đã chặn các kết nối ra cổng MQTT.

3.8.3. Độ trễ và tần suất cập nhật

a) Độ trễ của STM32 gửi dữ liệu tới ESP32

Để đo lường độ trễ một cách chính xác, phương pháp thử nghiệm "ping-pong" hay "echo test" (thử nghiệm phản hồi) đã được áp dụng. Phương pháp này yêu cầu cấu hình hai thiết bị với vai trò rõ rệt:

1. Thiết bị đo (Pinger): Vi điều khiển STM32 được lập trình để khởi tạo phép đo. Để đạt được độ phân giải thời gian cao nhất, bộ đếm chu kỳ CPU (DWT Cycle Counter) đã được sử dụng thay vì các bộ đếm thông thường (như SysTick, vốn có độ phân giải 1ms). Quy trình bắt đầu bằng việc STM32 gửi một gói tin dữ liệu ("PING_TEST\n") và đồng thời kích hoạt bộ đếm chu kỳ.
2. Thiết bị phản hồi (Ponger): Mô-đun ESP32 được lập trình với một tác vụ duy nhất là lắng nghe trên cổng UART. Ngay khi nhận được bất kỳ gói tin nào, nó sẽ lập tức gửi trả lại (echo) chính xác gói tin đó mà không qua xử lý phức tạp.

STM32 sẽ ở trạng thái chờ nhận. Ngay khi nhận lại thành công gói tin phản hồi, nó sẽ dừng bộ đếm chu kỳ. Giá trị của bộ đếm chính là tổng số chu kỳ CPU cho toàn bộ quá trình, được định nghĩa là Thời gian khứ hồi (Round-Trip Time - RTT).

Thử nghiệm được tiến hành với các thông số cấu hình: tốc độ baud là 9600 bit/giây (bps), gói tin thử nghiệm là "PING_TEST\n" (10 byte). Kết quả đo lường thực tế ghi nhận được một giá trị RTT ổn định, xấp xỉ 20.8 mili-giây (ms).

Để kiểm chứng tính hợp lệ của kết quả này, tiến hành phân tích lý thuyết. Thời gian truyền tải một gói tin qua UART phụ thuộc vào tốc độ baud và tổng số bit của gói tin. Trong giao thức UART tiêu chuẩn (8N1), mỗi byte dữ liệu (8 bit) yêu cầu thêm 1 bit bắt đầu (start bit) và 1 bit kết thúc (stop bit), nâng tổng số bit cho mỗi byte lên 10 bit.

- Tổng số bit truyền tải: $10 \text{ byte} * 10 \text{ bit/byte} = 100 \text{ bit}$.
- Thời gian truyền 1 chiều (One-way):

$$T = \frac{\sum \text{bit}}{\sum \text{Baud}} = \frac{100 \text{ bit}}{9600 \frac{\text{bit}}{\text{s}}} = 0.010416 \text{ s} \quad (8)$$

- Chuyển đổi sang mili-giây:

$$0.010416 * 1000 = 10.4 \text{ (ms)} \quad (9)$$

a) Độ trễ của ESP32 gửi dữ liệu tới STM32

Trong hệ thống nhúng, độ trễ truyền dữ liệu (communication latency) giữa hai vi điều khiển là một thông số quan trọng ảnh hưởng trực tiếp đến hiệu năng thời gian thực. Khi ESP32 truyền dữ liệu đến STM32 thông qua giao thức UART (Universal Asynchronous Receiver and Transmitter), tổng độ trễ T_{total} có thể được biểu diễn như sau:

$$T_{total} = T_{tx} + T_{rx} + T_{proc} \quad (10)$$

Trong đó:

- + T_{tx} : thời gian truyền dữ liệu từ ESP32 qua đường UART.
- + T_{rx} : thời gian STM32 nhận toàn bộ khung dữ liệu.
- + T_{proc} : thời gian xử lý phần mềm (bao gồm ngắt, lưu đệm, và đọc dữ liệu).

Đối với giao tiếp UART, thời gian truyền T_{tx} được tính dựa trên số lượng bit và tốc độ baud (baud rate) theo công thức:

$$T_{tx} = \frac{N_{bit}}{R_{baud}} \quad (11)$$

Trong đó:

- + $N_{bit} = N_{byte} \times (1 + N_{data} + N_{stop})$ là tổng số bit cần truyền (bao gồm bit start và bit stop).
- + R_{baud} : tốc độ baud (bit/s).

Ví dụ, với tốc độ truyền $R_{baud} = 9600$ bps và gói dữ liệu gồm $N_{byte} = 20$, trong đó mỗi byte chứa 8 bit dữ liệu, 1 bit start và 1 bit stop, ta có:

$$N_{bit} = 20 \times (1 + 8 + 1) = 200 \text{ bit} \quad (12)$$

$$T_{tx} = \frac{200}{9600} \approx 2.08 \text{ ms}$$

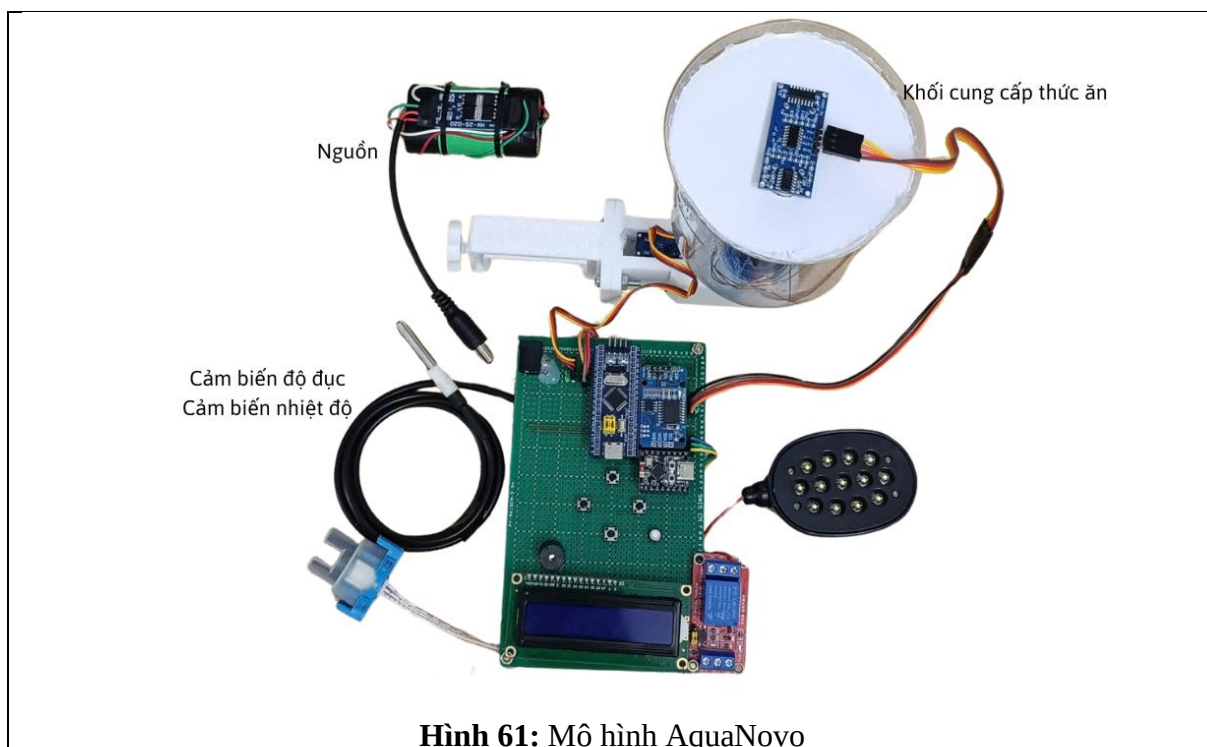
Khi tính thêm thời gian xử lý ngắt và ghi dữ liệu vào bộ đệm (buffer) ở cả hai vi điều khiển, tổng độ trễ thực tế có thể đạt khoảng 2–4 ms.

Các yếu tố ảnh hưởng đến độ trễ bao gồm: tốc độ baud, chế độ xử lý (ngắt hay polling), mức ưu tiên tác vụ trong FreeRTOS của ESP32, và cách STM32 xử lý nhận dữ liệu (DMA hoặc CPU polling).

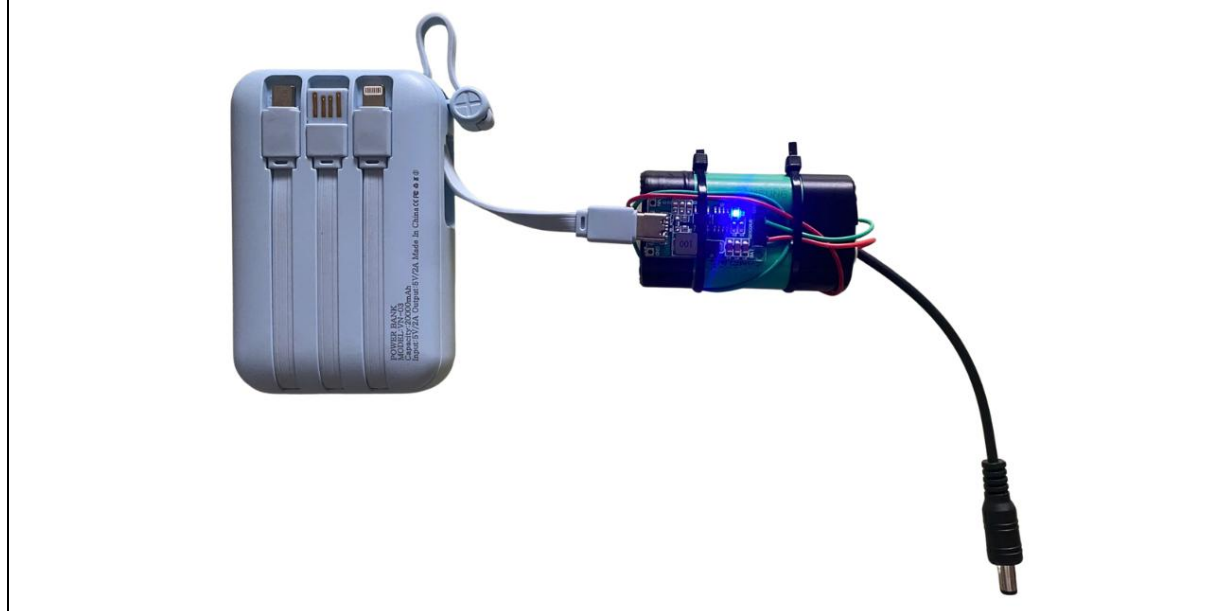
CHƯƠNG 4

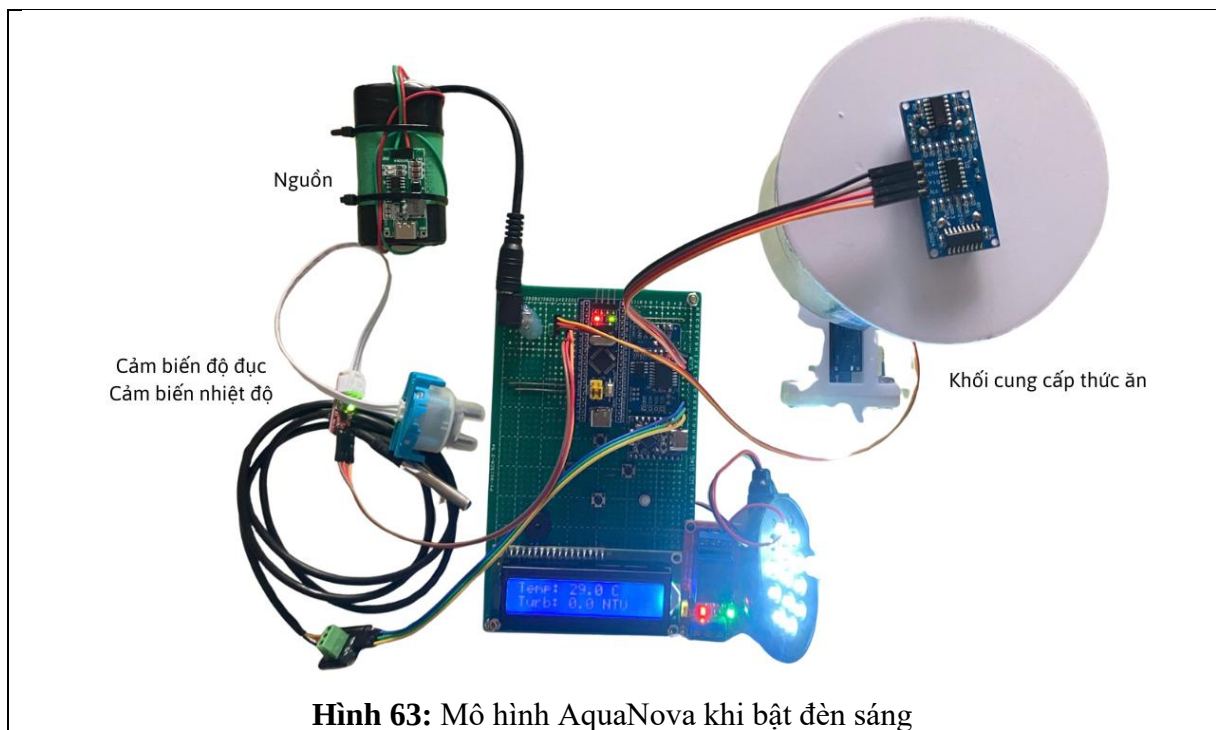
THỰC NGHIỆM, KIỂM CHỨNG VÀ ĐÁNH GIÁ

4.1. Kết quả thiết kế phần cứng



Hình 62: Khối nguồn hiện đèn báo khi cắm sạc





Hình 63: Mô hình AquaNova khi bật đèn sáng

Mô hình IoT AquaNova đặt trên hồ cá tại nhà với hình bên trái theo hướng trực diện và hình bên phải theo góc chụp từ trên xuống

Hình 64: Hình ảnh thực tế mô hình AquaNova vận hành thành công



Xem thêm kết quả trong video đính kèm phía dưới báo cáo

4.1. Kết quả thiết kế phần mềm

Kết quả khởi động AquaNova: Flask – Firestore – HiveMQ kết nối thành công

WARNING: This is a development [server](#). Do not use it in a production [deployment](#). Use a production WSGI server instead.

* Running on <http://127.0.0.1:5000>

Press CTRL+C to quit

* Restarting with stat

[Firebase Init] Using credentials:

C:\Users\ASUS\Documents\GitHub\IoT_AquaNova\IoT_AquaNova\serviceAccount.json

2025-10-18T02:39:51.821093 [MQTT]  Connecting to fdcab86156bf4f86973c13d0a7fe4e2c.s1.eu.hivemq.cloud:8883

2025-10-18T02:39:51.821433 [MQTT] LISTENING
fdcab86156bf4f86973c13d0a7fe4e2c.s1.eu.hivemq.cloud:8883
topic=aquanova/devices/telemetry

[Scheduler] Started – checking schedules every minute

* Debugger is active!

* Debugger PIN: 177-325-123

2025-10-18T02:39:52.077474 [MQTT] ☒ CONNECTED with code=Success

2025-10-18T02:39:52.078085 [MQTT] SUBSCRIBE sent

topic=aquanova/devices/telemetry mid=1 result=0

2025-10-18T02:39:52.335059 [MQTT] SUBACK mid=1,
granted_qos=[ReasonCode(Suback, 'Granted QoS 1')]

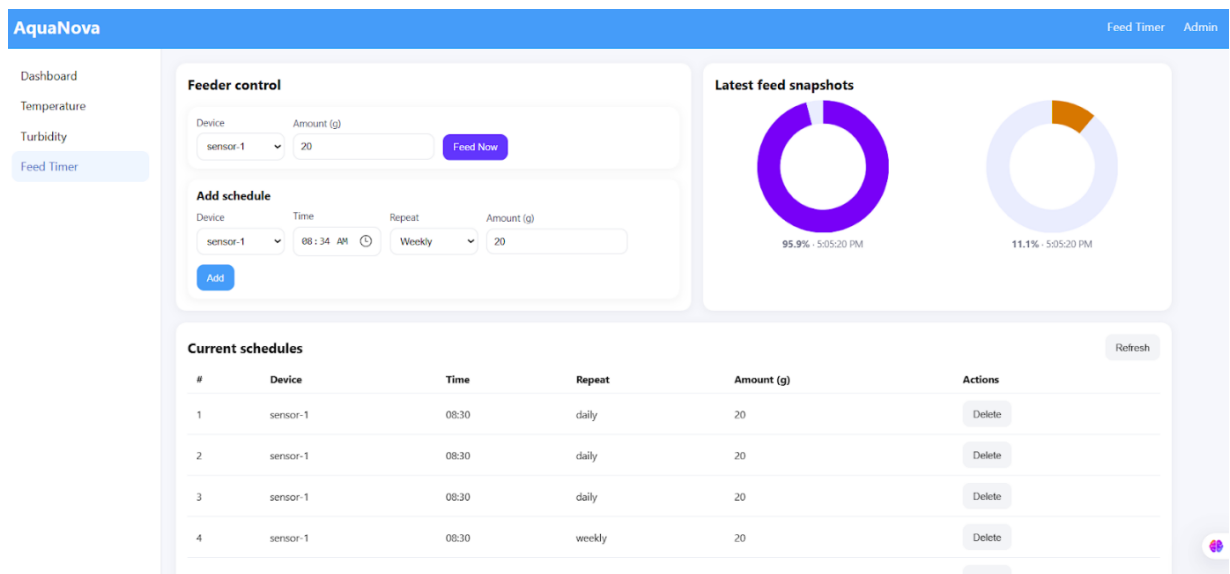
2025-10-18T02:39:53.355134 [MQTT] ☒ CONNECTED with code=Success

2025-10-18T02:39:53.356642 [MQTT] SUBSCRIBE sent

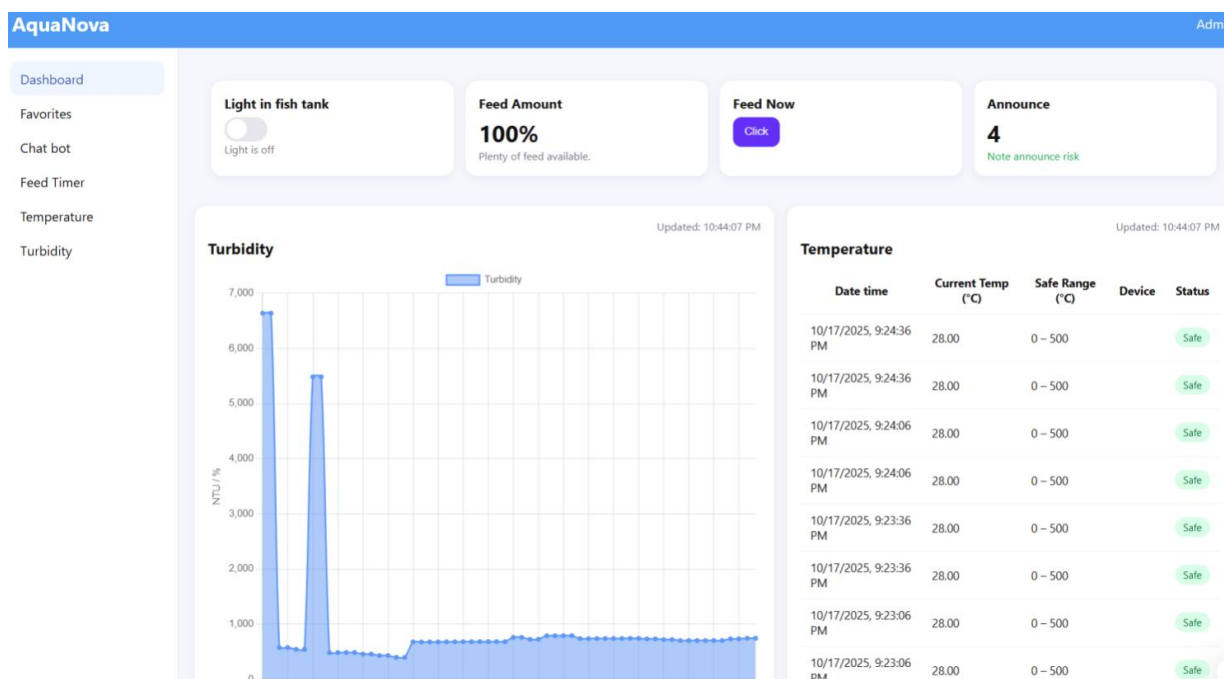
topic=aquanova/devices/telemetry mid=1 result=0

2025-10-18T02:39:55.665998 [MQTT] SUBACK mid=1,
granted_qos=[ReasonCode(Suback, 'Granted QoS 1')]

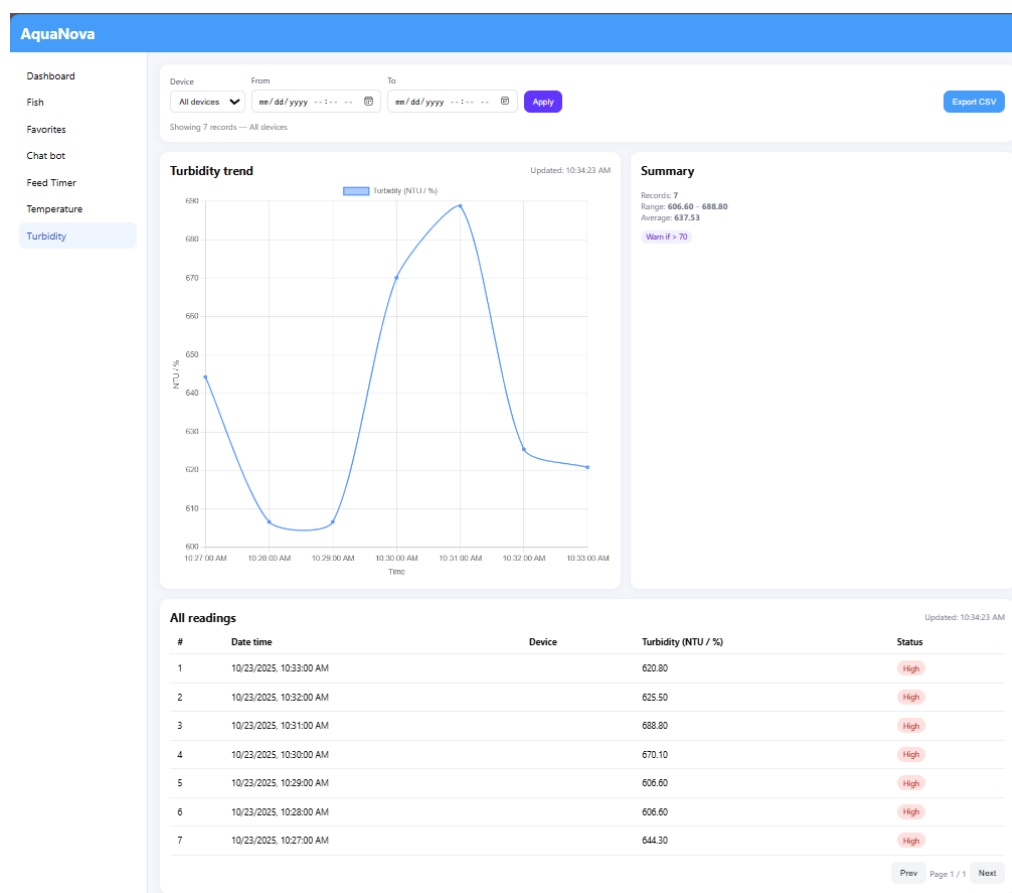
- Tiếp đến là giao diện hiển thị của mô hình AquaNova



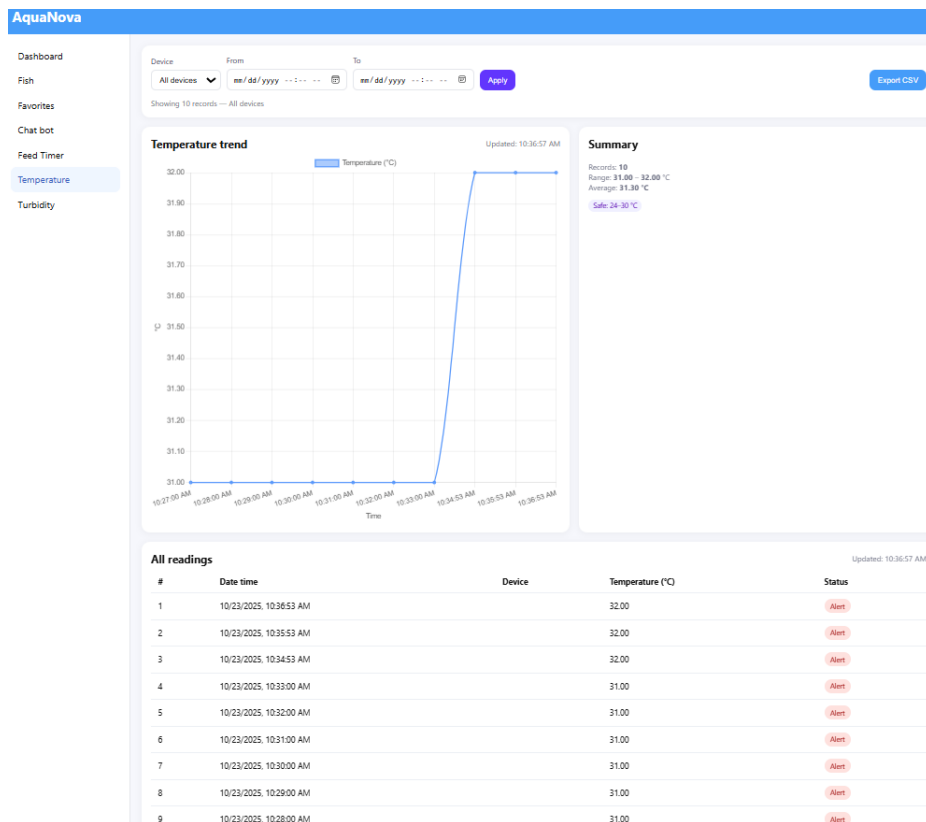
Hình 65: Giao diện web trang Feed Timer (menu bên trái và biểu đồ bên phải)



Hình 66: Giao diện home hiển thị chính của web AquaNova



Hình 67: Giao diện web trang Turbidity đo độ đục



Hình 68: Giao diện web trang hiển thị nhiệt độ

- Gói tin thực tế hiển thị trên HiveMQ và Firebase.

Subscriber web server: Dữ liệu này chứng minh ESP32 đã gửi dữ liệu thành công, Flask nhận được sau khoảng 50–60 ms (so sánh giữa thời gian của thiết bị và ts ghi tại server).

Ngoài ra, các dòng log GET /dashboard/latest và GET /dashboard/last biểu thị trình duyệt người dùng (web dashboard) liên tục gọi API để lấy dữ liệu mới nhất mỗi 3–5 giây, cho phép hiển thị gần thời gian thực.

```
127.0.0.1 - - [23/Oct/2025 10:34:15] "GET /dashboard/latest?n=60 HTTP/1.1" 200 -
127.0.0.1 - - [23/Oct/2025 10:34:20] "GET /dashboard/last HTTP/1.1" 200 -
127.0.0.1 - - [23/Oct/2025 10:34:20] "GET /dashboard/announce-count HTTP/1.1" 200 -
127.0.0.1 - - [23/Oct/2025 10:34:20] "GET /dashboard/latest?n=60 HTTP/1.1" 200 -
127.0.0.1 - - [23/Oct/2025 10:34:22] "GET /turbidity HTTP/1.1" 200 -
127.0.0.1 - - [23/Oct/2025 10:34:23] "GET /dashboard/latest?n=500 HTTP/1.1" 200 -
2025-10-23T10:34:53.153021 [MQTT] MESSAGE topic=aquanova/telemetry
payload={'turbidity': 575.7, 'temperature': 32, 'feed': 0,
'thoi_gian': '10:34:03', 'ts': '2025-10-23T03:34:53.061396+00:00'}
```

```
2025-10-23T10:35:53.203115 [MQTT] MESSAGE topic=aquanova/telemetry
payload={'turbidity': 590, 'temperature': 32, 'feed': 0,
'thoi_gian': '10:45:03', 'ts': '2025-10-23T03:35:53.077685+00:00'}
```

Publisher web server: Bảng log này thể hiện tiến trình gửi lệnh điều khiển từ web đến ESP32 qua MQTT topic aquanova/devices/telemetry hoặc aquanova/control/....

Các dòng POST /control/feed-now hoặc POST /control/light chứng tỏ người dùng đã thao tác trên giao diện web để:

- Feed now: bật cơ chế cho cá ăn tức thời.
- Light: bật/tắt đèn chiếu sáng hồ.
- Mỗi lệnh POST được Flask xử lý, sau đó publish một gói tin MQTT chứa thông tin lệnh (feeding=1, light=1, v.v.).
- Các dòng [MQTT] MESSAGE topic=aquanova/devices/telemetry là phản hồi (acknowledge) từ thiết bị gửi ngược về server, xác nhận lệnh được nhận thực thi.

```
[LIGHT] Publishing {'light': 0} → aquanova/control
[LIGHT] Publishing {'light': 1} → aquanova/control
127.0.0.1 - - [24/Oct/2025 19:59:44] "POST /control/light HTTP/1.1"
200 -
127.0.0.1 - - [24/Oct/2025 19:59:45] "POST /control/light HTTP/1.1"
200 -
[LIGHT] Publishing {'light': 0} → aquanova/control
127.0.0.1 - - [24/Oct/2025 19:59:49] "POST /control/light HTTP/1.1"
200 -
[LIGHT] Publishing {'light': 1} → aquanova/control
127.0.0.1 - - [24/Oct/2025 19:59:50] "POST /control/light HTTP/1.1"
200 -
[LIGHT] Publishing {'light': 0} → aquanova/control
127.0.0.1 - - [24/Oct/2025 19:59:52] "POST /control/light HTTP/1.1"
200 -
[FEED-NOW] Publishing {'feeding': 1} → aquanova/control
127.0.0.1 - - [24/Oct/2025 19:59:53] "POST /control/feed-now
HTTP/1.1" 200 -
2025-10-24T20:00:10.587820 [MQTT] MESSAGE topic=aquanova/telemetry
payload={'turbidity': 132.9, 'temperature': 29, 'feed': 0,
'thoi_gian': '19:59:06', 'ts': '2025-10-24T13:00:10.305273+00:00'}
```

Phản hồi ngược về server đã hoàn tất trong vòng dưới 0.5 giây. Terminal đặt lịch

```
2025-10-24T21:23:42.437787 [MQTT] MESSAGE topic=aquanova/telemetry
payload={'turbidity': 202.4, 'temperature': 28, 'feed': 0,
'thoi_gian': '21:22:39', 'ts': '2025-10-24T14:23:42.166958+00:00'}
```

```

127.0.0.1 - - [24/Oct/2025 21:23:49] "GET /dashboard/latest?n=200
HTTP/1.1" 200 -
127.0.0.1 - - [24/Oct/2025 21:24:18] "GET /dashboard/latest?n=200
HTTP/1.1" 200 -
127.0.0.1 - - [24/Oct/2025 21:24:20] "GET /dashboard/summary
HTTP/1.1" 200 -
[DEBUG] Checking schedules at 21:24:22
[SCHEDULE] Feed → aquanova/control (repeat=none)
[SCHEDULE] Published feed successfully.
[SCHEDULE] Deleted one-time schedule f8c0b403ba81

```

4.2. Mô phỏng một số kịch bản

Để kiểm chứng hoạt động của hệ thống, nhóm tiến hành mô phỏng một số kịch bản thực tế nhằm đánh giá khả năng đo đạc, phản hồi và điều khiển tự động của mô hình

Kịch bản 1 – Kiểm thử cảm biến độ đục

Mục tiêu: Đánh giá khả năng phát hiện và hiển thị sự thay đổi độ đục của nước.

Thực hiện:

- Bơm nước sạch vào hồ (nước trong).
- Ghi nhận giá trị cảm biến hiển thị trên LCD và web dashboard.
- Thêm các hạt cặn hoặc bột vào hồ để tạo độ đục nhân tạo.
- Quan sát sự thay đổi giá trị đo được từ cảm biến và phản hồi trên giao diện.

Kết quả mong đợi:

Trạng thái nước	Giá trị đo (NTU)	Hiển thị LCD	Kết luận
Nước trong	0–100 NTU	“Turb: 45.3 NTU”	Chính xác
Nước đục	>200 NTU	“Turb: 264.1 NTU”	Chính

Kịch bản 2 – Kiểm thử chức năng cảnh báo tự động

Mục tiêu: Đánh giá khả năng phát hiện và phản hồi cảnh báo khi các thông số môi trường (nhiệt độ, độ đục) vượt ngưỡng cho phép; xác minh việc hiển thị cảnh báo trên web dashboard.

Thực hiện:

1. Thiết lập hệ thống hoạt động bình thường, quan sát các giá trị ổn định trong khoảng:
 - Nhiệt độ: 26–29°C
 - Độ đục: <150 NTU

2. Làm nóng nước hoặc thêm vật liệu cần để làm tăng độ đục >250 NTU và nhiệt độ >31°C.

3. Theo dõi:

- Cảnh báo hiển thị trên dashboard (mục “Feed Status” hoặc phần thông báo).
- Số lượng bản ghi bất thường tăng lên trong Firestore collection readings.
- Log Flask hiển thị /dashboard/announce-count trả về count > 0.

Kết quả mong đợi:

Thông số	Ngưỡng bình thường	Giá trị đo	Cảnh báo hiển thị	Kết luận
Nhiệt độ	24–30°C	31.5°C	“High Temperature”	Hệ thống phát hiện chính xác
Độ đục	≤200 NTU	264 NTU	“High Turbidity”	Phản hồi nhanh, hiển thị đúng

Kịch bản 3 – Mất kết nối MQTT và tự động khôi phục

Mục tiêu: Đánh giá khả năng phát hiện và tự động khôi phục kết nối MQTT của hệ thống khi xảy ra gián đoạn mạng hoặc broker tạm ngắt kết nối. Đồng thời kiểm tra vi Thực hiện:

- Khởi chạy Flask server và MQTT Listener hoạt động bình thường.
- Xác nhận log hiển thị: việc duy trì nhận dữ liệu cảm biến liên tục từ ESP32
- Ngắt kết nối Internet hoặc tạm thời dừng broker HiveMQ Cloud trong 1–2 phút.
- Quan sát log trong file mqtt_sub_log.txt
- Sau khi khôi phục lại mạng/broker, theo dõi log tiếp theo:

Kết quả:

```
2025-10-15T16:11:17.610749 [MQTT] LISTENING
d2b2655505394c9b8e88c06c8d30d912.s1.eu.hivemq.cloud:8883
topic=aquanova/telemetry
2025-10-15T16:02:41.512233 [MQTT] SUBSCRIBE sent
topic=aquanova/telemetry mid=1 result=0
2025-10-15T16:02:41.799483 [MQTT] SUBACK mid=1,
granted_qos=[ReasonCode(Suback, 'Granted QoS 1')]
2025-10-15T16:02:45.595980 [MQTT] ☑ CONNECTED with code=Success
2025-10-15T16:02:45.597272 [MQTT] SUBSCRIBE sent
topic=aquanova/telemetry mid=1 result=0
2025-10-15T16:02:45.858955 [MQTT] SUBACK mid=1,
granted_qos=[ReasonCode(Suback, 'Granted QoS 1')]
2025-10-15T16:10:43.290035 [MQTT] ⚠ DISCONNECTED
(reason_code=Unspecified error) – attempting reconnect...
2025-10-15T16:10:44.243775 [MQTT] ☑ RECONNECTED successfully!
```

```
2025-10-15T16:10:46.410528 [MQTT] ☒ CONNECTED with code=Success
2025-10-15T16:10:46.412244 [MQTT] SUBSCRIBE sent
topic=aquanova/telemetry mid=2 result=0
2025-10-15T16:10:46.690518 [MQTT] SUBACK mid=2,
granted_qos=[ReasonCode(Suback, 'Granted QoS 1')]
```

Hệ thống MQTT hoạt động ổn định và tự động phục hồi sau sự cố ngắt kết nối. Cơ chế mid tăng dần giúp nhận biết từng lần subscribe, tránh trùng lặp trong các phiên khác nhau. Không có lỗi (result=0). Điều này chứng minh kết nối đến HiveMQ Cloud ổn định và được xác thực TLS thành công. Quá trình reconnect hoàn tất trong ~3 giây, đảm bảo tính liên tục cho hệ thống IoT thời gian thực.

4.3. Nhận xét, đánh giá và kết quả

Các cảm biến hoạt động ổn định, phản hồi nhanh và dữ liệu được cập nhật tức thời lên Firestore. Module MQTT Listener duy trì kết nối tin cậy với broker, tự động phục hồi khi mất mạng. Giao diện web Flask hiển thị chính xác, đồng bộ thời gian thực với dữ liệu Firebase. Bộ lập lịch (Scheduler) hoạt động chính xác, đảm bảo các tác vụ cho ăn được thực hiện đúng thời gian. Mô hình đạt tính toàn vẹn dữ liệu: mọi hành động (đo, cảnh báo, điều khiển, cho ăn) đều được ghi lại trong Firestore để phục vụ giám sát.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Trong suốt quá trình nghiên cứu, thiết kế, lập trình và thi công mô hình AquaNova đo độ đục và quản lý bể cá thông minh, tôi đã gặp không ít khó khăn, từ việc lựa chọn cảm biến phù hợp, thiết kế mạch phần cứng, đến xử lý truyền thông giữa STM32 và ESP32. Nhìn chung, hệ thống đã thể hiện được tính khả thi, hoạt động đúng theo nguyên lý và có tiềm năng phát triển thành một sản phẩm ứng dụng thực tế. Tuy nhiên, bên cạnh những ưu điểm đạt được, mô hình vẫn tồn tại một số hạn chế nhất định cần được cải thiện trong tương lai.

Ưu điểm:

- a) Hệ thống vận hành ổn định, đọc và xử lý dữ liệu độ đục chính xác, phản hồi nhanh.
- b) Kết nối UART giữa STM32 và ESP32 đơn giản, truyền dữ liệu ổn định, ít lỗi.
- c) Giao tiếp dữ liệu qua MQTT giúp dễ dàng giám sát từ xa, tiện cho ứng dụng IoT.
- d) Cấu trúc mạch gọn gàng, chi phí thi công thấp hơn nhiều so với các hệ thống thương mại có cùng chức năng.
- e) Có khả năng mở rộng thêm nhiều cảm biến khác (pH, DO, mực nước) và điều khiển thiết bị như bơm, quạt, hoặc máy lọc nước.
- f) Tự động hóa được các tác vụ như cho ăn, lọc và thay nước theo độ đục hoặc thời gian định sẵn, giảm thao tác thủ công.

Nhược điểm:

- g) Hệ thống chưa có cơ chế mã hóa hoặc xác thực dữ liệu, dẫn đến tính bảo mật chưa cao.
- h) Cảm biến độ đục dễ bị ảnh hưởng bởi môi trường (ánh sáng ngoài, cặn, rêu bám), cần hiệu chuẩn định kỳ.
- i) Chưa có khả năng tự động kết nối lại WiFi nếu mất mạng, cần reset thủ công để khởi động lại ESP32.
- j) Giao diện giám sát còn đơn giản, chưa có cảnh báo thông minh hoặc lưu trữ dữ liệu dài hạn.

- k) Thuật toán xử lý còn cơ bản, chưa tối ưu về thời gian và khả năng phân tích dữ liệu môi trường.

Đánh giá: qua quá trình thử nghiệm thực tế, mô hình đã đáp ứng được mục tiêu đề ra: đo và hiển thị giá trị độ đục, truyền dữ liệu ổn định và điều khiển các thiết bị trong hệ thống. Các chức năng cơ bản đều hoạt động đúng yêu cầu, cho thấy thiết kế mạch và phần mềm có tính ổn định cao. Tuy vậy, để mô hình trở thành một sản phẩm hoàn chỉnh và ứng dụng được trong thực tế, cần tiếp tục nâng cấp về bảo mật, khả năng tự động hóa, giao diện giám sát và độ chính xác của cảm biến.

5.2. Hướng phát triển

Trong tương lai, mô hình có thể được mở rộng và hoàn thiện hơn nhằm nâng cao tính ổn định, độ tin cậy và khả năng ứng dụng thực tế trong các mô hình bể cá thông minh hoặc hệ thống nuôi trồng thủy sản tự động. Để có thể hoàn thiện cũng như thương mại hóa được sản phẩm, cần phải nghiêm túc đặt ra những mục tiêu, hướng phát triển sau này cho hệ thống:

- a) Tăng cường bảo mật: mã hóa gói tin trước khi gửi qua MQTT để đảm bảo an toàn dữ liệu trong quá trình truyền.

Sử dụng giao thức bảo mật TLS/SSL để mã hóa toàn bộ luồng dữ liệu giữa client (ESP32) và MQTT Broker. Khi đó, dù người tấn công bắt được gói tin, nội dung vẫn không thể đọc hoặc giải mã. Tạo và cài đặt chứng chỉ số (SSL certificate) cho cả broker và client:

- Broker dùng file .crt và .key để xác thực.
- ESP32 nạp chứng chỉ CA vào flash (thông qua code hoặc SPIFFS).

Thay vì phụ thuộc vào tầng giao vận, ta mã hóa trực tiếp nội dung gói tin (payload) trước khi gửi đi. Khi đó, dữ liệu trong gói tin MQTT vẫn được gửi qua cổng 1883, nhưng phần nội dung bị mã hóa, không thể đọc được nếu không có khóa giải mã tương ứng. Các thuật toán có thể dùng:

- AES-128 hoặc AES-256 (Advanced Encryption Standard) — phù hợp cho ESP32 vì có hỗ trợ phần cứng.
- Base64 thường đi kèm để chuyển dữ liệu nhị phân thành chuỗi text.

- b) Tự động điều chỉnh lượng thức ăn dựa vào độ đục hoặc lịch định sẵn.

Dữ liệu đầu vào là độ đục (NTU) từ cảm biến có thể biểu thị mức độ trong của nước theo nguyên tắc nếu độ đục cao thì nước đục, có thể do dư thức ăn. Vì thế giảm hoặc ngưng cho ăn. Ngược lại nếu độ đục thấp → nước trong, có thể cho ăn thêm.

Thiết bị có thể kết hợp với thời gian định sẵn để không phụ thuộc hoàn toàn vào cảm biến. Việc tối ưu có thể thực hiện bằng cách sử dụng PID control hoặc fuzzy logic để điều chỉnh lượng (tính toán thời gian quay của động cơ thức ăn) hay học dần (machine learning nhẹ) dựa vào lịch sử độ đục sau mỗi lần cho ăn.

c) Tích hợp chức năng gửi lịch hẹn giờ cho ăn (Timer Scheduling) và phân luồng gói tin MQTT

Trong phiên bản hiện tại, hệ thống chỉ publish dữ liệu cảm biến dạng JSON duy nhất lên broker MQTT. Tuy nhiên, để hệ thống trở nên linh hoạt và mở rộng hơn trong tương lai, việc bổ sung chức năng gửi lịch hẹn giờ và phân luồng xử lý gói tin là cần thiết.

Về phía STM32:

Khi người dùng thiết lập lịch cho ăn, STM32 sẽ tạo một gói tin đặc tả theo cú pháp:

TIMER,08:00:50

Trong đó, 08:00:50 là thời gian hẹn giờ cho ăn được thiết lập, TIMER là tiền tố giúp phân luồng 2 loại gói tin, với gói tin đọc dữ liệu từ cảm biến tiền tố sẽ là DATA, việc này giúp ESP32 có thể dễ dàng nhận biết loại dữ liệu thông qua tiền tố.

Về phía ESP32 (Arduino):

ESP32 sẽ được mở rộng hàm giải mã chuỗi nhận từ UART. Khi phát hiện tiền tố:

- Nếu chuỗi bắt đầu bằng "DATA", ESP32 sẽ đóng gói thành JSON và publish lên topic: aquanova/telemetry
- Nếu chuỗi bắt đầu bằng "TIMER", ESP32 sẽ nhận diện là lịch hẹn giờ và publish lên topic khác, chẳng hạn: aquanova/set_timer

Việc phân luồng gói tin theo topic riêng giúp server dễ dàng xử lý, lưu trữ và hiển thị từng loại dữ liệu. Ngoài ra, cách thiết kế này cũng hỗ trợ mở rộng thêm các loại gói tin khác trong tương lai như "ALERT", "CONTROL" hoặc "STATUS" mà không ảnh hưởng đến cấu trúc chương trình hiện có.

Tính năng này tạo nền tảng cho việc nâng cấp AquaNova từ một hệ thống IoT cơ bản sang mô hình đa kênh dữ liệu (multi-topic IoT), đảm bảo khả năng mở rộng, bảo trì và tích hợp với các hệ thống giám sát thông minh trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] N. Hiền, "Quan, Thuế & Hải - Chuyên trang của tạp chí kinh tế - tài chính," 12 1 2024. [Online]. Available: <https://thuehaiquan.tapchikinhtetaichinh.vn//tphcm-xuat-khau-12-trieu-con-ca-canh-mang-ve-125-trieu-usd-182372.html>. [Accessed 12 9 2025].
- [2] N. M. Đ. Diệp Thị Quế Ngân, "Hiện trạng nuôi cá cảnh giải trí của người dân thành phố Hồ Chí Minh," Bộ môn Quản Lý và Phát triển Nghề Cá, Đại Học Nông Lâm TP. Hồ Chí Minh, 2011.
- [3] R. Burhan, R. A. Rehman, B. Khan, and B. Kim, "IoT Elements, Layered Architectures, and Security Issues: A Comprehensive Survey," *Sensors*, vol. 18, no. 9, p. 2796, Aug. 2018. [Online]. Available: <https://doi.org/10.3390/s18092796>
- [4] "ISO 7027-1:2016 - Water quality — Determination of turbidity," International Organization for Standardization, 2016, <https://www.iso.org/standard/62801.html>
- [5] J. Trevathan, T. Atchison, L. Read, P. Hitchens, and W. Read, "Towards the Development of an Affordable and Practical Turbidity Sensor," *Sensors*, vol. 20, no. 7, p. 1993, 2020, <https://doi.org/10.3390/s20071993>
- [6] J. Droujko and P. Molnar, "Open-source, low-cost, in-situ turbidity sensor for river network monitoring," *Scientific Reports*, vol. 12, no. 10341, 2022, <https://doi.org/10.1038/s41598-022-14228-4>
- [7] R. Isoyama, M. Taie, T. Kageyama, M. Miura, A. Maeda, A. Mori, and S.-S. Lee, "A Feasibility Study on the Simultaneous Sensing of Turbidity and Chlorophyll a Concentration Using a Simple Optical Measurement Method," *Micromachines*, vol. 8, no. 4, p. 112, 2017, <https://doi.org/10.3390/mi8040112>
- [8] L. C. Sperandio, M. S. Colombo, C. M. G. Andrade, and C. B. B. Costa, "Development of a Low-Cost Portable Turbidimeter for Processes," *arXiv preprint arXiv:2106.09491*, 2021, <https://arxiv.org/abs/2106.09491>
- [9] MULTICOMP, HC-SR04 Ultrasonic Distance Sensor Datasheet, Alldatasheet, 2024. [Online]. Available: <https://www.alldatasheet.com/datasheet-pdf/pdf/2216428/MULTICOMP/HC-SR04.htm>

- [10] “How to measure the turbidity in water using Arduino,” EngineersGarage, Apr. 19, 2021. [Online]. Available: <https://www.engineersgarage.com/how-to-measure-the-turbidity-in-water-using-arduino/>
- [11] Google Firebase, “Firebase Documentation,” Google Developers, 2025. [Online]. Available: <https://firebase.google.com/docs>
- [12] Google Firebase, “Cloud Firestore Overview,” Firebase Documentation, 2025. [Online]. Available: <https://firebase.google.com/docs/firestore>
- [13] Maxim Integrated, “*DS18B20 Programmable Resolution 1-Wire Digital Thermometer*,” Rev. 6, 2019. [Online]. Available: <https://datasheets.maximintegrated.com/en/ds/DS18B20.pdf>
- [14] DFRobot, “*Gravity: Analog Turbidity Sensor/SEN0189*,” 2019. [Online]. Available: https://wiki.dfrobot.com/Turbidity_sensor_SKU_SEN0189
- [15] Elecfreaks, “*HC-SR04 Ultrasonic Sensor Module*,” v2.0, 2018. [Online]. Available: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>
- [16] STMicroelectronics, “*STM32F103x8 Datasheet*,” Rev. 19, 2023. [Online]. Available: <https://www.st.com/resource/en/datasheet/stm32f103c8.pdf>
- [17] Winstar Display Co., “*1602A LCD Module Technical Data*,” 2018. [Online]. Available: <https://www.winstar.com.tw/products/character-lcd-display-module/wh1602a.html>
- [18] Espressif Systems, “*ESP32-C3-MINI-1 Datasheet*,” Ver. 1.4, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-c3-mini-1_datasheet_en.pdf
- [19] Maxim Integrated, “*DS3231 Extremely Accurate I²C-Integrated RTC/TCXO/Crystal*,” Rev. 10, 2020. [Online]. Available: <https://datasheets.maximintegrated.com/en/ds/DS3231.pdf>
- [20] TowerPro, “*Micro Servo SG90 Datasheet*,” Rev. 2.1, 2015. [Online]. Available: <https://cdn.sparkfun.com/datasheets/Robotics/SG90datasheet.pdf>

PHỤ LỤC

A: Tính toán tốc độ dòng chảy

Thông số	Giá trị giả định	Ghi chú
Lưu lượng bơm	400 L/h (0.000111 m ³ /s)	Bơm mini 12V DC
Đường kính ống ra	8 mm	ống silicon thường
Tốc độ tại đầu ra	~2.2 m/s	theo công thức Q/A
Tốc độ trung bình trong hồ	0.05 – 0.10 m/s	sau khi phân tán dòng
Áp lực tương đương	3–5 kPa (0.03–0.05 bar)	cột nước 0.3–0.5 m

Công thức: tốc độ dòng chảy

$$v = \frac{Q}{A}$$

Trong đó:

- Q : lưu lượng nước (m^3/s)
- A : tiết diện dòng chảy (m^2)
- B: Mã nguồn chương trình
- Code STM32, ESP32 và Website (HTML,CSS,JS, MQTT) được phát triển từ ngày 21/9/2025 đến 24/10/2025 trên github theo link sau: G. Giunzz et al., “IoT AquaNova: Smart Aquarium Monitoring and Feeding System,” GitHub Repository, 2024. [Online]. Available: https://github.com/giunzz/IoT_AquaNova [Accessed: Oct. 24, 2025].

